GOOGLE | 288 QUESTIONS | CONCISE EDITION

Short main solutions

The same 288 questions, numbers, and 43 pattern groups as the clean full-code study path. Each entry keeps the question card, one concise optimal implementation (or the required set of implementations for a multi-part prompt), a short idea, and time/space complexity. Brute force, tests, demos, and follow-up implementations are intentionally excluded.

38	197	53
EASY	MEDIUM	HARD

Contents

288 questions | tap a group or question to jump to it | E / M / H is difficulty

START HERE: groups stay in study-path order; questions stay in interview-priority order

1. Frontier search (BFS or DFS), Tier 1: spread, count and measure distance (9) Go >

E	1. Flood Fill	ColorStack * 2026-01-28 *
M	2. Number of Islands	1Point3Acres * 2026-09-30 *
M	3. Rotten Oranges / Multi-Source BFS (taxis)	1Point3Acres * 2026-09-08 *
M	4. Find shortest relationship path using BFS	PracHub * 2026-02-12 *
M	5. Compute distance to nearest taxi in grid	PracHub * 2026-01-10 *
M	6. Solve Grid and Counting Problems	PracHub * 2025-01-25 *
M	7. Farthest City for a New Data Center	ColorStack * 2025-10-10 *
H	8. Infection Spread on a Grid (Cellular Automaton)	PracHub * 2026-06-14 *
H	9. Shortest Distance From All Buildings	ColorStack * 2025-09-22 *

2. Sweep over events in time order (9) Go >

M	10. Meeting Rooms / Interval Scheduling Variants	1Point3Acres * 2026-09-29 *
M	11. Train / Flight Schedule Reachability	1Point3Acres * 2026-10-08 *
M	12. Solve Rooms and Top-K Streams	PracHub * 2026-06-05 *
M	13. Gate Entry / Exit Time Resolution with Tie-Break Rules	1Point3Acres * 2026-05-04 *
M	14. Compute servers needed for daily recurring jobs	PracHub * 2025-11-05 *
M	15. Meeting Rooms III Variant	HackTheRounds ND
H	16. Count Overlapping Rectangle Updates on a Grid	PracHub * 2026-07-10 *
H	17. Compute Turnstile Crossing Times	PracHub * 2026-05-04 *
H	18. Compute city skyline outline	PracHub * 2025-12-08 *

3. Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward (9) Go >

E	19. Find right-side view of binary tree	PracHub * 2025-11-01 *
E	20. Given two trees, determine if the pre-order traversals of two trees are same or not.	Exponent ND
E	21. Calculate the height of a binary tree.	Exponent ND
M	22. Expression Tree Single-Leaf Mutation	1Point3Acres * 2026-09-27 *
M	23. Longest Consecutive Path in Binary Tree	1Point3Acres * 2026-07-02 *
M	24. Implement zigzag level-order traversal	PracHub * 2025-08-01 *
M	25. Given the root of a binary tree, return the length of the diameter of the tree.	Exponent ND
H	26. Repeatedly Delete Leaves Matching a Target Value From a Binary Tree	PracHub * 2026-09-16 *
H	27. Delete an Entire Binary Tree	ColorStack * 2025-09-29 *

4. A heap of what is available: pop the best, put back the changed one (9) Go >

M	28. Streaming Median (LC 295) with Follow-ups	1Point3Acres * 2026-08-03 *
M	29. Compute minimum number of rooms needed	PracHub * 2026-02-12 *
M	30. Implement a Reusable Locker Allocation System	PracHub * 2026-08-14 *
M	31. Simulate room assignments for scheduled meetings	PracHub * 2026-01-06 *
M	32. Minimum Makespan on Identical CPUs	ColorStack * 2025-09-23 *
M	33. Design server allocation for multi-type nodes	PracHub * 2025-09-06 *
M	34. Trip Expense Settlement	HackTheRounds ND
M	35. Playlist Shuffle No Adjacent Artists	HackTheRounds ND
H	36. Huffman Tree Construction	1Point3Acres * 2026-06-01 *

5. Recurrence on a position or a range (memo or table) (9) Go >

M	37. Target Expressions from Ordered Digits	1Point3Acres * 2026-08-13 *
M	38. Count Word Decompositions into Chemical Element Symbols	1Point3Acres * 2026-07-14 *
M	39. Fence Painting Minimum Operations (Gemini Team)	1Point3Acres * 2026-04-27 *
M	40. Maximize 1D deliveries within distance and minimize total distance	PracHub * 2025-12-27 *
M	41. Find the longest palindromic subsequence using dynamic programming.	Exponent ND

M	42. Jump Game with Profit	1Point3Acres ND
H	43. Count Perfect Wake Numbers up to N	1Point3Acres * 2026-05-07 *
H	44. Word Break: All Sentences	ColorStack * 2026-01-28 *
H	45. Edit distance	Exponent ND

6. Best-first search with a heap: Dijkstra and its minimax twin (9) Go >

M	46. Path Minimizing the Maximum Cell (Swim in Rising Water)	1Point3Acres * 2026-09-29 *
M	47. Nearest Favorite City by Travel Time	ColorStack * 2025-10-17 *
M	48. Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time.	Exponent ND
M	49. Find the shortest path in a graph.	Exponent ND
M	50. Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat.	Exponent ND
H	51. Minimize maximum height along a grid path	PracHub * 2026-01-04 *
H	52. Find minimum threshold enabling grid path	PracHub * 2026-02-11 *
H	53. Minimize travel time with optional meeting point	PracHub * 2026-01-06 *
H	54. Compute minimax grid path and network delay	PracHub * 2026-01-22 *

7. Backtracking: choose, recurse, undo (9) Go >

M	55. Mahjong Winning Hand Detection (14 Tiles)	1Point3Acres * 2026-10-09 *
M	56. Count Prefix-Matching Simple Paths in a Labeled Graph	PracHub * 2026-08-09 *
M	57. Determine if a 14-tile hand is winning	PracHub * 2026-03-09 *
M	58. American Football Score Combinations	ColorStack * 2025-08-25 *
M	59. Count Passcode Patterns on a Keypad	ColorStack * 2025-11-07 *
M	60. Match people to questions using tags and priorities	PracHub * 2025-12-27 *
M	61. Generate Parentheses	Exponent ND
M	62. Permutations	Exponent ND
H	63. Matrix Flower Placement with House Adjacency	1Point3Acres * 2026-05-31 *

8. Runs and groups: extend while it fits, restart or close when it does not (9) Go >

E	64. First Out-of-Order Character	ColorStack * 2026-01-30 *
E	65. Group Timestamps Into Close Intervals	ColorStack * 2025-10-03 *
M	66. Longest Non-Decreasing Subarray with One Replacement	1Point3Acres * 2026-10-04 *
M	67. Piano Hand Movement Grouping	1Point3Acres * 2026-07-14 *
M	68. Find longest increasing contiguous subarray	PracHub * 2026-01-06 *
M	69. Sum of Good Subarrays (Adjacent Diff ±1)	1Point3Acres * 2026-02-08 *
M	70. Sustained-Error Time Window	ColorStack * 2025-09-16 *
M	71. Longest Non-Decreasing Subarray From Two Arrays	ColorStack * 2025-09-24 *
M	72. Count Super Streaks	ColorStack * 2025-10-03 *

9. Union-find: merge the pairs, then ask who is together (8) Go >

M	73. Count Islands in a Grid and Track the Count as Land Cells Are Added	PracHub * 2026-10-02 *
M	74. 2D Points Connected Components by Radius	1Point3Acres * 2026-08-04 *
M	75. Count Clusters of 2D Points Within a Radius	PracHub * 2026-06-18 *
M	76. Find A Threshold-Limited Path With Minimum Required Safety	PracHub * 2026-07-06 *
M	77. Determine Reporting Relationships	PracHub * 2026-02-22 *
M	78. Determine whether two nodes are related	PracHub * 2026-01-22 *
M	79. Movie Recommendations via Transitive Similarity	ColorStack * 2025-10-09 *
H	80. Friend Graph with Timeline (Friend / Unfriend Events)	1Point3Acres * 2026-09-29 *

10. A dict of per-key state that you update, expire, or rank (8) Go >

E	81. Robot Status Message Deduplication Window	1Point3Acres * 2026-09-10 *
M	82. Active Communication Top Users	1Point3Acres * 2026-09-22 *
M	83. Filter Repeated Robot Status Messages	PracHub * 2026-07-05 *
M	84. Next-Word Predictor	1Point3Acres * 2026-09-02 *
M	85. Batch and Streaming Log Deduplication	1Point3Acres * 2026-07-23 *

M	86. Design a logger that returns top-K messages	PracHub * 2026-01-06 *
M	87. Website Activity Analytics: Unique Users + Session Time	1Point3Acres * 2026-06-26 *
M	88. Hit Counter	HackTheRounds ND

11. Binary search on a yes/no condition (8) Go >

E	89. First Index Reaching a Threshold (Binary Search)	ColorStack * 2025-10-11 *
M	90. Cutting Ribbons (LC 1891)	1Point3Acres * 2026-08-16 *
M	91. First Bad Version with Parallel Search Follow-up	1Point3Acres * 2026-06-15 *
M	92. Solve Banana Speed and Interval Merge	PracHub * 2025-01-27 *
M	93. Given a list with response time for each server, calculate how long time it will take to process n requests.	Exponent ND
H	94. Median of Two Sorted Arrays (LC 4) with a Follow-up Variant	1Point3Acres * 2026-10-09 *
H	95. Find longest duplicated substring	PracHub * 2025-12-08 *
H	96. Find the median of two sorted arrays.	Exponent ND

12. Greedy: scan once, carry the best choice so far, act only when forced (8) Go >

E	97. Alarm for the Next Upcoming Birthday	ColorStack * 2025-10-07 *
E	98. Coins on the Table	ColorStack * 2025-09-20 *
E	99. Maximum Profit	Exponent ND
M	100. Visible People in a Line (Last Person, Then Everyone)	1Point3Acres * 2026-10-07 *
M	101. Bounded Knapsack Variant	1Point3Acres * 2026-10-16 *
M	102. Find the Largest Monotone Increasing Number	PracHub * 2026-07-04 *
M	103. Min boards to cover roof holes	PracHub * 2026-02-12 *
M	104. Jump Game II (Minimum Jumps)	ColorStack * 2025-08-19 *

13. Sets and counters: remember the first collection, check the second against it (8) Go >

E	105. Exclude First-K-Unique From Another List	ColorStack * 2025-10-17 *
E	106. Positional Matches Between Two Lists	ColorStack * 2025-10-15 *
E	107. Contains Duplicate	ColorStack * 2025-09-16 *
E	108. Find the Duplicates	Exponent ND
M	109. Reconstruct original array from doubled shuffle	PracHub * 2026-01-04 *
M	110. Min deletions to avoid overlap in first k	PracHub * 2026-02-12 *
M	111. Filter Common Elements Per Window (Two Lists)	ColorStack * 2025-10-08 *
H	112. Smallest Missing Positive Integer in Linear Time and Constant Extra Space	PracHub * 2026-09-09 *

14. bisect on a sorted list: find a position, count a range, keep it sorted (8) Go >

E	113. Count People in a Postal Code Range	ColorStack * 2025-10-19 *
M	114. Sliding-Window Mean After Removing the Largest K Values	PracHub * 2026-09-01 *
M	115. Streaming Insert + Find K-th Largest	1Point3Acres * 2026-02-04 *
M	116. Count Sorted Strings With a Given Prefix	ColorStack * 2025-10-08 *
M	117. Russian Doll Envelopes	1Point3Acres ND
M	118. Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.	Exponent ND
H	119. Find Maximum Envelope Nesting	PracHub * 2026-03-18 *
H	120. Count of Smaller Numbers After Self	HackTheRounds ND

15. Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) (8) Go >

E	121. Cake Balance Line	ColorStack * 2025-09-26 *
E	122. Find the Pivot Index (Balanced Split) in O(n)	ColorStack * 2025-09-21 *
E	123. Parity Alternating Subarray Query	HackTheRounds ND
M	124. Range Duplicate Check with Constant-Time Queries	1Point3Acres * 2026-09-29 *
M	125. Find a Horizontal Cut That Bisects Rectangle Area	PracHub * 2026-06-04 *
M	126. Check pivot after removing one element	PracHub * 2026-02-12 *
M	127. Gold Chain Split into Two Equal Halves	1Point3Acres * 2026-04-21 *
M	128. Count the number of ways to split a string.	Exponent ND

16. Sorted intervals: merge while they touch, then read the gaps or search (8) Go >

E	129. Check an Interval Against Existing Intervals for Conflicts	PracHub * 2026-08-08 *
M	130. Find Containing Range	PracHub * 2026-06-04 *
M	131. Union of Sorted Interval Lists	1Point3Acres * 2026-08-16 *
M	132. Range Module (Insert and Point Query)	ColorStack * 2026-01-20 *
M	133. Meetings Minus a Do-Not-Schedule Block	ColorStack * 2025-10-03 *
M	134. Merge Intervals	Exponent ND
H	135. Find safe travel intervals between planet influences	PracHub * 2025-11-22 *
H	136. Implement a function that processes a stream of interval data.	Exponent ND

17. Trie: nested dicts, one character per level (8) Go >

M	137. URL Request Handler Router	1Point3Acres * 2026-10-16 *
M	138. Trie Prefix Search / Autocomplete	1Point3Acres * 2026-07-02 *
M	139. Build Prefix Lookup with a Trie	PracHub * 2026-07-03 *
M	140. Greedy Longest-Match Dictionary Tokenizer	1Point3Acres * 2026-06-15 *
M	141. Check if all substrings are dictionary words	PracHub * 2026-03-11 *
M	142. Return dictionary words matching a prefix	PracHub * 2026-03-09 *
M	143. Implement Trie Insert and Exact-Word Search	PracHub * 2026-02-20 *
H	144. Implement Longest-Match Text Replacement	PracHub * 2026-01-20 *

18. A dict for lookup plus an ordered container for who is next (lazy deletion) (8) Go >

E	145. Implement Batched Undo/Redo Layer	PracHub * 2026-02-04 *
M	146. Design a restaurant waitlist system	PracHub * 2026-03-01 *
M	147. Implement Python's itertools.tee	1Point3Acres * 2026-08-20 *
M	148. Restaurant Waitlist: join / delete / find_first_match	1Point3Acres * 2026-02-01 *
M	149. Store With Cheapest-Item Removal	ColorStack * 2025-09-29 *
M	150. Round-Robin Job Scheduler	ColorStack * 2025-08-19 *
M	151. Aggregate FizzBuzz Offer Matches	ColorStack * 2025-10-13 *
H	152. Implement LRU Cache.	Exponent ND

19. Rolling state: the next state comes only from the previous one (7) Go >

E	153. Climbing Stairs	Exponent ND
E	154. Fibonacci Numbers	Exponent ND
M	155. Maximize sum without choosing adjacent elements	PracHub * 2026-01-06 *
M	156. Paint House II	ColorStack * 2025-10-17 *
M	157. Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.	Exponent ND
M	158. Write SQL code to publish the Fibonacci series.	Exponent ND
H	159. Constrained Dice Roll Sequences	ColorStack * 2025-10-06 *

20. Two pointers: move i and j toward each other (or along two lists) (7) Go >

E	160. Reverse a linked list.	Exponent ND
E	161. Sort a binary array in linear time.	Exponent ND
E	162. Is this a valid palindrome?	Exponent ND
E	163. Reverse a Sentence	Exponent ND
M	164. Find median of merged RLE arrays	PracHub * 2026-01-06 *
M	165. Run-Length Vector Storage + Dot Product	1Point3Acres * 2026-01-21 *
M	166. Count Unique Triplets That Sum to Zero	ColorStack * 2025-09-20 *

21. Walk two strings at once with two indices (7) Go >

E	167. Find the First Occurrence of a Substring	ColorStack * 2025-09-16 *
E	168. Determine if two words are sorted in lexicographic order.	Exponent ND
E	169. Return all unique substrings of a given string.	Exponent ND
M	170. Can board states be transformed?	PracHub * 2026-04-01 *
M	171. Decide string transform with directional tokens and walls	PracHub * 2025-07-15 *
M	172. Determine if two sentences are similar.	Exponent ND
H	173. Find Dictionary Matches for a Jammed String	PracHub * 2026-04-09 *

22. Simulate a machine: a class with state and one branch per kind of event (7) Go >

E	174. Mario-Style Score Tracker (OOP)	ColorStack * 2025-10-11 *
M	175. Tic-Tac-Toe System with K Players and 3-in-a-Row Win	1Point3Acres * 2026-05-18 *
M	176. Design Tic-Tac-Toe With K Players	PracHub * 2026-02-12 *
M	177. Implement iterator merging lists with filtering	PracHub * 2026-01-06 *
M	178. Infinite Deck Card Duel	ColorStack * 2025-09-26 *
M	179. Design an elevator control system	PracHub * 2025-09-06 *
H	180. Simulate In-Place Cellular State Updates	PracHub * 2026-05-04 *

23. Top-K by frequency: Counter plus nsmallest (6) Go >

M	181. Top-K Frequent in Large Logs	1Point3Acres * 2026-07-19 *
M	182. Rearrange Tasks With Cooldown	PracHub * 2026-06-03 *
M	183. Implement frequency + distance top-K queries	PracHub * 2026-02-12 *
M	184. Find top/bottom-k words in list or stream	PracHub * 2026-01-11 *
M	185. Solve chat, grid paths, and car rentals	PracHub * 2025-12-02 *
M	186. Top-K Chatters by Word Count	ColorStack * 2025-09-21 *

24. Digits and characters: peel one at a time and carry (6) Go >

E	187. Caesar Cipher: Recover Key, Encrypt, and Decrypt	ColorStack * 2026-02-08 *
M	188. Solve several coding interview problems	PracHub * 2026-03-04 *
M	189. Check Whether Digits Are Monotone Increasing	PracHub * 2026-07-04 *
M	190. Add two big integers from digit lists	PracHub * 2025-11-24 *
H	191. Check Digits and Combine Ranges	PracHub * 2026-03-29 *
H	192. Permutation Sequence	ColorStack * 2025-10-17 *

25. Look at the input once and keep only a small summary (and count your calls) (6) Go >

M	193. Streaming Points: Emit K Within Distance	1Point3Acres * 2026-06-23 *
M	194. Two-Day Log Intersection under Memory Limit	1Point3Acres * 2025-11-27 *
M	195. Find shared objects across two log files	PracHub * 2025-11-05 *
M	196. Minimize calls to find all bad test pairs	PracHub * 2025-11-05 *
H	197. Find All Bad Pairs Using runTest Oracle	1Point3Acres * 2025-11-27 *
H	198. Find maximum value in huge text file	PracHub * 2025-12-08 *

26. Grid DP: each cell from its neighbours above and to the left (5) Go >

M	199. Grid Path Count with Diagonal Moves and Checkpoints	1Point3Acres * 2026-09-29 *
M	200. Longest Same-Color Grid Sequence	1Point3Acres * 2026-09-08 *
M	201. Count Same-Color Squares in Matrix	1Point3Acres ND
H	202. Find the Longest Straight Run of One Color	PracHub * 2026-08-23 *
H	203. Count same-color squares in a character grid	PracHub * 2025-11-22 *

27. Nested structures: a stack of contexts, or recursion with a cache (5) Go >

M	204. Decode String	1Point3Acres * 2026-07-29 *
M	205. Template Variable Expansion with Follow-Ups	1Point3Acres * 2026-10-17 *
M	206. Expand Recursive Key-Value Placeholders in a Template	PracHub * 2026-08-04 *
M	207. Nested List Weight Sum (LC 339) with Follow-Ups	1Point3Acres * 2026-08-16 *
M	208. Nested Function Expression Evaluator	1Point3Acres * 2026-06-10 *

28. Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly (5) Go >

M	209. Shortest Path Nodes and Escaping a Pursuer	1Point3Acres * 2026-09-29 *
M	210. Radio Broadcast Reachability	ColorStack * 2026-01-30 *
M	211. Minimum Direction Violations in a Directed Graph	1Point3Acres * 2026-07-19 *
M	212. Compute shortest paths with blocked nodes	PracHub * 2025-09-06 *
M	213. Shortest Directed Cycle Through a Node	1Point3Acres ND

29. Topological sort (Kahn) and cycle detection (5) Go >

M	214. Course Scheduling and Dependency Variants	1Point3Acres * 2026-10-04 *

M	215. Validate Race Results (Cycle Detection)	ColorStack * 2025-10-01 *
M	216. Solve tree, graph, sliding-window problems	PracHub * 2025-08-04 *
M	217. Validate course catalog dependencies	PracHub * 2025-08-01 *
M	218. Course Schedule	Exponent ND

30. Counting pairs by looking up each partner in a Counter or a bucket (5) Go >

M	219. Progressive Array Partitioning	1Point3Acres * 2026-08-11 *
M	220. K-diff Pairs in an Array	ColorStack * 2025-09-16 *
M	221. Count Axis-Aligned Squares Among Grid Points	ColorStack * 2025-11-03 *
M	222. Sum of Two Squares in Two Ways	HackTheRounds ND
H	223. Maximum-Area Rectangle from Planar Points	1Point3Acres * 2026-09-29 *

31. Tree DP: build a parent order once, then add up from the leaves (5) Go >

M	224. Compute File System Total Size	1Point3Acres * 2026-09-02 *
M	225. Minimum-Cost Tree Disconnection	1Point3Acres * 2026-09-09 *
M	226. Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.	Exponent ND
H	227. Compute distance-sum from every tree node	PracHub * 2026-01-22 *
H	228. Tree Distance Sum (Re-root DP)	1Point3Acres * 2026-01-19 *

32. Counting or reaching an amount: the knapsack loop (5) Go >

M	229. Maximize Shelf Space (Unbounded Knapsack)	ColorStack * 2025-10-17 *
M	230. Coin Change	Exponent ND
M	231. Partition an array into two sub-arrays with equal sum.	Exponent ND
M	232. Explain how to find a target sum in an array.	Exponent ND
M	233. Reverse Coin Change	HackTheRounds ND

33. Prefix sums, Tier 2: running total plus a dict of earlier totals (5) Go >

M	234. Solve array-sum and city-community problems	PracHub * 2026-02-20 *
M	235. Maximum-Sum Subarray With Equal Endpoints	ColorStack * 2025-09-29 *
M	236. Maximum Subarray Sum With a Minimum Length Constraint	ColorStack * 2025-08-19 *
M	237. Find the maximum subarray sum.	Exponent ND
H	238. Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k.	Exponent ND

34. Group by a canonical key (5) Go >

E	239. Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.	Exponent ND
M	240. Check if all substrings are anagrams of words	PracHub * 2026-03-11 *
M	241. Group anagrams	Exponent ND
M	242. Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same.	Exponent ND
H	243. Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second?	Exponent ND

35. K-way merge: a heap of the next item from each source (5) Go >

E	244. Find k pairs with smallest sums	PracHub * 2025-11-01 *
M	245. K Largest Elements Across Sorted Lists	ColorStack * 2025-10-31 *
M	246. Sort a nearly sorted (or K sorted) array.	Exponent ND
H	247. Return top-k subset sums in descending order	PracHub * 2026-01-07 *
H	248. Merge k sorted linked lists.	Exponent ND

36. Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up (5) Go >

E	249. Next Birthday (BST In-Order Successor with Wraparound)	ColorStack * 2025-10-17 *
M	250. Find mode in a trinary search tree	PracHub * 2025-11-16 *
M	251. Next Upcoming Birthday in a Family Tree	ColorStack * 2025-10-02 *

M 252. Lowest Common Ancestor — HackTheRounds ND
M 253. Determine if a given binary tree is a binary search tree (BST). — Exponent ND

37. Random pick without repeats: swap the chosen item with the last one and pop (5) — Go >

M 254. Design set with O(1) random access — PracHub * 2026-01-01 *
M 255. Random Generator Without Repeats — ColorStack * 2025-10-10 *
M 256. Random Song Shuffler With a No-Repeat Window — ColorStack * 2025-10-06 *
M 257. Unique Random Number Generator — ColorStack * 2025-10-03 *
H 258. Shuffle array using random integer API — PracHub * 2025-12-08 *

38. Build and serialize: write a structure out, read it back with the same walk (5) — Go >

M 259. Print a Directory Tree — ColorStack * 2025-09-21 *
M 260. Construct a binary tree given its preorder and inorder traversals. — Exponent ND
M 261. Write functions to serialize and deserialize a list of strings. — Exponent ND
H 262. Serialize and Deserialize a Graph — ColorStack * 2025-09-24 *
H 263. Implement a decode and encode function of a general N-ary tree. — Exponent ND

39. Games: win or lose with a cache, and guessing with feedback (5) — Go >

M 264. Choose optimal guesses for green-only Wordle — PracHub * 2025-10-14 *
H 265. Determine winner in stack-merging game — PracHub * 2026-01-06 *
H 266. Find secret word with match-count feedback — PracHub * 2026-02-11 *
H 267. Wordle-Style 5-Letter Minimum-Guess Strategy — 1Point3Acres * 2025-12-15 *
H 268. Babylon Tile-Stack Merge Game — 1Point3Acres * 2025-12-29 *

40. Sliding window: grow on the right, shrink on the left while it is too bad (5) — Go >

M 269. Temperature Monitoring Data Structure — 1Point3Acres * 2026-09-10 *
M 270. Footpath Beauty After Repainting — ColorStack * 2025-09-22 *
M 271. Shortest Subarray with K Distinct Integers — HackTheRounds ND
M 272. Find the minimum window substring. — Exponent ND
H 273. Necklace Cut into Two Halves with Equal D/R Counts — 1Point3Acres * 2026-05-03 *

41. Parent pointers: store who is above each node and climb (5) — Go >

E 274. Tree Node. — Exponent ND
M 275. Employee Hierarchy Add / Delete — 1Point3Acres * 2026-06-10 *
M 276. Validate parent array forms a tree — PracHub * 2026-02-11 *
M 277. Root a Tree So It Becomes a Binary Tree — ColorStack * 2025-11-07 *
M 278. Lowest Common Ancestor of a Binary Tree III — HackTheRounds ND

42. Build or transform a grid with index arithmetic (5) — Go >

E 279. Filled ASCII Diamond — ColorStack * 2025-09-26 *
M 280. Connected Crop Arrangement on a Grid — 1Point3Acres * 2026-10-04 *
M 281. Solve matrix groups and recipe inventory — PracHub * 2026-01-22 *
M 282. Range Sum Query 2D (Immutable Matrix) — ColorStack * 2025-08-19 *
M 283. Rotate Any m×n 2D Matrix — 1Point3Acres ND

43. Split into pieces, fix each piece, join (and pack words into lines) (5) — Go >

E 284. Implement string formatting and request tracking tasks — PracHub * 2026-02-11 *
M 285. Snake Case to Camel Case with Repository-Wide Refactor — 1Point3Acres * 2026-07-14 *
M 286. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines) — 1Point3Acres * 2026-02-01 *
H 287. Convert Valid Snake Case to Lower Camel Case — PracHub * 2026-04-16 *
H 288. Implement a function to justify a list of words within a given width. — Exponent ND

GROUP 1 OF 43 | 9 QUESTIONS | SHORT MAIN CODE

Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

Pattern

Every one of these keeps a frontier of cells or nodes still to visit, takes one out, looks at its neighbours, and adds the new ones. A dict of distances doubles as the seen set. Change what you start from, which neighbours are allowed, and what you do when a node comes out.

1 Flood Fill — EASY

Template move: starts = [(sr, sc)]; usable = same colour as the start; repaint every cell you reach; return early if old colour == new colour

QUESTION

You are given an m x n grid image where image[r][c] is the integer color of the pixel at row r , column c . You are also given a starting pixel (sr, sc) and a new_color . Perform a flood fill: starting from (sr, sc) , repaint that pixel and every pixel connected to it 4-directionally (up, down, left, right) that shares the SAME original color as the starting pixel, changing them all to new_color . Pixels connected only diagonally are not filled. Return the modified grid.

Function signature

```
from typing import List
def flood_fill(image: List[List[int]], sr: int, sc: int, new_color: int) ->
List[List[int]]:
    ...
```

Follow-up (as reported): Implement with DFS (recursive or explicit stack) or BFS; discuss the base case that prevents re-visiting when new_color equals the original color.

EXAMPLES

Example 1	
Input	`image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, new_color = 2`
Output	`[[2,2,2],[2,2,0],[2,0,1]]`
Why	`Starting color is 1. All 1-pixels connected 4-directionally to (1,1) become 2. The bottom-right 1 is only diagonally reachable, so it stays.`

Example 2	
Input	`image = [[0,0,0],[0,0,0]], sr = 0, sc = 0, new_color = 5`
Output	`[[5,5,5],[5,5,5]]`
Why	`The whole grid is one connected region of color 0.`

Example 3	
Input	`image = [[0,0,0],[0,1,0]], sr = 1, sc = 1, new_color = 1`
Output	`[[0,0,0],[0,1,0]]`
Why	`The start pixel already equals new_color, so nothing changes.`

CONSTRAINTS

- 1 <= m, n <= 50
- 0 <= image[r][c], new_color < 2^16
- 0 <= sr < m , 0 <= sc < n
- If the starting pixel already has new_color , the grid is returned unchanged (guard against infinite recursion).

WHY IT WORKS

Repaint the current cell, then recursively visit each 4-directional neighbour that still has the old colour. Return immediately when old and new colours match.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Recursive DFS
# Idea: Repaint the start, then recursively spread to each
# 4-directional neighbour that still has the old colour.
def flood_fill(image, start_row, start_col, new_color):
    old_color = image[start_row][start_col]
    if old_color == new_color:
        return image

    def dfs(row, col):
        if not (0 <= row < len(image) and 0 <= col < len(image[0])):
            return
        if image[row][col] != old_color:
            return
        image[row][col] = new_color
        dfs(row + 1, col)
        dfs(row - 1, col)
        dfs(row, col + 1)
        dfs(row, col - 1)

    dfs(start_row, start_col)
    return image
```

COMPLEXITY | Time O(m * n) | Space O(m * n) recursion in the worst case

2 Number of Islands — MEDIUM

Template move: loop over every cell; each unseen land cell starts a new frontier_search (share one seen set); islands += 1 per start

QUESTION

Given a grid of 0s (water) and 1s (land), count the islands. An island is a group of 1s connected up, down, left or right (diagonal cells are not connected).

Function signature

def num_islands(grid: list) -> int

EXAMPLES

Example 1	
Input	`[[1, 1, 0, 0], [1, 1, 0, 0], [0, 0, 1, 1], [0, 0, 1, 1]]`
Output	`2`
Why	`Two 2x2 blocks of land that only touch the water between them: 2 islands.`

Example 2	
Input	`[[1, 0, 1], [0, 1, 0], [1, 0, 1]]`
Output	`5`
Why	`Land that touches only diagonally is not connected, so every 1 is its own island.`

Example 3	
Input	`[[0, 0], [0, 0]]`
Output	`0`
Why	`No land, no islands.`

WHY IT WORKS

Each unvisited land cell starts one island. DFS marks its complete 4-directional component before the scan continues.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Recursive DFS flood fill
# Idea: Every unvisited land cell starts one island. DFS
# marks its whole 4-directional component before scanning on.
def numIslands(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    seen = set()

    def dfs(row, col):
        if not (0 <= row < rows and 0 <= col < cols):
            return
        if grid[row][col] != 1 or (row, col) in seen:
            return
        seen.add((row, col))
        dfs(row + 1, col)
        dfs(row - 1, col)
        dfs(row, col + 1)
        dfs(row, col - 1)

    islands = 0
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] == 1 and (row, col) not in seen:
                islands += 1
                dfs(row, col)
    return islands
```

COMPLEXITY | Time O(m * n) | Space O(m * n) for seen and recursion

3 Rotten Oranges / Multi-Source BFS (taxis) MEDIUM

Template move: starts = every rotten orange; usable = fresh orange; answer = rounds taken, or -1 if a fresh orange stays unreached

QUESTION

Each cell of a grid is 0 (empty), 1 (a fresh orange) or 2 (a rotten orange). Every minute, each fresh orange that is next to (up, down, left or right) a rotten one becomes rotten. Return the number of minutes until no fresh orange is left, or -1 if some fresh orange can never rot.

Function signature

def rotting_minutes(grid: list) -> int

EXAMPLES

Example 1	
Input	`[[2, 1, 1], [1, 1, 0], [0, 1, 1]]`
Output	`4`
Why	`The rot spreads outward from the corner and reaches the last orange after 4 minutes.`

Example 2	
Input	`[[2, 1, 1], [0, 1, 1], [1, 0, 1]]`
Output	`-1`
Why	`The orange at the bottom left is cut off by empty cells, so it never rots.`

QUESTION

Example 3	
Input	`[[0, 2]]`
Output	`0`
Why	`Nothing fresh to rot: 0 minutes.`

PART 1: Rotten oranges

Start one BFS with every rotten orange; each level is one minute.

PART 2: Nearest taxi

Start one BFS with every taxi and store the first distance written to each open cell.

SHORT OPTIMAL CODE

```python
from collections import deque


def rotting_minutes(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    queue = deque((row, col) for row in range(rows) for col in range(cols)
                  if grid[row][col] == 2)
    fresh = sum(cell == 1 for row in grid for cell in row)
    minutes = 0
    while queue and fresh:
        minutes += 1
        for _ in range(len(queue)):
            row, col = queue.popleft()
            for row_step, col_step in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                next_row, next_col = row + row_step, col + col_step
                if 0 <= next_row < rows and 0 <= next_col < cols and grid[next_row][next_col] == 1:
                    grid[next_row][next_col] = 2
                    fresh -= 1
                    queue.append((next_row, next_col))
    return minutes if fresh == 0 else -1


def nearest_taxi(grid):
    if not grid:
        return []
    rows, cols = len(grid), len(grid[0])
    distance = [[-1] * cols for _ in range(rows)]
    queue = deque()
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] == "T":
                distance[row][col] = 0
                queue.append((row, col))
    while queue:
        row, col = queue.popleft()
        for row_step, col_step in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            next_row, next_col = row + row_step, col + col_step
            if (0 <= next_row < rows and 0 <= next_col < cols
                    and grid[next_row][next_col] != "#" and distance[next_row][next_col] == -1):
                distance[next_row][next_col] = distance[row][col] + 1
                queue.append((next_row, next_col))
    return distance
```

COMPLEXITY | Part 1: Time O(m * n) | Space O(m * n)
Part 2: Time O(m * n) | Space O(m * n)

4 Find shortest relationship path using BFS

MEDIUM

Template move: one start; also store parent[v] = u when you first reach v; stop at the target and walk parent back to rebuild the path

QUESTION

People are connected by relationships, given as an adjacency dict that maps each person to the people they are directly related to. shortest_path(adj, s, t) returns the shortest chain of people from s to t (both ends included), or None when t cannot be reached.

Function signature

def shortest_path(adj: dict, s, t) -> list | None

EXAMPLES

Example 1

Input	`adj = {'ann': ['bob', 'cy'], 'bob': ['dee'], 'cy': ['dee', 'eli'], 'dee': ['fay'], 'eli': ['fay'], 'fay': []}`
	`shortest_path(adj, 'ann', 'fay')`
	`shortest_path(adj, 'ann', 'ann')`
	`shortest_path(adj, 'fay', 'ann')`

Output	`shortest_path(adj, 'ann', 'fay') -> ['ann', 'bob', 'dee', 'fay']`
	`shortest_path(adj, 'ann', 'ann') -> ['ann']`
	`shortest_path(adj, 'fay', 'ann') -> null`

Why	Several chains of 4 people exist; BFS finds the first. A person is trivially related to themselves, and fay has no outgoing relationships.

WHY IT WORKS

Queue the source and mark it visited. Pop a person, queue each unvisited neighbour with parent = this person, and stop when the target is queued. Follow parents back to the source and reverse to get the path.

SHORT OPTIMAL CODE

```python
from collections import deque


# BASE QUESTION (optimal): Breadth-first search with a
# parent map
# Idea: Queue the source and mark it visited. Pop a
# person, queue each unvisited neighbour with parent =
# this person, and stop when the target is queued. Follow
# parents back to the source and reverse to get the path.
def shortest_relationship(connections, source, target):
    if source == target:
        return [source]
    parent, queue = {source: None}, deque([source])
    while queue:
        person = queue.popleft()
        for neighbour in connections.get(person, ()):
            if neighbour not in parent:
                parent[neighbour] = person
                if neighbour == target:
                    path = [neighbour]
                    while parent[path[-1]] is not None:
                        path.append(parent[path[-1]])
                    return path[::-1]
                queue.append(neighbour)
    return None
```

COMPLEXITY | Time O(V + E) | Space O(V)

5 Compute distance to nearest taxi in grid

MEDIUM

Template move: starts = every taxi cell; usable = not a wall; the dist table is the answer, cells never reached stay -1

QUESTION

A grid has 'T' for a taxi, '#' for a blocked cell and '.' for an open cell. For every cell return the number of steps (up, down, left or right, never through '#') to the nearest taxi: 0 for a taxi, and -1 for a blocked cell or a cell no taxi can reach.

Function signature

def taxi_distances(grid: list) -> list

EXAMPLES

Example 1	
Input	`['T.#', '...', '#.T']`
Output	`[[0, 1, -1], [1, 2, 1], [-1, 1, 0]]`
Why	The two taxis are in opposite corners; the middle cell is 2 steps from both; the blocked cells show -1.

Example 2	
Input	`['..', '..']`
Output	`[[-1, -1], [-1, -1]]`
Why	No taxi at all: every open cell is unreachable.

WHY IT WORKS

Initialise the queue with every taxi at distance 0 and expand in rounds. A cell receives its distance the first time it is reached; blocked cells are never entered; cells that no taxi can reach stay at -1.

SHORT OPTIMAL CODE

```
from collections import deque


# BASE QUESTION (optimal): One multi-source BFS
# Idea: Initialise the queue with every taxi at distance 0
# and expand in rounds. A cell receives its distance the
# first time it is reached; blocked cells are never
# entered; cells that no taxi can reach stay at -1.
def taxi_distances(grid):
    rows, cols = len(grid), len(grid[0])
    distance = [[-1] * cols for _ in range(rows)]
    queue = deque()
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] == "T":
                distance[row][col] = 0
                queue.append((row, col))
    while queue:
        row, col = queue.popleft()
        for next_row, next_col in ((row + 1, col), (row - 1, col), (row, col + 1), (row, col - 1)):
            if 0 <= next_row < rows and 0 <= next_col < cols and grid[next_row][next_col] != "#" and distance[n
ext_row][next_col] == -1:
                distance[next_row][next_col] = distance[row][col] + 1
                queue.append((next_row, next_col))
    return distance
```

COMPLEXITY | Time O(m * n) | Space O(m * n)

6 Solve Grid and Counting Problems

MEDIUM

Template move: the island count again (queue version); the second part of the question is a short counting formula, no search

QUESTION

A software engineering intern phone screen included two coding tasks: 1.
Count islands in a grid Given an `m x n` grid of characters where `'1'` represents land and `'0'` represents water, return the number of islands. An island is formed by connecting adjacent land cells horizontally or vertically. You may assume all four edges of the grid are surrounded by water. 2.
Count ways to form a target word Given an array of uppercase letters, count how many different ways you can choose letters from the array so that the chosen letters can be rearranged to spell **`GOOGLE`**. Each array element can be used at most once, and letters at different indices are considered different choices even if they contain the same character. Design efficient algorithms for both tasks, and discuss the time and space complexity of your approach.

Part 1: Count Islands in a GridGiven a 2D grid of characters where each cell is either '1' (land) or '0' (water), return the number of islands. An island is a group of adjacent land cells connected only horizontally or vertically. Diagonal cells do not count as connected. You may assume the grid edges are surrounded by water.
Constraints
0 <= number of rows <= 3000 <= number of columns <= 300grid[r][c] is either '0' or '1'All rows have the same length

EXAMPLES

```
Input: []
```

Expected Output:
Explanation: Empty grid contains no land, so there are no islands.

```
Input: [['1']]
```

Expected Output:
Explanation: A single land cell forms one island.

Hints
Whenever you find an unvisited land cell, start a DFS or BFS to mark the entire island.Only 4-directional neighbors matter: up, down, left, and right.
Part 2: Count Ways to Form the Word GOOGLEGiven an array of uppercase letters, count how many different ways you can choose letters so that the chosen letters can be rearranged to spell GOOGLE. Each array element can be used at most once. Letters at different indices count as different choices even if they contain the same character. The word GOOGLE requires exactly 2 G's, 2 O's, 1 L, and 1 E.
Constraints
0 <= len(letters) <= 200000Each element is an uppercase English letter from 'A' to 'Z'The answer may be large, but fits in Python's integer type
Examples
Input: []
Expected Output:
Explanation: No letters means it is impossible to form GOOGLE.
Input: ['G','O','O','G','L','E']
Expected Output:
Explanation: There is exactly one way to choose all six letters.
Hints
First count how many G, O, L, and E letters appear in the array.You need to choose 2 of the G positions and 2 of the O positions, then 1 L and 1 E.

PART 1: Islands: recursive DFS

Count each unvisited land cell once, then DFS through its whole component.

PART 2: Counting: product of combinations

Count the available letters and multiply the ways to choose the required copies of G, O, L and E.

SHORT OPTIMAL CODE

```python
from collections import Counter
from math import comb


# PART 1: Recursive DFS island count
def numIslands(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    seen = set()

    def dfs(row, col):
        if not (0 <= row < rows and 0 <= col < cols):
            return
        if grid[row][col] != "1" or (row, col) in seen:
            return
        seen.add((row, col))
        for row_step, col_step in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            dfs(row + row_step, col + col_step)

    islands = 0
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] == "1" and (row, col) not in seen:
                islands += 1
                dfs(row, col)
    return islands


# PART 2: choose the required copies of each letter
def google_ways(letters):
    count = Counter(letters)
    return comb(count["G"], 2) * comb(count["O"], 2) * count["L"] * count["E"]
```

COMPLEXITY | Part 1: Time O(m * n) | Space O(m * n)
Part 2: Time O(n) | Space O(1) for a fixed alphabet

7 Farthest City for a New Data Center — MEDIUM

Template move: same on a graph: starts = the data centres, neighbors = adjacency list; answer = the city with the largest distance

QUESTION

Cities are nodes 0..num_cities-1 of an undirected, unweighted graph; edges[i] = [u, v] is a direct connection. You are given a non-empty list dcs of cities that already host a data center. You want to build a new data center in the city that is as far as possible from the nearest existing data center — that is, the city whose shortest-path distance (in number of edges) to its closest data center is maximized. Return that city's index. Break ties by choosing the smallest index. It is guaranteed every city can reach at least one data center.

Function signature

def farthest_city(num_cities: int, edges: list[list[int]], dcs: list[int]) -> int:
    ...

Follow-up (as reported): Run a single BFS seeded from all data centers at once (distance 0) instead of a BFS per data center.

EXAMPLES

QUESTION

Example 1	
Input	num_cities = 5, edges = [[0,1],[1,2],[2,3],[3,4]], dcs = [0]
Output	4
Why	Distances from city 0 are 0,1,2,3,4; city 4 is farthest.

Example 2	
Input	num_cities = 5, edges = [[0,1],[1,2],[2,3],[3,4]], dcs = [0, 4]
Output	2
Why	Nearest-DC distances are 0->0, 1->1, 2->2, 3->1, 4->0; city 2 is farthest at distance 2.

CONSTRAINTS

- 1 <= num_cities <= 10**5 , 0 <= len(edges) <= 2*10**5 .
- 1 <= len(dcs) <= num_cities , dcs are distinct valid city indices.

WHY IT WORKS

Seed the queue with every data center at distance 0; the first visit is the nearest one.

SHORT OPTIMAL CODE

```python
from collections import deque


# BASE QUESTION (optimal): One multi-source BFS
# Idea: Seed the queue with every data center at distance
# 0; the first visit is the nearest one.
def farthest_city(num_cities, edges, data_centers):
    neighbours = [[] for _ in range(num_cities)]
    for from_city, to_city in edges:
        neighbours[from_city].append(to_city)
        neighbours[to_city].append(from_city)
    distance = [-1] * num_cities
    queue = deque(data_centers)
    for data_center in data_centers:
        distance[data_center] = 0
    while queue:
        from_city = queue.popleft()
        for to_city in neighbours[from_city]:
            if distance[to_city] < 0:
                distance[to_city] = distance[from_city] + 1
                queue.append(to_city)
    return distance.index(max(distance))
```

COMPLEXITY | Time O(V + E) | Space O(V + E)

8 Infection Spread on a Grid (Cellular Automaton) HARD

Template move: starts = every infected cell; usable = healthy cell; the answer is the largest distance reached (number of rounds)

QUESTION

A grid holds '.' for a healthy cell, 'I' for an infected cell and any other character (such as '#') for a barrier that is never infected. Every step, all infected cells infect their healthy neighbours (up, down, left, right) at the same time. infection_times(grid) returns the step at which each infected cell was infected (0 for the cells infected at the start); spread_steps(grid) returns the number of steps until nothing new gets infected.

QUESTION

Function signature

def infection_times(grid: list) -> dict

def spread_steps(grid: list) -> int

EXAMPLES

Example 1	
Input	`['I..', '...', '..#']`
Output	`3`
Why	`The infection travels from the corner: (0,1) and (1,0) at step 1, then three cells at step 2, and so on. The barrier at the bottom right is never infected.`

Example 2	
Input	`['I...']`
Output	`{(0, 0): 0, (0, 1): 1, (0, 2): 2, (0, 3): 3}`
Why	`A one-row grid: one cell per step.`

Example 3	
Input	`['I#.', 'I#.']`
Output	`0`
Why	`Both cells next to the barrier are already infected and the barrier blocks the right column, so nothing can spread: 0 steps.`

WHY IT WORKS

Put every initially infected cell in the frontier. Each step, look only at the neighbors of frontier cells, infect the healthy ones, and make them the next frontier. The loop ends when the next frontier is empty, which is the fixed point. Every cell enters the frontier at most once.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Frontier BFS (only recently
# infected cells spread)
# Idea: Put every initially infected cell in the frontier.
# Each step, look only at the neighbors of frontier cells,
# infect the healthy ones, and make them the next
# frontier. The loop ends when the next frontier is empty,
# which is the fixed point. Every cell enters the frontier
# at most once.
def spread_steps(grid):
    cells = [list(row) for row in grid]
    frontier = [(row, col) for row in range(len(cells)) for col in range(len(cells[0])) if cells[row][col] == "
I"]
    steps = 0
    while frontier:
        next_frontier = []
        for row, col in frontier:
            for row_step, col_step in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                next_row, next_col = row + row_step, col + col_step
                if 0 <= next_row < len(cells) and 0 <= next_col < len(cells[0]) and cells[next_row][next_col] =
= ".":
                    cells[next_row][next_col] = "I"
                    next_frontier.append((next_row, next_col))
        if not next_frontier:
            break
        frontier, steps = next_frontier, steps + 1
    return steps
```

COMPLEXITY | Time O(m * n) | Space O(m * n)

9 Shortest Distance From All Buildings HARD

Template move: run it once per building (one start each); add each run into a total table and count how many buildings reached each cell

QUESTION

You are given a 2D grid where each cell is:
0 = empty land (a school can be built here), 1 = a house, 2 = a tree (an obstacle that cannot be entered or crossed). You want to build one school on an empty cell so that the total travel distance from every house to the school is minimized. Distance is measured by 4-directional steps (up/down/left/right); trees block movement. A valid school location must be reachable from every house. Return the coordinates (row, col) of the empty cell with the minimum total distance to all houses. If several cells tie on total distance, return the one that comes first in row-major order (smallest row, then smallest column). If no empty cell can reach all houses, return (-1, -1) .

Function signature

```
def shortest_distance_from_all_buildings(grid: list[list[int]]) -> tuple[int, int]:
    ...
```

Follow-up (as reported): (Reported) Minimize the average distance instead of the total. Since the number of houses is fixed, the minimizing cell is the same, but track the count to report the average.

EXAMPLES

Example 1	
Input	`grid = [[1, 0, 2, 0, 1], [0, 0, 0, 0, 0], [0, 0, 1, 0, 0]]`
Output	`(1, 2)`
Why	`Cell (1,2) reaches all three houses with total distance 3 + 3 + 1 = 7, the minimum over all reachable empty cells.`

Example 2	
Input	`grid = [[1, 0, 1]]`
Output	`(0, 1)`
Why	`The only empty cell is distance 1 from each house; total 2.`

Example 3	
Input	`grid = [[1, 2, 1]]`
Output	`(-1, -1)`
Why	`There is no empty cell, so no valid location exists.`

CONSTRAINTS

- 1 <= rows, cols <= 50
- Each cell is 0 , 1 , or 2 .
- There is at least one house.

WHY IT WORKS

Every house adds its distance to the empty cells it reaches; valid cells were reached by all houses.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): One BFS per house
# Idea: Every house adds its distance to the empty cells
# it reaches; valid cells were reached by all houses.
from collections import deque


def school_totals(grid):
```

```
    rows, cols = len(grid), len(grid[0])
    total = [[0] * cols for _ in range(rows)]
    reached = [[0] * cols for _ in range(rows)]
    houses = 0
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] != 1:
                continue
            houses += 1
            seen, queue = {(row, col)}, deque([(row, col, 0)])
            while queue:
                cell_row, cell_col, distance = queue.popleft()
                for next_row, next_col in ((cell_row + 1, cell_col), (cell_row - 1, cell_col), (cell_row, cell_
col + 1), (cell_row, cell_col - 1)):
                    if 0 <= next_row < rows and 0 <= next_col < cols and (next_row, next_col) not in seen and g
rid[next_row][next_col] == 0:
                        seen.add((next_row, next_col))
                        total[next_row][next_col] += distance + 1
                        reached[next_row][next_col] += 1
                        queue.append((next_row, next_col, distance + 1))
    return total, reached, houses


def shortest_distance_from_all_buildings(grid):
    total, reached, houses = school_totals(grid)
    best, spot = float("inf"), (-1, -1)
    for row in range(len(grid)):
        for col in range(len(grid[0])):
            if grid[row][col] == 0 and reached[row][col] == houses and total[row][col] < best:
                best, spot = total[row][col], (row, col)
    return spot
```

COMPLEXITY | Time O(houses * R * C) | Space O(R * C)

GROUP 2 OF 43 | 9 QUESTIONS | SHORT MAIN CODE

Sweep over events in time order

Pattern

Turn every item into events (a start adds one, an end removes one, an arrival joins a queue), sort the events by time, and walk them keeping a small state: a counter, a queue, a heap or a running total. What you record at each event is the only thing that changes.

10 Meeting Rooms / Interval Scheduling Variants

MEDIUM

Template move: the template as written: the largest running count is the number of rooms

QUESTION

LC 253 (Meeting Rooms II) appears in many forms across phone screen and onsite coding rounds — minimum number of rooms, room assignment, cyclic time, daily server jobs, gate entry/exit. Common follow-ups push toward sweep line, min-heap, or distributed processing.

The base task in every variant: given a list of intervals (start, end) — or pickup/return pairs, meeting time pairs, server job runs — answer one or more of:

Min rooms / cars / servers needed to honor all intervals concurrently (the canonical min-meeting-rooms problem).
Assignment: which interval goes to which room/car, so the caller can use the rental record.
Single-interval overlap test between two intervals.

Variant: parking-lot capacity

Arrival and departure intervals represent cars occupying a parking lot. Compute the peak simultaneous occupancy using the same interval-overlap model.

Variant: cyclic time (24h wrap)

Intervals wrap around midnight; if an interval ends after the day boundary, it carries into the next day.
Follow-up: "if N is huge, how do you optimize?" — expect a sweep-line discussion.

Variant: daily-recurring server jobs

Same set of jobs runs every day. If a job can't finish in one day, truncate at midnight and resume next day.
A server can run multiple identical jobs that overlap on the same machine; different jobs need separate servers.
Follow-up: a single job that runs many days needs its own dedicated server.

Variant: on-call rotation consolidation

Input: on-call rotations {name, start, end} using half-open intervals.
Output maximal contiguous segments during which the active on-call set does not change; omit gaps with nobody on call.
Segments are sorted by start, and names in each segment are the active set for that time range.
Clean approach: sort starts, keep a min-heap by end time, and emit a segment whenever the next start or next end changes the active set.

Variant: gate entry / exit with priority rules

Array of (timestamp, action) where action in {enter, exit}; each index is a different person.
Return for each person what time they actually enter/exit, applying tie-break rules:

If the prior tick had an enter, enter wins on a tie.
If the prior tick had an exit, exit wins on a tie.
If the prior tick had no activity, exit wins.
Within the winning class, lower index wins.

Optimization: instead of stepping time tick-by-tick, jump through the sorted input timestamps to reach O(n).

Variant: Google Calendar free-time intersection

QUESTION

Input: requested attendees plus each person's busy calendar intervals.
Return every time range where all requested attendees are free.
Normalize each person's busy blocks into free gaps, then intersect the candidate ranges across attendees.

Variant: car-rental assignment

Input: rental orders containing an order ID, pickup time, and return time.
Assign every order to a car so that no assigned intervals overlap, minimize the number of cars, and return each car's complete order list. Model each car explicitly with its own rental record.
A current output variant instead returns a dictionary from each reservation ID to its assigned car ID; the follow-up asks for the final fleet size.
A return at time t and another pickup at time t may reuse the same car.
Follow-up: if the fleet is capped at K cars and all requests cannot be served, define how the allocator detects and handles the failure.

Variant: exact-concurrency intervals

Compute the maximum number of simultaneously active meetings.
Given a target concurrency X, also return every maximal time range during which exactly X meetings are active.
Define the event ordering when one meeting starts at the same timestamp another ends; this choice controls whether a zero-width overlap is counted.

Variant: maximum-profit job scheduling

Each job has startTime, endTime, and profit. Choose a set of non-overlapping jobs maximizing total profit. The candidate identified it as LC 1235 (Maximum Profit in Job Scheduling).
Asked as the first question of an L4 onsite round, followed by an unrelated range-query problem in the same slot.

On-call employee metadata variant

Each duty record includes employee ID, employee name, start time, and end time. Return the employee count and identifying information for each time segment.

EXAMPLES

```
Canonical min-meeting-rooms example: [[0,30],[5,10],[15,20]] -> 2.
On-call consolidation: A[10,50], B[20,60], C[30,40], D[30,40] -> [10,20]: A, [20,30]: A,B, [30,40]:
    A,B,C,D, [40,50]: A,B, [50,60]: B.
Single-interval overlap: intervalsOverlap([1,5],[4,7]) -> true.
```

Long-running job: a 36-hour daily job is impossible under the truncate rule; reserve a separate server.

PART 1: Minimum rooms

Sweep sorted start and end events and keep the largest active count.

PART 2: Room assignment

Process meetings by start time and reuse the room that becomes free first.

PART 3: On-call segments

Sweep change times and emit each interval with its current active people.

SHORT OPTIMAL CODE

```python
import heapq
from collections import defaultdict


def min_rooms(intervals):
    events = sorted((time, delta) for start, end in intervals
                    for time, delta in ((start, 1), (end, -1)))
    active = answer = 0
    for _, delta in events:
        active += delta
```

```
        answer = max(answer, active)
    return answer


def assign_rooms(intervals):
    active = []
    rooms = [0] * len(intervals)
    room_count = 0
    for start, end, index in sorted((begin, finish, position) for position, (begin, finish) in enumerate(interv
als)):
        if active and active[0][0] <= start:
            _, room = heapq.heappop(active)
        else:
            room = room_count
            room_count += 1
        rooms[index] = room
        heapq.heappush(active, (end, room))
    return rooms


def oncall_segments(rotations):
    changes = defaultdict(list)
    for name, start, end in rotations:
        changes[start].append((name, 1))
        changes[end].append((name, -1))
    active, answer = defaultdict(int), []
    times = sorted(changes)
    for time, next_time in zip(times, times[1:]):
        for name, delta in changes[time]:
            active[name] += delta
        people = frozenset(name for name, count in active.items() if count)
        if not people:
            continue
        if answer and answer[-1][1] == time and answer[-1][2] == people:
            answer[-1] = (answer[-1][0], next_time, people)
        else:
            answer.append((time, next_time, people))
    return answer
```

COMPLEXITY | Part 1: Time O(n log n) | Space O(n)
Part 2: Time O(n log n) | Space O(n)
Part 3: Time O(n log n) | Space O(n)

11 Train / Flight Schedule Reachability — MEDIUM

Template move: sort legs by departure; a leg is usable if you have already reached its origin before it leaves; keep the earliest arrival per place

QUESTION

Given trains (or flights) as (from, to, departure, arrival), decide whether you can travel from a source to a destination, boarding a later leg only if it departs after you arrive. Graph reachability with time state, solved with BFS / DFS.

Input: a schedule of legs, each with a departure station, an arrival station, a departure time, and an arrival time; plus a source and a destination.
Output: whether the destination is reachable from the source, where a leg can only be taken if it departs after you arrive at its departure station.
The same prompt appears as a flight schedule between airports in new-grad R1.

QUESTION

EXAMPLES

```
trains = [
  ("A", "Y", dep=1, arr=20),
  ("A", "Y", dep=2, arr=5),
  ("Y", "B", dep=6, arr=9)
]
source = "A", destination = "B"

Output: true — the first A->Y train arrives too late for the Y->B connection, but the second one makes it
.
```

WHY IT WORKS

Sort the legs by departure time. Keep earliest[station] = earliest known arrival, with the source at minus infinity. For each leg whose origin has an earliest arrival before (or at) its departure, relax earliest[destination] with its arrival. Sorting guarantees every leg that could feed this one has already been processed, because a leg arrives after it departs.

SHORT OPTIMAL CODE

```python
INFINITY = float("inf")


# BASE QUESTION (optimal): Sort by departure and keep
# earliest arrivals
# Idea: Sort the legs by departure time. Keep
# earliest[station] = earliest known arrival, with the
# source at minus infinity. For each leg whose origin has
# an earliest arrival before (or at) its departure, relax
# earliest[destination] with its arrival. Sorting
# guarantees every leg that could feed this one has
# already been processed, because a leg arrives after it
# departs.
def earliest_arrivals(legs, source, strict=True):
    earliest = {source: -INFINITY}
    for origin, destination, departure, arrival in sorted(legs, key=lambda leg: leg[2]):
        earliest_at_origin = earliest.get(origin, INFINITY)
        if earliest_at_origin < departure or (not strict and earliest_at_origin == departure):
            if arrival < earliest.get(destination, INFINITY):
                earliest[destination] = arrival
    return earliest
```

COMPLEXITY | Time O(L log L) | Space O(S)

12 Solve Rooms and Top-K Streams — MEDIUM

Template move: sort starts and ends separately and walk two pointers: a start before the earliest end needs a new room; the stream part is a size-K heap

QUESTION

Two small tasks. (1) min_rooms(intervals): the minimum number of rooms needed to hold all meetings; a meeting may start exactly when another ends. (2) TopKStream(k): numbers arrive one at a time with add(x); top() returns the k largest numbers seen so far, largest first.

Function signature

def min_rooms(intervals: list) -> int
class TopKStream:
  def __init__(self, k: int)
  def add(self, x: int) -> None
  def top(self) -> list

EXAMPLES

QUESTION

Example 1	
Input	`[(0, 30), (5, 10), (15, 20)]`
Output	`2`
Why	`(5, 10) and (15, 20) both overlap the long meeting (0, 30): 2 rooms.`

Example 2	
Input	`[(7, 10), (2, 4)]`
Output	`1`
Why	`The second meeting ends before the first starts: one room is enough.`

Example 3	

Input	`s = TopKStream(3)`
	`s.add(5)`
	`s.add(1)`
	`s.add(9)`
	`s.add(7)`
	`s.top()`

Output	`s.top() -> [9, 7, 5]`
Why	`Of 5, 1, 9, 7 the three largest are 9, 7 and 5.`

PART 1: Rooms: sorted starts and ends with two pointers

Sort starts and ends separately. For each start: if it is at or after the earliest unfinished end, that room is reused (advance the end pointer); otherwise a new room is needed. The count of rooms opened is the maximum overlap.

PART 2: Stream: min-heap of size K

Push each value; if the heap has more than K items pop the smallest. The root is then the K-th largest. Memory is K, regardless of how many values have been seen.

SHORT OPTIMAL CODE

```
import heapq


# Rooms: sorted starts and ends with two pointers
# Idea: Sort starts and ends separately. For each start:
# if it is at or after the earliest unfinished end, that
# room is reused (advance the end pointer); otherwise a
# new room is needed. The count of rooms opened is the
# maximum overlap.
def minMeetingRooms(intervals):
    starts, ends = sorted(start_time for start_time, _ in intervals), sorted(end_time for _, end_time in interv
als)
    rooms = next_end_index = 0
    for start_time in starts:
        if start_time >= ends[next_end_index]:
            next_end_index += 1
        else:
            rooms += 1
    return rooms


# Stream: min-heap of size K
# Idea: Push each value; if the heap has more than K items
# pop the smallest. The root is then the K-th largest.
# Memory is K, regardless of how many values have been
# seen.
class KthLargestStream:
```

```
    def __init__(self, top_count, nums=()):
        self.top_count, self.smallest_heap = top_count, []
        for value in nums:
            self.add(value)

    def add(self, value):
        heapq.heappush(self.smallest_heap, value)
        if len(self.smallest_heap) > self.top_count:
            heapq.heappop(self.smallest_heap)
        return self.smallest_heap[0] if len(self.smallest_heap) == self.top_count else None
```

COMPLEXITY | Part 1: Time O(n log n) | Space O(n)
Part 2: Time O(log k) per add | Space O(k)

13 Gate Entry / Exit Time Resolution with Tie-Break Rules — MEDIUM

Template move: the same two-queue simulation with the tie-break rules from the statement

QUESTION

A gate lets one person through per second. events[i] = (time, action) for person i, where action is 'enter' or 'exit'. When several people want the gate in the same second: if the gate was not used in the previous second, people who exit go first; if it was used, the direction used in the previous second goes first again. Within one direction, people go in order of arrival (person id). Return the second at which each person passes.

Function signature

def gate_pass_times(events: list) -> list

EXAMPLES

Example 1	
Input	`[(0, 'enter'), (0, 'exit'), (1, 'enter'), (5, 'enter')]`
Output	`[1, 0, 2, 5]`
Why	At second 0 the gate was idle, so the exit (person 1) goes first. Person 0 follows at 1, person 2 (arrived at 1) at 2, and person 3 arrives alone at 5.

Example 2	
Input	`[(0, 'enter'), (1, 'enter'), (1, 'exit')]`
Output	`[0, 1, 2]`
Why	The gate was used for entering in the previous second, so the waiting enter keeps priority over the exit.

WHY IT WORKS

Sort people by (arrival time, id) and keep one FIFO queue for waiting enters and one for exits. At each second pick from the queues using the previous-second state. When both queues are empty jump straight to the next arrival; if that jump leaves a gap of at least one idle second the previous state resets to 'none', which is the subtle part of the optimization.

SHORT OPTIMAL CODE

```
from collections import deque


# BASE QUESTION (optimal): Event-driven simulation with
# two queues
# Idea: Sort people by (arrival time, id) and keep one
# FIFO queue for waiting enters and one for exits. At each
# second pick from the queues using the previous-second
# state. When both queues are empty jump straight to the
# next arrival; if that jump leaves a gap of at least one
```

```python
# idle second the previous state resets to 'none', which
# is the subtle part of the optimization.
def gate_pass_times(events):
    num_people = len(events)
    people_by_arrival = sorted(range(num_people), key=lambda person: (events[person][0], person))
    enter_queue, exit_queue = deque(), deque()
    pass_time = [0] * num_people
    time, next_arrival, last_direction, last_busy_second = 0, 0, None, -10
    while next_arrival < num_people or enter_queue or exit_queue:
        if not enter_queue and not exit_queue:
            time = max(time, events[people_by_arrival[next_arrival]][0])
            if time > last_busy_second + 1:
                last_direction = None
        while next_arrival < num_people and events[people_by_arrival[next_arrival]][0] <= time:
            arrived_person = people_by_arrival[next_arrival]
            (enter_queue if events[arrived_person][1] == "enter" else exit_queue).append(arrived_person)
            next_arrival += 1
        if enter_queue and exit_queue:
            queue = enter_queue if last_direction == "enter" else exit_queue
        else:
            queue = enter_queue or exit_queue
        person = queue.popleft()
        pass_time[person] = time
        last_direction = events[person][1]
        last_busy_second = time
        time += 1
    return pass_time
```

COMPLEXITY | Time O(n log n) for the sort, O(n) simulation | Space O(n)

14 Compute servers needed for daily recurring jobs — MEDIUM

Template move: merge each job's own intervals first so one job counts once, then sweep all of them together

QUESTION

You operate a cluster of identical servers that run recurring **daily** jobs. For a single day, you are given a list of job execution intervals. Each interval has:

- **`job_id`** (string or integer),
- **`start_time`** (timestamp within that day),
- **`end_time`** (timestamp within that day), with **`start_time < end_time`**. Interpret each interval as a half-open time range **`[start_time, end_time)`** during which that job must be running on some server **on that day**. Rules and constraints:
- Each physical server can execute at most **one job at a time**.
- However, if multiple intervals in the input correspond to the **same `job_id`**, they may overlap in time **on the same server** (they are considered parts or retries of the same job and do not consume extra capacity).
- If a job's true execution would extend past midnight into the next day, it is conceptually **truncated at the end of the current day** for scheduling purposes; for each day you only see the portion that falls within that day's 24-hour window.

Tasks

1. Given all job intervals for a single day (potentially with repeated **`job_id`**s and overlapping intervals), compute the **minimum number of servers** required to schedule all the jobs under the above rules.
2. Describe the time and space complexity of your algorithm in terms of the number of intervals (m).

Follow-up

Some rare jobs are extremely long-running and conceptually span **multiple days** without restart. For such a multi-day job, you decide to reserve a **dedicated server** that is not shared with any other jobs, but which may still handle different intervals of that same **`job_id`**. Explain how you would extend or adapt your solution to:

QUESTION

- Detect which jobs require a dedicated server based on their multi-day duration or configuration.
- Account for these dedicated servers when computing the total number of servers needed for the system.

Part 1: Minimum Servers for One Day with Overlapping Same-Job IntervalsYou are given all job execution intervals for a single day. Each interval is half-open, meaning [start_time, end_time), and represents a time range during which a job must be running. A server can run at most one job at a time. However, if multiple intervals have the same job_id, they are considered parts or retries of the same job, so overlapping intervals with the same job_id consume only one server total during the overlap. Return the minimum number of servers required to run all intervals for the day.
Constraints
0 <= len(intervals) <= 2000000 <= job_id <= 10^90 <= start_time < end_time <= 1440Intervals are half-open: an interval ending at time t does not overlap an interval starting at time t.Multiple intervals may have the same job_id.

EXAMPLES

```
Input: ([],)
Expected Output: 0
```

Explanation: There are no jobs, so no servers are needed.

```
Input: ([[5, 100, 200], [5, 150, 250], [5, 240, 300]],)
Expected Output: 1
```

Explanation: All intervals belong to the same job_id. Even though they overlap, they can share one server.

Hints

At any moment, one server is needed for each distinct active job_id, not for each active interval.Try merging all intervals belonging to the same job_id first, then run a sweep-line algorithm over the merged intervals.
Part 2: Total Servers with Dedicated Multi-Day JobsYou are given true job execution intervals that may span multiple days. Some jobs require dedicated servers. A job is dedicated if its job_id is listed in configured_dedicated, or if any one of its true intervals has duration at least min_dedicated_duration minutes. A dedicated job uses one reserved server for that job_id and does not share that server with other jobs, though multiple intervals of the same dedicated job_id may use the same dedicated server. For a given target_day, compute the total number of

WHY IT WORKS

Group the intervals by job_id, sort and merge each group's overlapping or touching intervals, then collect all merged intervals. Sort the +1/-1 events (ends before starts at equal times for half-open intervals) and take the maximum running count.

SHORT OPTIMAL CODE

```python
from collections import defaultdict


# BASE QUESTION (optimal): Merge each job's intervals,
# then sweep start and end events
# Idea: Group the intervals by job_id, sort and merge each
# group's overlapping or touching intervals, then collect
# all merged intervals. Sort the +1/-1 events (ends before
# starts at equal times for half-open intervals) and take
# the maximum running count.
def servers_needed(intervals):
    by_job = defaultdict(list)
    for job_id, start, end in intervals:
        by_job[job_id].append((start, end))
    events = []
    for job_intervals in by_job.values():
        merged_start, merged_end = None, None
        for start, end in sorted(job_intervals):
            if merged_end is not None and start <= merged_end:
                merged_end = max(merged_end, end)
            else:
```

```
            if merged_end is not None:
                events += [(merged_start, 1), (merged_end, -1)]
            merged_start, merged_end = start, end
        events += [(merged_start, 1), (merged_end, -1)]
    running_count = most_servers = 0
    for _, change in sorted(events):
        running_count += change
        most_servers = max(most_servers, running_count)
    return most_servers
```

COMPLEXITY | Time O(m log m) | Space O(m)

15 Meeting Rooms III Variant — MEDIUM

Template move: split meetings that cross midnight into two pieces first, then the same sweep

QUESTION

Given an array of meeting time intervals where each interval is [start, end], determine the minimum number of conference rooms required to hold all meetings without overlap. Variant: times are cyclic within a 24-hour period (meetings can wrap around midnight).

EXAMPLES

Example 1	
Input	intervals = [[0, 30], [5, 10], [15, 20]]
Output	2
Why	Meeting [0,30] overlaps with [5,10] and [15,20], so we need 2 rooms.

Example 2	
Input	intervals = [[7, 10], [2, 4]]
Output	1
Why	No overlap, 1 room is sufficient.

Example 3	
Input	intervals = [[22, 2], [23, 1], [0, 3]]
Output	3
Why	All three meetings overlap around midnight.

CONSTRAINTS

- 1 <= intervals.length <= 10^4
- 0 <= start < end <= 24 * 60 (minutes in a day)
- Meetings may wrap around (e.g., [23:00, 01:00])

WHY IT WORKS

Turn every meeting into one or two ordinary intervals, create +1 events at starts and -1 events at ends, sort them (ends before starts at the same minute, because intervals are half-open), and take the maximum running count. This equals the minimum number of rooms if wrapped meetings may switch rooms at midnight, and it is always a lower bound on the exact answer.

SHORT OPTIMAL CODE

```
DAY = 24


def _pieces(meetings, day):
    for start, end in meetings:
        if end > start:
```

```
            yield start, end
        elif end < start:
            yield start, day
            yield 0, end


# BASE QUESTION (optimal): Split wrapped meetings and
# sweep (maximum overlap)
# Idea: Turn every meeting into one or two ordinary
# intervals, create +1 events at starts and -1 events at
# ends, sort them (ends before starts at the same minute,
# because intervals are half-open), and take the maximum
# running count. This equals the minimum number of rooms
# if wrapped meetings may switch rooms at midnight, and it
# is always a lower bound on the exact answer.
def min_rooms_sweep(meetings, day=DAY):
    events = []
    for start, end in _pieces(meetings, day):
        events.append((start, 1))
        events.append((end, -1))
    events.sort()
    running_count = most_rooms = 0
    for _, change in events:
        running_count += change
        most_rooms = max(most_rooms, running_count)
    return most_rooms
```

COMPLEXITY | Time O(n log n) | Space O(n)

16 Count Overlapping Rectangle Updates on a Grid — HARD

Template move: the same idea in two dimensions: a difference array (+1, -1, -1, +1 at the corners) and two running-sum passes

QUESTION

Count Overlapping Rectangle Updates on a Grid Implement

`range_add_counts(n, rectangles)`. Start with an `n x n` matrix of zeroes. Each rectangle is represented as `[row1, col1, row2, col2]` with inclusive, zero-based coordinates. For every rectangle, add one to each matrix cell inside that rectangle. Return the final matrix, where each cell therefore contains the number of input rectangles covering it.

Example

For `n = 3` and `rectangles = [[0, 0, 1, 1], [1, 1, 2, 2]]`, return:

```
[ [1, 1, 0], [1, 2, 1], [0, 1, 1] ]
```

Constraints

- `1 <= n <= 500`
- `0 <= len(rectangles) <= 100_000`
- `0 <= row1 <= row2 < n`
- `0 <= col1 <= col2 < n`

EXAMPLES

Example 1	
Input	`3, [(0, 0, 1, 1), (1, 1, 2, 2)]`
Output	`[[1, 1, 0], [1, 2, 1], [0, 1, 1]]`
Why	`The cell (1, 1) is covered by both rectangles, so it is counted twice.`

QUESTION

Example 2	
Input	`2, []`
Output	`[[0, 0], [0, 0]]`
Why	`No rectangles: all zeros.`

WHY IT WORKS

For each rectangle add +1 at (r1, c1), -1 at (r1, c2 + 1), -1 at (r2 + 1, c1) and +1 at (r2 + 1, c2 + 1) in an (n + 1) x (n + 1) array. A prefix sum over rows and then over columns turns these markers into the number of rectangles covering each cell.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): 2D difference array and prefix
# sums
# Idea: For each rectangle add +1 at (r1, c1), -1 at (r1,
# c2 + 1), -1 at (r2 + 1, c1) and +1 at (r2 + 1, c2 + 1)
# in an (size + 1) x (size + 1) array. A prefix sum over rows
# and then over columns turns these markers into the
# number of rectangles covering each cell.
def range_add_counts(size, rectangles):
    marks = [[0] * (size + 1) for _ in range(size + 1)]
    for r1, c1, r2, c2 in rectangles:
        marks[r1][c1] += 1
        marks[r1][c2 + 1] -= 1
        marks[r2 + 1][c1] -= 1
        marks[r2 + 1][c2 + 1] += 1
    for row in range(size + 1):
        for col in range(1, size + 1):
            marks[row][col] += marks[row][col - 1]
    for col in range(size + 1):
        for row in range(1, size + 1):
            marks[row][col] += marks[row - 1][col]
    return [row[:size] for row in marks[:size]]
```

COMPLEXITY | Time O(R + n^2) | Space O(n^2)

17 Compute Turnstile Crossing Times — HARD

Template move: sort people by arrival time; two queues (entering, exiting) with a rule for who goes first at each second

QUESTION

A turnstile lets one person through per second. events[i] = (arrival time, direction) for person i, with direction 'enter' or 'exit'. When people in both directions wait in the same second: if the turnstile was not used in the previous second, people exiting go first; otherwise the direction used in the previous second goes first again. People of the same direction go in order of arrival (person id). Return the second at which each person crosses.

Function signature

def turnstile_times(events: list) -> list

EXAMPLES

Example 1	
Input	`[(0, 'exit'), (0, 'enter'), (0, 'enter'), (3, 'exit')]`
Output	`[0, 1, 2, 3]`
Why	`The idle turnstile lets the exit (person 0) go first at second 0, then the two entering people follow at 1 and 2; person 3 arrives alone at 3.`

QUESTION

Example 2	
Input	`[(0, 'enter'), (0, 'exit'), (0, 'exit'), (0, 'enter')]`
Output	`[2, 0, 1, 3]`
Why	`Idle turnstile: the exits (persons 1 and 2) go first at seconds 0 and 1, then the entering people (persons 0 and 3) at seconds 2 and 3.`

WHY IT WORKS

Sort arrivals once. When both queues are empty, jump straight to the next arrival time; if the jump leaves at least one idle second, reset the previous state to 'none'. Otherwise apply the rule, record the time, and move one second ahead. Work depends on n, not on the size of the timestamps.

SHORT OPTIMAL CODE

```
from collections import deque


# BASE QUESTION (optimal): Event-driven simulation with
# two queues
# Idea: Sort arrivals once. When both queues are empty,
# jump straight to the next arrival time; if the jump
# leaves at least one idle second, reset the previous
# state to 'none'. Otherwise apply the rule, record the
# time, and move one second ahead. Work depends on n, not
# on the size of the timestamps.
def turnstile_times(events):
    num_people = len(events)
    people_by_arrival = sorted(range(num_people), key=lambda person: (events[person][0], person))
    enter_queue, exit_queue = deque(), deque()
    crossing_time = [0] * num_people
    time, next_arrival, last_direction, last_busy_second = 0, 0, None, -10
    while next_arrival < num_people or enter_queue or exit_queue:
        if not enter_queue and not exit_queue:
            time = max(time, events[people_by_arrival[next_arrival]][0])
            if time > last_busy_second + 1:
                last_direction = None
        while next_arrival < num_people and events[people_by_arrival[next_arrival]][0] <= time:
            (enter_queue if events[people_by_arrival[next_arrival]][1] == "enter" else exit_queue).append(peopl
e_by_arrival[next_arrival])
            next_arrival += 1
        queue = (enter_queue if last_direction == "enter" else exit_queue) if (enter_queue and exit_queue) else
 (enter_queue or exit_queue)
        person = queue.popleft()
        crossing_time[person], last_direction, last_busy_second = time, events[person][1], time
        time += 1
    return crossing_time
```

COMPLEXITY | Time O(n log n) | Space O(n)

18 Compute city skyline outline — HARD

Template move: events at each building's left and right edge; a max-heap of active heights; record a point when the top height changes

QUESTION

You are given a list of rectangular buildings in a 2D city skyline. Each building is represented by three integers **`[L, R, H]`**:

- **`L`**: the x-coordinate of the left edge of the building,
- **`R`**: the x-coordinate of the right edge of the building (**`L < R`**),

QUESTION

- **`H`**: the height of the building. All buildings stand on a flat horizontal line (y = 0) and extend vertically up to height **`H`**. Buildings may overlap in the x-dimension. Your task is to compute the *skyline* formed by these buildings, as seen from a distance. The skyline should be represented as a list of **key points `[x, height]`** that describe the outer contour of the union of all buildings. A **key point** is a point where the height of the skyline changes. The output list must satisfy all of the following:

1. The list is sorted by increasing **`x`**.
2. No two consecutive points have the same height (i.e., the height must change at each key point).
3. The first point has a height greater than 0.
4. The last point has height 0, marking the end of the skyline.

Input

- An integer **`n`** — the number of buildings.
- A list of **`n`** buildings, where each building is given as a triple **`[L, R, H]`** of integers. You may assume:
- **`1 <= n <= 10^4`**
- **`0 <= L < R <= 10^9`**
- **`1 <= H <= 10^9`**

Output

Return a list of key points representing the skyline. Each key point is a pair **`[x, height]`** (or a 2-element array/tuple) of integers.

Example

Input:

- **`n = 3`** - buildings = **`[[2, 9, 10], [3, 7, 15], [5, 12, 12]]`** One valid output:
- **`[[2, 10], [3, 15], [7, 12], [12, 0]]`** Explanation (informal):
- From x = 2 to 3, the highest building has height 10.
- From x = 3 to 7, the highest building has height 15.
- From x = 7 to 12, the remaining highest building has height 12.
- After x = 12, there are no buildings, so the height drops to 0. Design an algorithm to compute the skyline key points for the given buildings.

EXAMPLES

Example 1	
Input	`[[2, 9, 10], [3, 7, 15], [5, 12, 12], [15, 20, 10], [19, 24, 8]]`
Output	`[[2, 10], [3, 15], [7, 12], [12, 0], [15, 10], [20, 8], [24, 0]]`
Why	**`Buildings are [left, right, height]; the outline lists the points where the height changes.`**

Example 2	
Input	`[[0, 2, 3]]`
Output	`[[0, 3], [2, 0]]`
Why	**`One building: a point at its left edge and one at its right edge, where the height returns to 0.`**

WHY IT WORKS

Make events (x, -height) for left edges and (x, 0, right) for right edges, sorted so that at the same x taller starts come first and ends come after starts. Keep a heap of (-height, right edge). Before reading the maximum, pop buildings whose right edge is at or before the current x. When the maximum height differs from the last output height, emit [x, new height].

SHORT OPTIMAL CODE

```python
import heapq


# BASE QUESTION (optimal): Sweep line with a max-heap and
# lazy deletion
# Idea: Make events (position, -height) for left edges and (position, 0,
# right) for right edges, sorted so that at the same x
# taller starts come first and ends come after starts.
# Keep a heap of (-height, right edge). Before reading the
# maximum, pop buildings whose right edge is at or before
# the current x. When the maximum height differs from the
# last output height, emit [position, new height].
def getSkyline(buildings):
    events = sorted([(left, -height, right) for left, right, height in buildings] + [(right, 0, 0) for _, right
, _ in buildings])
    outline, heap = [], [(0, float("inf"))]
    for position, negative_height, right_edge in events:
        while heap[0][1] <= position:
            heapq.heappop(heap)
        if negative_height:
            heapq.heappush(heap, (negative_height, right_edge))
        current_height = -heap[0][0]
        if not outline or outline[-1][1] != current_height:
            outline.append([position, current_height])
    return outline
```

COMPLEXITY | Time O(n log n) | Space O(n)

GROUP 3 OF 43 | 9 QUESTIONS | SHORT MAIN CODE

Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

Pattern

Write dfs(node, depth). An empty node returns a base value, a real node calls dfs on both children, updates the best answer so far, and returns one number or flag to its parent. Only three lines change: the base value, the update, and what you return.

19 Find right-side view of binary tree

EASY

Template move: pass depth down; the first node you reach at a new depth (visiting right before left) is the one you can see

QUESTION

You are given the root of a binary tree. From the **right side** of the tree, at each depth you can see exactly one node: the rightmost node at that depth. Write a function that returns a list of the values of the nodes visible when looking at the tree from the right side, ordered from **top to bottom**. ---

Function Signature

(one possible form)

```
rightSideView(root: TreeNode) -> List[int]
```

Where **`TreeNode`** is defined as:

```
class TreeNode { int val; TreeNode left; TreeNode right; }
```

Example

Consider this binary tree:

```
1 / \ 2 3 \ \ 5 4
```

The right-side view is:

```
[1, 3, 4]
```

Explain the algorithm you would use, including its time and space complexity. You may use either breadth-first search (BFS) or depth-first search (DFS).

EXAMPLES

Example 1	
Input	`[1, 2, 3, null, 5, null, 4]`
Output	`[1, 3, 4]`
Why	`Looking from the right you see 1, then 3 (hiding 2), then 4 (hiding 5).`

Example 2	
Input	`[1, 2]`
Output	`[1, 2]`
Why	`A left child is visible when there is no right child at that depth.`

Example 3	
Input	`[]`
Output	`[]`
Why	`An empty tree shows nothing.`

WHY IT WORKS

Visit the right child first. The first node you meet at a depth that is not yet recorded is the rightmost one at that depth. Only O(height) memory is needed.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): DFS visiting right before left,
# first node at each depth
# Idea: Visit the right child first. The first node you
# meet at a depth that is not yet recorded is the
# rightmost one at that depth. Only O(height) memory is
# needed.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def rightSideView(root):
    right_view = []

    def dfs(node, depth):
        if node is None:
            return
        if depth == len(right_view):
            right_view.append(node.val)
        dfs(node.right, depth + 1)
        dfs(node.left, depth + 1)

    dfs(root, 0)
    return right_view
```

COMPLEXITY | Time O(n) | Space O(h)

20 Given two trees, determine if the pre-order traversals of two trees are same or not. EASY

Template move: walk both trees in preorder with an explicit stack (push right, then left) and compare value by value; stop at the first difference

QUESTION

Given two binary trees, determine whether their pre-order traversals (node, then left subtree, then right subtree) produce the same sequence of values. The trees themselves may have different shapes.

Function signature

def same_preorder(a: TreeNode, b: TreeNode) -> bool

EXAMPLES

Example 1	
Input	`[1, 2, 3], [1, 2, 3]`
Output	`true`
Why	`Identical trees.`

Example 2	
Input	`[1, 2, null, 3], [1, null, 2, null, 3]`
Output	`true`
Why	`Different shapes (a chain going left versus a chain going right) but both read 1, 2, 3.`

QUESTION

Example 3	
Input	`[1, 2, 3], [1, 3, 2]`
Output	`false`
Why	`1, 2, 3 versus 1, 3, 2.`

WHY IT WORKS

Yield node, left subtree and right subtree from both trees, then compare the two streams until a mismatch or the end.

SHORT OPTIMAL CODE

```python
from itertools import zip_longest


# BASE QUESTION (optimal): Recursive pre-order generators
# Idea: Yield node, left subtree, then right subtree from
# both trees and compare the streams until a mismatch.
def same_preorder(first_root, second_root):
    def preorder(node):
        if node is not None:
            yield node.val
            yield from preorder(node.left)
            yield from preorder(node.right)

    missing = object()
    return all(first_value == second_value for first_value, second_value in zip_longest(
        preorder(first_root), preorder(second_root), fillvalue=missing
    ))
```

COMPLEXITY | Time O(n) | Space O(h) recursion

21 Calculate the height of a binary tree. EASY

Template move: no update needed; return 1 + max(left, right) and read the value at the root

QUESTION

Given the root of a binary tree, return its height: the number of nodes on the longest path from the root down to a leaf (an empty tree has height 0).

Function signature

def height(root: TreeNode) -> int

EXAMPLES

Example 1	
Input	`[3, 9, 20, null, null, 15, 7]`
Output	`3`
Why	`Levels: [3], [9, 20], [15, 7]: height 3.`

Example 2	
Input	`[1, 2]`
Output	`2`
Why	`A root with one child.`

QUESTION

Example 3	
Input	`[]`
Output	`0`
Why	`An empty tree.`

WHY IT WORKS

An empty tree has height 0. Every other node has height 1 plus the larger height of its two children.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Recursive height
# Idea: A node's height is one plus the larger height of
# its two children. An empty tree has height zero.
def height(root):
    if root is None:
        return 0
    return 1 + max(height(root.left), height(root.right))
```

COMPLEXITY | Time O(n) | Space O(h) recursion

22 Expression Tree Single-Leaf Mutation — MEDIUM

Template move: post-order evaluation (children first, then apply the operator); to test one leaf flip, re-evaluate with that leaf changed

QUESTION

A boolean expression tree has leaves holding True or False and internal nodes AND, OR, XOR (two children) or NOT (one child). For each leaf in turn, imagine flipping only that leaf and report the value of the whole tree after the flip (every flip is independent of the others). Return the leaves and the list of answers, in leaf order.

Function signature

def flip_answers(root) -> (list, list)

(the leaves, the root value

after flipping each one)

EXAMPLES

Example 1	

Input	`t = Node('AND', Node(val=True), Node('OR', Node(val=False), Node(val=True)))`
	`evaluate(t)`
	`[leaf.val for leaf in flip_answers(t)[0]]`
	`flip_answers(t)[1]`

Output	`evaluate(t) -> true`
	`[leaf.val for leaf in flip_answers(t)[0]] -> [true, false, true]`
	`flip_answers(t)[1] -> [false, true, false]`

Why	`The tree is True AND (False OR True) = True. Flipping the first leaf gives False, flipping the False leaf changes nothing (the OR is still True), flipping the last leaf makes the OR False.`

WHY IT WORKS

One traversal stores every node's value and its parent. To flip a leaf, toggle it, then walk up recomputing each ancestor from its children's cached values, stopping as soon as a node's value does not change (then the root is unchanged too). Undo by restoring the saved values on the walked path.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Cache each value and parent
# Idea: Build the cache with DFS. For each leaf flip, only
# recompute its path to the root, then restore that path.
def apply(op, left, right=None):
    if op == "AND":
        return left and right
    if op == "OR":
        return left or right
    if op == "XOR":
        return left != right
    return not left


def flip_answers(root):
    parent = {root: None}
    value = {}
    leaves = []

    def build(node):
        if node.op is None:
            value[node] = node.val
            leaves.append(node)
            return
        parent[node.left] = node
        build(node.left)
        right_value = None
        if node.right is not None:
            parent[node.right] = node
            build(node.right)
            right_value = value[node.right]
        value[node] = apply(node.op, value[node.left], right_value)

    build(root)
    answers = []
    for leaf in leaves:
        changed = []
        value[leaf] = not value[leaf]
        changed.append(leaf)
        node = parent[leaf]
        while node is not None:
            right_value = value[node.right] if node.right is not None else None
            new_value = apply(node.op, value[node.left], right_value)
            if new_value == value[node]:
                break
            value[node] = new_value
            changed.append(node)
            node = parent[node]
        answers.append(value[root])
        for node in changed:
            value[node] = not value[node] if node.op is None else apply(
                node.op, value[node.left],
                value[node.right] if node.right is not None else None,
            )
    return answers
```

COMPLEXITY | Time O(n + L * depth) | Space O(n)

23 Longest Consecutive Path in Binary Tree

MEDIUM

Template move: pass the parent's value and the run length down; extend the run when value == parent + 1, else restart at 1

QUESTION

Given a binary tree, return the length of the longest path that goes from a parent down to a child where every value is exactly 1 more than the previous one (counted in nodes).

Function signature

def longestConsecutive(root: TreeNode) -> int

EXAMPLES

Example 1	
Input	[1, null, 3, 2, 4, null, null, null, 5]
Output	3
Why	3 -> 4 -> 5 is the longest increasing-by-one downward path.

Example 2	
Input	[2, null, 3, 2, null, 1]
Output	2
Why	2 -> 3 is the longest; 3 -> 2 and 2 -> 1 go the wrong way.

WHY IT WORKS

Pass the length of the run ending at the parent. If the child is exactly parent + 1 the run grows, otherwise it restarts at 1. Track the global best. Length is counted in nodes; state that choice out loud.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Top-down DFS carrying the
# current run
# Idea: Pass the length of the run ending at the parent.
# If the child is exactly parent + 1 the run grows,
# otherwise it restarts at 1. Track the global best.
# Length is counted in nodes; state that choice out loud.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def longestConsecutive(root):
    best = 0

    def dfs(node, parent_val, run):
        nonlocal best
        if node is None:
            return
        run = run + 1 if parent_val is not None and node.val == parent_val + 1 else 1
        best = max(best, run)
        dfs(node.left, node.val, run)
        dfs(node.right, node.val, run)

    dfs(root, None, 0)
    return best
```

COMPLEXITY | Time O(n) | Space O(h)

24 Implement zigzag level-order traversal

MEDIUM

Template move: collect values by depth; reverse every odd level (or fill each level from the other end)

QUESTION

Given the root of a binary tree, return its values level by level, reading the first level from left to right, the next from right to left, and so on, alternating. The result is a list of lists, one per level. Aim for O(n) time.

Function signature

def zigzag_level_order(root: TreeNode) -> list

EXAMPLES

Example 1	
Input	`[3, 9, 20, null, null, 15, 7]`
Output	`[[3], [20, 9], [15, 7]]`
Why	`Level 0 is [3], level 1 is read right to left [20, 9], level 2 left to right [15, 7].`

Example 2	
Input	`[1]`
Output	`[[1]]`
Why	`A single node.`

Example 3	
Input	`[]`
Output	`[]`
Why	`An empty tree has no levels.`

WHY IT WORKS

Keep a left-to-right flag. For each level build a deque; append on the right when going left to right and on the left when going right to left. Flip the flag after each level.

SHORT OPTIMAL CODE

```
from collections import deque


# BASE QUESTION (optimal): BFS filling each level from the
# left or the right
# Idea: Keep a left-to-right flag. For each level build a
# deque; append on the right when going left to right and
# on the left when going right to left. Flip the flag
# after each level.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def zigzagLevelOrder(root):
    levels, queue, left_to_right = [], deque([root] if root else []), True
    while queue:
        level = deque()
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val) if left_to_right else level.appendleft(node.val)
            queue.extend(child for child in (node.left, node.right) if child)
        levels.append(list(level))
        left_to_right = not left_to_right
    return levels
```

COMPLEXITY | Time O(n) | Space O(width)

25 Given the root of a binary tree, return the length of the diameter of the tree. MEDIUM

Template move: best = max(best, left + right); return 1 + max(left, right) (the diameter)

QUESTION

Given the root of a binary tree, create a function diameterOfTree that determines and returns the length of the diameter of the tree, where the diameter of a tree is defined as the number of nodes along the longest path between any two nodes in the tree. For example, given the following binary tree: // tree 1 / \ 2 3 / \ 4 5 The diameter is 4, which is the length of the path [4, 2, 1, 3] or [5, 2, 1, 3].Watch Ali, SDE @ Amazon, answer the question: "Given the root of a binary tree, return the length of the diameter of the tree."No answers contributed by the community yet.Interview experiences6 shared‹›Uber · 3 months ago"interview process is very lengthy took around 5 weeks, and expectations is very high in each round, for LLD working code is required, also dont mess your HM round since i felt like it contained more weightage"Software Engineer · Mid Level / L4Meta · 4 months ago"The interview went smoothly overall. 1 round of phone screen followed by the full loop of 4 rounds packed in a day. 2 coding 1 behavior 1 system design."Software Engineer · Senior / L5Google · 9 months ago"I interviewed for an AI Engineer internship, but the process was mostly DSA-heavy. I had an online coding round first, then a technical round with binary tree and DP questions, then another harder DSA round with DP and linked list questions, and finally an HR round. The online round covered arrays and strings, and the later rounds got progressively more difficult. The only non-DSA part was the last

EXAMPLES

Example 1	
Input	`[1, 2, 3, 4, 5]`
Output	`4`
Why	`The longest path 4 - 2 - 1 - 3 has 4 nodes.`

Example 2	
Input	`[1]`
Output	`1`
Why	`A single node.`

WHY IT WORKS

depth(node) returns the height of the node's subtree and, on the way, updates best = max(best, left + right). Every node is visited once, and the answer is best when the traversal ends (edges, not nodes).

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): One post-order pass returning
# heights and updating the best
# Idea: depth(node) returns the height of the node's
# subtree and, on the way, updates best = max(best, left +
# right). Every node is visited once, and the answer is
# best when the traversal ends (edges, not nodes).
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def diameterOfBinaryTree(root):
    best = 0

    def depth(node):
        nonlocal best
        if node is None:
            return 0
        left_height, right_height = depth(node.left), depth(node.right)
        best = max(best, left_height + right_height)
        return 1 + max(left_height, right_height)
```

```
    depth(root)
    return best
```

COMPLEXITY | Time O(n) | Space O(h)

26 Repeatedly Delete Leaves Matching a Target Value From a Binary Tree — HARD

Template move: set node.left = dfs(node.left) and node.right likewise; return None when the node became a leaf with the target value

QUESTION

Given a binary tree and an integer **`target`**, delete every leaf whose value equals **`target`**. Deleting a leaf can turn its parent into a leaf; if that parent's value also equals **`target`**, it must be deleted as well, and so on, until no leaf with value **`target`** remains. Return the resulting tree, which is empty if the root itself ends up deleted.

Function Signature

```
remove_target_leaves(tree: list[int | None], target: int) -> list[int | None]
```

Tree Encoding

The input and the output use the same level-order encoding:

- An empty tree is **`[]`**.
- Otherwise the first element is the root's value. Present nodes are then visited in breadth-first order, left to right, and for each one its left child and then its right child are appended, using **`None`** for an absent child. Children of absent nodes are not listed.
- Trailing **`None`** values are omitted. The input list is always well formed, and the output must be in exactly this canonical form.

Rules

- A leaf is a node with no children.
- A node whose value equals **`target`** but that still has at least one child after all deletions is kept.
- Kept nodes never move: every kept node keeps its original parent and its side (left or right).
- The returned tree contains no leaf whose value equals **`target`**.

Constraints

- The input tree has between 1 and 5000 nodes, and its height is at most 500.
- Node values and **`target`** are integers in **`[-1000000000, 1000000000]`**.

Examples

Input: **`tree = [5,7,7,7,3,None,7], target = 7`**
Output: **`[5,7,None,None,3]`** The leaf 7 under the root's left child is deleted, but that left child still has the child 3, so it stays. On the right side, deleting the lower 7 turns the root's right child into a leaf with value 7, so it is deleted too.
Input: **`tree = [3,3,1], target = 3`**
Output: **`[3,None,1]`** Only the left leaf is deleted. The root's value is 3, but it still has a child.
Input: **`tree = [4,4,4], target = 4`**
Output: **`[]`**

EXAMPLES

Example 1	
Input	`tree = [1, 2, 3, 2, null, 2, 4], target = 2 and tree = [1, 3, 3, 3, 2], target = 3`
Output	`values(remove_leaves(tree([1, 2, 3, 2, None, 2, 4]), 2)[0]) -> [1, null, 3, null, 4]` `values(remove_leaves(tree([1, 3, 3, 3, 2]), 3)[0]) -> [1, 3, null, null, 2]`
Why	`remove_leaves returns (the new root, how many leaves were removed). Deleting a leaf can turn its parent into a leaf, which is deleted too if it also equals the target: in the first tree the 2s go and the root's left branch disappears entirely.`

WHY IT WORKS

Process both children first, then look at the node: if it is now a leaf and its value is the target, return None to its parent. Because children are fixed before the parent is judged, a whole matching chain collapses in one traversal.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): One post-order pass
# Idea: Process both children first, then look at the
# node: if it is now a leaf and its value is the target,
# return None to its parent. Because children are fixed
# before the parent is judged, a whole matching chain
# collapses in one traversal.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def removeLeafNodes(root, target):
    def dfs(node):
        if node is None:
            return None
        node.left = dfs(node.left)
        node.right = dfs(node.right)
        if node.left is None and node.right is None and node.val == target:
            return None
        return node

    return dfs(root)
```

COMPLEXITY | Time O(n) | Space O(h)

27 Delete an Entire Binary Tree — HARD

Template move: post-order: free the children first, then the node itself

QUESTION

You are given the root of a binary tree (not a BST); each node has only left and right child pointers. You must "free" every node so that after the call the tree is empty. Because a node must not be freed before its children, the correct freeing order is a post-order traversal. Implement delete_tree(root) that frees all nodes and returns the list of node values in the exact order they are freed (post-order: left subtree, right subtree, then node).

Function signature

```
from typing import Optional
class TreeNode:
    def __init__(self, val: int = 0,
                 left: "Optional[TreeNode]" = None,
                 right: "Optional[TreeNode]" = None) -> None:
        self.val = val
        self.left = left
        self.right = right
def delete_tree(root: Optional[TreeNode]) -> list[int]:
    """Free every node and return the values in the order freed (post-order)."""
    ...
```

Follow-up (as reported): Recursive DFS uses O(h) stack space. Achieve O(1) extra space by repeatedly rotating the tree (Morris-style): while the root has a left child, rotate it up; once the root has no left child, free the root and move to its right child.

EXAMPLES

QUESTION

Example 1	
Input	`root = 1 / \ 2 3`
Output	`[2, 3, 1]`
Why	`Post-order frees the left child, then the right child, then the root.`

Example 2	
Input	`root = None`
Output	`[]`
Why	`An empty tree frees nothing.`

Example 3	
Input	`root = 1 / 2 / 3`
Output	`[3, 2, 1]`
Why	`A left-leaning chain frees deepest-first.`

CONSTRAINTS

- 0 <= number of nodes <= 10^5 .
- Node values are integers.

WHY IT WORKS

Free the left subtree, the right subtree and then the node.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Recursive post-order
# Idea: Free the left subtree, the right subtree and then
# the node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def delete_tree(root):
    freed = []

    def free(node):
        if node is None:
            return
        free(node.left)
        free(node.right)
        freed.append(node.val)
        node.left = node.right = None

    free(root)
    return freed
```

COMPLEXITY | Time O(n) | Space O(h)

GROUP 4 OF 43 | 9 QUESTIONS | SHORT MAIN CODE

A heap of what is available: pop the best, put back the changed one

Pattern

The heap holds the things you can choose from right now: machines and when they are free, rooms, lockers, the biggest remaining count, the two smallest weights. Each step pops the best choice, uses it, and pushes it back changed (or a combined item).

28 Streaming Median (LC 295) with Follow-ups

MEDIUM

Template move: two heaps: a max-heap for the lower half and a min-heap for the upper half; keep them balanced

QUESTION

Design a structure that receives numbers one at a time and can report the median of everything received so far at any moment. addNum(x) adds a number; findMedian() returns the middle value, or the mean of the two middle values when the count is even.

Function signature

```
class MedianFinder:
    def addNum(self, x: float) -> None
    def findMedian(self) -> float
```

EXAMPLES

Example 1

Input	`m = MedianFinder()`
	`m.addNum(5)`
	`m.addNum(2)`
	`m.findMedian()`
	`m.addNum(8)`
	`m.findMedian()`
	`m.addNum(1)`
	`m.findMedian()`

Output	`m.findMedian() -> 3.5`
	`m.findMedian() -> 5`
	`m.findMedian() -> 3.5`

Why	`Sorted so far: [2, 5] -> 3.5; [2, 5, 8] -> 5; [1, 2, 5, 8] -> 3.5.`

PART 1: Two heaps

Lower half in a max-heap, upper half in a min-heap, sizes differing by at most one and every lower value <= every upper value. The median is the larger heap's root, or the average of both roots.

PART 2: Small value domain: counting array

If values are integers in [0, V), keep a count per value. Adding is O(1); the median scans the counts once (or maintains a moving pointer to make it amortized O(1)).

PART 3: Any k-th smallest: Fenwick tree over the value range

A Fenwick (binary indexed) tree stores counts per value. The k-th smallest is found by descending the tree with binary lifting, so any rank, not just the median, costs O(log V). Compress coordinates if V is huge.

SHORT OPTIMAL CODE

```
import heapq


# BASE QUESTION (optimal): Two heaps
# Idea: Lower half in a max-heap, upper half in a min-
```

```python
# heap, sizes differing by at most one and every lower
# value <= every upper value. The median is the larger
# heap's root, or the average of both roots.
class MedianFinder:
    def __init__(self):
        self.low = []        # max-heap via negation
        self.high = []       # min-heap

    def addNum(self, number):
        heapq.heappush(self.low, -number)
        heapq.heappush(self.high, -heapq.heappop(self.low))
        if len(self.high) > len(self.low):
            heapq.heappush(self.low, -heapq.heappop(self.high))

    def findMedian(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2
```

COMPLEXITY | Part 1: Time O(log n) add, O(1) median | Space O(n)
Part 2: Time O(1) add, O(V) median | Space O(V)
Part 3: Time O(log V) add and query | Space O(V)

29 Compute minimum number of rooms needed

MEDIUM

Template move: heap of end times; a meeting reuses the room that frees first if it ended, otherwise push a new room

QUESTION

Problem

You are given a list of meetings, each with a start time and end time. A single room can host only one meeting at a time. Two meetings **overlap** if one starts before the other ends (treat meetings as half-open intervals **[start, end)** so a meeting ending at time **`t`** does not conflict with another starting at time **`t`**).

Task

Return the **minimum number of rooms** required to host all meetings.

Input

- **`intervals`**: an array of **`n`** pairs **`[start, end]`** where **`0 <= start <= end`**.

Output

- An integer: the minimum number of rooms needed.

Constraints (typical interview bounds)

- **`1 <= n <= 200000`**
- Times are integers in a reasonable range (e.g., **`0 . 10^9`**).

Example

- Input: **`[[0,30],[5,10],[15,20]]`**
- Output: **`2`** (One room can host **`[0,30]`**, the other can host **`[5,10]`** then **`[15,20]`**.)

You are given a list of meetings, where each meeting is represented as [start, end]. A single room can host only one meeting at a time. Meetings are treated as half-open intervals [start, end), so a meeting ending at time t does not conflict with another meeting starting at time t. Return the minimum number of rooms required to host all meetings. If start == end, the meeting occupies no time and does not require a room.
Constraints
0 <= n <= 2000000 <= start <= end <= 10^9

QUESTION

EXAMPLES

```
Input: ([[0,30],[5,10],[15,20]],)
Expected Output: 2
Explanation: [0,30] overlaps with both [5,10] and [15,20], so at most 2 rooms are needed at the same
    time.
Input: ([[7,10],[2,4],[4,7],[10,12]],)
Expected Output: 1
```

Explanation: These meetings only touch at endpoints, so one room can host all of them in sequence.

Hints

Try turning each meeting into two events: one when it starts and one when it ends.Because intervals are half-open, if a meeting ends at the same time another starts, process the ending event first.

PART 1: Sort by start with a min-heap of end times

Process meetings by start time. If the earliest-ending room has finished by this meeting's start, reuse it (pop and push the new end); otherwise open a new room (push). The heap size at the end is the number of rooms. Zero-length meetings (start == end) occupy no time under half-open intervals, so they are ignored.

PART 2: Sorted starts and ends with two pointers

Sort the starts and the ends separately. For each start, if it is at or after the earliest unfinished end, a room has been freed (move the end pointer); otherwise a new room is needed. No heap and no pairing of starts with ends is required.

SHORT OPTIMAL CODE

```python
import heapq


# BASE QUESTION (optimal): Sort by start with a min-heap
# of end times
# Idea: Process meetings by start time. If the earliest-
# ending room has finished by this meeting's start, reuse
# it (pop and push the new end); otherwise open a new room
# (push). The heap size at the end is the number of rooms.
# Zero-length meetings (start == end) occupy no time under
# half-open intervals, so they are ignored.
def min_meeting_rooms(intervals):
    ends = []
    for start, end in sorted(interval for interval in intervals if interval[0] < interval[1]):          # a zer
o-length meeting [t, t) occupies no time, so it needs no room
        if ends and ends[0] <= start:
            heapq.heapreplace(ends, end)
        else:
            heapq.heappush(ends, end)
    return len(ends)
```

COMPLEXITY | Part 1: Time O(n log n) | Space O(n)
Part 2: Time O(n log n) | Space O(n)

30 Implement a Reusable Locker Allocation System — MEDIUM

Template move: heap of freed locker numbers; take the smallest freed one before handing out a new number

QUESTION

Problem

A locker bank has lockers numbered from 1 through **n**. Process operations in order. **`store(customer)`** assigns the smallest available locker and returns its number, or **`-1`** if that customer already has a locker or none is available. **`pickup(customer)`** releases and returns that customer's locker, or **`-1`** if no locker is assigned. **`last()`** returns the locker number from the most recent successful **`store`**, or **`-1`** if there has not been one. Return one result per operation.

Constraints & Assumptions

QUESTION

- `1 <= n <= 200,000`.
- There are at most 200,000 operations.
- Customer IDs are non-empty strings.
- A released locker becomes immediately available for reuse.

Clarifications

- Failed store and pickup operations do not change `last()`.
- The allocation rule is always the smallest currently free locker number.
- A customer can store again after a successful pickup.

Examples

```
n = 3 operations = [store("a"), store("b"), pickup("a"), store("c"), last()] output = [1, 2, 1, 1, 1]
```

Hints

```
Track free lockers Use an ordered structure that can remove and reinsert the minimum locker efficiently.
```

```
Track ownership A map from customer ID to locker supports validation and pickup.
```

Process a fresh locker bank with lockers numbered 1 through n. Operations are ["store", customer], ["pickup", customer], or ["last"]. Store assigns and returns the smallest available locker, or -1 if the customer already owns one or none is free. Pickup releases and returns that customer's locker, or -1 if none is assigned. Last reports the locker from the most recent successful store, or -1 before any success. Return one integer per operation.
Constraints
1 <= n <= 200000.operations contains at most 200000 encoded store, pickup, or last lists.Customer IDs are non-empty strings.A released locker becomes immediately available for reuse.Failed store and pickup operations do not change last.A customer can store again after a successful pickup.

EXAMPLES

```
Input: (3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])
Expected Output: [1, 2, 1, 1, 1]
```

Explanation: This is the source example; the released smallest locker is reused.

```
Input: (5, [])
Expected Output: []
```

Explanation: An empty batch produces no per-operation results.

Hints

Keep available locker numbers in a min-oriented structure.Use a customer-to-locker map to validate stores and pickups.

WHY IT WORKS

store: if the customer already has a locker return -1; take the smallest released number from the heap if any, else the next never-used number if it is at most n, else -1. pickup: remove the customer's record and push the locker number on the heap. The heap never holds a number that is currently in use.

SHORT OPTIMAL CODE

```python
import heapq


# BASE QUESTION (optimal): Min-heap of released numbers
# plus a next-unused counter
# Idea: store: if the customer already has a locker return
# -1; take the smallest released number from the heap if
# any, else the next never-used number if it is at most n,
```

```python
# else -1. pickup: remove the customer's record and push
# the locker number on the heap. The heap never holds a
# number that is currently in use.
class Lockers:
    def __init__(self, locker_count):
        self.locker_count, self.next_new, self.released_lockers, self.locker_of = locker_count, 1, [], {}

    def store(self, customer):
        if customer in self.locker_of:
            return -1
        if self.released_lockers:
            locker_number = heapq.heappop(self.released_lockers)
        elif self.next_new <= self.locker_count:
            locker_number, self.next_new = self.next_new, self.next_new + 1
        else:
            return -1
        self.locker_of[customer] = locker_number
        return locker_number

    def pickup(self, customer):
        locker_number = self.locker_of.pop(customer, None)
        if locker_number is None:
            return -1
        heapq.heappush(self.released_lockers, locker_number)
        return locker_number
```

COMPLEXITY | Time O(log n) per operation | Space O(lockers released and not reused)

31 Simulate room assignments for scheduled meetings — MEDIUM

Template move: a heap of free room numbers and a heap of (end time, room); a delayed meeting takes the earliest-ending room and shifts later

QUESTION

There are n rooms numbered 0 to n - 1 and a list of meetings (start, end). Meetings are handled in order of start time. A meeting takes the lowest-numbered free room; if every room is busy it waits until the room that frees up first becomes free and then runs for its full original length, so it ends later. Return the room that hosted the most meetings (the lowest room number on a tie).

Function signature

def mostBookedRoom(n: int, meetings: list) -> int

EXAMPLES

Example 1	
Input	2, [[0, 10], [1, 5], [2, 7], [3, 4]]
Output	0
Why	Rooms 0 and 1 each host two meetings; the lower number wins the tie.

Example 2	
Input	3, [[1, 20], [2, 10], [3, 5], [4, 9], [6, 8]]
Output	1
Why	With three rooms, room 1 hosts two of the five meetings and no room hosts more.

WHY IT WORKS

Process meetings in start order. Move rooms that finished by the meeting's start into the free heap (lowest index first). If a room is free, use it. Otherwise delay the meeting until the earliest busy room finishes and keep its original length. Count meetings per room and return the busiest (lowest index on ties).

SHORT OPTIMAL CODE

```python
import heapq


# BASE QUESTION (optimal): Two heaps: free rooms and busy
# rooms
# Idea: Process meetings in start order. Move rooms that
# finished by the meeting's start into the free heap
# (lowest index first). If a room is free, use it.
# Otherwise delay the meeting until the earliest busy room
# finishes and keep its original length. Count meetings
# per room and return the busiest (lowest index on ties).
def mostBookedRoom(room_count, meetings):
    free_rooms = list(range(room_count))
    heapq.heapify(free_rooms)
    busy_rooms = []                       # (end time, room)
    meetings_per_room = [0] * room_count
    for start, end in sorted(meetings):
        while busy_rooms and busy_rooms[0][0] <= start:
            heapq.heappush(free_rooms, heapq.heappop(busy_rooms)[1])
        if free_rooms:
            room = heapq.heappop(free_rooms)
            heapq.heappush(busy_rooms, (end, room))
        else:
            finish, room = heapq.heappop(busy_rooms)
            heapq.heappush(busy_rooms, (finish + end - start, room))
        meetings_per_room[room] += 1
    return meetings_per_room.index(max(meetings_per_room))
```

COMPLEXITY | Time O(M log M + M log N) | Space O(N)

32 Minimum Makespan on Identical CPUs — MEDIUM

Template move: the template as written: heap of the times the CPUs become free

QUESTION

You have C identical CPUs and a list release_times , where release_times[i] is the earliest time job i may start. Every job runs non-preemptively for the same fixed duration S . When a job's release time arrives it may start on any free CPU; if all CPUs are busy it waits and starts on the first CPU that becomes free (never before its release time). Each CPU runs one job at a time. Scheduling optimally, compute the minimum possible makespan: the time at which the last job finishes.

Function signature

def min_makespan(release_times: list[int], num_cpus: int, duration: int) -> int:
    ...

Follow-up (as reported): (Reported) The naive approach tracked an O(n*C) table of CPU availability; reduce it to a min-heap of the C next-free times, popping the earliest-free CPU per job after sorting jobs by release time.

EXAMPLES

Example 1	
Input	`release_times = [0, 1, 2, 3], num_cpus = 2, duration = 3`
Output	`7`
Why	`CPU-A: job@0 -> [0,3), job@2 starts at 3 -> [3,6). CPU-B: job@1 -> [1,4), job@3 starts at 4 -> [4,7). Last finish = 7.`

Example 2	
Input	`release_times = [0, 0, 0], num_cpus = 1, duration = 2`
Output	`6`
Why	`One CPU runs them back to back: [0,2), [2,4), [4,6).`

QUESTION

Example 3	
Input	`release_times = [5], num_cpus = 3, duration = 4`
Output	`9`
Why	`The single job starts at its release time 5, finishes at 9.`

CONSTRAINTS

- 1 <= C <= 10^5
- 0 <= len(release_times) <= 10^5
- 0 <= release_times[i] <= 10^9
- 1 <= S <= 10^9
- If there are no jobs, the makespan is 0 .
- Target complexity: O(n log C) time, O(C) space.

WHY IT WORKS

Each job takes the CPU that frees first, but never starts before its release.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Sorted jobs and a heap of free
# times
# Idea: Each job takes the CPU that frees first, but never
# starts before its release.
import heapq


def min_makespan(release_times, num_cpus, duration):
    cpu_free_at = [0] * num_cpus
    makespan = 0
    for release_time in sorted(release_times):
        start = max(release_time, heapq.heappop(cpu_free_at))
        heapq.heappush(cpu_free_at, start + duration)
        makespan = max(makespan, start + duration)
    return makespan
```

COMPLEXITY | Time O(n log n) | Space O(C)

33 Design server allocation for multi-type nodes

MEDIUM

Template move: a heap per type of (flexibility, server); skip servers already allocated (lazy deletion)

QUESTION

Design a service that manages a pool of server nodes. Each node supports several instance types. getServer(count, types) hands out up to count servers that together cover all the requested types, preferring the least flexible servers (the ones that support the fewest types) so that versatile servers stay free, and returns None (allocating nothing) if the request cannot be met. releaseServer(ids) returns servers to the pool.

Function signature

class Servers:

def __init__(self, servers: dict)

id -> set of instance types

def getServer(self, count: int, types: list) -> list | None

def releaseServer(self, ids: list) -> None

EXAMPLES

QUESTION

Example 1

Input	`s = Servers({1: {'a'}, 2: {'a', 'b'}, 3: {'b'}, 4: {'a', 'b', 'c'}})`
	`s.getServer(2, ['a', 'b'])`
	`s.getServer(2, ['a', 'b'])`
	`s.getServer(1, ['a'])`
	`s.releaseServer([1])`
	`s.getServer(1, ['a'])`

Output	`s.getServer(2, ['a', 'b']) -> [1, 3]`
	`s.getServer(2, ['a', 'b']) -> [2, 4]`
	`s.getServer(1, ['a']) -> null`
	`s.getServer(1, ['a']) -> [1]`

Why	The first request takes the two least flexible servers (1 for a, 3 for b); the second takes 2 and 4; nothing is left for the third; after server 1 is released it can serve type a again.

WHY IT WORKS

For each type keep a heap of (number of supported types, id) for the servers that support it. A request first covers each requested type (taking the best server not already chosen), then fills the rest from the best heap tops among the requested types. Allocating a server sets its allocated flag, so stale entries in other types' heaps are discarded when they reach the top. If the request cannot be met, every claim is rolled back.

SHORT OPTIMAL CODE

```
from collections import defaultdict
import heapq


# BASE QUESTION (optimal): Per-type heaps ordered by
# flexibility, with lazy deletion
# Idea: For each type keep a heap of (number of supported
# types, id) for the servers that support it. A request
# first covers each requested type (taking the best server
# not already chosen), then fills the rest from the best
# heap tops among the requested types. Allocating a server
# sets its allocated flag, so stale entries in other
# types' heaps are discarded when they reach the top. If
# the request cannot be met, every claim is rolled back.
class Servers:
    def __init__(self, servers):
        self.types_of_server = {server_id: frozenset(server_types) for server_id, server_types in servers.items
()}
        self.allocated = set()
        self.heaps = defaultdict(list)
        for server_id, supported_types in self.types_of_server.items():
            for server_type in supported_types:
                heapq.heappush(self.heaps[server_type], (len(supported_types), server_id))

    def _top(self, server_type, skipped_ids):
        heap, skipped_entries = self.heaps[server_type], []
        while heap:
            flexibility, server_id = heap[0]
            if server_id in self.allocated:
                heapq.heappop(heap)
            elif server_id in skipped_ids:
                skipped_entries.append(heapq.heappop(heap))
            else:
                break
```

```
        result = heap[0] if heap else None            # read the best valid entry before putting the skipped on
es back
```

```
        for item in skipped_entries:
            heapq.heappush(heap, item)
        return result

    def getServer(self, count, types):
        chosen = []
        for server_type in types:
            if any(server_type in self.types_of_server[chosen_id] for chosen_id in chosen):
                continue
            top = self._top(server_type, set(chosen))
            if top is None:
                return None
            chosen.append(top[1])
        while len(chosen) < count:
            best = None
            for server_type in types:
                top = self._top(server_type, set(chosen))
                if top and (best is None or top < best):
                    best = top
            if best is None:
                return None
            chosen.append(best[1])
        if len(chosen) > count:
            return None
        self.allocated |= set(chosen)
        return chosen

    def releaseServer(self, ids):
        for server_id in ids:
            if server_id in self.allocated:
                self.allocated.discard(server_id)
                for server_type in self.types_of_server[server_id]:
                    heapq.heappush(self.heaps[server_type], (len(self.types_of_server[server_id]), server_id))
```

COMPLEXITY | Time O((count + types) log N) amortized per request | Space O(total type memberships)

34 Trip Expense Settlement

MEDIUM

Template move: a heap of creditors and a heap of debtors; the biggest of each settles min(amount) and the remainder goes back

QUESTION

A group of friends go on a trip and each friend may pay for others. Every record gives the payer, the amount and the list of people who share it equally (the payer may be one of them). Compute a small set of payments that settles everyone's debts: each payment is {payer, amount, payee}.

Function signature

def settle_greedy(records: list) -> list

EXAMPLES

Example 1

Input	`trip = [{'payer': 'alice', 'amount': 4000, 'payees': ['bob', 'jess', 'alice', 'sam']}, {'payer': 'jess', 'amount': 2000, 'payees': ['jess', 'alice']}]` `settle_greedy(trip)`
Output	`settle_greedy(trip) -> [{'payer': 'bob', 'amount': 1000, 'payee': 'alice'}, {'payer': 'sam', 'amount': 1000, 'payee': 'alice'}]`
Why	`Alice paid 4000 shared four ways and owes 1000 of Jess's 2000; Bob and Sam each owe 1000, and Jess is even. Two payments settle everything.`

QUESTION

Example 2	
Input	`trip = [{'payer': 'alice', 'amount': 3000, 'payees': ['alice', 'bob', 'cy']}]` `settle_greedy(trip)`
Output	`settle_greedy(trip) -> [{'payer': 'bob', 'amount': 1000, 'payee': 'alice'}, {'payer': 'cy', 'amount': 1000, 'payee': 'alice'}]`
Why	`Bob and Cy each owe Alice their 1000 share.`

CONSTRAINTS

- 1 <= payments.length <= 1000
- 1 <= amount <= 10^6
- 1 <= payees.length <= 100

PART 1: Heaps: match the largest debtor with the largest creditor

Push creditors (positive net) in a max-heap and debtors (negative net) in another. Repeatedly pay min(debt, credit) from the biggest debtor to the biggest creditor, and push back whoever still has a remainder. Each payment settles at least one person, so there are at most n - 1 payments, but it is not always the minimum possible number.

PART 2: Exact minimum with a bitmask DP over zero-sum groups

A group of k people whose balances sum to zero can settle among themselves with k - 1 payments, so the minimum is (people with a balance) minus (the maximum number of disjoint zero-sum groups). dp[mask] is the largest number of zero-sum groups inside mask: add one person at a time and gain 1 when the mask's total is zero. Feasible for about 20 people.

SHORT OPTIMAL CODE

```python
import heapq


# BASE QUESTION (optimal): Heaps: match the largest debtor
# with the largest creditor
# Idea: Push creditors (positive net) in a max-heap and
# debtors (negative net) in another. Repeatedly pay
# min(debtors, credit) from the biggest debtor to the biggest
# creditor, and push back whoever still has a remainder.
def balances(records):
    # net balance per person: what they paid minus their
    # equal share of every record they are part of
    net = {}
    for record in records:
        payer, amount, people = record["payer"], record["amount"], record["payees"]
        share = amount // len(people)
        net[payer] = net.get(payer, 0) + amount
        for person in people:
            net[person] = net.get(person, 0) - share
    return {person: balance for person, balance in net.items() if balance}


def settle_greedy(records):
    net = balances(records)
    creditors = [(-balance, person) for person, balance in net.items() if balance > 0]
    debtors = [(balance, person) for person, balance in net.items() if balance < 0]
    heapq.heapify(creditors)
    heapq.heapify(debtors)
    payments = []
    while creditors and debtors:
        neg_credit, creditor = heapq.heappop(creditors)
        neg_debt, debtor = heapq.heappop(debtors)
        payment = min(-neg_credit, -neg_debt)
        payments.append({"payer": debtor, "amount": payment, "payee": creditor})
        if -neg_credit > payment:
            heapq.heappush(creditors, (neg_credit + payment, creditor))
```

```
        if -neg_debt > payment:
            heapq.heappush(debtors, (neg_debt + payment, debtor))
    return payments
```

COMPLEXITY | Part 1: Time O(n log n) | Space O(n)
Part 2: Time O(2^n * n) | Space O(2^n)

35 Playlist Shuffle No Adjacent Artists

MEDIUM

Template move: pop the artist with the most songs left, hold it for one turn, then push it back with a smaller count

QUESTION

Given the artist of every song in a playlist, return an order of the artists in which as few neighbours as possible are by the same artist: with no repeats next to each other whenever that is possible.

Function signature

def shuffle_no_adjacent(artists: list) -> list

EXAMPLES

Example 1	
Input	`['a', 'a', 'b', 'b', 'c']`
Output	`['a', 'b', 'a', 'b', 'c']`
Why	Two a's and two b's alternate; c fits wherever it likes.

Example 2	
Input	`['a', 'a', 'a', 'b']`
Output	`['a', 'b', 'a', 'a']`
Why	Three songs by 'a' cannot all be separated by a single 'b': the best is to keep two a's together at the end.

Example 3	
Input	`['a', 'a']`
Output	`['a', 'a']`
Why	A single artist: nothing can be done.

CONSTRAINTS

- 1 <= songs.length <= 10^4

WHY IT WORKS

Keep a heap of (remaining songs, artist). Each step pop the artist with the most songs left; play one of their songs, and push them back only after the next artist has played (so they cannot repeat). If only the previous artist is left, play it anyway: that is an unavoidable adjacency. For feasible inputs the result has no adjacent pair; otherwise the number of adjacent pairs equals the minimum, 2m - n - 1 for a dominant artist with m songs.

SHORT OPTIMAL CODE

```
from collections import Counter
import heapq


# BASE QUESTION (optimal): Greedy with a max-heap, holding
# back the last artist
# Idea: Keep a heap of (remaining songs, artist). Each
# step pop the artist with the most songs left; play one
# of their songs, and push them back only after the next
# artist has played (so they cannot repeat). If only the
```

```
# previous artist is left, play it anyway: that is an
# unavoidable adjacency. For feasible inputs the result
# has no adjacent pair; otherwise the number of adjacent
# pairs equals the minimum, 2m - n - 1 for a dominant
# artist with m songs.
def shuffle_no_adjacent(artists):
    counts = Counter(artists)
    heap = [(-songs_left, artist) for artist, songs_left in counts.items()]
    heapq.heapify(heap)
    playlist, held = [], None
    while heap or held:
        if not heap:
            songs_left, artist = held
            playlist.append(artist)
            held = (songs_left + 1, artist) if songs_left + 1 else None
            continue
        songs_left, artist = heapq.heappop(heap)
        playlist.append(artist)
        if held:
            heapq.heappush(heap, held)
        held = (songs_left + 1, artist) if songs_left + 1 else None
    return playlist
```

COMPLEXITY | Time O(n log A) | Space O(A)

36 Huffman Tree Construction — HARD

Template move: pop the two smallest weights, push their sum as a new node; repeat until one node remains

QUESTION

Given symbol frequencies, build the Huffman code: repeatedly merge the two least frequent items into one until a single tree is left; the path from the root gives each symbol its bit string. Return the code of every symbol. To make the answer unique, the codes are assigned canonically: shorter codes first, then alphabetical order.

Function signature

def huffman_codes(freq: dict) -> dict

EXAMPLES

Example 1	
Input	{'a': 5, 'b': 9, 'c': 12, 'd': 13, 'e': 16, 'f': 45}
Output	{'f': '0', 'c': '100', 'd': '101', 'e': '110', 'a': '1110', 'b': '1111'}
Why	The frequent symbol f gets a 1-bit code; the rare ones get longer codes.

Example 2	
Input	{'x': 3, 'y': 8}
Output	{'x': '0', 'y': '1'}
Why	Two symbols always get one bit each.

Example 3	
Input	{'z': 10}
Output	{'z': '0'}
Why	A single symbol still needs a code of one bit.

WHY IT WORKS

Put every symbol in a min-heap keyed by frequency, repeatedly pop the two lightest nodes, merge them under a new internal node with the summed weight and push it back. The last node is the root. A counter is the second heap key so ties never compare tree nodes. Codes come from a DFS: left edge '0', right edge '1'. A single-symbol alphabet needs the special case of

code '0'.

SHORT OPTIMAL CODE

```python
import heapq


# BASE QUESTION (optimal): Min-heap merge of the two
# lightest nodes
# Idea: Put every symbol in a min-heap keyed by frequency,
# repeatedly pop the two lightest nodes, merge them under
# a new internal node with the summed weight and push it
# back. The last node is the root. A counter is the second
# heap key so ties never compare tree nodes. Codes come
# from a DFS: left edge '0', right edge '1'. A single-
# symbol alphabet needs the special case of code '0'.
def huffman_codes(freq):
    heap = [(weight, insertion_order, symbol, None, None) for insertion_order, (symbol, weight) in enumerate(so
rted(freq.items()))]
    if not heap:
        return {}
    heapq.heapify(heap)
    counter = len(heap)
    while len(heap) > 1:
        lightest = heapq.heappop(heap)
        second_lightest = heapq.heappop(heap)
        heapq.heappush(heap, (lightest[0] + second_lightest[0], counter, None, lightest, second_lightest))
        counter += 1
    root = heap[0]
    codes = {}
    stack = [(root, "")]
    while stack:
        node, prefix = stack.pop()
        if node[2] is not None:
            codes[node[2]] = prefix or "0"
        else:
            stack.append((node[4], prefix + "1"))
            stack.append((node[3], prefix + "0"))
    return codes
```

COMPLEXITY | Time O(n log n) | Space O(n)

GROUP 5 OF 43 | 9 QUESTIONS | SHORT MAIN CODE

Recurrence on a position or a range (memo or table)

Pattern

Define best(i): the answer for everything from position i onward (or best(i, j) for a range, or a table over two sequences). Write what an empty or finished state returns, then combine the answers of the states you can jump to. Cache it, or fill a table in the right order.

37 Target Expressions from Ordered Digits — MEDIUM

Template move: best(l, r) = every value reachable from digits l..r; split at m and combine the two sides with +, - and *

QUESTION

Given a list of digits in order and a target number, put +, - or * between the digits (never reorder them and never glue two digits into one number) and add parentheses wherever you like. Return every distinct expression that evaluates to the target.

Function signature

def expressions_for_target(digits: list, target: int) -> list

EXAMPLES

Example 1	
Input	`[2, 3, 4], 14`
Output	`['2*(3+4)', '2+3*4']`
Why	`2 * (3 + 4) = 14 and 2 + 3 * 4 = 14 are the only two ways.`

Example 2	
Input	`[1, 2, 3], 6`
Output	`['1*2*3', '1+2+3']`
Why	`Three digits can reach 6 several ways.`

Example 3	
Input	`[1, 2], 100`
Output	`[]`
Why	`No way to reach 100 with these digits.`

WHY IT WORKS

Every expression over digits[l:r] is two sub-expressions over a split point joined by +, - or *. Memoize, for each interval, a map from numeric value to the set of expression strings that produce it. Keep expression construction separate from evaluation, and add parentheses only where precedence demands them.

SHORT OPTIMAL CODE

```
from collections import defaultdict


# BASE QUESTION (optimal): Interval DP: all expressions
# for digits[start:end]
# Idea: Every expression over digits[start:end] is two sub-
# expressions over a split point joined by +, - or *.
# Memoize, for each interval, a map from numeric value to
# the set of expression strings that produce it. Keep
# expression construction separate from evaluation, and
# add parentheses only where precedence demands them.
```

```python
def expressions_for_target(digits, target):
    memo = {}

    def build(start, end):
        if (start, end) in memo:
            return memo[(start, end)]
        by_value = defaultdict(set)                    # value -> {(text, precedence)}
        if end - start == 1:
            by_value[digits[start]].add((str(digits[start]), 3))
        for split in range(start + 1, end):
            for left_value, left_texts in build(start, split).items():
                for right_value, right_texts in build(split, end).items():
                    for left_text, left_precedence in left_texts:
                        for right_text, right_precedence in right_texts:
                            for op, precedence, value in (("+", 1, left_value + right_value), ("-", 1, left_val
ue - right_value), ("*", 2, left_value * right_value)):
                                left = f"({left_text})" if left_precedence < precedence else left_text
                                right = f"({right_text})" if (right_precedence < precedence or (right_precedenc
e == precedence and op == "-")) else right_text
                                by_value[value].add((f"{left}{op}{right}", precedence))
        memo[(start, end)] = by_value
        return by_value

    return sorted(text for text, _ in build(0, len(digits)).get(target, ()))
```

COMPLEXITY | Time exponential in the number of results; polynomial in the number of distinct values | Space O(distinct values per interval)

38 Count Word Decompositions into Chemical Element Symbols — MEDIUM

Template move: bottom-up ways[i]: add ways[i+1] if the one-letter symbol exists and ways[i+2] if the two-letter symbol exists

QUESTION

Chemical elements have symbols of one or two letters (H, He, Si, Cs ...). Given a word, count the ways to write it as a sequence of element symbols (ignore upper and lower case). Different splits count as different ways even if they spell the same word.

Function signature

def count_decompositions(word: str) -> int

EXAMPLES

Example 1	
Input	'Physics'
Output	4
Why	P-H-Y-Si-C-S, P-H-Y-Si-Cs, P-H-Y-S-I-C-S and P-H-Y-S-I-Cs: 4 ways.

Example 2	
Input	'Ho'
Output	2
Why	'Ho' is one symbol (holmium) and also H + O: 2 ways.

Example 3	
Input	'xqz'
Output	0
Why	No split works (x, y... cannot all be symbols).

WHY IT WORKS

ways[i] is the number of decompositions of word[i:]. A symbol has one or two letters, so ways[i] = ways[i + 1] (if word[i] is a symbol) + ways[i + 2] (if word[i:i + 2] is a symbol). ways[n] = 1 for the empty suffix. Matching is case-insensitive.

SHORT OPTIMAL CODE

```
ELEMENTS = set("""h he li be b c n o f ne na mg al si
p s cl ar k ca sc ti v cr mn fe co ni cu zn ga ge as
se br kr rb sr y zr nb mo tc ru rh pd ag cd in sn sb
te i xe cs ba la ce pr nd pm sm eu gd tb dy ho er tm
yb lu hf ta w re os ir pt au hg tl pb bi po at rn fr
ra ac th pa u np pu am cm bk cf es fm md no lr rf db
sg bh hs mt ds rg cn nh fl mc lv ts og""".split())


# BASE QUESTION (optimal): Suffix-count DP
# Idea: ways[position] is the number of decompositions of
# word[position:]. A symbol has one or two letters, so ways[position] =
# ways[position + 1] (if word[position] is a symbol) + ways[position + 2] (if
# word[position:position + 2] is a symbol). ways[word_length] = 1 for the empty
# suffix. Matching is case-insensitive.
def count_element_decompositions(word, symbols=ELEMENTS):
    lowered_word = word.lower()
    word_length = len(lowered_word)
    ways = [0] * (word_length + 2)
    ways[word_length] = 1
    for position in range(word_length - 1, -1, -1):
        if lowered_word[position] in symbols:
            ways[position] += ways[position + 1]
        if position + 2 <= word_length and lowered_word[position:position + 2] in symbols:
            ways[position] += ways[position + 2]
    return ways[0]
```

COMPLEXITY | Time O(n) | Space O(n)

39 Fence Painting Minimum Operations (Gemini Team) — MEDIUM

Template move: range (l, r): paint one stripe at the lowest height, then solve the pieces on both sides; a sparse table finds the lowest point fast

QUESTION

A fence has n planks, plank i is a[i] units tall. One stroke is either horizontal (paints one row of height h across a run of neighbouring planks that are all at least h tall) or vertical (paints one plank completely, bottom to top). Return the smallest number of strokes needed to paint the whole fence.

Function signature

def paint(a: list) -> int

EXAMPLES

Example 1	
Input	`[2, 2, 1, 2, 1]`
Output	`3`
Why	`Row 1 across all five planks, row 2 across the first two planks, and one vertical stroke on plank 4: 3 strokes.`

Example 2	
Input	`[1, 1, 1]`
Output	`1`
Why	`One horizontal stroke of height 1 across all three planks paints the whole fence.`

QUESTION

Example 3	
Input	`[5]`
Output	`1`
Why	`One tall plank: a single vertical stroke beats five horizontal ones.`

WHY IT WORKS

The recursion is correct; only the minimum lookup is slow. A sparse table answers range-minimum-index queries in O(1), and each plank becomes the split point at most once, so the recursion visits n ranges. Splitting at one minimum plank (instead of all of them) keeps ranges disjoint and makes the count obvious.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Same recursion with a sparse
# table for range minimum
# Idea: The recursion is correct; only the minimum lookup
# is slow. A sparse table answers range-minimum-index
# queries in O(1), and each plank becomes the split point
# at most once, so the recursion visits n ranges.
# Splitting at one minimum plank (instead of all of them)
# keeps ranges disjoint and makes the count obvious.
def paint(heights):
    plank_count = len(heights)
    if plank_count == 0:
        return 0
    table = [list(range(plank_count))]
    level = 1
    while (1 << level) <= plank_count:
        previous_row, next_row = table[-1], []
        for start in range(plank_count - (1 << level) + 1):
            left_candidate, right_candidate = previous_row[start], previous_row[start + (1 << (level - 1))]
            next_row.append(left_candidate if heights[left_candidate] <= heights[right_candidate] else right_ca
ndidate)
        table.append(next_row)
        level += 1

    def argmin(left, right):
        span_level = (right - left + 1).bit_length() - 1
        left_candidate, right_candidate = table[span_level][left], table[span_level][right - (1 << span_level)
+ 1]
        return left_candidate if heights[left_candidate] <= heights[right_candidate] else right_candidate

    def solve(left, right, base):
        if left > right:
            return 0
        lowest_index = argmin(left, right)
        return min(right - left + 1, heights[lowest_index] - base + solve(left, lowest_index - 1, heights[lowes
t_index]) + solve(lowest_index + 1, right, heights[lowest_index]))

    return solve(0, plank_count - 1, 0)
```

COMPLEXITY | Time O(n log n) | Space O(n log n)

40 Maximize 1D deliveries within distance and minimize total distance

MEDIUM

Template move: table over (orders used, drivers used): skip an order, skip a driver, or match them when they are within D (count, then minus distance)

QUESTION

Orders and drivers are points on a line. A driver can deliver an order that is at most D away; every driver takes at most one order and every order is delivered by at most one driver. First maximize the number of deliveries; among all plans with that many deliveries, minimize the total distance driven. Return (number of deliveries, total distance).

Function signature

def deliveries(orders: list, drivers: list, D: int) -> tuple

EXAMPLES

Example 1	
Input	`[1, 5, 9], [2, 6, 20], 2`
Output	`(2, 2)`
Why	`1 is served by the driver at 2, and 5 by the driver at 6: 2 deliveries with total distance 1 + 1 = 2.`

Example 2	
Input	`[1, 5, 9], [2, 6, 20], 11`
Output	`(3, 13)`
Why	`With D = 11 the driver at 20 can also take the order at 9 (distance 11): 1 + 1 + 11 = 13 for 3 deliveries.`

Example 3	
Input	`[3, 4], [], 5`
Output	`(0, 0)`
Why	`No drivers, no deliveries.`

WHY IT WORKS

Sort both lists. dp[i][j] is the best (matches, -total distance) using the first i orders and the first j drivers: skip order i, skip driver j, or match them if their distance is at most D. Because an optimal matching can be taken non-crossing, matching in sorted order loses nothing. Compare pairs lexicographically.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): DP over sorted orders and
# drivers, maximising (matches, -distance)
# Idea: Sort both lists. dp[orders_used][drivers_used] is the best (matches,
# -total distance) using the first i orders and the first
# j drivers: skip order i, skip driver j, or match them if
# their distance is at most D. Because an optimal matching
# can be taken non-crossing, matching in sorted order
# loses nothing. Compare pairs lexicographically.
def deliveries(orders, drivers, max_distance):
    sorted_orders, sorted_drivers = sorted(orders), sorted(drivers)
    dp = [[(0, 0)] * (len(sorted_drivers) + 1) for _ in range(len(sorted_orders) + 1)]
    for orders_used in range(1, len(sorted_orders) + 1):
        for drivers_used in range(1, len(sorted_drivers) + 1):
            best = max(dp[orders_used - 1][drivers_used], dp[orders_used][drivers_used - 1])
            if abs(sorted_orders[orders_used - 1] - sorted_drivers[drivers_used - 1]) <= max_distance:
                matches, negative_distance = dp[orders_used - 1][drivers_used - 1]
                best = max(best, (matches + 1, negative_distance - abs(sorted_orders[orders_used - 1] - sorted_
drivers[drivers_used - 1])))
            dp[orders_used][drivers_used] = best
    matches, negative_distance = dp[-1][-1]
```

```
    return (matches, -negative_distance)
```

COMPLEXITY | Time O(n * m) | Space O(n * m)

41 Find the longest palindromic subsequence using dynamic programming. MEDIUM

Template move: table over (i, j): equal ends add 2 to the inside; otherwise the better of dropping either end

QUESTION

Given a string s, return the length of its longest palindromic subsequence: the longest sequence of letters of s, taken in order but not necessarily next to each other, that reads the same forwards and backwards.

Function signature

def lps_length(s: str) -> int

EXAMPLES

Example 1	
Input	'bbbab'
Output	4
Why	'bbbb' is a subsequence of 'bbbab' and a palindrome.

Example 2	
Input	'cbbd'
Output	2
Why	'bb' (the two b's) is the longest.

Example 3	
Input	'a'
Output	1
Why	A single letter.

WHY IT WORKS

dp[i][j] is the length of the longest palindromic subsequence of s[i..j]. Fill i from the end to the start and j from i + 1 up: equal ends give dp[i + 1][j - 1] + 2, otherwise max(dp[i + 1][j], dp[i][j - 1]). Only the next row and the current row are needed.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Interval DP with a rolling row
# Idea: dp[start][end] is the length of the longest palindromic
# subsequence of text[start..j]. Fill i from the end to the start
# and j from start + 1 up: equal ends give dp[start + 1][end - 1] +
# 2, otherwise max(dp[start + 1][end], dp[start][end - 1]). Only the
# next row and the current row are needed.
def longestPalindromeSubseq(text):
    length = len(text)
    if length == 0:
        return 0
    row_below = [0] * length                                    # dp[start + 1][*]
    for start in range(length - 1, -1, -1):
        current_row = [0] * length
        current_row[start] = 1
        for end in range(start + 1, length):
            current_row[end] = row_below[end - 1] + 2 if text[start] == text[end] else max(row_below[end], curr
ent_row[end - 1])
        row_below = current_row
    return row_below[length - 1]
```

COMPLEXITY | Time O(n^2) | Space O(n)

42 Jump Game with Profit

MEDIUM

Template move: best[i] = max over every j before i that can reach i (j + arr[j] >= i) of best[j] + profit[i]

QUESTION

You stand at index 0 of an array. From index i you may jump to any index from i + 1 up to i + arr[i]. Landing on index i earns profit[i] (the start earns profit[0] too). Return the largest total profit of a path that ends exactly at the last index, or -1 if the last index cannot be reached.

Function signature

def max_profit_fast(arr: list, profit: list) -> int

EXAMPLES

Example 1	
Input	`[2, 3, 1, 1, 4], [1, 5, 2, 3, 4]`
Output	`15`
Why	`Jumping one index at a time visits (and collects) everything: 1 + 5 + 2 + 3 + 4 = 15.`

Example 2	
Input	`[1, 0, 1], [1, 1, 1]`
Output	`-1`
Why	`From index 1 you cannot jump at all (arr[1] = 0), so the last index is unreachable.`

PART 1: Reading A: arr[i] is a maximum jump, profit[i] is gained on landing, maximize total profit to the last index

Let best[i] be the largest profit achievable when standing on i. Landing on i from any j < i with j + arr[j] >= i gives best[i] = profit[i] + max(best[j]). Unreachable cells stay at minus infinity. Return best[n - 1], or -1 if the end cannot be reached. Pull-style DP keeps the reachability test explicit.

PART 2: Reading B: arr[i] is an exact jump; at each index you may jump (gaining profit[i]) or skip one cell for free

Memoize from the right. dp[i] is the best profit starting at i, with dp[i] = 0 for i >= n (past the end ends the game). dp[i] = max(skip: dp[i + 1], jump: profit[i] + dp[i + arr[i]]). This is the 'exact jump, not range cap' reading the original report warns about.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Reading A: max_jump_from[cell] is a maximum
# jump, profit[cell] is gained on landing, maximize total
# profit to the last index
# Idea: Let best[cell] be the largest profit achievable when
# standing on i. Landing on i from any earlier_cell < cell with earlier_cell +
# max_jump_from[earlier_cell] >= cell gives best[cell] = profit[cell] + max(best[earlier_cell]).
# Unreachable cells stay at minus infinity. Return best[cell_count
# - 1], or -1 if the end cannot be reached. Pull-style DP
# keeps the reachability test explicit.
def max_profit_jump(max_jump_from, profit):
    cell_count = len(max_jump_from)
    MINUS_INFINITY = float("-inf")
    best = [MINUS_INFINITY] * cell_count
    best[0] = profit[0]
    for cell in range(1, cell_count):
        for earlier_cell in range(cell):
            if best[earlier_cell] != MINUS_INFINITY and earlier_cell + max_jump_from[earlier_cell] >= cell:
                best[cell] = max(best[cell], best[earlier_cell] + profit[cell])
    return -1 if best[-1] == MINUS_INFINITY else best[-1]
```

COMPLEXITY | Part 1: Time O(n * maxJump) | Space O(n)
Part 2: Time O(n) | Space O(n)

43 Count Perfect Wake Numbers up to N

HARD

Template move: digit DP: cache best(remaining digits, used-digit mask, last digit, falling?) and walk along N with a tight flag

QUESTION

A number is perfect if its digits are all different, it contains no 0, and no digit is a valley (a digit strictly smaller than both of its neighbours). Part 1: decide whether a number is perfect. Part 2: count the perfect numbers from 1 to N.

Function signature

def is_perfect(n: int) -> bool

def count_up_to(N: int) -> int

EXAMPLES

Example 1	
Input	196
Output	true
Why	Digits 1, 9, 6 are different, no zero, and 9 is a peak, not a valley.

Example 2	
Input	1546
Output	false
Why	The 4 in 1546 sits between 5 and 6, both larger: a valley.

Example 3	
Input	99
Output	81
Why	Every number 1-9 and every two-digit number with different non-zero digits: 9 + 72 = 81.

PART 1: Check one number

Reject zero, repeated digits, or an increase after the digits have started decreasing.

PART 2: Count through N

Digit DP tracks used digits, the last digit, whether the sequence is falling, and the upper-bound flag.

SHORT OPTIMAL CODE

```
from functools import lru_cache


def is_perfect(number):
    digits = [int(digit_char) for digit_char in str(number)]
    if 0 in digits or len(set(digits)) != len(digits):
        return False
    falling = False
    for previous_digit, next_digit in zip(digits, digits[1:]):
        if falling and next_digit > previous_digit:
            return False
        falling |= next_digit < previous_digit
    return True


def count_up_to(number):
    if number < 1:
        return 0
    digits = [int(digit_char) for digit_char in str(number)]

    @lru_cache(None)
```

```
    def dp(position, used_digits, last_digit, falling, tight, started):
        if position == len(digits):
            return int(started)
        limit = digits[position] if tight else 9
        total = dp(position + 1, 0, -1, False, tight and limit == 0, False) if not started else 0
        for digit in range(1, limit + 1):
            if used_digits & (1 << digit):
                continue
            if started and falling and digit > last_digit:
                continue
            total += dp(position + 1, used_digits | (1 << digit), digit,
                        falling or (started and digit < last_digit),
                        tight and digit == limit, True)
        return total

    return dp(0, 0, -1, False, True, False)
```

COMPLEXITY | Part 1: Time O(d) | Space O(d)
Part 2: Time O(d * 2^10 * 10^2) | Space O(d * 2^10 * 10)

44 Word Break: All Sentences — HARD

Template move: best(start) = the list of sentences for the suffix; for every word that matches at start, put it in front of each sentence of best(end)

QUESTION

You are given a string s and a dictionary word_dict of distinct words. Return ALL possible ways to segment s into a sequence of space-separated words such that every word in the sequence appears in word_dict . A dictionary word may be reused any number of times. Each returned sentence is a single string of the chosen words joined by single spaces. The sentences may be returned in any order.

Function signature

from typing import List
def word_break_all(s: str, word_dict: List[str]) -> List[str]:
    ...

Follow-up (as reported): Solve with backtracking: at each position try every dictionary word that is a prefix of the remaining string, then recurse. Add memoization keyed on the suffix (or start index) to reuse partial results and avoid exponential re-work.

EXAMPLES

Example 1	
Input	s = "catsanddog", word_dict = ["cat","cats","and","sand","dog"]
Output	["cats and dog", "cat sand dog"]
Why	Two segmentations use only dictionary words.

Example 2	
Input	s = "pineapplepenapple", word_dict = ["apple","pen","applepen","pine","pineapple"]
Output	["pine apple pen apple", "pineapple pen apple", "pine applepen apple"]
Why	All three segmentations consist entirely of dictionary words.

Example 3	
Input	s = "catsandog", word_dict = ["cats","dog","sand","and","cat"]
Output	[]
Why	No segmentation covers the whole string with dictionary words.

CONSTRAINTS

- 1 <= len(s) <= 20

QUESTION

- 1 <= len(word_dict) <= 1000
- 1 <= len(word) <= 10 for each word; words consist of lowercase letters and are distinct.
- If no valid segmentation exists, return an empty list [] .

WHY IT WORKS

The sentences of a suffix do not depend on how you reached it, so compute them once.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Memoized DFS on the start index
# Idea: The sentences of a suffix do not depend on how you
# reached it, so compute them once.
def word_break_all(text, word_dict):
    words = set(word_dict)
    memo = {}

    def sentences(start):
        if start == len(text):
            return [""]
        if start in memo:
            return memo[start]
        sentences_found = []
        for end in range(start + 1, len(text) + 1):
            if text[start:end] in words:
                for rest in sentences(end):
                    sentences_found.append(text[start:end] + (" " + rest if rest else ""))
        memo[start] = sentences_found
        return sentences_found

    return sentences(0)
```

COMPLEXITY | Time O(n^2 + output) | Space O(n + output)

45 Edit distance — HARD

Template move: table over two strings, two rows: equal letters copy the diagonal, otherwise 1 + the smallest of the three neighbours

QUESTION

Given two words a and b, return the edit distance: the smallest number of single-letter operations (insert a letter, delete a letter, or replace a letter) that turn a into b.

Function signature

def edit_distance(a: str, b: str) -> (int, list)

(the distance, the whole DP

table)

EXAMPLES

Example 1	
Input	a = 'horse', b = 'ros'
Output	edit_distance('horse', 'ros')[0] -> 3
Why	horse -> rorse (replace h) -> rose (delete r) -> ros (delete e): 3 operations.

Example 2	
Input	a = 'intention', b = 'execution'
Output	edit_distance('intention', 'execution')[0] -> 5
Why	intention -> execution takes 5 operations.

QUESTION

Example 3	
Input	`a = '', b = 'abc'`
Output	`edit_distance('', 'abc')[0] -> 3`
Why	`From the empty word, insert every letter.`

WHY IT WORKS

dp[j] holds the distance from the current prefix of a to the first j characters of b. Each cell is the diagonal value if the characters match, otherwise 1 + the minimum of the diagonal (replace), above (delete) and left (insert). Keeping only the previous row reduces memory to one row.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): DP table with a rolling row
# Idea: dp[second_length] holds the distance from the current prefix
# of a to the first j characters of b. Each cell is the
# diagonal value if the characters match, otherwise 1 +
# the minimum of the diagonal (replace), above (delete)
# and left (insert). Keeping only the previous row reduces
# memory to one row.
def minDistance(first, second):
    if len(first) < len(second):
        first, second = second, first
    previous_row = list(range(len(second) + 1))
    for first_length, first_char in enumerate(first, 1):
        current_row = [first_length]
        for second_length, second_char in enumerate(second, 1):
            current_row.append(previous_row[second_length - 1] if first_char == second_char else 1 + min(previo
us_row[second_length - 1], previous_row[second_length], current_row[second_length - 1]))
        previous_row = current_row
    return previous_row[-1]
```

COMPLEXITY | Time O(m * n) | Space O(min(m, n))

GROUP 6 OF 43 | 9 QUESTIONS | SHORT MAIN CODE

Best-first search with a heap: Dijkstra and its minimax twin

Pattern

Keep a heap of (cost so far, node). Pop the cheapest, skip stale entries, and push any neighbour you improved. Dijkstra adds the edge weight to the cost; the minimax questions take the maximum of the cost and the edge instead. Everything else is the same loop.

46 Path Minimizing the Maximum Cell (Swim in Rising Water) — MEDIUM

Template move: the same minimax loop with a given start and end cell (Swim in Rising Water)

QUESTION

You get a grid of heights, a start cell and an end cell, and you may move to any of the four neighbouring cells. The cost of a path is the largest height on it (start and end included). Return the smallest cost over all paths from start to end.

Function signature

def min_max_path(g: list, start: tuple, end: tuple) -> int

EXAMPLES

Example 1	
Input	`[[0, 2], [1, 3]], (0, 0), (1, 1)`
Output	`3`
Why	`Both routes pass the cell of height 3 at the end, so the best cost is 3.`

Example 2	
Input	`[[0, 1, 2, 3, 4], [24, 23, 22, 21, 5], [12, 13, 14, 15, 16], [11, 17, 18, 19, 20], [10, 9, 8, 7, 6]], (0, 0), (4, 4)`
Output	`16`
Why	`A 5x5 board: the best route has to climb to 16 somewhere, and no route stays lower.`

WHY IT WORKS

Heap entries are (cost so far, cell). Popping a cell fixes its cost; each neighbour gets max(cost, neighbour value). Stop when the end cell is popped. Skip stale heap entries. The answer is the cost of the end cell.

SHORT OPTIMAL CODE

```
import heapq


# BASE QUESTION (optimal): Dijkstra on the running maximum
# Idea: Heap entries are (cost so far, cell). Popping a
# cell fixes its cost; each neighbour gets max(cost,
# neighbour value). Stop when the end cell is popped. Skip
# stale heap entries. The answer is the cost of the end
# cell.
def min_max_path(grid, start, end):
    rows, cols = len(grid), len(grid[0])
    best = {start: grid[start[0]][start[1]]}
    heap = [(best[start], start)]
    while heap:
        cost, (row, col) = heapq.heappop(heap)
        if cost > best[(row, col)]:
            continue
        if (row, col) == end:
            return cost
```

```
        for next_row, next_col in ((row + 1, col), (row - 1, col), (row, col + 1), (row, col - 1)):
            if 0 <= next_row < rows and 0 <= next_col < cols:
                new_cost = max(cost, grid[next_row][next_col])
                if new_cost < best.get((next_row, next_col), float("inf")):
                    best[(next_row, next_col)] = new_cost
                    heapq.heappush(heap, (new_cost, (next_row, next_col)))
    return None
```

COMPLEXITY | Time O(V log V) | Space O(V)

47 Nearest Favorite City by Travel Time — MEDIUM

Template move: combine = cost + w; stop when a favourite is popped (keep popping equal distances to break ties on the smaller index)

QUESTION

You are given a network of num_cities cities (0..num_cities-1) connected by weighted, undirected roads roads[i] = [u, v, w] where w > 0 is the travel time. Given a start city and a non-empty list favorites of favorite cities, return the favorite city reachable from start in the shortest total travel time. Break ties by smallest city index. If none of the favorite cities is reachable, return -1 .

Function signature

```
def nearest_favorite(num_cities: int, roads: list[list[int]],
                     start: int, favorites: list[int]) -> int:
    ...
```

EXAMPLES

Example 1	
Input	num_cities = 5, roads = [[0,1,2],[0,2,5],[1,3,1],[2,4,1]], start = 0, favorites = [3, 4]
Output	3
Why	Shortest times from 0: to 3 = 2+1 = 3, to 4 = 5+1 = 6. City 3 is nearer.

Example 2	
Input	num_cities = 3, roads = [[0,1,4]], start = 0, favorites = [2]
Output	-1
Why	City 2 is disconnected from start, so no favorite is reachable.

CONSTRAINTS

- 1 <= num_cities <= 10**5 , 0 <= len(roads) <= 2*10**5 , 1 <= w <= 10**9 .
- The interviewer required the optimal solution (Dijkstra with a priority queue).

WHY IT WORKS

Cities are finalised in order of distance, so the first favourite reached is the nearest.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Dijkstra from the start
# Idea: Cities are finalised in order of distance, so the
# first favourite reached is the nearest.
import heapq


def nearest_favorite(num_cities, roads, start, favorites):
    neighbours = [[] for _ in range(num_cities)]
    for city, next_city, minutes in roads:
        neighbours[city].append((next_city, minutes))
        neighbours[next_city].append((city, minutes))
```

```
    best_time = [float("inf")] * num_cities
    best_time[start] = 0
    heap, wanted = [(0, start)], set(favorites)
    nearest_time, nearest_city = None, -1
    while heap:
        time, city = heapq.heappop(heap)
        if time > best_time[city]:
            continue
        if nearest_time is not None and time > nearest_time:
            break
        if city in wanted and (nearest_time is None or city < nearest_city):
            nearest_time, nearest_city = time, city
        for next_city, minutes in neighbours[city]:
            if time + minutes < best_time[next_city]:
                best_time[next_city] = time + minutes
                heapq.heappush(heap, (best_time[next_city], next_city))
    return nearest_city
```

COMPLEXITY | Time O((V + E) log V) | Space O(V + E)

48 Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time.

MEDIUM

Template move: cost = arrival time; neighbors only offers trains that leave at or after the current time; combine = the train's arrival time

QUESTION

A schedule lists trains as (from, to, departure time, arrival time). Starting in city src at time start, find the sequence of trains that gets you to dst at the earliest possible time. You can only board a train that leaves at or after the time you arrive in its departure city. Return (arrival time, the trains you take), or None if dst cannot be reached.

Function signature

def earliest_arrival(trains: list, src, dst, start=0) -> tuple | None

EXAMPLES

Example 1	
Input	`[('A', 'B', 1, 3), ('B', 'C', 4, 6), ('A', 'C', 2, 9), ('B', 'C', 2, 5)], 'A', 'C'`
Output	`(6, [('A', 'B', 1, 3), ('B', 'C', 4, 6)])`
Why	`A -> B arrives at 3; the train B -> C that leaves at 2 is already gone, so take the one leaving at 4 and arrive at 6 (the direct A -> C train arrives later, at 9).`

Example 2	
Input	`[('A', 'B', 1, 3), ('B', 'C', 4, 6)], 'C', 'A'`
Output	`null`
Why	`No train leaves C, so A cannot be reached from C.`

WHY IT WORKS

best[city] is the earliest time you can be there. Pop the city with the smallest time, and for every train from it that departs at or after that time, improve best[to] to the train's arrival time and remember the train. Because waiting is free, a later arrival never beats an earlier one, so the first pop of the destination is optimal.

SHORT OPTIMAL CODE

```
from collections import defaultdict
import heapq
```

```python
# BASE QUESTION (optimal): Dijkstra on earliest arrival
# times, with the train sequence
# Idea: earliest_at[city] is the earliest time you can be there.
# Pop the city with the smallest time, and for every train
# from it that departs at or after that time, improve
# earliest_at[next_city] to the train's arrival time and remember the
# train. Because waiting is free, a later arrival never
# beats an earlier one, so the first pop of the
# destination is optimal.
def earliest_arrival(trains, source, destination, start=0):
    by_city = defaultdict(list)
    for train in trains:
        by_city[train[0]].append(train)
    earliest_at, came_by, heap = {source: start}, {}, [(start, source)]
    while heap:
        time_now, city = heapq.heappop(heap)
        if time_now > earliest_at[city]:
            continue
        if city == destination:
            route = []
            while city != source:
                train, previous_city = came_by[city]
                route.append(train)
                city = previous_city
            return time_now, route[::-1]
        for train in by_city[city]:
            origin, next_city, departure, arrival = train
            if departure >= time_now and arrival < earliest_at.get(next_city, float("inf")):
                earliest_at[next_city], came_by[next_city] = arrival, (train, city)
                heapq.heappush(heap, (arrival, next_city))
    return None
```

COMPLEXITY | Time O((T + C) log C) with trains sorted by departure per city | Space O(T + C)

49 Find the shortest path in a graph. MEDIUM

Template move: combine = cost + w; also store parent[v]; stop at the target and rebuild the path

QUESTION

A directed graph is given as a list of weighted edges (u, v, w) with non-negative weights. Return the total length and the nodes of a shortest path from s to t, or None if t cannot be reached.

Function signature

def dijkstra_path(edges: list, s, t) -> tuple | None

(distance, path)

EXAMPLES

Example 1	
Input	`[('A', 'B', 1), ('A', 'C', 4), ('B', 'C', 2), ('C', 'D', 1)], 'A', 'D'`
Output	`(4, ['A', 'B', 'C', 'D'])`
Why	`A -> B -> C -> D costs 1 + 2 + 1 = 4; the direct edge A -> C costs 4, which is longer overall.`

Example 2	
Input	`[('A', 'B', 1), ('B', 'C', 2), ('C', 'D', 1)], 'D', 'A'`
Output	`null`
Why	`There is no edge leading from D back to A.`

WHY IT WORKS

Keep the best known distance to each node and a min-heap of (distance, node). Pop the closest unsettled node, relax its edges, and remember the parent of every improvement. The first time the target is popped its distance is final; rebuild the path by following parents. It needs non-negative weights.

SHORT OPTIMAL CODE

```python
from collections import defaultdict
import heapq


# BASE QUESTION (optimal): Dijkstra with a heap (non-
# negative weights), returning the path
# Idea: Keep the best known distance to each node and a
# min-heap of (distance, node). Pop the closest unsettled
# node, relax its edges, and remember the parent of every
# improvement. The first time the target is popped its
# distance is final; rebuild the path by following
# parents. It needs non-negative weights.
def shortest_path(node_count, edges, source, target):
    graph = defaultdict(list)
    for node, neighbour, weight in edges:
        graph[node].append((neighbour, weight))
    shortest, parent, heap = {source: 0}, {source: None}, [(0, source)]
    while heap:
        distance_so_far, node = heapq.heappop(heap)
        if distance_so_far > shortest[node]:
            continue
        if node == target:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return distance_so_far, path[::-1]
        for neighbour, weight in graph[node]:
            if distance_so_far + weight < shortest.get(neighbour, float("inf")):
                shortest[neighbour], parent[neighbour] = distance_so_far + weight, node
                heapq.heappush(heap, (distance_so_far + weight, neighbour))
    return None
```

COMPLEXITY | Time O((V + E) log V) | Space O(V)

50 Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat. MEDIUM

Template move: maximise the minimum distance to the cat along the path: push negated values, combine = min(value, cat distance of the cell)

QUESTION

A grid contains a rat R, a piece of bread B, a cat C, walls

and open cells '.'. Move

the rat from R to B through open cells (up, down, left, right). The safety of a path is the smallest distance to the cat of any cell on it (distances count steps around the walls). Return the largest safety you can achieve, or -1 if the bread cannot be reached.

Function signature

def safest_path(grid: list) -> int

EXAMPLES

Example 1	
Input	`['R..', '...', 'C.B']`
Output	**2**
Why	**The path along the top and right side keeps at least 2 steps from the cat at the bottom left.**

QUESTION

Example 2	
Input	`['R#B']`
Output	`-1`
Why	The wall blocks the only route to the bread.

WHY IT WORKS

BFS from every cat gives the distance of each cell. Then run Dijkstra from the rat where the 'cost' of a path is the minimum cat distance along it and we maximise it: pop the cell with the best bottleneck so far, and extend to a neighbour with min(bottleneck, neighbour distance). The first time the bread is popped its bottleneck is optimal.

SHORT OPTIMAL CODE

```
import heapq
from collections import deque


def _cat_dist(grid):
    rows, cols = len(grid), len(grid[0])
    cat_distance = {(row, col): 0 for row in range(rows) for col in range(cols) if grid[row][col] == "C"}
    queue = deque(cat_distance)
    while queue:
        row, col = queue.popleft()
        for next_row, next_col in ((row + 1, col), (row - 1, col), (row, col + 1), (row, col - 1)):
            if 0 <= next_row < rows and 0 <= next_col < cols and grid[next_row][next_col] != "#" and (next_row,
 next_col) not in cat_distance:
                cat_distance[(next_row, next_col)] = cat_distance[(row, col)] + 1
                queue.append((next_row, next_col))
    return cat_distance


def _find(grid, symbol):
    return next((row, col) for row in range(len(grid)) for col in range(len(grid[0])) if grid[row][col] == symb
ol)


# BASE QUESTION (optimal): Multi-source BFS from the cat,
# then a max-heap bottleneck search
# Idea: BFS from every cat gives the distance of each
# cell. Then run Dijkstra from the rat where the 'cost' of
# a path is the minimum cat distance along it and we
# maximise it: pop the cell with the best bottleneck so
# far, and extend to a neighbour with min(bottleneck,
# neighbour distance). The first time the bread is popped
# its bottleneck is optimal.
def safest_path(grid):
    rows, cols = len(grid), len(grid[0])
    cat = _cat_dist(grid)
    start, goal = _find(grid, "R"), _find(grid, "B")
    best = {start: cat.get(start, 10**9)}
    heap = [(-best[start], start)]
    while heap:
        negative_bottleneck, position = heapq.heappop(heap)
        if -negative_bottleneck < best[position]:
            continue
        if position == goal:
            return -negative_bottleneck
        row, col = position
        for next_row, next_col in ((row + 1, col), (row - 1, col), (row, col + 1), (row, col - 1)):
            if 0 <= next_row < rows and 0 <= next_col < cols and grid[next_row][next_col] != "#":
                value = min(-negative_bottleneck, cat.get((next_row, next_col), 10**9))
                if value > best.get((next_row, next_col), -1):
                    best[(next_row, next_col)] = value
```

```
                heapq.heappush(heap, (-value, (next_row, next_col)))
    return -1
```

COMPLEXITY | Time O(m * n log(m * n)) | Space O(m * n)

51 Minimize maximum height along a grid path

HARD

Template move: the same minimax loop on a height grid

QUESTION

You are given a grid of heights and walk from the top-left cell to the bottom-right cell, moving up, down, left or right. The cost of a path is the largest height of any cell on it. Return the smallest possible cost (and, with with_path=True, a path that achieves it).

Function signature

def min_max_path(H: list, with_path=False)

EXAMPLES

Example 1	
Input	`[[1, 9], [2, 3]]`
Output	`3`
Why	`Down then right passes cells 1, 2, 3: the largest is 3; going right first would have to cross the 9.`

Example 2	
Input	`[[1, 2], [4, 3]], with_path = true`
Output	`(3, [(0, 0), (0, 1), (1, 1)])`
Why	`With with_path=True you also get the route.`

PART 1: Dijkstra with cost = maximum cell on the path

Heap of (largest cell so far, cell). Pop the cell with the smallest value, extend to each neighbour with cost max(current, neighbour), and keep only improvements. The first time the target is popped its cost is the answer.

PART 2: Alternative: binary search the limit with a flood fill

Sort the distinct values; the check 'can we walk from start to goal using only cells at or below this value' is monotone in the value, so binary search the smallest value for which it is true.

SHORT OPTIMAL CODE

```
import heapq


# BASE QUESTION (optimal): Dijkstra with cost = maximum
# cell on the path
# Idea: Heap of (largest cell so far, cell). Pop the cell
# with the smallest value, extend to each neighbour with
# cost max(current, neighbour), and keep only
# improvements. The first time the target is popped its
# cost is the answer.
def min_max_path(heights):
    rows, cols = len(heights), len(heights[0])
    best = {(0, 0): heights[0][0]}
    heap = [(heights[0][0], 0, 0)]
    while heap:
        cost, row, col = heapq.heappop(heap)
        if cost > best[(row, col)]:
            continue
        if (row, col) == (rows - 1, cols - 1):
```

```
            return cost
        for next_row, next_col in ((row + 1, col), (row - 1, col), (row, col + 1), (row, col - 1)):
            if 0 <= next_row < rows and 0 <= next_col < cols:
                new_cost = max(cost, heights[next_row][next_col])
                if new_cost < best.get((next_row, next_col), float("inf")):
                    best[(next_row, next_col)] = new_cost
                    heapq.heappush(heap, (new_cost, next_row, next_col))
```

COMPLEXITY | Part 1: Time O(m * n log(m * n)) | Space O(m * n)
Part 2: Time O(m * n log(m * n)) | Space O(m * n)

52 Find minimum threshold enabling grid path — HARD

Template move: the same minimax loop: the smallest threshold that lets you walk from corner to corner

QUESTION

You are given a grid of heights. Find the smallest threshold T such that you can walk from the top-left cell to the bottom-right cell, moving up, down, left or right, using only cells of height at most T. Return T together with one path that achieves it.

Function signature

def min_max_path(H: list) -> (int, list)

EXAMPLES

Example 1	
Input	`[[1, 3], [2, 2]]`
Output	`(2, [(0, 0), (1, 0), (1, 1)])`
Why	`Going down then right uses cells 1, 2, 2: the largest is 2, and no path stays below that.`

Example 2	
Input	`[[7]]`
Output	`(7, [(0, 0)])`
Why	`A single cell: the threshold is its own height.`

PART 1: Dijkstra with cost = maximum cell on the path

Heap of (largest cell so far, cell). Pop the cell with the smallest value, extend to each neighbour with cost max(current, neighbour), and keep only improvements. The first time the target is popped its cost is the answer.

PART 2: Alternative: binary search the limit with a flood fill

Sort the distinct values; the check 'can we walk from start to goal using only cells at or below this value' is monotone in the value, so binary search the smallest value for which it is true.

SHORT OPTIMAL CODE

```
import heapq


# BASE QUESTION (optimal): Dijkstra with cost = maximum
# cell on the path
# Idea: Heap of (largest cell so far, cell). Pop the cell
# with the smallest value, extend to each neighbour with
# cost max(current, neighbour), and keep only
# improvements. The first time the target is popped its
# cost is the answer.
def min_max_path(heights):
    rows, cols = len(heights), len(heights[0])
    best = {(0, 0): heights[0][0]}
```

```
    heap = [(heights[0][0], 0, 0)]
    while heap:
        cost, row, col = heapq.heappop(heap)
        if cost > best[(row, col)]:
            continue
        if (row, col) == (rows - 1, cols - 1):
            return cost
        for next_row, next_col in ((row + 1, col), (row - 1, col), (row, col + 1), (row, col - 1)):
            if 0 <= next_row < rows and 0 <= next_col < cols:
                new_cost = max(cost, heights[next_row][next_col])
                if new_cost < best.get((next_row, next_col), float("inf")):
                    best[(next_row, next_col)] = new_cost
                    heapq.heappush(heap, (new_cost, next_row, next_col))
```

COMPLEXITY | Part 1: Time O(m * n log(m * n)) | Space O(m * n)
Part 2: Time O(m * n log(m * n)) | Space O(m * n)

53 Minimize travel time with optional meeting point

HARD

Template move: three runs: from a, from b, and from the destination on the reversed graph; answer = min over meeting points m of max(da, db) + dd

QUESTION

A weighted graph has n nodes and edges (u, v, w) (undirected unless directed=True). Two people start at nodes a and b and both must end at dest. They may meet at some node m and walk the rest of the way together, so the total time is max(time a to m, time b to m) + time m to dest. Return [the minimum total time, the meeting node] (or [-1, -1] when dest cannot be reached). Meeting at dest itself, or at a or b, is allowed.

Function signature

def meeting_point(n: int, edges: list, a: int, b: int, dest: int, directed=False) -> list

EXAMPLES

Example 1	
Input	5, [(0, 1, 2), (3, 1, 1), (1, 4, 5)], 0, 3, 4
Output	[7, 1]
Why	They meet at node 1: a needs 2, b needs 1, then 5 more to node 4: 2 + 5 = 7. Meeting at node 4 itself also takes 7, and the smaller node number is reported.

Example 2	
Input	3, [(0, 1, 1)], 0, 1, 2
Output	[-1, -1]
Why	Node 2 is not connected to anything.

WHY IT WORKS

distA and distB come from forward searches. The distance from every node to the destination is a search from the destination on the reversed graph (the same graph if undirected). The best meeting node minimises max(distA, distB) + distToDest, ties to the smaller node id.

SHORT OPTIMAL CODE

```
import heapq


def _dijkstra_all(node_count, neighbours, start):
    distance = [float("inf")] * node_count
    distance[start] = 0
    heap = [(0, start)]
```

```
    while heap:
        distance_so_far, from_node = heapq.heappop(heap)
        if distance_so_far > distance[from_node]:
            continue
        for to_node, weight in neighbours[from_node]:
            if distance_so_far + weight < distance[to_node]:
                distance[to_node] = distance_so_far + weight
                heapq.heappush(heap, (distance_so_far + weight, to_node))
    return distance


# BASE QUESTION (optimal): Three Dijkstra runs (from A,
# from B, and into the destination)
# Idea: distA and distB come from forward searches. The
# distance from every node to the destination is a search
# from the destination on the reversed graph (the same
# graph if undirected). The best meeting node minimises
# max(distA, distB) + distToDest, ties to the smaller node
# id.
def meeting_point(node_count, edges, start_a, start_b, destination, directed=False):
    forward, reverse = [[] for _ in range(node_count)], [[] for _ in range(node_count)]
    for from_node, to_node, weight in edges:
        forward[from_node].append((to_node, weight))
        reverse[to_node].append((from_node, weight))
        if not directed:
            forward[to_node].append((from_node, weight))
            reverse[from_node].append((to_node, weight))
    distance_from_a, distance_from_b, distance_to_destination = _dijkstra_all(node_count, forward, start_a), _d
ijkstra_all(node_count, forward, start_b), _dijkstra_all(node_count, reverse, destination)
    best = (float("inf"), -1)
    for meeting_node in range(node_count):
        travel_time = max(distance_from_a[meeting_node], distance_from_b[meeting_node]) + distance_to_destinati
on[meeting_node]
        if travel_time < best[0]:
            best = (travel_time, meeting_node)
    return [-1, -1] if best[0] == float("inf") else [best[0], best[1]]
```

COMPLEXITY | Time O((V + E) log V) | Space O(V + E)

54 Compute minimax grid path and network delay — HARD

Template move: combine = max(cost, height of the next cell); the answer is the cost at the bottom-right cell

QUESTION

Two tasks. (1) minimax_path(H): given a grid of heights, find a path from the top-left to the bottom-right cell (moving up, down, left or right) whose largest height is as small as possible, and return that largest height. (2) network_delay(n, edges, source, node_delay=None): a signal starts at source and travels along directed weighted edges (u, v, w); return the time until every node has received it, or -1 if some node never does. node_delay[u], when given, is a pause at node u before it forwards the signal.

Function signature

def minimax_path(H: list) -> int

def network_delay(n: int, edges: list, source: int, node_delay=None) -> int

EXAMPLES

Example 1	
Input	`[[1, 5, 1], [2, 2, 5]]`
Output	`5`
Why	`Both ways around pass a cell of height 5, so the answer is 5.`

QUESTION

Example 2	
Input	4, [(0, 1, 2), (1, 2, 3), (0, 3, 10)], 0
Output	10
Why	The signal reaches node 1 at 2, node 2 at 5, node 3 at 10: everyone has it after 10.

Example 3	
Input	3, [(0, 1, 1)], 0
Output	-1
Why	Node 2 has no incoming edge: it is never reached.

Example 4	
Input	4, [(0, 1, 2), (1, 2, 3), (0, 3, 10)], 0, node_delay = [1, 0, 0, 0]
Output	11
Why	With a pause of 1 at node 0, every arrival is 1 later: node 3 is reached at 11.

PART 1: Part 1: Dijkstra where the cost of a path is its maximum cell

Heap of (best maximum so far, cell). Pop the cell with the smallest bottleneck, and for each neighbour the new bottleneck is max(current, neighbour). Improve if smaller. The first time the goal is popped, its bottleneck is optimal.

PART 2: Part 2: Dijkstra from the source, answer is the largest distance

Compute shortest distances with a heap. The signal reaches everyone at the time of the farthest node; if fewer than n nodes are reached the answer is -1.

SHORT OPTIMAL CODE

```
from collections import defaultdict
import heapq


# Part 1: Dijkstra where the cost of a path is its maximum
# cell
# Idea: Heap of (best maximum so far, cell). Pop the cell
# with the smallest bottleneck, and for each neighbour the
# new bottleneck is max(current, neighbour). Improve if
# smaller. The first time the goal is popped, its
# bottleneck is optimal.
def minimax_path(heights):
    rows, cols = len(heights), len(heights[0])
    best = {(0, 0): heights[0][0]}
    heap = [(heights[0][0], 0, 0)]
    while heap:
        cost, row, col = heapq.heappop(heap)
        if cost > best[(row, col)]:
            continue
        if (row, col) == (rows - 1, cols - 1):
            return cost
        for next_row, next_col in ((row + 1, col), (row - 1, col), (row, col + 1), (row, col - 1)):
            if 0 <= next_row < rows and 0 <= next_col < cols:
                new_cost = max(cost, heights[next_row][next_col])
                if new_cost < best.get((next_row, next_col), float("inf")):
                    best[(next_row, next_col)] = new_cost
                    heapq.heappush(heap, (new_cost, next_row, next_col))
    return -1


# Part 2: Dijkstra from the source, answer is the largest
# distance
```

```python
# Idea: Compute shortest distances with a heap. The signal
# reaches everyone at the time of the farthest node; if
# fewer than n nodes are reached the answer is -1.
def networkDelayTime(times, node_count, source):
    graph = defaultdict(list)
    for from_node, to_node, delay in times:
        graph[from_node].append((to_node, delay))
    arrival_time, heap = {}, [(0, source)]
    while heap:
        time, from_node = heapq.heappop(heap)
        if from_node in arrival_time:
            continue
        arrival_time[from_node] = time
        for to_node, delay in graph[from_node]:
            if to_node not in arrival_time:
                heapq.heappush(heap, (time + delay, to_node))
    return max(arrival_time.values()) if len(arrival_time) == node_count else -1
```

COMPLEXITY | Part 1: Time O(m * n log(m * n)) | Space O(m * n)
Part 2: Time O((V + E) log V) | Space O(V + E)

GROUP 7 OF 43 | 9 QUESTIONS | SHORT MAIN CODE

Backtracking: choose, recurse, undo

Pattern

Build the answer one choice at a time in a list. When the list is complete, record it. Otherwise loop over the allowed choices: append one, recurse, and then remove exactly what you added so the next choice starts from the same state. Pruning is just a check that skips a choice.

55 Mahjong Winning Hand Detection (14 Tiles) — MEDIUM

Template move: take the smallest tile; try it as a triple, then as the start of a run; undo the counts after each try

QUESTION

Mountain View onsite coding round 1: given a hand of 14 tiles, decide whether it is a winning mahjong hand. The candidate must clarify the rule set (typically: 4 melds of three identical tiles or three consecutive ranks + 1 pair).

Input: array of 14 integers representing tile values (clarify suit / range during the interview).
A winning hand consists of:

One pair (two identical tiles), and
Four melds, each being either:

A triplet (3 identical tiles), or
A run (3 consecutive ranks in the same suit).

Minority variant: A newer version instead specifies one pair plus two melds while retaining a 14-tile input. Because that accounts for only eight tiles, clarify the intended meld count and how the remaining six tiles should be handled before coding.

Output: boolean.

EXAMPLES

Example 1	
Input	`[1, 1, 1, 2, 3, 4, 11, 12, 13, 21, 21, 21, 31, 31]`
Output	`true`
Why	`Tiles are numbers: 1-9 one suit, 11-19 a second suit, 21-29 a third, 31 and up honours. Pair 31-31, triplets 1-1-1 and 21-21-21, and sequences 2-3-4 and 11-12-13.`

Example 2	
Input	`[1, 1, 1, 2, 3, 4, 11, 12, 13, 21, 21, 21, 31, 32]`
Output	`false`
Why	`31 and 32 are different honours, so there is no pair to finish the hand.`

WHY IT WORKS

Tiles are encoded as suit * 10 + rank for the numbered suits (ranks 1 to 9, suits 0 to 2) and 30 and above for honours, which never form runs. Count the tiles, remove each possible pair, then take the smallest tile left and try a triplet of it or a run starting at it, backtracking if neither works.

SHORT OPTIMAL CODE

```
from collections import Counter


# BASE QUESTION (optimal): Try each pair, then greedy-
# smallest with two options
# Idea: Tiles are encoded as suit * 10 + rank for the
```

```
# numbered suits (ranks 1 to 9, suits 0 to 2) and 30 and
# above for honours, which never form runs. Count the
# tiles, remove each possible pair, then take the smallest
# tile left and try a triplet of it or a run starting at
# it, backtracking if neither works.
def is_winning_hand(hand):
    def melds(tile_counts):
        smallest_tile = next((tile for tile in sorted(tile_counts) if tile_counts[tile] > 0), None)
        if smallest_tile is None:
            return True
        if tile_counts[smallest_tile] >= 3:
            tile_counts[smallest_tile] -= 3
            rest_works = melds(tile_counts)
            tile_counts[smallest_tile] += 3
            if rest_works:
                return True
        if smallest_tile < 30 and smallest_tile % 10 <= 7 and tile_counts.get(smallest_tile + 1, 0) > 0 and til
e_counts.get(smallest_tile + 2, 0) > 0:
            for run_tile in (smallest_tile, smallest_tile + 1, smallest_tile + 2):
                tile_counts[run_tile] -= 1
            rest_works = melds(tile_counts)
            for run_tile in (smallest_tile, smallest_tile + 1, smallest_tile + 2):
                tile_counts[run_tile] += 1
            if rest_works:
                return True
        return False
    tile_counts = Counter(hand)
    for pair_tile in list(tile_counts):
        if tile_counts[pair_tile] >= 2:
            tile_counts[pair_tile] -= 2
            rest_works = melds(tile_counts)
            tile_counts[pair_tile] += 2
            if rest_works:
                return True
    return False
```

COMPLEXITY | Time O(1) for 14 tiles (about 2^4 branches per pair) | Space O(distinct tiles)

56 Count Prefix-Matching Simple Paths in a Labeled Graph

MEDIUM

Template move: extend the path along edges whose label matches the next query letter, with a bitmask of visited nodes; count per depth

QUESTION

A directed graph has labelled vertices (labels[i] is the label of vertex i) and edges (u, v). A simple path never visits a vertex twice. For a query word, count for every prefix length L (1 up to the length of the word) the number of simple paths with L vertices whose labels spell the first L letters of the word. Return the list of counts.

Function signature

def prefix_paths(labels: list, edges: list, query: str) -> list

EXAMPLES

Example 1	
Input	`['a', 'b', 'c', 'a'], [(0, 1), (1, 2), (3, 1)], 'abc'`
Output	`[2, 2, 2]`
Why	`Two paths spell 'a' (vertices 0 and 3), and both continue to b (vertex 1) and then c (vertex 2).`

QUESTION

Example 2	
Input	`['a', 'a'], [(0, 1), (1, 0)], 'aaa'`
Output	`[2, 2, 0]`
Why	`Two paths spell 'a' and two spell 'aa' (0 -> 1 and 1 -> 0); none spells 'aaa' because that would reuse a vertex.`

WHY IT WORKS

Start from every vertex whose label equals the first query letter. Extend to an unvisited neighbour only if its label equals the next query letter, adding one to the count for the current length at each step. Branches that stop matching are cut at once.

SHORT OPTIMAL CODE

```
from collections import defaultdict


# BASE QUESTION (optimal): DFS with label pruning and a
# visited bitmask
# Idea: Start from every vertex whose label equals the
# first query letter. Extend to an unvisited neighbour
# only if its label equals the next query letter, adding
# one to the count for the current length at each step.
# Branches that stop matching are cut at once.
def prefix_paths(labels, edges, query):
    neighbours = defaultdict(list)
    for node, neighbour in edges:
        neighbours[node].append(neighbour)
    counts = [0] * len(query)

    def go(node, depth, visited_mask):
        counts[depth] += 1
        if depth + 1 < len(query):
            for next_node in neighbours[node]:
                if not visited_mask >> next_node & 1 and labels[next_node] == query[depth + 1]:
                    go(next_node, depth + 1, visited_mask | 1 << next_node)

    for start_vertex in range(len(labels)):
        if query and labels[start_vertex] == query[0]:
            go(start_vertex, 0, 1 << start_vertex)
    return counts
```

COMPLEXITY | Time O(number of matching paths * degree) | Space O(path)

57 Determine if a 14-tile hand is winning

MEDIUM

Template move: the same try-triple-then-run recursion over a count array, after choosing the pair

QUESTION

A hand is 14 tiles with values 1 to 9 of one suit (you may reorder them). It wins if it splits into one pair (two identical tiles) and four melds of three tiles, where a meld is either a triplet (three identical tiles) or a sequence (three consecutive values). Return whether the hand is winning.

Function signature

def is_winning_hand(tiles: list) -> bool

EXAMPLES

QUESTION

Example 1	
Input	`[1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9, 9]`
Output	`true`
Why	`Pair 9-9, triplets 1-1-1 and 5-5-5, sequences 2-3-4 and 6-7-8.`

Example 2	
Input	`[1, 2, 3, 4, 5, 6, 7, 8, 9, 1, 1, 2, 4, 6]`
Output	`false`
Why	`These tiles cannot be split into a pair and four melds.`

WHY IT WORKS

Keep counts of 1..9. For each value with at least two tiles, take it as the pair. Then repeatedly look at the smallest value still present: it must start a triplet (count at least 3) or a sequence (v, v + 1, v + 2 all present); try both. Because the smallest value has to be used by one of those melds, no grouping is missed and no order is repeated.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Counts, try each pair, then
# always group the smallest remaining tile
# Idea: Keep counts of 1..9. For each value with at least
# two tiles, take it as the pair. Then repeatedly look at
# the smallest value still present: it must start a
# triplet (count at least 3) or a sequence (rank, rank + 1, rank +
# 2 all present); try both. Because the smallest value has
# to be used by one of those melds, no grouping is missed
# and no order is repeated.
def is_winning_hand(tiles):
    count_of_rank = [0] * 12

    for tile in tiles:
        count_of_rank[tile] += 1

    def melds(rank):
        while rank <= 9 and count_of_rank[rank] == 0:
            rank += 1
        if rank > 9:
            return True
        if count_of_rank[rank] >= 3:
            count_of_rank[rank] -= 3
            rest_works = melds(rank)
            count_of_rank[rank] += 3
            if rest_works:
                return True
        if rank <= 7 and count_of_rank[rank + 1] and count_of_rank[rank + 2]:
            count_of_rank[rank] -= 1; count_of_rank[rank + 1] -= 1; count_of_rank[rank + 2] -= 1
            rest_works = melds(rank)
            count_of_rank[rank] += 1; count_of_rank[rank + 1] += 1; count_of_rank[rank + 2] += 1
            if rest_works:
                return True
        return False

    for pair in range(1, 10):
        if count_of_rank[pair] >= 2:
            count_of_rank[pair] -= 2
            rest_works = melds(1)
            count_of_rank[pair] += 2
            if rest_works:
                return True
    return False
```

COMPLEXITY | Time O(9 * small branching) = constant for 14 tiles | Space O(1)

58 American Football Score Combinations — MEDIUM

Template move: choices are the scoring plays from index i onward (the same play may repeat); complete when the remaining score is 0

QUESTION

Bryan's DP question was: enumerate all the ways a football team can accumulate a given total number of points. In American football a single scoring play is worth one of: 2 (safety), 3 (field goal), 6 (touchdown), 7 (touchdown + extra point), or 8 (touchdown + 2-point conversion). Given a target total , return every distinct combination of scoring plays whose values sum to total . Two combinations are the same if they use the same multiset of play values (order does not matter). Each combination should be returned as a list of play values sorted in non-decreasing order. The overall list of combinations may be in any order. A play value may be used any number of times.

Function signature

```
def football_combinations(total: int) -> list[list[int]]:
    """Return all distinct multisets of {2,3,6,7,8} that sum to total, each sorted
ascending."""
    ...
```

Follow-up (as reported): If you only need the count of combinations (not the lists), do it with an O(total) DP table (unbounded coin-change counting).

EXAMPLES

Example 1	
Input	`total = 6`
Output	`[[2, 2, 2], [3, 3], [6]]`
Why	`2+2+2 = 3+3 = 6 = 6.`

Example 2	
Input	`total = 7`
Output	`[[2, 2, 3], [7]]`
Why	`2+2+3 = 7 and the single play 7.`

Example 3	
Input	`total = 0`
Output	`[[]]`
Why	`The empty combination is the only way to score 0.`

CONSTRAINTS

- 0 <= total <= 60
- Allowed play values are exactly [2, 3, 6, 7, 8] .
- If total == 0 , the only combination is the empty one, so return [[]] .

WHY IT WORKS

Choosing plays in non-decreasing order lists every multiset exactly once.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Backtracking in increasing
# order
# Idea: Choosing plays in non-decreasing order lists every
# multiset exactly once.
def football_combinations(total):
    plays = (2, 3, 6, 7, 8)
    combinations = []
```

```
    def dfs(start, remaining, path):
        if remaining == 0:
            combinations.append(path[:])
            return
        for play_index in range(start, len(plays)):
            if plays[play_index] <= remaining:
                path.append(plays[play_index])
                # the same play may be used again, so i stays
                dfs(play_index, remaining - plays[play_index], path)
                path.pop()

    dfs(0, total, [])
    return combinations
```

COMPLEXITY | Time O(number of combinations * length) | Space O(total) recursion

59 Count Passcode Patterns on a Keypad

MEDIUM

Template move: the choice is the next adjacent unused key; complete at k keys; count instead of storing

QUESTION

A keypad is a rows x cols grid of keys (the classic phone dial is 3x3):
1 2 3 4 5 6 7 8 9 A passcode pattern is a path across keys where each move goes only to an orthogonally adjacent key (horizontal or vertical — no diagonals), and no key is visited more than once. The order matters: 1 -> 2 -> 3 -> 6 and 6 -> 3 -> 2 -> 1 are different patterns. Given the grid dimensions and an integer k , return the number of distinct passcode patterns that use exactly k keys.

Function signature

def count_patterns(rows: int, cols: int, k: int) -> int:
"""Return the number of distinct passcode patterns using exactly k keys,
moving only to orthogonally adjacent, unvisited keys on a rows x cols grid."""
...

Follow-up (as reported): Count patterns of length at least k instead of exactly k . Hint: sum the DFS counts over all lengths from k to rows*cols .

EXAMPLES

Example 1	
Input	`rows = 3, cols = 3, k = 1`
Output	`9`
Why	`Each of the 9 keys alone is one pattern.`

Example 2	
Input	`rows = 3, cols = 3, k = 2`
Output	`24`
Why	`There are 12 adjacent key pairs; each can be traversed in 2 directions, giving 24 patterns.`

Example 3	
Input	`rows = 3, cols = 3, k = 3`
Output	`44`
Why	`Summing over each possible middle key b, deg(b)*(deg(b)-1) directed 3-key paths: corners give 4*2=8, edge-mids 4*6=24, center 12; total 44.`

CONSTRAINTS

QUESTION

- 1 <= rows, cols <= 4 .
- 1 <= k <= rows * cols .
- A single key (k = 1) is a valid pattern of length 1.
- Count directed patterns (a path and its reverse are counted separately).

WHY IT WORKS

Extend the path one adjacent unused key at a time and undo the choice afterwards.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): DFS with backtracking from
# every key
# Idea: Extend the path one adjacent unused key at a time
# and undo the choice afterwards.
def count_patterns(rows, cols, pattern_length):
    def dfs(row, col, keys_used, seen):
        if keys_used == pattern_length:
            return 1
        total = 0
        for row_step, col_step in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            next_row, next_col = row + row_step, col + col_step
            if 0 <= next_row < rows and 0 <= next_col < cols and (next_row, next_col) not in seen:
                seen.add((next_row, next_col))
                total += dfs(next_row, next_col, keys_used + 1, seen)
                seen.remove((next_row, next_col))
        return total

    return sum(dfs(row, col, 1, {(row, col)})
               for row in range(rows) for col in range(cols))
```

COMPLEXITY | Time O(number of simple paths) | Space O(k)

60 Match people to questions using tags and priorities

MEDIUM

Template move: try each question of a person; if it is taken, recursively ask its owner to move (augmenting path); undo by not recording on failure

QUESTION

A set of people and a set of interview questions: adj[p] lists the questions person p is allowed to answer (their tags match). Each person is given at most one question and each question goes to at most one person. Return the largest number of people that can be matched.

Function signature

def max_matching(adj: list) -> int

EXAMPLES

Example 1	
Input	`[[0, 1], [0], [1, 2]]`
Output	`3`
Why	`Person 0 takes question 0, person 1 has only question 0 so person 0 moves to question 1, person 2 takes question 2: all three are matched.`

Example 2	
Input	`[[0], [0], [0]]`
Output	`1`
Why	`Three people can only answer the same single question: just one match.`

WHY IT WORKS

For each person run a DFS that tries each acceptable question: if it is free take it, otherwise try to move its current owner to another question (recursively), using a visited set so no question is tried twice in one attempt. Each successful DFS increases the matching by one. Hopcroft-Karp improves this to O(E * sqrt(V)).

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Augmenting paths (Kuhn's
# algorithm)
# Idea: For each person run a DFS that tries each
# acceptable question: if it is free take it, otherwise
# try to move its current owner to another question
# (recursively), using a visited set so no question is
# tried twice in one attempt. Each successful DFS
# increases the matching by one. Hopcroft-Karp improves
# this to O(E * sqrt(V)).
def max_matching(acceptable):
    owner_of = {}

    def try_assign(person, seen):
        for question in acceptable[person]:
            if question in seen:
                continue
            seen.add(question)
            if question not in owner_of or try_assign(owner_of[question], seen):
                owner_of[question] = person
                return True
        return False

    return sum(try_assign(person, set()) for person in range(len(acceptable)))
```

COMPLEXITY | Time O(P * E) | Space O(P + Q)

61 Generate Parentheses — MEDIUM

Template move: choices are '(' and ')'; allowed when opened < n, and ')' only when closed < opened; complete at length 2n

QUESTION

Generate all combinations of well-formed parentheses. A parenthesis combination is well-formed if each opening parenthesis "(" is closed by a matching closing parenthesis ")" in the correct order. For example, "()", "()()", and "((()))" are all combinations of well-formed parentheses, while ")(", "((" and "(()" are not. Examples n = 1 output: ["()"] n = 2 output: ["(())", "()()"] n = 3 output: ["((()))", "(()())", "(())()", "()(())", "()()()"] n = 4 output should contain: ["(((())))", "((()()))", "((())())", "((()))()", "(()(()))", "(()()())", "(()())()", "(())(())", "(())()()", "()((()))", "()(()())", "()(())()", "()()(())", "()()()()"]The problem is to generate all combinations of well-formed parentheses given n pairs of parentheses. A combination is well-formed if every opening parenthesis has a corresponding closing parenthesis and they are properly nested. Recursive Approach The most common and intuitive way to solve this problem is by using recursion with backtracking. Here's a step-by-step breakdown of the approach: Define the Problem: We need to build strings of length 2n using n opening (and n closing) parentheses. At any point in the string, the number of closing parentheses should not exceed the number of opening parentheses to ensure the sequence is valid. Recursive Function: We use a recursive helper function to build the strings character by character. The function tracks the remaining number of opening and closing parentheses that can still be added. It also

EXAMPLES

Example 1	
Input	`3`
Output	`['((()))', '(()())', '(())()', '()(())', '()()()']`
Why	`Five well-formed combinations of three pairs.`

QUESTION

Example 2	
Input	`1`
Output	`['()']`
Why	`One pair.`

WHY IT WORKS

Track how many '(' and ')' have been placed. Add '(' if open < n; add ')' if close < open. When the string reaches length 2n it is valid. Only valid prefixes are extended, so no work is wasted; the number of results is the Catalan number C(n) = (2n choose n) / (n + 1).

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Backtracking with open and
# close counters
# Idea: Track how many '(' and ')' have been placed. Add
# '(' if open < pair_count; add ')' if close < open. When the
# string reaches length 2n it is valid. Only valid
# prefixes are extended, so no work is wasted; the number
# of results is the Catalan number C(pair_count) = (2n choose pair_count) /
# (pair_count + 1).
def generateParenthesis(pair_count):
    results, current = [], []

    def go(opened, closed):
        if len(current) == 2 * pair_count:
            results.append("".join(current))
            return
        if opened < pair_count:
            current.append("(")
            go(opened + 1, closed)
            current.pop()
        if closed < opened:
            current.append(")")
            go(opened, closed + 1)
            current.pop()

    go(0, 0)
    return results
```

COMPLEXITY | Time O(4^n / sqrt(n)) | Space O(n) beyond the output

62 Permutations — MEDIUM

Template move: the template as written

QUESTION

Given an array of distinct integers, return all possible permutations of the array (every rearrangement of its elements), in any order.

Function signature

def permute(nums: list) -> list

EXAMPLES

Example 1	
Input	`[1, 2, 3]`
Output	`[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]`
Why	`Six orders of three numbers.`

QUESTION

Example 2	
Input	`[0, 1]`
Output	`[[0, 1], [1, 0]]`
Why	`Two numbers have two orders.`

Example 3	
Input	`[1]`
Output	`[[1]]`
Why	`One number has a single permutation.`

WHY IT WORKS

Fill positions left to right. At each position try every unused value, mark it, recurse, and unmark. Only valid prefixes are ever extended, so every step leads to a permutation.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Backtracking with a used array
# Idea: Fill positions left to right. At each position try
# every unused value, mark it, recurse, and unmark. Only
# valid prefixes are ever extended, so every step leads to
# a permutation.
def permute(nums):
    results, current, used = [], [], [False] * len(nums)

    def go():
        if len(current) == len(nums):
            results.append(current[:])
            return
        for index, value in enumerate(nums):
            if not used[index]:
                used[index] = True
                current.append(value)
                go()
                current.pop()
                used[index] = False

    go()
    return results
```

COMPLEXITY | Time O(n * n!) | Space O(n) beyond the output

63 Matrix Flower Placement with House Adjacency — HARD

Template move: place row by row; reject a row early when the houses two rows above are still not next to a flower

QUESTION

A grid has houses (H) and empty cells (0). Place flowers (F) on empty cells so that every row and every column holds at most one flower (exactly one of each when exactly_one is True) and every house has a flower directly beside it (up, down, left or right). Return one valid grid as a list of strings, or None if there is none.

Function signature

def place_flowers(grid: list, exactly_one=False) -> list | None

EXAMPLES

QUESTION

Example 1	
Input	['H000', 'H000', '000H', '0000']
Output	['HF00', 'H0F0', 'F00H', '000F']
Why	Flowers at (0,1), (2,0) and (1,3): one per row and column, and each house touches a flower.

Example 2	
Input	['H000', 'H000', '000H', '0000'], exactly_one = true
Output	['HF00', 'H0F0', 'F00H', '000F']
Why	Exactly one flower in every row and column: a fourth flower at (3,2) completes it.

Example 3	
Input	['H', 'H']
Output	null
Why	The two houses can only be served by the same column, so no valid placement exists.

WHY IT WORKS

Place at most one flower per row, tracking used columns, and only on empty cells ('0'). A house at row h has neighbours in rows h - 1, h and h + 1, so its requirement is final once row h + 1 is decided; failing there prunes the whole subtree. At the last row check all remaining houses. The 'exactly one' variant just forbids skipping a row and a column. Collecting every solution instead of the first one only changes whether the recursion stops at the first success.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Row-by-row backtracking with
# used-column set and delayed house checks
# Idea: Place at most one flower per row, tracking used
# columns, and only on empty cells ('0'). A house at row h
# has neighbours in rows h - 1, h and h + 1, so its
# requirement is final once row h + 1 is decided; failing
# there prunes the whole subtree. At the last row check
# all remaining houses. The 'exactly one' variant just
# forbids skipping a row and a column. Collecting every
# solution instead of the first one only changes whether
# the recursion stops at the first success.
def place_flowers(grid, exactly_one=False, find_all=False):
    rows, cols = len(grid), len(grid[0])
    cells = [list(row) for row in grid]
    houses = [(row, col) for row in range(rows) for col in range(cols) if cells[row][col] == "H"]
    solutions = []

    def house_ok(row, col):
        return any(0 <= row + row_step < rows and 0 <= col + col_step < cols and cells[row + row_step][col + co
l_step] == "F"
                   for row_step, col_step in ((1, 0), (-1, 0), (0, 1), (0, -1)))

    def check_row_houses(row):
        return all(house_ok(house_row, house_col) for house_row, house_col in houses if house_row == row)

    def solve(row, used_cols):
        if not find_all and solutions:
            return
        if row >= 2 and not check_row_houses(row - 2):          # rows up to row - 1 are decided, so row row -
2 houses are final
            return
        if row == rows:
            if check_row_houses(rows - 1) and (not exactly_one or len(used_cols) == cols):
```

```
                solutions.append(["".join(row) for row in cells])
            return
        for col in range(cols):
            if col not in used_cols and cells[row][col] == "0":
                cells[row][col] = "F"
                solve(row + 1, used_cols | {col})
                cells[row][col] = "0"
        if not exactly_one:
            solve(row + 1, used_cols)

    solve(0, frozenset())
    return solutions if find_all else (solutions[0] if solutions else None)
```

COMPLEXITY | Time exponential worst case, heavily pruned | Space O(rows + cols)

GROUP 8 OF 43 | 9 QUESTIONS | SHORT MAIN CODE

Runs and groups: extend while it fits, restart or close when it does not

Pattern

Walk the sequence once with a counter. If the next item fits the rule, the run grows; if not, the run restarts (or the group closes). Longest run, first break, number of groups, and sums over runs are all small changes to what you record.

64 First Out-of-Order Character — EASY

Template move: fits = s[i] >= s[i-1]; return i at the first item that does not fit

QUESTION

You are given a string of lowercase English letters. Scanning left to right, the string is considered "in order" as long as each character is greater than or equal to the character before it (a non-decreasing sequence; equal adjacent characters are allowed). Return the index of the FIRST character that breaks this rule — that is, the first index i >= 1 where s[i] < s[i-1] . If the entire string is non-decreasing, return -1 .

Function signature

```
def first_out_of_order(s: str) -> int:
    ...
```

Follow-up (as reported): Solve in a single O(N) pass with O(1) extra space; state the time/space complexity as the interviewer expected.

EXAMPLES

Example 1	
Input	`"abcda"`
Output	`4`
Why	`a<=b<=c<=d holds, but at index 4 'a' < 'd' (s[3]), so 4 is returned.`

Example 2	
Input	`"aabbz"`
Output	`-1`
Why	`Every character is >= the previous one; fully non-decreasing.`

Example 3	
Input	`"cba"`
Output	`1`
Why	`At index 1, 'b' < 'c', the first violation.`

Example 4	
Input	`""`
Output	`-1`
Why	`An empty string is trivially in order.`

CONSTRAINTS

- 0 <= len(s) <= 10^5 s consists only of lowercase English letters.
- An empty string or a single-character string is trivially in order -> return -1 .

WHY IT WORKS

Order is broken exactly where a letter is smaller than the one before it.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): One pass, compare with the
# previous letter
# Idea: Order is broken exactly where a letter is smaller
# than the one before it.
def first_out_of_order(text):
    for index in range(1, len(text)):
        if text[index] < text[index - 1]:
            return index
    return -1
```

COMPLEXITY | Time O(n) | Space O(1)

65 Group Timestamps Into Close Intervals

EASY

Template move: close the group when the gap to the next timestamp is at least k; otherwise extend it

QUESTION

You are given an array of positive integer timestamps in ascending order and an integer k . Partition the timestamps into maximal runs such that within each run, the difference between every pair of adjacent values is strictly less than k . A run breaks wherever an adjacent difference is >= k . Return each run as an interval [start, end] (its first and last value), in order.

Function signature

def group_into_intervals(timestamps: list[int], k: int) -> list[list[int]]:

...

Follow-up (as reported): One pass, O(n) time and O(1) extra space (besides the output).

EXAMPLES

Example 1	
Input	`timestamps = [1,2,3,4,11,14,15,17,21,22,23], k = 2`
Output	`[[1,4],[11,11],[14,15],[17,17],[21,23]]`
Why	`Adjacent diffs of 1 stay in a run; a diff of >=2 (e.g. 4->11, 11->14, 15- >17, 17->21) starts a new run.`

Example 2	
Input	`timestamps = [5], k = 100`
Output	`[[5,5]]`
Why	`A single timestamp forms a run by itself.`

Example 3	
Input	`timestamps = [1,2,3], k = 1`
Output	`[[1,1],[2,2],[3,3]]`
Why	`A diff of 1 is not strictly less than k=1, so every element stands alone.`

CONSTRAINTS

- 1 <= len(timestamps) <= 10^5 , values strictly increasing.
- k >= 1 .

WHY IT WORKS

Close the run whenever the gap to the previous value reaches k.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): One pass with a run start
# Idea: Close the run whenever the gap to the previous
# value reaches k.
def group_into_intervals(timestamps, gap_limit):
    if not timestamps:
        return []
    runs, start = [], timestamps[0]
    for previous, current in zip(timestamps, timestamps[1:]):
        if current - previous >= gap_limit:
            runs.append([start, previous])
            start = current
    runs.append([start, timestamps[-1]])
    return runs
```

COMPLEXITY | Time O(n) | Space O(1) extra

66 Longest Non-Decreasing Subarray with One Replacement — MEDIUM

Template move: left[i] and right[i] run lengths; the best answer joins the run before and after one changed element

QUESTION

Find the maximum length of a contiguous non-decreasing subarray if at most one element may be changed to any value. Some rounds start from the plain longest-increasing-subarray question and add the one-replacement follow-up; those versions ask for an increasing run, so clarify strict vs non-strict.

Input: an integer array nums.
You may change at most one element to any value.
Return the maximum length of a contiguous subarray that is non-decreasing (nums[i] <= nums[i + 1] for every adjacent pair inside it) after the change.

Variant: other rounds ask for a strictly increasing run and open with the no-replacement version (longest increasing subarray) before adding the one-replacement follow-up — clarify strict vs non-strict before coding.

Expect to discuss and compare the complexity of several approaches; a brute-force first pass is usually followed by a request to optimize.

EXAMPLES

```
nums = [1, 0, 3, 4, 5, 2, 3]
Output: 5
Change nums[1] from 0 to 1 -> [1, 1, 3, 4, 5, 2, 3]
[1, 1, 3, 4, 5] is a non-decreasing subarray of length 5.
```

WHY IT WORKS

left[i] and right[i] are the non-decreasing run lengths ending and starting at i. Changing a[i] gives left[i-1] + 1 + right[i+1] if a[i-1] <= a[i+1], otherwise max(left[i-1], right[i+1]) + 1. The ends need only one side. Because changing a value to itself is allowed, this also covers the no-change answer.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Left and right run lengths
# Idea: left[index] and right[index] are the non-decreasing run
# lengths ending and starting at i. Changing values[index] gives
# left[index-1] + 1 + right[index+1] if values[index-1] <= values[index+1],
# otherwise max(left[index-1], right[index+1]) + 1. The ends need
# only one side. Because changing a value to itself is
# allowed, this also covers the no-change answer.
def longest_with_one_change(values):
    length = len(values)
    if length <= 2:
        return length
    left, right = [1] * length, [1] * length
    for index in range(1, length):
```

```
        if values[index - 1] <= values[index]:
            left[index] = left[index - 1] + 1
    for index in range(length - 2, -1, -1):
        if values[index] <= values[index + 1]:
            right[index] = right[index + 1] + 1
    best = 1
    for index in range(length):
        run_before = left[index - 1] if index else 0
        run_after = right[index + 1] if index + 1 < length else 0
        best = max(best, run_before + 1 + run_after if (not run_before or not run_after or values[index - 1] <=
values[index + 1]) else max(run_before, run_after) + 1)
    return best
```

COMPLEXITY | Time O(n) | Space O(n)

67 Piano Hand Movement Grouping — MEDIUM

Template move: start a new group when adding the note would make max - min exceed k

QUESTION

A pianist plays a sequence of notes in order (a bigger number is a higher key). The hand can stay in one position while every note it plays lies within a span of k (highest minus lowest at most k). Starting a new hand position costs a move. Return the smallest number of hand positions needed to play all the notes in order.

Function signature

def groups(notes: list, k: int) -> int

EXAMPLES

Example 1	
Input	[1, 3, 5, 7, 9, 11], 4
Output	2
Why	[1, 3, 5] fits in a span of 4, then [7, 9, 11]: 2 positions.

Example 2	
Input	[1, 2, 3, 4, 5], 4
Output	1
Why	All five notes fit in a span of 4: one position.

Example 3	
Input	[5, 9, 1], 4
Output	2
Why	5 and 9 fit together, then 1 is too far from them: 2 positions.

WHY IT WORKS

Keep lo and hi of the current group. Extend while hi - lo stays within k, otherwise close the group and start a new one at this note. Cutting as late as possible cannot be worse because any later cut leaves a suffix that is a subset of what an earlier cut would have to cover.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Greedy scan, tracking the
# group's smallest and largest note
# Idea: Keep lo and hi of the current group. Extend while
# highest - lowest stays within k, otherwise close the group and
# start a new one at this note. Cutting as late as
# possible cannot be worse because any later cut leaves a
```

```python
# suffix that is a subset of what an earlier cut would
# have to cover.
def groups(notes, max_spread):
    if not notes:
        return 0
    count, lowest, highest = 1, notes[0], notes[0]
    for note in notes[1:]:
        new_lowest, new_highest = min(lowest, note), max(highest, note)
        if new_highest - new_lowest <= max_spread:
            lowest, highest = new_lowest, new_highest
        else:
            count += 1
            lowest = highest = note
    return count
```

COMPLEXITY | Time O(n) | Space O(1)

68 Find longest increasing contiguous subarray — MEDIUM

Template move: fits = a[i] > a[i-1]; also keep a left run and a right run per index so one change can join two runs

QUESTION

Given an array you may change at most one element to any value (an integer when integers=True, any real number otherwise). Return the length of the longest strictly increasing contiguous subarray you can obtain.

Function signature

def longest_one_change(a: list, integers: bool = True) -> int

EXAMPLES

Example 1	
Input	[1, 2, 3, 2, 5, 6]
Output	6
Why	Change the second 2 to 4 and the whole array 1, 2, 3, 4, 5, 6 increases.

Example 2	
Input	[1, 3, 2, 4]
Output	3
Why	Between 1 and 2 there is no integer, so the array 1, 3, 2, 4 only improves to a run of 3.

Example 3	
Input	[1, 3, 2, 4], integers = false
Output	4
Why	With real numbers, 3 can become 1.5 and the whole array increases.

WHY IT WORKS

left[i] is the length of the strictly increasing run ending at i and right[i] the length of the one starting at i. If you change a[i], you can attach the run before it and the run after it when a[i + 1] - a[i - 1] >= 2 (there is an integer strictly between), giving left[i - 1] + 1 + right[i + 1]. Otherwise you can only extend one side: left[i - 1] + 1 or right[i + 1] + 1. Also consider the unchanged runs.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Left and right run lengths, try
# every position as the changed element
# Idea: left[index] is the length of the strictly increasing
# run ending at i and right[index] the length of the one
```

```python
# starting at i. If you change values[index], you can attach the
# run before it and the run after it when values[index + 1] - values[index -
# 1] >= 2 (there is an integer strictly between), giving
# left[index - 1] + 1 + right[index + 1]. Otherwise you can only
# extend one side: left[index - 1] + 1 or right[index + 1] + 1.
# Also consider the unchanged runs.
def longest_one_change(values):
    length = len(values)
    if length == 0:
        return 0
    left, right = [1] * length, [1] * length
    for index in range(1, length):
        if values[index] > values[index - 1]:
            left[index] = left[index - 1] + 1
    for index in range(length - 2, -1, -1):
        if values[index] < values[index + 1]:
            right[index] = right[index + 1] + 1
    best = max(left)
    for index in range(length):
        if index > 0:
            best = max(best, left[index - 1] + 1)
        if index < length - 1:
            best = max(best, right[index + 1] + 1)
        if 0 < index < length - 1 and values[index + 1] - values[index - 1] >= 2:
            best = max(best, left[index - 1] + 1 + right[index + 1])
    return min(best, length)
```

COMPLEXITY | Time O(n) | Space O(n)

69 Sum of Good Subarrays (Adjacent Diff ±1) — MEDIUM

Template move: keep the running length and running sum of the current up-run and down-run; add both to the total at every index

QUESTION

MLE L5 coding round: sum over all contiguous subarrays whose elements form a strictly monotonic ±1 step sequence (every adjacent pair differs by exactly +1, OR every adjacent pair differs by exactly -1). Required time complexity is O(N).

Input: integer array arr of length N.
A subarray is good if its consecutive differences are either all +1 (monotone increasing by 1) or all -1 (monotone decreasing by 1).
Singleton subarrays are always good.
Output: the sum of element-sums across all good subarrays (each good subarray contributes the sum of its elements once).
Required: O(N) time.

EXAMPLES

```
arr = [3, 5, 6, 7, 6]
```

Good subarrays

```
  singletons:           [3], [5], [6], [7], [6]
  monotone +1 runs:     [5, 6], [6, 7], [5, 6, 7]
  monotone -1 runs:     [7, 6]
Not good: [3, 5] (diff 2), [6, 7, 6] (mixes +1 then -1).
```

WHY IT WORKS

For an increasing run of length L ending at i let S be the sum of all suffix sums of the run. When the next element x extends it, S becomes S + (L + 1) * x and L becomes L + 1; a break restarts at S = x, L = 1. Do the same for decreasing runs. Add S_up + S_down - x at each index, because the singleton [x] appears in both totals.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Two running run totals
# Idea: For an increasing run of length L ending at i let
# S be the sum of all suffix sums of the run. When the
# next element x extends it, S becomes S + (L + 1) * value and
# L becomes L + 1; a break restarts at S = value, L = 1. Do
# the same for decreasing runs. Add S_up + S_down - value at
# each index, because the singleton [value] appears in both
# totals.
def good_subarray_sum(values):
    total = 0
    up_total = up_length = down_total = down_length = 0
    for index, value in enumerate(values):
        step = value - values[index - 1] if index else None
        up_total, up_length = (up_total + (up_length + 1) * value, up_length + 1) if step == 1 else (value, 1)
        down_total, down_length = (down_total + (down_length + 1) * value, down_length + 1) if step == -1 else
(value, 1)
        total += up_total + down_total - value
    return total
```

COMPLEXITY | Time O(n) | Space O(1)

70 Sustained-Error Time Window

MEDIUM

Template move: from the given second extend left and right while errors >= baseline; compare the time span with the interval

QUESTION

Reconstructing the "two arrays - Error and Time" question. You are given two parallel arrays: times (timestamps in seconds, strictly increasing) and errors (the error reading recorded at each timestamp), where errors[i] was recorded at times[i] . You are also given three integers: second , interval , and baseline . Determine whether there exists a contiguous run of readings that:

1. Includes the reading taken at timestamp second (guaranteed to exist exactly once in times), 2. Has every reading in the run satisfy error >= baseline , and 3. Spans a duration of at least interval seconds — that is, times[right] - times[left] >= interval , where left and right are the endpoints of the run. Return True if such a run exists, otherwise False . Note that if the reading at second is itself below baseline , the answer is immediately False .

Function signature

```
def has_sustained_error_window(
    times: list[int],
    errors: list[int],
    second: int,
    interval: int,
    baseline: int,
) -> bool:
    """Return True if a run of readings all >= baseline includes second and spans >=
interval seconds."""
    ...
```

EXAMPLES

Example 1	
Input	`times = [1, 2, 3, 4, 5], errors = [10, 20, 5, 30, 40], second = 1, interval = 1, baseline = 8`
Output	`True`
Why	`Starting at time 1 (err 10 >= 8) the run extends to time 2 (err 20 >= 8) but stops at time 3 (err 5 < 8). The run covers times 1..2, a span of 1 >= interval 1.`

QUESTION

Example 2	
Input	`times = [1, 2, 3, 4, 5], errors = [10, 20, 5, 30, 40], second = 4, interval = 2, baseline = 8`
Output	`False`
Why	`Time 4 (err 30 >= 8) can only extend right to time 5 (err 40 >= 8); time 3 (err 5) blocks going left. The run covers times 4..5, span 1 < interval 2.`

Example 3	
Input	`times = [1, 2, 3], errors = [10, 4, 10], second = 2, interval = 0, baseline = 8`
Output	`False`
Why	`The reading at second=2 is 4 < baseline 8, so no valid run includes it.`

CONSTRAINTS

- 1 <= len(times) == len(errors) <= 10^5 times is strictly increasing; 0 <= times[i] <= 10^9
- -10^9 <= errors[i] <= 10^9 , baseline in the same range second is one of the values in times ; 0 <= interval <= 10^9

WHY IT WORKS

The longest valid run around the index is the best one, so only its span matters.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Expand around the reading
# Idea: The longest valid run around the index is the best
# one, so only its span matters.
def has_sustained_error_window(times, errors, second, interval, baseline):
    index = times.index(second)
    if errors[index] < baseline:
        return False
    left = right = index
    while left > 0 and errors[left - 1] >= baseline:
        left -= 1
    while right + 1 < len(times) and errors[right + 1] >= baseline:
        right += 1
    return times[right] - times[left] >= interval
```

COMPLEXITY | Time O(n) | Space O(1)

71 Longest Non-Decreasing Subarray From Two Arrays — MEDIUM

Template move: two run lengths per index, one for each array you may take the value from

QUESTION

You are given two integer arrays nums1 and nums2 , both of length n . You must construct a third array nums3 of length n where, for each index i , nums3[i] is chosen to be either nums1[i] or nums2[i] (chosen independently per index). Return the length of the longest non-decreasing subarray (a contiguous run) achievable in nums3 over all valid choices.

Function signature

def longest_non_decreasing_subarray(nums1: list[int], nums2: list[int]) -> int:

...

Follow-up (as reported): (Reported) The interviewer first asked for the longest contiguous non-decreasing subarray of a single array, then generalized to picking per index from two arrays; also asked for the time complexity and to write test cases.

EXAMPLES

QUESTION

Example 1	
Input	nums1 = [2, 3, 1], nums2 = [1, 2, 1]
Output	2
Why	Choosing nums3 = [2, 3, ...] gives the run [2, 3] of length 2. No choice yields a non-decreasing run of length 3.

Example 2	
Input	nums1 = [1, 3, 2, 1], nums2 = [2, 2, 3, 4]
Output	4
Why	nums3 = [1, 2, 3, 4] (pick nums1[0], nums2[1], nums2[2], nums2[3]) is non-decreasing over all 4 positions.

Example 3	
Input	nums1 = [1, 1], nums2 = [2, 2]
Output	2
Why	Any choice, e.g. [1, 1], is non-decreasing across both.

CONSTRAINTS

- 1 <= n <= 10^5
- nums1 and nums2 have the same length n .
- 1 <= nums1[i], nums2[i] <= 10^9
- Target complexity: O(n) time, O(1) extra space.

WHY IT WORKS

dp1 and dp2 are the best run ending at i using nums1[i] or nums2[i].

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Two-state DP
# Idea: dp1 and dp2 are the best run ending at i using
# nums1[index] or nums2[index].
def longest_non_decreasing_subarray(nums1, nums2):
    dp1 = dp2 = best = 1
    for index in range(1, len(nums1)):
        new_dp1 = new_dp2 = 1
        for previous_value, previous_length in ((nums1[index - 1], dp1), (nums2[index - 1], dp2)):
            if previous_value <= nums1[index]:
                new_dp1 = max(new_dp1, previous_length + 1)
            if previous_value <= nums2[index]:
                new_dp2 = max(new_dp2, previous_length + 1)
        dp1, dp2 = new_dp1, new_dp2
        best = max(best, dp1, dp2)
    return best
```

COMPLEXITY | Time O(n) | Space O(1)

72 Count Super Streaks

MEDIUM

Template move: sort by time; the same type within T continues the streak; when it breaks, count it if size >= N and span >= X

QUESTION

A player produces a list of events, each (event_id, event_type, event_time) . Sort events by event_time . A streak is a maximal run of events, consecutive in time order, in which every event has the same event_type AND the gap between each event and the previous one in the run is at most T seconds. A streak ends when the next event has a different event_type or its gap from the previous event exceeds T . A streak is a Super Streak if it contains at least N events AND spans at least X seconds (span = last event time minus first event time within the streak). Return the number of Super Streaks.

Function signature

```
def count_super_streaks(events: list[tuple[int, str, int]],
                        N: int, T: int, X: int) -> int:
    ...
```

Follow-up (as reported): Follow-up 1: events are tagged with user_id ; return a per-user count of Super Streaks. Follow-up 2: after processing, support count_streaks(user_id, time_A, time_B) returning how many of that user's Super Streaks completed entirely within [time_A, time_B] . Store each user's Super-Streak end-times sorted and answer with two binary searches in O(log n) per query.

EXAMPLES

Example 1	
Input	`events = [(1,"A",0),(2,"A",10),(3,"A",20),(4,"A",30),(5,"A",40)], N=4, T=10, X=30`
Output	`1`
Why	`One streak of 5 A-events, gaps all 10 (<=T), span 40 (>=30), size 5 (>=4). Super.`

Example 2	
Input	`events = [(1,"A",0),(2,"A",10),(3,"A",20),(4,"A",30),(5,"A",40), (6,"A",55),(7,"A",56),(8,"A",57)], N=4, T=10, X=30`
Output	`1`
Why	`The jump 40->55 (gap 15 > T) breaks the run. The first streak [0..40] is Super; the second [55,56,57] has only 3 events and span 2, so not Super.`

Example 3	
Input	`events = [(1,"A",0),(2,"B",5),(3,"A",10)], N=2, T=10, X=1`
Output	`0`
Why	`Type changes each step, so every streak has length 1 — none reaches N=2.`

CONSTRAINTS

- 0 <= len(events) <= 10^5 .
- N >= 1 , T >= 0 , X >= 0 ; times are non-negative integers, possibly with duplicate timestamps.

WHY IT WORKS

Keep only the open streak's start, last time, size and type.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): One pass over the sorted events
# Idea: Keep only the open streak's start, last time, size
# and type.
def count_super_streaks(events, min_events, max_gap, min_duration):
    count = 0
    start = last_time = streak_type = None
    size = 0
```

```
    for _, event_type, event_time in sorted(events, key=lambda event: event[2]):
        if streak_type == event_type and event_time - last_time <= max_gap:
            size += 1
        else:
            if size >= min_events and last_time - start >= min_duration:
                count += 1
            start, streak_type, size = event_time, event_type, 1
        last_time = event_time
    if size >= min_events and last_time - start >= min_duration:
        count += 1
    return count
```

COMPLEXITY | Time O(n log n) | Space O(1) extra

Union-find: merge the pairs, then ask who is together

Pattern

Keep a parent array. find climbs to the root (with path compression), union joins two roots. The only thing that changes between questions is where the pairs to merge come from and what you read at the end: same group, group size, number of groups, or the moment everything became one group.

73 Count Islands in a Grid and Track the Count as Land Cells Are Added — MEDIUM

Template move: each new land cell: groups += 1, then union it with every land neighbour (groups -= 1 per real merge); report groups after each addition

QUESTION

A map is a grid with **`m`** rows and **`n`** columns in which every cell is land (**`1`**) or water (**`0`**). An island is a maximal group of land cells connected horizontally or vertically. Cells that touch only at a corner are not connected. You are given the initial map and a sequence of additions, each of which turns one cell into land. Report the number of islands before any addition, and again after each addition.

Function Signature

```
def island_counts(grid: list[list[int]], additions: list[tuple[int, int]]) -> list[int]:
```

Rules

- **`grid[r][c]`** is **`1`** for land and **`0`** for water, where **`r`** is the row index and **`c`** is the column index.
- An addition **`(r, c)`** turns cell **`(r, c)`** into land. If that cell is already land, either initially or because of an earlier addition, the map does not change.
- Additions are applied in the given order. Land never turns back into water.
- Return a list of length **`len(additions) + 1`**. Element **`0`** is the number of islands in the initial grid. For **`i >= 1`**, element **`i`** is the number of islands after the first **`i`** additions have been applied.

Constraints

- **`1 <= m <= 1000`**, **`1 <= n <= 1000`** and **`m * n <= 200000`**
- Every row of **`grid`** has exactly **`n`** entries, each **`0`** or **`1`**.
- **`0 <= len(additions) <= 200000`**
- Every addition **`(r, c)`** satisfies **`0 <= r < m`** and **`0 <= c < n`**. The same cell may appear more than once.
- Every count is at most **`m * n`**, so all values fit in a 32-bit signed integer.

Examples

Example 1

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = []
Output: [3]
```

The islands are {(0, 0), (0, 1)}, {(1, 3), (2, 3)} and {(2, 0)}. With no additions, the only element is the initial count.

Example 2

```
Input: grid = [[0, 0, 0], [0, 0, 0], [0, 0, 0]], additions = [(0, 0), (0, 1), (1, 2), (2, 1), (1, 1), (1, 1)]
Output: [0, 1, 1, 2, 3, 1, 1]
```

The grid starts with no land. (0, 0) creates an island and (0, 1) joins it. (1, 2) and (2, 1) each touch no land, so each creates a new island. (1, 1) touches all three islands and merges them into one. The second (1, 1) is already land, so nothing changes.

Example 3

QUESTION

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = [(0, 2), (0, 3), (1, 0)]
Output: [3, 3, 2, 1]
```

(0, 2) joins the top-left island. It touches (1, 3) only at a corner, so the count stays 3. (0, 3) connects the top-left island to the island at (1, 3) and (2, 3), giving 2. (1, 0) connects the merged island to (2, 0), giving 1.

WHY IT WORKS

Build the initial islands with unions (count them), then for each addition: ignore it if the cell is already land; otherwise make it a new set (count + 1) and union it with each land neighbour, subtracting one from the count every time two different sets merge. Path compression and union by size make each step near-constant.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Union-Find updated cell by cell
# Idea: Build the initial islands with unions (count
# them), then for each addition: ignore it if the cell is
# already land; otherwise make it a new set (count + 1)
# and union it with each land neighbour, subtracting one
# from the count every time two different sets merge. Path
# compression and union by size make each step near-
# constant.
def island_counts(grid, additions):
    rows, cols = len(grid), len(grid[0])
    parent = {}

    def find(cell):
        while parent[cell] != cell:
            parent[cell] = parent[parent[cell]]
            cell = parent[cell]
        return cell

    count = 0

    def add(row, col):
        nonlocal count
        if (row, col) in parent:
            return
        parent[(row, col)] = (row, col)
        count += 1
        for next_row, next_col in ((row + 1, col), (row - 1, col), (row, col + 1), (row, col - 1)):
            if (next_row, next_col) in parent:
                root_a, root_b = find((row, col)), find((next_row, next_col))
                if root_a != root_b:
                    parent[root_a] = root_b
                    count -= 1

    for row in range(rows):
        for col in range(cols):
            if grid[row][col] == 1:
                add(row, col)
    counts = [count]
    for row, col in additions:
        add(row, col)
        counts.append(count)
    return counts
```

COMPLEXITY | Time O(m * n + A * alpha(m * n)) | Space O(m * n)

74 2D Points Connected Components by Radius

MEDIUM

Template move: pairs = points within distance r, found by bucketing points into cells of size r and checking only the 9 nearby cells; answer = groups

QUESTION

Given 2D points and radius **r**, connect two points when their distance is at most **r**; connectivity is transitive, and the output is the final number of clusters. The interviewer accepted or preferred a brute-force neighbor scan after discussing optimization ideas.

Input: a list of 2D points and a radius r.
Two points are connected if their Euclidean distance is <= r.
Connectivity is transitive: if A connects to B and B connects to C, all three belong to the same cluster.
Return the number of final clusters.

EXAMPLES

Example 1	
Input	`[(0, 0), (1, 0), (10, 10), (10, 11), (20, 20)], 2`
Output	`3`
Why	`Points within distance 2 of each other form a cluster: two near the origin, two near (10, 10), one alone.`

Example 2	
Input	`[(0, 0), (1, 0), (2, 0), (3, 0)], 1`
Output	`1`
Why	`A chain is one cluster even though its ends are far apart.`

WHY IT WORKS

Put each point in the cell (x // r, y // r). Two points within distance r are in the same or adjacent cells, so only those 9 cells are compared. Union the pairs whose squared distance is <= r^2 and count the roots. With r = 0 only identical points are linked, which the same code handles by using cell size 1.

SHORT OPTIMAL CODE

```python
from collections import defaultdict


# BASE QUESTION (optimal): Union-find with a spatial grid
# Idea: Put each point in the cell (point_x // radius, point_y // radius). Two
# points within distance r are in the same or adjacent
# cells, so only those 9 cells are compared. Union the
# pairs whose squared distance is <= radius^2 and count the
# roots. With radius = 0 only identical points are linked,
# which the same code handles by using cell size 1.
def count_components(points, radius):
    cell_size = radius if radius > 0 else 1
    parent = list(range(len(points)))
    def find(node):
        while parent[node] != node:
            parent[node] = parent[parent[node]]
            node = parent[node]
        return node
    cells = defaultdict(list)
    for point_index, (point_x, point_y) in enumerate(points):
        cells[(point_x // cell_size, point_y // cell_size)].append(point_index)
    for point_index, (point_x, point_y) in enumerate(points):
        cell_x, cell_y = point_x // cell_size, point_y // cell_size
        for cell_dx in (-1, 0, 1):
            for cell_dy in (-1, 0, 1):
                for other_index in cells.get((cell_x + cell_dx, cell_y + cell_dy), ()):
```

```
                    if other_index > point_index and (point_x - points[other_index][0]) ** 2 + (point_y - point
s[other_index][1]) ** 2 <= radius * radius:
                        parent[find(point_index)] = find(other_index)
    return len({find(point_index) for point_index in range(len(points))})
```

COMPLEXITY | Time about O(n) when points are spread out (O(n^2) if they all share a cell) | Space O(n)

75 Count Clusters of 2D Points Within a Radius — MEDIUM

Template move: identical to the line above: cell buckets give the candidate pairs, answer = groups

QUESTION

Given 2D points and a radius r, two points are connected when their distance is at most r. A cluster is a group of points connected directly or through chains of other points. Return the number of clusters.

Function signature

def count_clusters(points: list, r: float) -> int

EXAMPLES

Example 1	
Input	`[(0, 0), (1, 0), (10, 10), (10, 11), (20, 20)], 2`
Output	`3`
Why	`Three groups: the two points near the origin, the two near (10, 10), and the lone point at (20, 20).`

Example 2	
Input	`[(0, 0), (1, 0), (2, 0)], 1`
Output	`1`
Why	`A chain: (0,0)-(1,0)-(2,0) are linked neighbour to neighbour, so it is one cluster.`

WHY IT WORKS

Put each point in the cell (x // r, y // r). A point within distance r of another can only be in the same cell or one of the 8 neighbouring cells, so compare only with those. Union the pairs within r and count the distinct roots.

SHORT OPTIMAL CODE

```
from collections import defaultdict


def _find_factory(point_count):
    parent = list(range(point_count))

    def find(node):
        while parent[node] != node:
            parent[node] = parent[parent[node]]
            node = parent[node]
        return node

    return parent, find


# BASE QUESTION (optimal): Grid hashing with cells of side
# r, then Union-Find
# Idea: Put each point in the cell (point_x // radius, point_y // radius). A
# point within distance r of another can only be in the
# same cell or one of the 8 neighbouring cells, so compare
# only with those. Union the pairs within r and count the
# distinct roots.
```

```
def count_clusters(points, radius):
    cells = defaultdict(list)
    for point_index, (point_x, point_y) in enumerate(points):
        cells[(point_x // radius, point_y // radius)].append(point_index)
    parent, find = _find_factory(len(points))
    for (cell_x, cell_y), members in cells.items():
        for cell_dx in (-1, 0, 1):
            for cell_dy in (-1, 0, 1):
                for neighbour_index in cells.get((cell_x + cell_dx, cell_y + cell_dy), ()):
                    for member_index in members:
                        if member_index < neighbour_index and (points[member_index][0] - points[neighbour_index
][0]) ** 2 + (points[member_index][1] - points[neighbour_index][1]) ** 2 <= radius * radius:
                            parent[find(member_index)] = find(neighbour_index)
    return len({find(point_index) for point_index in range(len(points))})
```

COMPLEXITY | Time O(n) expected for evenly spread points | Space O(n)

76 Find A Threshold-Limited Path With Minimum Required Safety

MEDIUM

Template move: pairs = edges sorted from the highest safety down; union until find(s) == find(t); the answer is the safety of that last edge

QUESTION

A graph has n nodes and edges (u, v, safety). The safety of a path is the smallest safety among its edges. Using only edges whose safety is at least threshold (no limit when threshold is None), return the best (largest) safety you can reach going from s to t, or -1 if no such path exists. A path from a node to itself has infinite safety.

Function signature

def best_path_safety(n: int, edges: list, s: int, t: int, threshold=None) -> int

EXAMPLES

Example 1	
Input	3, [(0, 1, 5), (1, 2, 3), (0, 2, 1)], 0, 2
Output	3
Why	Direct edge 0-2 has safety 1; the route 0-1-2 has safety min(5, 3) = 3, which is better.

Example 2	
Input	3, [(0, 1, 5), (1, 2, 3), (0, 2, 1)], 0, 2, threshold = 4
Output	-1
Why	Only edges with safety at least 4 may be used: 0-1 is allowed but nothing leads to node 2.

Example 3	
Input	3, [(0, 1, 5)], 1, 1
Output	infinity
Why	Source and target are the same node.

WHY IT WORKS

Sort the edges by safety, strongest first, and union their endpoints one by one. The moment the source and the target become connected, the safety of the edge just added is the best possible minimum along a path. If they never connect return -1.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Union-Find adding edges from
# strongest to weakest
# Idea: Sort the edges by safety, strongest first, and
```

```python
# union their endpoints one by one. The moment the source
# and the target become connected, the safety of the edge
# just added is the best possible minimum along a path. If
# they never connect return -1.
def best_path_safety(node_count, edges, source, target):
    if source == target:
        return float("inf")
    parent = list(range(node_count))

    def find(node):
        while parent[node] != node:
            parent[node] = parent[parent[node]]
            node = parent[node]
        return node

    for from_node, to_node, safety in sorted(edges, key=lambda edge: -edge[2]):
        parent[find(from_node)] = find(to_node)
        if find(source) == find(target):
            return safety
    return -1
```

COMPLEXITY | Time O(E log E) | Space O(V)

77 Determine Reporting Relationships — MEDIUM

Template move: pairs = peer links; keep the boss at the root and carry it over when two groups merge

QUESTION

Build an in-memory organization model that supports three operations on employee IDs:

1. **`add_manager(a, b)`**: employee **a** is the direct manager of employee **b**.
2. **`add_peer(a, b)`**: employees **a** and **b** share the same direct manager.
3. **`is_manager(a, b)`**: return whether employee **a** is a manager of employee **b**. The answer should account for relationships that can be inferred from prior operations. For example, if **`add_manager(m, x)`** and **`add_peer(x, y)`** were previously recorded, then **`is_manager(m, y)`** should return **`true`**. You may assume the input is logically consistent and does not create invalid cycles in the org chart. Explain how you would design the data structures and implement these operations efficiently.

Implement an in-memory organization model that processes operations on employee IDs in order. Each operation is a tuple **`(op, a, b)`** where:

- **`("add_manager", a, b)`**: employee **a** becomes the direct manager of employee **b**.
- **`("add_peer", a, b)`**: employees **a** and **b** share the same direct manager.
- **`("is_manager", a, b)`**: ask whether employee **a** is a direct or indirect manager of employee **b**. Relationships can be inferred from earlier operations. For example, after **`add_manager(m, x)`** and **`add_peer(x, y)`**, **`is_manager(m, y)`** should be **`True`**. Process the operations sequentially and return a list of answers for all **`is_manager`** queries, in order. Notes:
- The input is logically consistent.
- No operation creates a cycle in the org chart.
- Repeating the same valid relationship is allowed.
- An employee is not considered their own manager.

Constraints
0 <= len(operations) <= 20000Employee IDs are integers in the range [-10^9, 10^9]At most 10000 distinct employee IDs appearThe sequence of operations is logically consistent and does not create cycles

EXAMPLES

```
Input: ([("add_manager", 1, 2), ("add_peer", 2, 3), ("is_manager", 1, 3)])
Expected Output: [True]
```

QUESTION

Explanation: Employee 2 reports to 1. Since 2 and 3 are peers, 3 also reports to 1.

```
Input: ([("add_manager", 2, 4), ("is_manager", 1, 4), ("add_manager", 1, 2), ("is_manager", 1, 4),
    ("is_manager", 2, 4), ("is_manager", 4, 2)])
Expected Output: [False, True, True, False]
```

Explanation: Before 2 gets a manager, 1 is not a manager of 4. After adding 1 as manager of 2, 1 becomes an indirect manager of 4. Hints**`add_peer(a, b)`** means **a** and **b** share one direct manager, but they are not interchangeable as managers. Group such employees with Union-Find.Once a peer group learns its direct manager, every current member of that group gets the same parent pointer. To answer **`is_manager(a, b)`**, walk upward from **b** through parent pointers.

WHY IT WORKS

Each employee starts in its own group. add_peer unions the two groups (and keeps the manager, if either has one). add_manager sets the manager of b's group to a. is_manager(a, b) repeatedly takes the manager of the current employee's group and moves up; it is true if a appears in that chain.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Union-Find for peer groups with
# a manager stored on each group
# Idea: Each employee starts in its own group. add_peer
# unions the two groups (and keeps the manager, if either
# has one). add_manager sets the manager of b's group to
# a. is_manager(a, b) repeatedly takes the manager of the
# current employee's group and moves up; it is true if a
# appears in that chain.
class Org:
    def __init__(self):
        self.parent, self.manager_of = {}, {}

    def _find(self, person):
        self.parent.setdefault(person, person)
        while self.parent[person] != person:
            self.parent[person] = self.parent[self.parent[person]]
            person = self.parent[person]
        return person

    def add_peer(self, peer_a, peer_b):
        root_a, root_b = self._find(peer_a), self._find(peer_b)
        if root_a != root_b:
            self.parent[root_b] = root_a
            if root_b in self.manager_of and root_a not in self.manager_of:
                self.manager_of[root_a] = self.manager_of[root_b]

    def add_manager(self, manager, employee):
        self._find(manager)
        self.manager_of[self._find(employee)] = manager

    def is_manager(self, manager, employee):
        seen, current = set(), employee
        while current not in seen:
            seen.add(current)
            root = self._find(current)
            if root not in self.manager_of:
                return False
            current = self.manager_of[root]
            if current == manager:
                return True
        return False
```

COMPLEXITY | Time O(alpha) per update, O(depth) per query | Space O(employees)

78 Determine whether two nodes are related

MEDIUM

Template move: pairs = the given edges; the query is find(a) == find(b)

QUESTION

Nodes 0 to n - 1 are linked by edges, and two nodes are related when a chain of edges joins them (links work in both directions; the graph may be disconnected and may contain cycles). Support union(a, b) to add a link and related(a, b) to answer the query.

Function signature

class Relations:
  def __init__(self, n: int, edges=())
  def union(self, a: int, b: int) -> None
  def related(self, a: int, b: int) -> bool

EXAMPLES

Example 1

Input	`r = Relations(5, [(0, 1), (1, 2)])`
	`r.related(0, 2)`
	`r.related(0, 3)`
	`r.union(3, 4)`
	`r.related(3, 4)`
	`r.related(2, 4)`

Output	`r.related(0, 2) -> true`
	`r.related(0, 3) -> false`
	`r.related(3, 4) -> true`
	`r.related(2, 4) -> false`

Why	`0, 1, 2 form one group and 3, 4 another until they are joined; the two groups are never linked to each other.`

WHY IT WORKS

Union the endpoints of every relationship; two nodes are related exactly when they have the same root. Path compression and union by size keep every operation near constant time, and the structure also handles cycles and disconnected components without any visited set.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Union-Find built once,
# constant-time queries
# Idea: Union the endpoints of every relationship; two
# nodes are related exactly when they have the same root.
# Path compression and union by size keep every operation
# near constant time, and the structure also handles
# cycles and disconnected components without any visited
# set.
class Relations:
    def __init__(self, person_count, edges):
        self.parent, self.size = list(range(person_count)), [1] * person_count
        for person_a, person_b in edges:
            self.union(person_a, person_b)

    def find(self, person):
        while self.parent[person] != person:
            self.parent[person] = self.parent[self.parent[person]]
            person = self.parent[person]
        return person

    def union(self, person_a, person_b):
        root_a, root_b = self.find(person_a), self.find(person_b)
        if root_a != root_b:
```

```
            if self.size[root_a] < self.size[root_b]:
                root_a, root_b = root_b, root_a
            self.parent[root_b] = root_a
            self.size[root_a] += self.size[root_b]

    def related(self, person_a, person_b):
        return self.find(person_a) == self.find(person_b)
```

COMPLEXITY | Time O(E alpha(V)) to build, O(alpha(V)) per query | Space O(V)

79 Movie Recommendations via Transitive Similarity — MEDIUM

Template move: pairs = similar movies (use a dict instead of a list); group = every movie with the same root as the target, sorted by rating

QUESTION

You are given a target movie name, a dictionary ratings mapping each movie to its numeric rating, an integer n , and a list similarities of [movieX, movieY] pairs meaning the two movies are directly similar. Similarity is transitive: if A is similar to B and B is similar to C, then A, B, and C all belong to the same similarity group. Return up to n recommended movies from the target's similarity group, excluding the target itself, ordered by rating descending (break ties alphabetically by name). If n == 1 , this returns the single highest- rated similar movie. If the group has fewer than n other movies, return all of them.

Function signature

def recommend(target: str, ratings: dict[str, float], n: int,
similarities: list[list[str]]) -> list[str]:
...

EXAMPLES

Example 1	
Input	`target = "A", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 1, similarities = [["A", "B"], ["B", "C"]]`
Output	`["C"]`
Why	`A, B, C form one group. Excluding A, the highest rated is C (9).`

Example 2	
Input	`target = "A", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 2, similarities = [["A", "B"], ["B", "C"]]`
Output	`["C", "B"]`
Why	`Similar to A are {B (5), C (9)}; sorted by rating descending -> [C, B].`

Example 3	
Input	`target = "D", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 3, similarities = [["A", "B"], ["B", "C"]]`
Output	`[]`
Why	`D is in no similarity pair, so its group is just {D}; excluding D leaves nothing.`

CONSTRAINTS

- Every movie mentioned appears as a key in ratings .
- 1 <= n ; ratings are numbers, names are distinct strings.

WHY IT WORKS

The target's component is its group; rank it by rating.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Union-find and a sort
# Idea: The target's component is its group; rank it by
# rating.
def recommend(target, ratings, how_many, similarities):
    parent = {}

    def find(movie):
        parent.setdefault(movie, movie)
        while parent[movie] != movie:
            parent[movie] = parent[parent[movie]]
            movie = parent[movie]
        return movie

    for movie_a, movie_b in similarities:
        parent[find(movie_a)] = find(movie_b)
    root = find(target)
    group = [movie for movie in list(parent) if movie != target and find(movie) == root]
    group.sort(key=lambda movie: (-ratings[movie], movie))
    return group[:how_many]
```

COMPLEXITY | Time O(P alpha + g log g) | Space O(P)

80 Friend Graph with Timeline (Friend / Unfriend Events) — HARD

Template move: pairs = friend events sorted by time; the answer is the time of the union that makes groups == 1

QUESTION

There are n people numbered 0 to n - 1. Each event is (time, a, b, kind). A 'friend' event makes a and b friends from that time on, and two people are connected when a chain of friends links them. Return the earliest time at which all n people are connected, or -1 if that never happens.

Function signature

def earliest_all_connected(n: int, events: list) -> int

EXAMPLES

Example 1	
Input	`3, [(1, 0, 1, 'friend'), (5, 1, 2, 'friend')]`
Output	`5`
Why	`After time 1, {0, 1} are connected; at time 5 person 2 joins, so everyone is connected at 5.`

Example 2	
Input	`4, [(1, 0, 1, 'friend'), (2, 1, 2, 'friend')]`
Output	`-1`
Why	`Person 3 is never befriended, so the group never becomes one.`

WHY IT WORKS

Sort by time and union the two people for each friend event. The first time the component count reaches 1 is the answer; -1 if it never does. Union by size with path compression keeps each operation near constant.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Base: earliest time everyone is
# connected (Union-Find on sorted events)
# Idea: Sort by time and union the two people for each
# friend event. The first time the component count reaches
# 1 is the answer; -1 if it never does. Union by size with
# path compression keeps each operation near constant.
def earliest_all_connected(people_count, events):
```

```
    parent = list(range(people_count))

    def find(person):
        while parent[person] != person:
            parent[person] = parent[parent[person]]
            person = parent[person]
        return person

    components = people_count
    if people_count == 1:
        return 0
    for time, person_a, person_b, kind in sorted(events, key=lambda event: event[0]):
        if kind != "friend":
            continue
        root_a, root_b = find(person_a), find(person_b)
        if root_a != root_b:
            parent[root_a] = root_b
            components -= 1
            if components == 1:
                return time
    return -1
```

COMPLEXITY | Time O(E log E) | Space O(N)

GROUP 10 OF 43 | 8 QUESTIONS | SHORT MAIN CODE

A dict of per-key state that you update, expire, or rank

Pattern

Keep a dict from a key (message, user, word, activity) to the little bit of state you need: the last time seen, a Counter, a set, a list of timestamps. Every event updates one key's state; the rule that decides what to accept, expire or report is the only part that changes.

81 Robot Status Message Deduplication Window — EASY

Template move: the template as written: show a message only if it was not shown in the last 10 seconds

QUESTION

Status events arrive from a robot as (time, message). Show a message only if the same message was not shown during the previous 10 seconds; a hidden message does not restart the timer, and exactly 10 seconds later it is shown again. show(t, msg) returns True when the message should be displayed.

Function signature

class Dedup:
    def show(self, t: int, msg: str) -> bool

EXAMPLES

Example 1

Input	`d = Dedup()`
	`d.show(0, 'low battery')`
	`d.show(4, 'low battery')`
	`d.show(6, 'door open')`
	`d.show(9, 'low battery')`
	`d.show(10, 'low battery')`

Output	`d.show(0, 'low battery') -> true`
	`d.show(4, 'low battery') -> false`
	`d.show(6, 'door open') -> true`
	`d.show(9, 'low battery') -> false`
	`d.show(10, 'low battery') -> true`

Why	The repeats at 4 and 9 are hidden and do not move the timer; at time 10 it is exactly 10 seconds after the last display, so it shows again.

WHY IT WORKS

If the message was displayed less than `window` seconds ago hide it without changing the stored time; otherwise display it and store the new time. The comparison t - last < window means exactly 10 seconds later is allowed; change to <= for the other reading.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Hash map of last displayed time
# Idea: If the message was displayed less than `window`
# seconds ago hide it without changing the stored time;
# otherwise display it and store the new time. The
# comparison time - last < window means exactly 10 seconds
# later is allowed; change to <= for the other reading.
class Dedup:
    def __init__(self, window=10):
        self.window, self.last_shown = window, {}

    def show(self, time, message):
        if message in self.last_shown and time - self.last_shown[message] < self.window:
            return False
```

```
        self.last_shown[message] = time
        return True
```

COMPLEXITY | Time O(1) per event | Space O(distinct messages)

82 Active Communication Top Users

MEDIUM

Template move: a set of seen pairs, a Counter of partners per user, and buckets by count to read the top users in O(1)

QUESTION

A stream of messages like 'A-B' means users A and B talked. Each distinct pair counts once (A-B and B-A are the same pair) and a pair of a user with themselves is ignored. A user's count is how many different partners they have. Support add(pair), most_active() (the set of users with the highest count) and top_k(k) (k users by count, ties broken by name).

Function signature

class Active:
  def add(self, pair: str) -> None
  def most_active(self) -> set
  def top_k(self, k: int) -> list

EXAMPLES

Example 1

Input	**a = Active()** **a.add('A-B')** **a.add('B-C')** **a.add('B-A')** **a.most_active()** **a.top_k(2)**
Output	**a.most_active() -> {'B'}** **a.top_k(2) -> ['B', 'A']**
Why	**B talked to A and C (2 partners). The repeated pair B-A adds nothing. A and C have 1 each, so the top 2 are B, then A (ties by name).**

WHY IT WORKS

A new edge increments the count of both users. Buckets map each count to the set of users with it, so a user moves from bucket c to bucket c + 1. Because counts never decrease, the highest non-empty bucket index only grows, which makes the most active users available in O(1). Top-K uses a heap over the count dict.

SHORT OPTIMAL CODE

```
from collections import Counter
from collections import defaultdict
import heapq


def _norm(pair):
    user_a, user_b = pair.split("-")
    return (user_a, user_b) if user_a <= user_b else (user_b, user_a)


# BASE QUESTION (optimal): Edge set, counts per user,
# count buckets for the maximum
# Idea: A new edge increments the count of both users.
# Buckets map each count to the set of users with it, so a
# user moves from bucket c to bucket old_count + 1. Because counts
# never decrease, the highest non-empty bucket index only
# grows, which makes the most active users available in
# O(1). Top-K uses a heap over the count dict.
```

```
class Active:
    def __init__(self, count_self=False):
        self.edges, self.connection_count, self.buckets, self.highest_count, self.count_self = set(), Counter()
, defaultdict(set), 0, count_self

    def _bump(self, user):
        old_count = self.connection_count[user]
        if old_count:
            self.buckets[old_count].discard(user)
        self.connection_count[user] = old_count + 1
        self.buckets[old_count + 1].add(user)
        self.highest_count = max(self.highest_count, old_count + 1)

    def add(self, pair):
        user_a, user_b = _norm(pair)
        if (user_a == user_b and not self.count_self) or (user_a, user_b) in self.edges:
            return
        self.edges.add((user_a, user_b))
        self._bump(user_a)
        if user_a != user_b:
            self._bump(user_b)

    def top_k(self, how_many):
        return heapq.nsmallest(how_many, self.connection_count, key=lambda user: (-self.connection_count[user],
 user))

    def most_active(self):
        return set(self.buckets[self.highest_count]) if self.highest_count else set()
```

COMPLEXITY | Time O(1) per pair, O(U log k) for top-K, O(1) for the most active user | Space O(E)

83 Filter Repeated Robot Status Messages

MEDIUM

Template move: the same rule applied to a stream of (timestamp, message) pairs; keep what is accepted

QUESTION

A robot status stream is a list of (timestamp, message). Show a message when it has never been shown, or when strictly more than gap seconds (default 10) have passed since it was last shown. Messages are compared exactly (case-sensitive), a hidden repeat does not restart the interval, and the shown messages keep their input order. Return the shown (timestamp, message) pairs.

Function signature

def filter_status(stream: list, gap: int = 10) -> list

EXAMPLES

Example 1	
Input	`[(0, 'ok'), (5, 'ok'), (10, 'ok'), (11, 'ok'), (11, 'OK')]`
Output	`[(0, 'ok'), (11, 'ok'), (11, 'OK')]`
Why	`'ok' at 5 and 10 are hidden (not more than 10 seconds after 0); at 11 it is 11 seconds later, so it shows; 'OK' is a different message.`

Example 2	
Input	`[(0, 'low'), (3, 'low'), (7, 'low')], gap = 2`
Output	`[(0, 'low'), (3, 'low'), (7, 'low')]`
Why	`A smaller gap lets repeats through sooner.`

WHY IT WORKS

For each message in order: if it was never shown or now - last > 10, emit it and set last = now; otherwise drop it and leave last unchanged. One dictionary lookup per message.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Dictionary from message to the
# time it was last shown
# Idea: For each message in order: if it was never shown
# or now - last_shown > 10, emit it and set last_shown = now;
# otherwise drop it and leave last unchanged. One
# dictionary lookup per message.
def filter_status(stream):
    last_shown, shown = {}, []
    for time, message in stream:
        if message not in last_shown or time - last_shown[message] > 10:
            last_shown[message] = time
            shown.append((time, message))
    return shown
```

COMPLEXITY | Time O(n) | Space O(distinct messages)

84 Next-Word Predictor — MEDIUM

Template move: defaultdict(Counter): counts[a][b] += 1 for neighbouring words; remember the best next word per word

QUESTION

Train on sentences, then predict the word that most often follows a given word. train(sentence) learns every pair of neighbouring words; predict(word) returns the word that followed it most often (ties go to the alphabetically smaller word) or None if nothing ever followed it. predict must run in O(1).

Function signature

class Predictor:
    def train(self, sentence: str) -> None
    def predict(self, word: str) -> str | None

EXAMPLES

Example 1

Input	`p = Predictor()`
	`p.train('I love apples')`
	`p.train('I love pears')`
	`p.train('you love pears too')`
	`p.predict('love')`
	`p.predict('I')`
	`p.predict('too')`

Output	`p.predict('love') -> 'pears'`
	`p.predict('I') -> 'love'`
	`p.predict('too') -> null`

Why	`After 'love' came 'apples' once and 'pears' twice, so 'pears' wins. Nothing ever followed 'too'.`

WHY IT WORKS

counts[word][next] is incremented for every adjacent pair. After the increment compare that one count with the current best for the word (higher count wins, then the smaller word). Because only one count changed, no other successor can overtake the best, so one comparison keeps it correct.

SHORT OPTIMAL CODE

```
from collections import Counter
from collections import defaultdict


# BASE QUESTION (optimal): Counts plus a maintained best
```

```python
# successor
# Idea: counts[word][next] is incremented for every
# adjacent pair. After the increment compare that one
# count with the current best for the word (higher count
# wins, then the smaller word). Because only one count
# changed, no other successor can overtake the best, so
# one comparison keeps it correct.
class Predictor:
    def __init__(self):
        self.counts = defaultdict(Counter)
        self.best = {}

    def train(self, sentence):
        words = sentence.split()
        for word, following_word in zip(words, words[1:]):
            self.counts[word][following_word] += 1
            current_best = self.best.get(word)
            if current_best is None or self.counts[word][following_word] > self.counts[word][current_best] or (
self.counts[word][following_word] == self.counts[word][current_best] and following_word < current_best):
                self.best[word] = following_word

    def predict(self, word):
        return self.best.get(word)
```

COMPLEXITY | Time O(1) per pair when training and per prediction | Space O(number of distinct pairs)

85 Batch and Streaming Log Deduplication

MEDIUM

Template move: dict message -> first (or latest) timestamp depending on the policy; sort the result at the end

QUESTION

Given log entries (timestamp, message), keep one entry per message and return the kept entries sorted by timestamp. With policy 'first' keep the entry seen first for each message; with policy 'latest' keep the one with the latest timestamp.

Function signature

def dedupe(logs: list, policy: str = 'first') -> list

EXAMPLES

Example 1	
Input	`[(5, 'a'), (1, 'b'), (9, 'a'), (3, 'c'), (2, 'b')]`
Output	`[(1, 'b'), (3, 'c'), (5, 'a')]`
Why	`Messages 'a' and 'b' each appear twice; the first one seen is kept.`

Example 2	
Input	`[(5, 'a'), (1, 'b'), (9, 'a'), (3, 'c'), (2, 'b')], policy = 'latest'`
Output	`[(2, 'b'), (3, 'c'), (9, 'a')]`
Why	`With 'latest', the later timestamp of each repeated message wins.`

PART 1: Dictionary on the message, then sort

One pass with a dict from message to the retained timestamp (first seen: keep the first; latest: replace when the new timestamp is larger), then sort the retained entries for the output.

PART 2: Streaming: map + min-heap with lazy deletion

current[msg] holds the retained timestamp. add pushes (timestamp, msg) on the heap only when it becomes the retained value (new message, or a larger timestamp under the 'latest' policy), leaving the older heap entry behind. get_next pops entries until one matches current[msg] (the others are stale), returns it and removes the message.

SHORT OPTIMAL CODE

```python
import heapq


# PART 1: Keep one batch entry per message, then sort by time.
def dedupe(logs, policy="first"):
    kept = {}
    for timestamp, message in logs:
        if message not in kept or (policy == "latest" and timestamp > kept[message]):
            kept[message] = timestamp
    return sorted((timestamp, message) for message, timestamp in kept.items())


# PART 2: Stream entries in time order with lazy heap deletion.
class LogStream:
    def __init__(self, policy="latest"):
        self.policy = policy
        self.current = {}
        self.heap = []

    def add(self, timestamp, message):
        if (message not in self.current
                or self.policy == "latest" and timestamp > self.current[message]):
            self.current[message] = timestamp
            heapq.heappush(self.heap, (timestamp, message))

    def get_next(self):
        while self.heap:
            timestamp, message = heapq.heappop(self.heap)
            if self.current.get(message) == timestamp:
                del self.current[message]
                return timestamp, message
        return None
```

COMPLEXITY | Part 1: Time O(n log n) | Space O(n)
Part 2: Time O(log n) amortized per add and get_next | Space O(n)

86 Design a logger that returns top-K messages

MEDIUM

Template move: dict message -> list of timestamps; top_k counts each list inside the time range with bisect, then takes the k largest

QUESTION

Design a logger. log(ts, msg) records a message at time ts (timestamps arrive in non-decreasing order). top_k(k, start, end) returns the k most frequent messages whose timestamps fall in [start, end]; leave start and end out to use the whole history. Ties are broken alphabetically.

Function signature

class Logger:
    def log(self, ts: int, msg: str) -> None
    def top_k(self, k: int, start=None, end=None) -> list

EXAMPLES

QUESTION

Example 1

Input	`lg = Logger()`
	`lg.log(1, 'disk full')`
	`lg.log(2, 'timeout')`
	`lg.log(3, 'timeout')`
	`lg.log(4, 'disk full')`
	`lg.log(5, 'timeout')`
	`lg.top_k(1)`
	`lg.top_k(2, 4, 5)`
	`lg.top_k(2, 1, 1)`

Output	`lg.top_k(1) -> ['timeout']`
	`lg.top_k(2, 4, 5) -> ['disk full', 'timeout']`
	`lg.top_k(2, 1, 1) -> ['disk full']`

Why	`Overall 'timeout' occurred 3 times. In the window [4, 5] 'disk full' and 'timeout' occur once each (tie, alphabetical). The window [1, 1] only holds 'disk full'.`

WHY IT WORKS

log appends the timestamp to the message's list (timestamps arrive in order, so lists stay sorted). A window count for one message is bisect_right(end) - bisect_left(start). top_k computes the counts for the d distinct messages and takes the K best with a heap, ordering by (-count, message).

SHORT OPTIMAL CODE

```python
from bisect import bisect_left
from bisect import bisect_right
from collections import defaultdict
import heapq


# BASE QUESTION (optimal): Per-message sorted timestamps
# plus a heap for the top K
# Idea: log appends the timestamp to the message's list
# (timestamps arrive in order, so lists stay sorted). A
# window count for one message is bisect_right(end) -
# bisect_left(start). top_k computes the counts for the d
# distinct messages and takes the K best with a heap,
# ordering by (-count, message).
class Logger:
    def __init__(self):
        self.times = defaultdict(list)

    def log(self, timestamp, message):
        self.times[message].append(timestamp)

    def top_k(self, how_many, start=None, end=None):
        counts = {}
        for message, message_times in self.times.items():
            first_in_window = 0 if start is None else bisect_left(message_times, start)
            after_window = len(message_times) if end is None else bisect_right(message_times, end)
            if after_window > first_in_window:
                counts[message] = after_window - first_in_window
        return heapq.nsmallest(how_many, counts, key=lambda message: (-counts[message], message))
```

COMPLEXITY | Time O(1) per log, O(d log n + d log k) per query | Space O(n)

87 Website Activity Analytics: Unique Users + Session Time

MEDIUM

Template move: dict activity -> set of users for unique counts; dict user -> sorted events to add up login-to-logout session time

QUESTION

Each record has a user_id, a timestamp ('YYYY-MM-DD HH:MM:SS') and an activity. Return (1) the number of distinct users per activity and (2) the average number of seconds a user spends on the site. A user's time is the sum of the gaps from each 'login' to the next 'logout'; a login without a logout counts until the user's last event.

Function signature

def activity_stats(records: list) -> (dict, float)

EXAMPLES

Example 1

Input	`rec = lambda u, t, a: {'user_id': u, 'timestamp': '2026-01-01 ' + t, 'activity': a}`
	`records = [rec(1, '10:00:00', 'login'), rec(1, '10:10:00', 'click'), rec(1, '10:30:00', 'logout'), rec(2, '10:05:00', 'login'), rec(2, '10:20:00', 'logout')]`
	`activity_stats(records)`

Output	`activity_stats(records) -> ({'click': 1, 'login': 2, 'logout': 2}, 1350)`
Why	`User 1 spent 30 minutes (1800 s), user 2 spent 15 minutes (900 s): the average is 1350 s. Both users logged in and out; only user 1 clicked.`

WHY IT WORKS

Parse each record once. unique[activity] collects user ids in a set. Each user's events are collected and sorted by time; a login opens a session and the next logout closes it, and an open session ends at the user's last event. The average is the total seconds divided by the number of users (state the denominator you chose).

SHORT OPTIMAL CODE

```
from collections import defaultdict


# BASE QUESTION (optimal): One pass to group, then
# sessions per user
# Idea: Parse each record once. unique[activity] collects
# user ids in a set. Each user's events are collected and
# sorted by time; a login opens a session and the next
# logout closes it, and an open session ends at the user's
# last event. The average is the total seconds divided by
# the number of users (state the denominator you chose).
def activity_stats(records):
    from datetime import datetime
    unique, events = defaultdict(set), defaultdict(list)
    for record in records:
        unique[record["activity"]].add(record["user_id"])
        events[record["user_id"]].append((datetime.strptime(record["timestamp"], "%Y-%m-%d %H:%M:%S"), record["
activity"]))
    total = 0
    for user_events in events.values():
        user_events.sort()
        start = None
        for event_time, activity in user_events:
            if activity == "login" and start is None:
                start = event_time
            elif activity == "logout" and start is not None:
                total += (event_time - start).total_seconds()
                start = None
        if start is not None:
            total += (user_events[-1][0] - start).total_seconds()
    return {activity: len(users) for activity, users in sorted(unique.items())}, (total / len(events) if events
 else 0.0)
```

COMPLEXITY | Time O(N log N) | Space O(N)

88 Hit Counter

MEDIUM

Template move: one deque of hit times; drop the ones older than 300 seconds; the answer is the length

QUESTION

Design a hit counter. hit(timestamp) records a hit at that second, and get_hits(timestamp) returns the number of hits in the past 300 seconds (timestamps never go backwards).

Function signature

class QueueCounter:
    def hit(self, ts: int) -> None
    def get_hits(self, ts: int) -> int

EXAMPLES

Example 1

Input	**h = QueueCounter()** **h.hit(1)** **h.hit(2)** **h.hit(3)** **h.get_hits(4)** **h.hit(300)** **h.get_hits(300)** **h.get_hits(301)**
Output	**h.get_hits(4) -> 3** **h.get_hits(300) -> 4** **h.get_hits(301) -> 3**
Why	**At time 301 the hit at time 1 is exactly 300 seconds old and no longer counts.**

CONSTRAINTS

- 1 <= timestamp <= 2 * 10^9
- Timestamps are monotonically increasing

WHY IT WORKS

Keep a deque of timestamps in arrival order. getHits pops from the front while the oldest is outside the window, and the answer is the deque length. Each timestamp is pushed and popped once.

SHORT OPTIMAL CODE

```python
from collections import deque


# BASE QUESTION (optimal): Queue of timestamps with
# eviction
# Idea: Keep a deque of timestamps in arrival order.
# getHits pops from the front while the oldest is outside
# the window, and the answer is the deque length. Each
# timestamp is pushed and popped once.
class HitCounter:
    def __init__(self):
        self.timestamps = deque()

    def hit(self, time):
        self.timestamps.append(time)

    def getHits(self, time):
        while self.timestamps and self.timestamps[0] <= time - 300:
            self.timestamps.popleft()
        return len(self.timestamps)
```

COMPLEXITY | Time O(1) amortized | Space O(hits in the window)

GROUP 11 OF 43 | 8 QUESTIONS | SHORT MAIN CODE

Binary search on a yes/no condition

Pattern

If answers go no, no, no, yes, yes, yes as the value grows, binary search finds the first yes. Write ok(x), set lo and hi, and halve. The questions only differ in what x is (a length, a speed, a time, an index, a version) and how ok(x) is checked.

89 First Index Reaching a Threshold (Binary Search) EASY

Template move: x = an index; ok(x) = arr[x] >= k on a sorted array

QUESTION

Given a non-decreasing (sorted ascending, duplicates allowed) array arr and a target value k , return the smallest index i such that arr[i] >= k — the first position where the array "reaches" the value k . If no element is >= k , return -1 . Solve in O(log n) time and O(1) extra space.

Function signature

def first_index_at_least(arr: list[int], k: int) -> int:

...

Follow-up (as reported): The interviewer probed space complexity; note the solution is already O(1) space. If arr is a huge sorted stream/file, describe reading via random access rather than loading it all.

EXAMPLES

Example 1	
Input	`arr = [1, 2, 2, 3, 5, 7], k = 2`
Output	`1`
Why	`arr[1] = 2 is the first element >= 2.`

Example 2	
Input	`arr = [1, 2, 2, 3, 5, 7], k = 4`
Output	`4`
Why	`The first element >= 4 is 5 at index 4.`

Example 3	
Input	`arr = [1, 2, 2, 3, 5, 7], k = 8`
Output	`-1`
Why	`No element is >= 8.`

CONSTRAINTS

- 0 <= len(arr) <= 10**7
- Values and k are integers.

WHY IT WORKS

Halve the range, keeping the boundary between values below k and values at least k.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Binary search for the lower
# bound
# Idea: Halve the range, keeping the boundary between
# values below k and values at least k.
def first_index_at_least(sorted_values, threshold):
    low, high = 0, len(sorted_values)
```

```
    while low < high:
        middle = (low + high) // 2
        if sorted_values[middle] >= threshold:
            high = middle
        else:
            low = middle + 1
    return low if low < len(sorted_values) else -1
```

COMPLEXITY | Time O(log n) | Space O(1)

90 Cutting Ribbons (LC 1891) MEDIUM

Template move: x = the piece length; ok(x) = the ribbons give at least k pieces; you want the LARGEST ok, so search the first x that fails and subtract 1

QUESTION

You have ribbons of given lengths and an integer k. Each ribbon may be cut into pieces of one common integer length L (leftover bits are thrown away). Return the largest L that gives at least k pieces in total, or 0 if even length 1 cannot give k pieces.

Function signature

def max_length(ribbons: list, k: int) -> int

EXAMPLES

Example 1	
Input	`[9, 7, 5], 3`
Output	`5`
Why	`Length 5 gives 1 + 1 + 1 = 3 pieces, length 6 only 2.`

Example 2	
Input	`[5, 7], 12`
Output	`1`
Why	`Length 1 already gives 12 pieces.`

Example 3	
Input	`[1, 2], 10`
Output	`0`
Why	`Total length is 3 but 10 pieces are needed: impossible, so 0.`

PART 1: Binary search the answer (integer lengths)

If you can cut k pieces of length L you can cut k pieces of any shorter length, so feasibility is monotone. Binary search the largest L in [1, max(ribbons)] with sum(r // L) >= k, and return 0 if even L = 1 fails.

PART 2: Floating-point lengths with a fixed iteration count

With real-valued lengths there is no 'next integer', so search the real line and stop after a fixed number of halvings (each halves the error) instead of looping on an epsilon comparison, which can fail to terminate near rounding noise. Feasibility uses floor(r / L) as before.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Binary search the answer
# (integer lengths)
# Idea: If you can cut k pieces of length L you can cut k
# pieces of any shorter length, so feasibility is
# monotone. Binary search the largest L in [1,
```

```
# max(ribbons)] with sum(r // L) >= pieces_needed, and return 0 if
# even L = 1 fails.
def maxLength(ribbons, pieces_needed):
    low_length, high_length, best_length = 1, max(ribbons), 0
    while low_length <= high_length:
        trial_length = (low_length + high_length) // 2
        if sum(ribbon // trial_length for ribbon in ribbons) >= pieces_needed:
            best_length, low_length = trial_length, trial_length + 1
        else:
            high_length = trial_length - 1
    return best_length
```

COMPLEXITY | Part 1: Time O(n log(max length)) | Space O(1)
Part 2: Time O(n * iterations) | Space O(1)

91 First Bad Version with Parallel Search Follow-up

MEDIUM

Template move: x = the version number; ok(x) = is_bad(x)

QUESTION

Versions are numbered 1 to n and from some version on every version is bad. The function is_bad(v) tells whether version v is bad. Find the first bad version with as few calls to is_bad as possible.

Function signature

def first_bad_version(n: int, is_bad) -> int

EXAMPLES

Example 1	
Input	bad_from = 4 first_bad_version(10, lambda v: v >= bad_from)
Output	first_bad_version(10, lambda v: v >= bad_from) -> 4
Why	Versions 4 to 10 are bad, so 4 is the first; binary search needs about log2(10) calls.

Example 2	
Input	bad_from = 1 first_bad_version(1, lambda v: v >= bad_from)
Output	first_bad_version(1, lambda v: v >= bad_from) -> 1
Why	With a single version that is bad, the answer is 1.

PART 1: Base problem: binary search

isBad is monotone (all good, then all bad), so keep the invariant 'the first bad version is in [lo, hi]' and halve the interval. Use lo + (hi - lo) // 2 in languages with fixed-width integers.

PART 2: Parallel: probe k points at once, then recurse into the bucket that flips

Each round fires up to `workers` probes concurrently at evenly spaced points. Because the predicate is monotone, exactly one gap between consecutive probes contains the flip, so the interval shrinks by a factor of workers + 1 per round. This trades more total calls for fewer sequential rounds, which is the right trade when each probe is slow (a network call).

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Base problem: binary search
# Idea: isBad is monotone (all good, then all bad), so
# keep the invariant 'the first bad version is in [low,
# high]' and halve the interval. Use low + (high - low) // 2 in
# languages with fixed-width integers.
def first_bad_version(version_count, is_bad):
    low, high = 1, version_count
    while low < high:
```

```
        middle = low + (high - low) // 2
        if is_bad(middle):
            high = middle
        else:
            low = middle + 1
    return low
```

COMPLEXITY | Part 1: Time O(log n) calls | Space O(1)
Part 2: Time O(log_{k+1} n) rounds | Space O(k)

92 Solve Banana Speed and Interval Merge — MEDIUM

Template move: x = the eating speed; ok(x) = total hours (ceil of pile / x summed) <= h; first_true over 1..max pile

QUESTION

During two technical phone screens for a new graduate software engineering role, the candidate was asked to solve two coding problems: minimum eating speed and merging overlapping intervals.

Constraints & Assumptions

- Provide clear algorithms, correctness reasoning, complexity, and edge cases.
- For minimum eating speed, each hour can be spent on only one pile.
- For interval merging, intervals overlap when the next start is less than or equal to the current merged end.
- Use sorted or binary-search approaches where appropriate.

Clarifying Questions

to Ask - Are **`piles`** and **h** always positive?

- Is **h** at least the number of piles?
- Are interval endpoints inclusive?
- Can intervals be empty or unsorted?

Part 1

- Minimum Eating Speed Given **`piles`** and **h**, return the minimum integer speed **k** such that all piles can be eaten within **h** hours.

What This Part Should Cover

- Binary search on answer from 1 to **`max(piles)`**.
- Feasibility function using sum of **`ceil(pile / k)`**.
- Why feasibility is monotonic.
- Complexity and edge cases.

Part 2

- Merge Overlapping Intervals Given intervals **`[start, end]`**, merge all overlapping intervals and return non-overlapping intervals sorted by start.

What This Part Should Cover

- Sort by start.
- Maintain current merged interval.
- Extend on overlap, otherwise append and start a new interval.
- Complexity and edge cases.

What a Strong

Answer Covers - Recognizes monotonic binary search for eating speed.

- Uses integer-safe ceiling division.
- Correctly defines interval overlap.
- Gives clean code and complexity for both problems.

QUESTION

Follow-up Questions

- How would you avoid floating-point division in the banana problem?
- What if **h** is smaller than the number of piles?
- How do you merge touching intervals like **`[1,3]`** and **`[3,5]`**?
- How would you handle streaming intervals?Minimum Eating Speed (Koko Eating Bananas)Koko loves to eat bananas. There are **n** piles of bananas; the i-th pile has **`piles[i]`** bananas. The guards will return in **h** hours. Koko can decide her constant bananas-per-hour eating speed **k**. Each hour she chooses one pile and eats **k** bananas from it. If that pile has fewer than **k** bananas, she eats all of them and does not eat more during that hour. An hour is spent on only one pile. Return the minimum integer speed **k** such that she can finish all the bananas within **h** hours. Example:
- **`piles = [3, 6, 7, 11]`**, **`h = 8`** -> **`4`**
- **`piles = [30, 11, 23, 4, 20]`**, **`h = 5`** -> **`30`**
- **`piles = [30, 11, 23, 4, 20]`**, **`h = 6`** -> **`23`** Constraints: **`1 <= piles.length <= h`**, **`1 <= piles[i] <= 10^9`**.

Constraints

1 <= piles.length <= h1 <= piles[i] <= 10^9h can be as large as 10^9

EXAMPLES

```
Input: ([3, 6, 7, 11], 8)
Expected Output: 4
Explanation: At k=4: ceil(3/4)+ceil(6/4)+ceil(7/4)+ceil(11/4) = 1+2+2+3 = 8 hours. k=3 needs 1+2+3+4 =
    10 > 8, so 4 is minimal.
Input: ([30, 11, 23, 4, 20], 5)
Expected Output: 30
```

Explanation: With exactly 5 piles and 5 hours, each pile must be eaten in one hour, so k must be at least max(piles) = 30.

Hints

If speed k finishes in time, any larger speed also finishes in time. This monotonicity lets you binary search on the answer.The answer lies in [1, max(piles)]: speed 1 is the slowest, and max(piles) finishes every pile in at most one hour.Hours needed at speed k is sum of ceil(pile / k). Use integer-safe ceiling: (pile + k - 1) // k to avoid floating point.Merge Overlapping IntervalsGiven an array of intervals where **`intervals[i] = [start_i, end_i]`**, merge all overlapping intervals and return an array of the non-overlapping intervals that cover all the intervals in the input, sorted by start. Two intervals overlap when the next start is less than or equal to the current merged end

PART 1: Bananas: binary search the speed

Hours needed are non-increasing in the speed, so keep lo = 1 and hi = max(piles) and halve: if speed mid finishes within h, try slower, otherwise faster. Use ceiling division (an hour finishes at most one pile).

PART 2: Intervals: sort and sweep

Sort by start. If an interval starts at or before the end of the last merged one, extend that end with max; otherwise start a new interval.

SHORT OPTIMAL CODE

```python
# Bananas: binary search the speed
# Idea: Hours needed are non-increasing in the speed, so
# keep low_speed = 1 and high_speed = max(piles) and halve: if speed mid
# finishes within h, try slower, otherwise faster. Use
# ceiling division (an hour finishes at most one pile).
def minEatingSpeed(piles, hours_allowed):
    low_speed, high_speed = 1, max(piles)
    while low_speed < high_speed:
        speed = (low_speed + high_speed) // 2
        if sum(-(-pile // speed) for pile in piles) <= hours_allowed:
```

```
            high_speed = speed
        else:
            low_speed = speed + 1
    return low_speed


# Intervals: sort and sweep
# Idea: Sort by start. If an interval starts at or before
# the end of the last merged one, extend that end with
# max; otherwise start a new interval.
def merge_intervals(intervals):
    merged = []
    for start, end in sorted(intervals):
        if merged and start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged
```

COMPLEXITY | Part 1: Time O(n log max) | Space O(1)
Part 2: Time O(n log n) | Space O(n)

93 Given a list with response time for each server, calculate how long time it will take to process n requests. MEDIUM

Template move: x = the elapsed time; ok(x) = sum(x // time_i) >= n requests

QUESTION

Servers work in parallel and server i needs times[i] time units to process one request (optionally server i only starts at starts[i]). Return the earliest time at which n requests have been processed in total.

Function signature

def time_for(times: list, n: int, starts=None) -> int

EXAMPLES

Example 1	
Input	[1, 2], 3
Output	2
Why	By time 2 the first server has done 2 requests and the second one: 3 in total.

Example 2	
Input	[3], 4
Output	12
Why	One server needs 3 units per request: 4 requests take 12.

Example 3	
Input	[2, 1], 6, starts = [0, 5]
Output	8
Why	The second server only starts at time 5, so the first one does most of the work.

WHY IT WORKS

In time T server i completes T // r_i requests, so the servers finish at least n requests exactly when sum(T // r_i) >= n, which is monotone in T. Search T between 0 and min(times) * n (one fastest server alone is enough).

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Binary search on the total time
# Idea: In time T server i completes T // r_i requests, so
# the servers finish at least n requests exactly when
# sum(T // r_i) >= request_count, which is monotone in T. Search T
# between 0 and min(response_times) * request_count (one fastest server alone
# is enough).
def time_for_requests(response_times, request_count):
    if request_count == 0:
        return 0
    low, high = 1, min(response_times) * request_count
    while low < high:
        time_limit = (low + high) // 2
        if sum(time_limit // response_time for response_time in response_times) >= request_count:
            high = time_limit
        else:
            low = time_limit + 1
    return low
```

COMPLEXITY | Time O(S log(min(times) * n)) | Space O(1)

94 Median of Two Sorted Arrays (LC 4) with a Follow-up Variant

HARD

Template move: x = how many items come from the shorter array's left side; ok(x) = the left part's largest <= the right part's smallest

QUESTION

Given two sorted arrays of numbers, return the median of all their elements together, without merging them into one array. The median is the middle element when the total count is odd, and the average of the two middle elements when it is even. Aim for O(log(min(m, n))) time.

Function signature

def median_two(a: list, b: list) -> float

EXAMPLES

Example 1	
Input	[1, 3], [2]
Output	2
Why	Combined [1, 2, 3]: the middle element is 2.

Example 2	
Input	[1, 2], [3, 4]
Output	2.5
Why	Combined [1, 2, 3, 4]: the two middle elements are 2 and 3, so the median is 2.5.

Example 3	
Input	[], [7, 9]
Output	8
Why	One array may be empty: the median is just the other array's median.

WHY IT WORKS

Make a the shorter array. Binary search i in a, set j = half - i in b so the left side holds (m + n + 1) // 2 elements. A cut is valid when a[i-1] <= b[j] and b[j-1] <= a[i]; use minus and plus infinity at the ends. Odd total: the answer is the larger left value. Even total: the average of the larger left value and the smaller right value.

SHORT OPTIMAL CODE

```python
INF = float("inf")
```

```python
# BASE QUESTION (optimal): Binary search on the partition
# Idea: Make a the shorter array. Binary search i in a,
# set second_cut = half - first_cut in b so the left side holds (first_length + second_length + 1)
# // 2 elements. A cut is valid when first[first_cut-1] <= second[second_cut] and
# second[second_cut-1] <= first[first_cut]; use minus and plus infinity at the ends.
# Odd total: the answer is the larger left value. Even
# total: the average of the larger left value and the
# smaller right value.
def median_two(first, second):
    if len(first) > len(second):
        first, second = second, first
    first_length, second_length = len(first), len(second)
    if first_length + second_length == 0:
        raise ValueError("no elements")
    low, high, half = 0, first_length, (first_length + second_length + 1) // 2
    while low <= high:
        first_cut = (low + high) // 2
        second_cut = half - first_cut
        a_left = first[first_cut - 1] if first_cut > 0 else -INF
        a_right = first[first_cut] if first_cut < first_length else INF
        b_left = second[second_cut - 1] if second_cut > 0 else -INF
        b_right = second[second_cut] if second_cut < second_length else INF
        if a_left <= b_right and b_left <= a_right:
            if (first_length + second_length) % 2:
                return max(a_left, b_left)
            return (max(a_left, b_left) + min(a_right, b_right)) / 2
        if a_left > b_right:
            high = first_cut - 1
        else:
            low = first_cut + 1
```

COMPLEXITY | Time O(log min(m, n)) | Space O(1)

95 Find longest duplicated substring — HARD

Template move: x = the substring length; ok(x) = some substring of that length appears twice (rolling hash); search the largest ok

QUESTION

You are given a string **s** consisting of lowercase English letters. A **substring** of **s** is any contiguous sequence of characters, i.e., **`s[i..j)`** for **`0 <= i < j <= len(s)`**. Design an algorithm to find **any one** of the longest substrings that appears **at least twice** in **s**. The two occurrences may overlap. If no substring appears at least twice, return the empty string.

Requirements

- Input: a string **s** of length **n** (e.g., **`1 <= n <= 2 × 10^5`**).
- Output: a string representing one of the longest duplicated substrings.
- Aim for an algorithm substantially better than the naive **`O(n^2)`** substring comparison approach.

Examples

- **`s = "banana"`**
- Possible duplicated substrings: "a", "an", "ana" - Longest length is 3 ("ana"), so acceptable outputs include **`"ana"`**.
- **`s = "abcd"`**
- No duplicated substring, so output should be **`""`** (empty string). Describe your algorithm, its time and space complexity, and how it scales for large **n**.

EXAMPLES

QUESTION

Example 1	
Input	'banana'
Output	'ana'
Why	'ana' occurs twice (overlapping is allowed).

Example 2	
Input	'abcd'
Output	''
Why	No substring repeats.

Example 3	
Input	'aaaaa'
Output	'aaaa'
Why	'aaaa' appears at positions 0 and 1.

WHY IT WORKS

For a fixed length L compute the polynomial hash of every window in O(1) each by sliding it, store hashes with their start positions, and when a hash repeats compare the actual substrings (to rule out a collision). Binary search the largest L for which a duplicate exists. A suffix array with the LCP array gives a deterministic O(n log n) alternative.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Binary search on the length
# with a rolling hash (Rabin-Karp)
# Idea: For a fixed length L compute the polynomial hash
# of every window in O(1) each by sliding it, store hashes
# with their start positions, and when a hash repeats
# compare the actual substrings (to rule out a collision).
# Binary search the largest L for which a duplicate
# exists. A suffix array with the LCP array gives a
# deterministic O(text_length log text_length) alternative.
def longestDupSubstring(text):
    text_length, base, modulus = len(text), 131, (1 << 61) - 1
    codes = [ord(char) for char in text]

    def find(window_length):
        window_hash, power = 0, pow(base, window_length, modulus)
        for window_start in range(window_length):
            window_hash = (window_hash * base + codes[window_start]) % modulus
        seen = {window_hash: [0]}
        for window_start in range(1, text_length - window_length + 1):
            window_hash = (window_hash * base - codes[window_start - 1] * power + codes[window_start + window_l
ength - 1]) % modulus
            for earlier_start in seen.get(window_hash, ()):
                if text[earlier_start:earlier_start + window_length] == text[window_start:window_start + window
_length]:
                    return text[window_start:window_start + window_length]
            seen.setdefault(window_hash, []).append(window_start)
        return None

    low_length, high_length, best = 1, text_length - 1, ""
    while low_length <= high_length:
        trial_length = (low_length + high_length) // 2
        duplicate = find(trial_length)
        if duplicate:
            best, low_length = duplicate, trial_length + 1
        else:
```

```
            high_length = trial_length - 1
    return best
```

COMPLEXITY | Time O(n log n) expected | Space O(n)

96 Find the median of two sorted arrays. HARD

Template move: the same partition search on the shorter array

QUESTION

Given two sorted arrays, return the median of all their elements together. The median is the middle value when the total count is odd and the average of the two middle values when it is even. The best solutions run in O(log(min(m, n))).

Function signature

def findMedianSortedArrays(nums1: list, nums2: list) -> float

EXAMPLES

Example 1	
Input	[1, 3], [2]
Output	2
Why	Combined: 1, 2, 3.

Example 2	
Input	[1, 2], [3, 4]
Output	2.5
Why	Combined: 1, 2, 3, 4: the middle two are 2 and 3.

Example 3	
Input	[], [5]
Output	5
Why	One empty array.

WHY IT WORKS

Binary search how many elements of the shorter array go in the left half. A partition is right when every left element is <= every right element, which only needs the four border values to be checked.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Binary search on the partition
# Idea: Binary search how many elements of the shorter
# array go in the left half. A partition is right when
# every left element is <= every right element, which only
# needs the four border values to be checked.
def findMedianSortedArrays(nums1, nums2):
    shorter, longer = (nums1, nums2) if len(nums1) <= len(nums2) else (nums2, nums1)
    shorter_length, longer_length = len(shorter), len(longer)
    half = (shorter_length + longer_length + 1) // 2
    low, high = 0, shorter_length
    while low <= high:
        shorter_cut = (low + high) // 2
        longer_cut = half - shorter_cut
        a_left = shorter[shorter_cut - 1] if shorter_cut > 0 else float("-inf")
        a_right = shorter[shorter_cut] if shorter_cut < shorter_length else float("inf")
        b_left = longer[longer_cut - 1] if longer_cut > 0 else float("-inf")
        b_right = longer[longer_cut] if longer_cut < longer_length else float("inf")
        if a_left <= b_right and b_left <= a_right:
```

```
            if (shorter_length + longer_length) % 2:
                return float(max(a_left, b_left))
            return (max(a_left, b_left) + min(a_right, b_right)) / 2
        if a_left > b_right:
            high = shorter_cut - 1
        else:
            low = shorter_cut + 1
    raise ValueError("inputs must be sorted")
```

COMPLEXITY | Time O(log(min(m, n))) | Space O(1)

GROUP 12 OF 43 | 8 QUESTIONS | SHORT MAIN CODE

Greedy: scan once, carry the best choice so far, act only when forced

Pattern

Decide locally and never look back: sort or scan, carry one value (how far you can reach, how far you are covered, the largest seen so far), and take an action only at the moment the current choice stops being enough.

97 Alarm for the Next Upcoming Birthday — EASY

Template move: minimum over people of the circular gap (their day - today) modulo 365, ties by name

QUESTION

You maintain a list of people and their birthdays. Given today's date and a list of (name, month, day) records, return the name of the person whose birthday comes soonest starting from today. Birthdays recur every year, so if no birthday remains later this calendar year, the soonest one is the earliest birthday next year. A birthday that falls exactly on today counts as 0 days away (soonest). Ignore leap years; treat every date simply as a (month, day) pair. If two people share the soonest birthday, return the one that is alphabetically first by name.

Function signature

```
def next_birthday(today: tuple[int, int], people: list[tuple[str, int, int]]) -> str:
    ...
```

EXAMPLES

Example 1

Input	`today = (10, 7), people = [("Alice", 10, 9), ("Bob", 1, 15), ("Carol", 12, 25)]`
Output	`"Alice"`
Why	`Alice's birthday is 2 days away; Carol's is later this year; Bob's is next January. Alice is soonest.`

Example 2

Input	`today = (12, 31), people = [("Alice", 1, 1), ("Dan", 12, 31)]`
Output	`"Dan"`
Why	`Dan's birthday is today (0 days away), which beats Alice's tomorrow.`

Example 3

Input	`today = (6, 1), people = [("Eve", 5, 30), ("Finn", 5, 30)]`
Output	`"Eve"`
Why	`Both birthdays already passed this year, so both are ~next-year and tie; alphabetical order picks "Eve".`

CONSTRAINTS

- 1 <= len(people) <= 10**4
- 1 <= month <= 12 , 1 <= day <= 31 (assume valid calendar dates). today is a valid (month, day) pair.

WHY IT WORKS

Days until the birthday is (doy - today) mod 365; take the smallest, ties by name.

SHORT OPTIMAL CODE

```
MONTH_DAYS = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]      # a non-leap year


# BASE QUESTION (optimal): Day of the year and a cyclic
```

```python
# minimum
# Idea: Days until the birthday is (doy - today) mod 365;
# take the smallest, ties by name.
def day_of_year(month, day):
    return sum(MONTH_DAYS[:month - 1]) + day


def next_birthday(today, people):
    today_number = day_of_year(*today)
    return min(people, key=lambda person: ((day_of_year(person[1], person[2]) - today_number) % 365, person[0])
)[0]
```

COMPLEXITY | Time O(p) | Space O(1)

98 Coins on the Table

EASY

Template move: scan the events with one counter: add on '+', subtract on '-' but never below 0

QUESTION

There is a table that starts with 0 coins on it. You are given a string events , where each character is a single event applied left to right: '+' places one coin on the table. '-' removes one coin from the table. The table can never hold a negative number of coins: a '-' event when the table is empty has no effect (it is ignored). After processing every event, return the number of coins on the table.

Function signature

def coins_on_table(events: str) -> int:

...

Follow-up (as reported): Generalize so each event is an integer delta (e.g. "+3" , "-2"), still clamping the running total at 0. Hint: parse tokens instead of single characters.

EXAMPLES

Example 1	
Input	events = "+++-"
Output	2
Why	+1, +1, +1, then -1 -> 2 coins remain.

Example 2	
Input	events = "----"
Output	0
Why	Every removal happens on an empty table and is ignored.

Example 3	
Input	events = "+-+-+"
Output	1
Why	1, 0, 1, 0, 1 -> 1 coin remains.

CONSTRAINTS

- 0 <= len(events) <= 10^5
- Every character of events is either '+' or '-' .
- An empty string returns 0 .

WHY IT WORKS

total = max(0, total + delta) for every event.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): One clamped counter
# Idea: total = max(0, total + delta) for every event.
def coins_on_table(events):
    total = 0
    for event in events:
        total = total + 1 if event == "+" else max(0, total - 1)
    return total
```

COMPLEXITY | Time O(n) | Space O(1)

99 Maximum Profit — EASY

Template move: pick the cheapest and the dearest city with a fixed tie-break (min and max once)

QUESTION

You and your friend have decided to start an international trading business. You want to earn a profit through some ol' fashioned arbitrage: You'll buy widgets in one country and sell them in another at a higher price. Your friend has already found the current market prices for widgets in cities around the world, and you'd like to use this data to pick which cities to buy and sell widgets in. Write a function max_profit(prices) that efficiently finds the two cities that maximize your profit (i.e. largest difference in prices).
Input: A dictionary with cities as keys and prices as values Output: An array containing the names of the cities (min, max) If the prices dictionary is empty, return None/null (depending on the programming language used). If there is 0 profit to be earned, return None/null as well. If you are using C++, return {"", ""} in such cases. Example Pythonprices = {'London': 72, 'New York': 70, 'Tokyo': 67, 'Miami': 62} max_profit(prices)

=> ['Miami', 'London']Hint 1You can think of this as two sub-problems

: Finding the lowest price, and finding the highest price.Hint 2While you could compare every pair of cities, it's more efficient to find the min and max values.Hint 3You can keep track of the lowest and highest values and update them as you iterate through the list.Challenge 1Does your solution work in O(N) time and O(1) space? If not, you can be more efficient!Challenge 2What happens if the price is the same in every city? Does your function still return

EXAMPLES

Example 1	
Input	{'A': 3, 'B': 10, 'C': 1}
Output	9
Why	Buy in the cheapest city (C, price 1) and sell in the dearest (B, price 10): profit 9.

Example 2	
Input	{'A': 5}
Output	0
Why	With a single city there is nothing to trade.

WHY IT WORKS

Find the city with the lowest price and the city with the highest price in one scan (ties by name). They are the best pair because profit is sell minus buy and the order of cities does not matter. Handle all-equal prices and fewer than two cities.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): One pass for the cheapest and
# the dearest city
# Idea: Find the city with the lowest price and the city
# with the highest price in one scan (ties by name). They
# are the best pair because profit is sell minus buy and
# the order of cities does not matter. Handle all-equal
```

```python
# prices and fewer than two cities.
def max_profit(prices):
    if len(prices) < 2:
        raise ValueError("need at least two cities")
    cheapest_city = min(prices, key=lambda city: (prices[city], city))
    dearest_city = min(prices, key=lambda city: (-prices[city], city))
    if cheapest_city == dearest_city:
        return sorted(prices)[:2]
    return [cheapest_city, dearest_city]
```

COMPLEXITY | Time O(n) | Space O(1)

100 Visible People in a Line (Last Person, Then Everyone) — MEDIUM

Template move: scan from the end carrying the tallest person seen between; count a person when no taller one blocks the view

QUESTION

Given heights of people in a line, A can see B if nobody between them is taller than both. Part 1: count how many people the last person can see. Part 2: return that count for every person. Written in a plain editor with no compilation.

Input: an array of heights representing a line of people.
Visibility rule: person A can see person B if no one standing between them is taller than both A and B.
Part 1: return how many people the last person in line can see.
Part 2: for the same array, return, for every person in the line, how many people they can see.

EXAMPLES

```
[1, 10, 6, 7, 9, 8, 2, 4, 3, 5] -> 6 (the last person, height 5, can see 3, 4, 2, 8, 9, 10).
```

PART 1: One pass with a running maximum (last person)

Walk from the person next to the last one towards the front, keeping the largest height seen strictly between the last person and the current one. The current person is visible when that maximum is at most the larger of the two heights. Then fold the current height into the maximum.

PART 2: Monotonic stack for everyone

Count what each person sees to the right, then run the same code on the reversed line for the left. To the right of i: everyone up to and including the first strictly taller person t is visible, and after t only the people at least as tall as everyone from t onward. next_ge[k] is the next person at least as tall as k, and chain[k] counts how far that chain runs, so the right count is (t - i) + chain[t], or the whole rest of the line when nobody is taller.

SHORT OPTIMAL CODE

```python
# PART 1: Count the people visible to the last person.
def last_sees(heights):
    if not heights:
        return 0
    last = heights[-1]
    tallest_between = float("-inf")
    visible = 0
    for index in range(len(heights) - 2, -1, -1):
        if tallest_between <= max(last, heights[index]):
            visible += 1
        tallest_between = max(tallest_between, heights[index])
    return visible


# PART 2: Monotonic stacks handle every person, including equal heights.
def visible_to_right(heights):
    people_count = len(heights)
    next_at_least_as_tall = [None] * people_count
    next_taller = [None] * people_count
    stack = []
```

```
    for index in range(people_count - 1, -1, -1):
        while stack and heights[stack[-1]] < heights[index]:
            stack.pop()
        next_at_least_as_tall[index] = stack[-1] if stack else None
        stack.append(index)
    stack = []
    for index in range(people_count - 1, -1, -1):
        while stack and heights[stack[-1]] <= heights[index]:
            stack.pop()
        next_taller[index] = stack[-1] if stack else None
        stack.append(index)
    chain = [0] * people_count
    for index in range(people_count - 1, -1, -1):
        if next_at_least_as_tall[index] is not None:
            chain[index] = 1 + chain[next_at_least_as_tall[index]]
    answer = []
    for index in range(people_count):
        taller = next_taller[index]
        answer.append(people_count - 1 - index if taller is None else taller - index + chain[taller])
    return answer
```

COMPLEXITY | Part 1: Time O(n) | Space O(1)
Part 2: Time O(n) | Space O(n)

101 Bounded Knapsack Variant

MEDIUM

Template move: sort items by value, highest first; take min(available count, remaining capacity) of each

QUESTION

Each item type has a value and an available count; every item weighs 1. Given capacity W, maximize total value. DP or DFS + memo works at O(n·W), greedy by value after sorting is O(n log n), and the best answer uses quickselect for O(n).

Input: for each item type, its value and the number of copies available. Every item has weight 1.
Input: a maximum total weight W.
Output: the maximum total value achievable without exceeding W.

EXAMPLES

Example 1	
Input	`[(10, 2), (6, 3), (3, 5)], 4`
Output	`32`
Why	`Items are (value, available count), each weighs 1. Take both 10s and two 6s: 20 + 12 = 32.`

Example 2	
Input	`[(10, 2), (6, 3), (3, 5)], 7`
Output	`44`
Why	`A bigger capacity takes more items.`

PART 1: Sort by value and take greedily

Sort the item types by value, highest first. For each type take min(copies, slots left) and add value times that. Stop when the bag is full. Greedy is correct because all items weigh 1: any optimal bag can swap an item for a more valuable one without breaking the limit.

PART 2: Quickselect on the value

Pick a random pivot value and split the types into higher, equal and lower. If the higher types alone fill the bag, recurse into them; otherwise take all of them, take as many equal ones as fit, and recurse into the lower ones with the slots that remain. Each round discards a constant fraction in expectation.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Sort by value and take greedily
# Idea: Sort the item types by value, highest first. For
# each type take min(copies, slots left) and add value
# times that. Stop when the bag is full. Greedy is correct
# because all items weigh 1: any optimal bag can swap an
# item for a more valuable one without breaking the limit.
def max_value_sorted(items, capacity):
    total = 0
    for value, copies in sorted(items, reverse=True):
        take = min(copies, capacity)
        total += value * take
        capacity -= take
        if capacity == 0:
            break
    return total
```

COMPLEXITY | Part 1: Time O(n log n) | Space O(1) extra
Part 2: Time O(n) expected | Space O(n)

102 Find the Largest Monotone Increasing Number

MEDIUM

Template move: scan the digits right to left; when a digit is smaller than the one before it, lower the earlier digit and fill 9s after it

QUESTION

Given an integer n, return the largest number not greater than n whose digits never decrease from left to right (for example 1223 qualifies, 1232 does not).

Function signature

def largest_monotone(n: int) -> int

EXAMPLES

Example 1	
Input	10
Output	9
Why	10 itself has a drop (1 then 0); the largest valid number below it is 9.

Example 2	
Input	1234
Output	1234
Why	1234 is already monotone.

Example 3	
Input	332
Output	299
Why	332: lower the first 3 to 2 and fill the rest with 9s: 299.

WHY IT WORKS

Walk i from the last digit to the first. When digit[i - 1] > digit[i], decrease digit[i - 1] by one and remember i as the start of the nines. After the scan set every digit from the marked position to the end to 9. The scan goes right to left so a decrement that breaks the pair on its left is repaired on the next step.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Right-to-left scan, then fill
# with nines
```

```python
# Idea: Walk i from the last digit to the first. When
# digit[position - 1] > digit[position], decrease digit[position - 1] by one
# and remember i as the start of the nines. After the scan
# set every digit from the marked position to the end to
# 9. The scan goes right to left so a decrement that
# breaks the pair on its left is repaired on the next
# step.
def monotoneIncreasingDigits(number):
    digits = list(str(number))
    nines_start = len(digits)
    for position in range(len(digits) - 1, 0, -1):
        if digits[position - 1] > digits[position]:
            digits[position - 1] = str(int(digits[position - 1]) - 1)
            nines_start = position
    for position in range(nines_start, len(digits)):
        digits[position] = "9"
    return int("".join(digits))
```

COMPLEXITY | Time O(d) | Space O(d)

103 Min boards to cover roof holes

MEDIUM

Template move: the template as written

QUESTION

Holes in a roof are at integer positions along a line (the same position may be listed twice). A board of the given length covers that many consecutive positions. Return the fewest boards that cover every hole.

Function signature

def min_boards(holes: list, length: int) -> int

EXAMPLES

Example 1	
Input	`[1, 2, 4, 7, 8], 3`
Output	`3`
Why	`Boards cover [1..3], [4..6] and [7..9].`

Example 2	
Input	`[1, 10], 5`
Output	`2`
Why	`The holes are far apart: one board each.`

Example 3	
Input	`[5, 5, 5], 1`
Output	`1`
Why	`Repeated positions need only one board.`

WHY IT WORKS

Sort the holes. Place a board starting at the first uncovered hole, which covers every hole below start + L; skip those and repeat. Each board is placed as far right as it can be while still covering the leftmost uncovered hole, so no better placement exists (exchange argument).

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Greedy: start a board at the
# leftmost uncovered hole
```

```python
# Idea: Sort the holes. Place a board starting at the
# first uncovered hole, which covers every hole below
# start + board_length; skip those and repeat. Each board is placed
# as far right as it can be while still covering the
# leftmost uncovered hole, so no better placement exists
# (exchange argument).
def min_boards(holes, board_length):
    boards, covered_until = 0, float("-inf")
    for hole in sorted(set(holes)):
        if hole > covered_until:
            boards += 1
            covered_until = hole + board_length - 1
    return boards
```

COMPLEXITY | Time O(n log n) | Space O(1) extra

104 Jump Game II (Minimum Jumps) — MEDIUM

Template move: scan the indices carrying the farthest reach; when you hit the end of the current jump, you must jump again: jumps += 1, end = farthest

QUESTION

You are given a 0-indexed array of non-negative integers nums . You start at index 0. From any index i , you can jump to any index in the range [i + 1, i + nums[i]] (i.e., nums[i] is the maximum forward jump length from i). Return the minimum number of jumps required to reach the last index len(nums)

- 1 . The test cases guarantee that the last index is always reachable.

Function signature

def jump(nums: list[int]) -> int:
    """Return the minimum number of jumps to reach the last index."""
    ...

Follow-up (as reported): Solve it in O(n) time and O(1) extra space with a greedy "current reach vs. farthest reach" scan rather than O(n) DP.

EXAMPLES

Example 1	
Input	`nums = [2, 3, 1, 1, 4]`
Output	`2`
Why	`Jump 1 step from index 0 to 1, then 3 steps to the last index.`

Example 2	
Input	`nums = [2, 3, 0, 1, 4]`
Output	`2`
Why	`0 -> 1 -> 4.`

Example 3	
Input	`nums = [0]`
Output	`0`
Why	`Already at the last index.`

CONSTRAINTS

- 1 <= len(nums) <= 10^4
- 0 <= nums[i] <= 1000
- If the array has length 1 you are already at the last index, so the answer is 0.

WHY IT WORKS

Each jump extends the covered range to the farthest index seen so far.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Greedy by ranges
# Idea: Each jump extends the covered range to the
# farthest index seen so far.
def jump(nums):
    jumps, current_end, farthest = 0, 0, 0
    for position in range(len(nums) - 1):
        farthest = max(farthest, position + nums[position])
        if position == current_end:
            # the range of the current jump is used up: take another
            jumps += 1
            current_end = farthest
    return jumps
```

COMPLEXITY | Time O(n) | Space O(1)

GROUP 13 OF 43 | 8 QUESTIONS | SHORT MAIN CODE

Sets and counters: remember the first collection, check the second against it

Pattern

Build a set (or a Counter) from one collection, then walk the other and ask 'is this in it?'. Membership in a set is O(1), which is the whole trick. Duplicates, intersections, filters, and missing values are the same loop with a different question.

105 Exclude First-K-Unique From Another List — EASY

Template move: the set holds the first k distinct values of list 1; keep list 2's values that are not in it

QUESTION

You are given two lists list1 and list2 and an integer k . Scan list1 left to right and collect the first k distinct values it contains (in order of first appearance; if list1 has fewer than k distinct values, take all of them). Then return list2 with every occurrence of any of those collected values removed, preserving the order of the remaining elements.

Function signature

```
def exclude_first_k_unique(list1: list[int], list2: list[int], k: int) -> list[int]:
    ...
```

EXAMPLES

Example 1	
Input	`list1 = [3, 3, 1, 4, 1, 5], list2 = [1, 2, 3, 4, 3, 5], k = 2`
Output	`[2, 4, 5]`
Why	`First 2 distinct values of list1 are {3, 1}. Removing all 3s and 1s from list2 leaves [2, 4, 5].`

Example 2	
Input	`list1 = [7], list2 = [7, 8, 9], k = 3`
Output	`[8, 9]`
Why	`list1 has only one distinct value (7); remove all 7s from list2.`

CONSTRAINTS

- 0 <= len(list1), len(list2) <= 10**5 , 1 <= k .
- Values are integers.

WHY IT WORKS

Membership in a set makes the filter linear.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): A set of the first k distinct
# values
# Idea: Membership in a set makes the filter linear.
def exclude_first_k_unique(list1, list2, distinct_count):
    blocked = set()
    for value in list1:
        if len(blocked) == distinct_count:
            break
        blocked.add(value)
    return [value for value in list2 if value not in blocked]
```

COMPLEXITY | Time O(n + m) | Space O(k)

106 Positional Matches Between Two Lists

EASY

Template move: no set needed: zip both lists and keep a where a == b

QUESTION

You are given two lists of equal length, first and second . Return, in order, the values at every position where the two lists agree — that is, all first[i] such that first[i] == second[i] .

Function signature

def positional_matches(first: list[int], second: list[int]) -> list[int]:

...

EXAMPLES

Example 1	
Input	`first = [3, 1, 4, 1, 5, 5, 7, 9], second = [1, 2, 3, 4, 5, 6, 7, 8]`
Output	`[5, 7]`
Why	`Positions 4 (5 == 5) and 6 (7 == 7) match. (The report described "matches" between a digits-of-pi list and an index list.)`

Example 2	
Input	`first = [1, 2, 3], second = [4, 5, 6]`
Output	`[]`
Why	`No position matches.`

CONSTRAINTS

- len(first) == len(second) , 0 <= length <= 10**6 .
- Values are integers.

WHY IT WORKS

Walk both lists together and keep the agreeing values.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): zip and a filter
# Idea: Walk both lists together and keep the agreeing
# values.
def positional_matches(first, second):
    return [first_value for first_value, second_value in zip(first, second) if first_value == second_value]
```

COMPLEXITY | Time O(n) | Space O(matches)

107 Contains Duplicate

EASY

Template move: add values to a set as you go; the first value already in the set means a duplicate

QUESTION

Given an integer array nums , return True if any value appears at least twice in the array, and False if every element is distinct.

Function signature

def contains_duplicate(nums: list[int]) -> bool:

"""Return True if any value appears more than once."""

...

EXAMPLES

QUESTION

Example 1	
Input	`nums = [1, 2, 3, 1]`
Output	`True`

Example 2	
Input	`nums = [1, 2, 3, 4]`
Output	`False`

Example 3	
Input	`nums = [1, 1, 1, 3, 3, 4, 3, 2, 4, 2]`
Output	`True`

CONSTRAINTS

- 1 <= len(nums) <= 10^5
- -10^9 <= nums[i] <= 10^9

WHY IT WORKS

Add each value to a set; seeing one again means a duplicate.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): A set of seen values
# Idea: Add each value to a set; seeing one again means a
# duplicate.
def contains_duplicate(nums):
    seen = set()
    for value in nums:
        if value in seen:
            return True
        seen.add(value)
    return False
```

COMPLEXITY | Time O(n) | Space O(n)

108 Find the Duplicates

EASY

Template move: values that appear in both lists, each reported once (set intersection, or two pointers on sorted lists)

QUESTION

Given two sorted arrays arr1 and arr2 of passport numbers, implement a function findDuplicates that returns an array of all passport numbers that are both arrays. Note that the output array should be sorted in ascending order. Let N and M be the lengths of arr1 and arr2, respectively. Solve for two cases and analyze the time & space complexities of your solutions: M ~ N - the array lengths are approximately the same M >> N - arr2 is much bigger than arr1. Example input: arr1 = [1, 2, 3, 5, 6, 7], arr2 = [3, 6, 7, 8, 20] output: [3, 6, 7]

since only these three values

are both in arr1 and arr2Hint 1Start with a brute force solution and optimize from there.Hint 2Note that the question asks for a sorted array. What techniques work well on sorted arrays?Hint 3If M >> N (case #2) is giving you trouble, consider how you might use the values of N, the shorter array, to increase the efficiency of your answer.Challenge 1Have you taken care to avoid array out-of-bound runtime errors?First, the brute force solution. Try two nested loops. For every value arr1[i] in the outer loop, we look it up in arr2 in the inner loop. The time complexity of this algorithm in the worst case scenario is O(N*M). Not very efficient. We can do better than that. Case 1 (M ~ N) We can use the fact the arrays are sorted to traverse both of them in an in-order manner at the same time. The general idea of the algorithm is to use two indices, i and j, for arr1 and arr2, respectively. Every time one of indices,

QUESTION

EXAMPLES

Example 1	
Input	`[1, 3, 5, 7], [3, 4, 5, 8]`
Output	`[3, 5]`
Why	`Two sorted lists of passport numbers: 3 and 5 appear in both.`

Example 2	
Input	`[1, 2], [3, 4]`
Output	`[]`
Why	`Nothing in common.`

PART 1: Similar sizes

Walk both sorted arrays with two pointers.

PART 2: One list is much smaller

Binary-search each distinct value from the smaller list in the larger list.

SHORT OPTIMAL CODE

```python
from bisect import bisect_left
from math import log2


def common_two_pointers(first, second):
    answer, first_index, second_index = [], 0, 0
    while first_index < len(first) and second_index < len(second):
        if first[first_index] < second[second_index]:
            first_index += 1
        elif first[first_index] > second[second_index]:
            second_index += 1
        else:
            if not answer or answer[-1] != first[first_index]:
                answer.append(first[first_index])
            first_index += 1
            second_index += 1
    return answer


def common_binary_search(small, big):
    answer, start = [], 0
    for value in small:
        start = bisect_left(big, value, start)
        if start == len(big):
            break
        if big[start] == value and (not answer or answer[-1] != value):
            answer.append(value)
    return answer


def common(first, second):
    small, big = (first, second) if len(first) <= len(second) else (second, first)
    if len(small) * log2(max(2, len(big))) < len(first) + len(second):
        return common_binary_search(small, big)
    return common_two_pointers(first, second)
```

COMPLEXITY | Part 1: Time O(n + m) | Space O(1) excluding output
Part 2: Time O(s log b) | Space O(1) excluding output

109 Reconstruct original array from doubled shuffle

MEDIUM

Template move: Counter of the values; go from the smallest, pairing each x with 2x and using both counts up

QUESTION

An array of numbers was doubled: for every element x of the original, the value 2x was added, and then all the numbers were shuffled. Given the shuffled array changed, return an original array (in any order), or an empty list if changed cannot be a doubled array. Zero and negative numbers are possible.

Function signature

def find_original(changed: list) -> list

EXAMPLES

Example 1	
Input	`[1, 3, 4, 2, 6, 8]`
Output	`[1, 3, 4]`
Why	`Original [1, 3, 4] gives doubles [2, 6, 8].`

Example 2	
Input	`[1, 3, 2]`
Output	`[]`
Why	`An odd number of values cannot be doubled.`

Example 3	
Input	`[0, 0, 0, 0]`
Output	`[0, 0]`
Why	`Zeros pair with zeros.`

Example 4	
Input	`[-2, -1, -4, -2]`
Output	`[-1, -2]`
Why	`Negatives work too: -2 is the double of -1.`

WHY IT WORKS

Count the values. For each value x in order of absolute value with a remaining count, it needs count[x] copies of 2x: if count[2x] is smaller, fail; otherwise consume them and append x that many times. Zero is handled by the same rule because 2 * 0 = 0 (the check becomes 'the count is even' via consuming pairs).

SHORT OPTIMAL CODE

```
from collections import Counter


# BASE QUESTION (optimal): Counter processed in order of
# absolute value
# Idea: Count the values. For each value x in order of
# absolute value with a remaining count, it needs count[value]
# copies of 2x: if count[2x] is smaller, fail; otherwise
# consume them and append x that many times. Zero is
# handled by the same rule because 2 * 0 = 0 (the check
# becomes 'the count is even' via consuming pairs).
def findOriginalArray(changed):
    if len(changed) % 2:
        return []
    count, original = Counter(changed), []
    for value in sorted(count, key=lambda value: (abs(value), value)):
```

```
        if count[value] == 0:
            continue
        if value == 0:
            if count[0] % 2:
                return []
            original += [0] * (count[0] // 2)
            count[0] = 0
            continue
        if count[2 * value] < count[value]:
            return []
        original += [value] * count[value]
        count[2 * value] -= count[value]
        count[value] = 0
    return original
```

COMPLEXITY | Time O(n log n) | Space O(n)

110 Min deletions to avoid overlap in first k

MEDIUM

Template move: Counter of the left part and of the right part; for each value in both, delete min(left count, right count)

QUESTION

Split an array into its first k elements and the rest. Return the minimum number of elements to delete so that no value appears on both sides (for every shared value delete all copies from the side that has fewer).

Function signature

def min_deletions(a: list, k: int) -> int

EXAMPLES

Example 1	
Input	`[1, 2, 2, 3, 2, 1], 3`
Output	`2`
Why	`Left [1, 2, 2], right [3, 2, 1]: value 1 appears once on each side (delete 1), value 2 appears twice on the left and once on the right (delete 1): 2 deletions.`

Example 2	
Input	`[1, 2, 3, 4], 2`
Output	`0`
Why	`No value is shared between [1, 2] and [3, 4].`

WHY IT WORKS

Count each value in the first k elements and in the rest. For every value with a positive count on both sides, one side has to lose all of its copies, so add the smaller count. Different values never affect each other.

SHORT OPTIMAL CODE

```
from collections import Counter


# BASE QUESTION (optimal): Sum of min(count in the first
# k, count in the rest) over shared values
# Idea: Count each value in the first k elements and in
# the rest. For every value with a positive count on both
# sides, one side has to lose all of its copies, so add
# the smaller count. Different values never affect each
# other.
def min_deletions(values, prefix_length):
    left, right = Counter(values[:prefix_length]), Counter(values[prefix_length:])
```

```
    return sum(min(left[value], right[value]) for value in left.keys() & right.keys())
```

COMPLEXITY | Time O(n) | Space O(n)

111 Filter Common Elements Per Window (Two Lists) MEDIUM

Template move: build a set per window of the first list; keep the matching window of the second list minus that set

QUESTION

You are given two lists a and b of equal length n and a window size k that divides n . Split both lists into consecutive, non-overlapping windows of size k (window 0 is indices 0..k-1 , window 1 is k..2k-1 , and so on). For each window, remove from b 's window every value that also appears in the corresponding window of a . Concatenate the surviving elements of b , preserving their original order, and return that list.

Function signature

def filter_windows(a: list[int], b: list[int], k: int) -> list[int]:

    ...

Follow-up (as reported): The interviewer asked to do the removal in place / O(1) extra space. Naive in-place removal becomes O(n*k); discuss compaction with a write pointer to keep it O(n) time.

EXAMPLES

Example 1	
Input	`a = [1, 2, 3, 4, 5, 6], b = [3, 7, 8, 6, 9, 5], k = 3`
Output	`[7, 8, 9]`
Why	`Window 0: a={1,2,3}, b=[3,7,8] -> remove 3 -> [7,8]. Window 1: a={4,5,6}, b=[6,9,5] -> remove 6 and 5 -> [9]. Concatenate -> [7,8,9].`

Example 2	
Input	`a = [1, 1], b = [1, 2], k = 2`
Output	`[2]`
Why	`Single window: a={1}, b=[1,2] -> remove 1 -> [2].`

CONSTRAINTS

- 0 <= n <= 10**5 , n % k == 0 , 1 <= k <= n (for n > 0).
- Values are integers.

WHY IT WORKS

Membership in the window of a is a set lookup.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): A set per window
# Idea: Membership in the window of a is a set lookup.
def filter_windows(first, second, window_size):
    kept = []
    for window_start in range(0, len(first), window_size):
        seen = set(first[window_start:window_start + window_size])
        kept.extend(value for value in second[window_start:window_start + window_size] if value not in seen)
    return kept
```

COMPLEXITY | Time O(n) | Space O(k)

112 Smallest Missing Positive Integer in Linear Time and Constant Extra Space HARD

Template move: use the array itself as the set: swap each value x into slot x - 1, then the first slot with the wrong value is the answer

QUESTION

You are given an unsorted list of integers **nums**, which may contain negative numbers, zeros, duplicates, and values far larger than the length of the list. Return the smallest positive integer, that is the smallest integer that is at least **1**, that does not appear in **nums**. In the interview, a first solution that ran in linear time but used extra memory proportional to the input was accepted as a starting point, and the interviewer then asked for the same result using only constant extra space. Aim for that tighter target.

Function Signature

```
def first_missing_positive(nums: list[int]) -> int:
```

Rules

- Target complexity: **O(n)** time and **O(1)** auxiliary space, where **n = len(nums)**. The list may be rearranged or overwritten in place; its contents after the call do not matter.
- Return only the missing value.

Constraints

- **1 <= len(nums) <= 10^5**
- **-2147483648 <= nums[i] <= 2147483647**
- The answer is a positive integer that fits in a signed 32-bit integer.

Examples

Example 1

- Input: **nums = [4, -2, 1, 2]**
- Output: **3**
- Explanation: **1** and **2** are present and **3** is not.

Example 2

- Input: **nums = [5, 6, 100, -3]**
- Output: **1**
- Explanation: **1** does not appear, so it is the answer, however large the other values are.

Example 3

- Input: **nums = [1, 2, 2, 3]**
- Output: **4**
- Explanation: **1**, **2** and **3** all appear (**2** twice), so the smallest missing positive integer is **4**.

WHY IT WORKS

For each position, while nums[i] is in 1..n and not already at its home index, swap it to index nums[i] - 1. Each swap puts one value in its final place, so the total number of swaps is at most n. Then scan for the first index whose value is not i + 1; if none, the answer is n + 1.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Cyclic placement: put each
# value v at index v - 1
# Idea: For each position, while nums[position] is in 1..n and
# not already at its home index, swap it to index nums[position]
# - 1. Each swap puts one value in its final place, so the
# total number of swaps is at most n. Then scan for the
# first index whose value is not position + 1; if none, the
# answer is length + 1.
def firstMissingPositive(nums):
```

```
    values, length = list(nums), len(nums)
    for position in range(length):
        while 1 <= values[position] <= length and values[values[position] - 1] != values[position]:
            home_index = values[position] - 1
            values[position], values[home_index] = values[home_index], values[position]
    for position in range(length):
        if values[position] != position + 1:
            return position + 1
    return length + 1
```

COMPLEXITY | Time O(n) | Space O(1)

bisect on a sorted list: find a position, count a range, keep it sorted

Pattern

bisect_left and bisect_right give the position of a value in a sorted list in O(log n). Subtract two positions to count a range, slice between them to list a range, insort to keep inserting in sorted order, and replace an element at a position for the 'longest increasing' trick.

113 Count People in a Postal Code Range

EASY

Template move: sort the codes once; the answer is the template as written, never below 0

QUESTION

You are given a list of people, each as a tuple (name, postal_code) . Build a structure that can answer many queries of the form: "how many people have a postal code within the inclusive range [low, high] ?" The interviewer explicitly wanted the optimal query implementation — not a linear scan of the people (or a scan over the range and a hash-map lookup for each value). Preprocess once, then answer each query in O(log n) . Implement a class with: a constructor that accepts the list of people, and count_in_range(low, high) returning the number of people whose postal code is in [low, high] inclusive.

Function signature

```
from typing import List, Tuple
class PostalRegistry:
    def __init__(self, people: List[Tuple[str, int]]) -> None:
        """Preprocess the (name, postal_code) pairs for fast range counts."""
        ...
    def count_in_range(self, low: int, high: int) -> int:
        """Return how many people have postal_code in [low, high] inclusive."""
        ...
```

Follow-up (as reported): Also return the list of matching names. Hint: sort by (postal_code, name) and slice between bisect_left(low) and bisect_right(high) .

EXAMPLES

Example 1	
Input	`people = [("Alice", 90210), ("Bob", 10001), ("Cara", 90211), ("Dan", 30301)]` `count_in_range(90000, 99999)`
Output	`2`
Why	`Alice (90210) and Cara (90211) fall in the range.`

Example 2	
Input	`same people; count_in_range(10001, 30301)`
Output	`2`
Why	`Bob (10001) and Dan (30301); range is inclusive on both ends.`

Example 3	
Input	`same people; count_in_range(50000, 60000)`
Output	`0`
Why	`No postal codes in that range.`

CONSTRAINTS

- 0 <= number of people <= 10^5 ; the same postal code may appear for multiple people (duplicates allowed).

QUESTION

- Postal codes are integers, 0 <= postal_code <= 10^9 .
- 0 <= low <= high <= 10^9 .
- Each count_in_range call must run in O(log n) after an O(n log n) preprocessing step.

WHY IT WORKS

The codes in the range form one block of the sorted list.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Sorted codes and two bisects
# Idea: The codes in the range form one block of the
# sorted list.
from bisect import bisect_left, bisect_right


class PostalRegistry:
    def __init__(self, people):
        self.codes = sorted(code for _, code in people)

    def count_in_range(self, low, high):
        return max(0, bisect_right(self.codes, high) - bisect_left(self.codes, low))
```

COMPLEXITY | Time O(log n) per query | Space O(n)

114 Sliding-Window Mean After Removing the Largest K Values

MEDIUM

Template move: keep the window in a sorted list; insort the new value, remove the old one with bisect_left, and track the sum of the top-K region

QUESTION

For every window of a fixed size over the array, remove the k largest values of the window (repeated values count separately) and compute the mean of the rest. Return the means in window order as reduced fractions written 'numerator/denominator'. Update the windows incrementally instead of sorting each one again.

Function signature

def trimmed_window_means(nums: list, window: int, k: int) -> list

EXAMPLES

Example 1	
Input	[1, 2, 3, 4, 5], 3, 1
Output	['3/2', '5/2', '7/2']
Why	Window [1,2,3] without its largest value 3 averages (1+2)/2 = 3/2; then 5/2 and 7/2.

Example 2	
Input	[2, 4, 6, 8], 2, 0
Output	['3/1', '5/1', '7/1']
Why	With k = 0 nothing is removed: plain window means.

WHY IT WORKS

Keep the window values in a sorted list, the total window sum, and top = the sum of its K largest elements. Inserting x at position p: if p falls into the top K region, add x and subtract the element that was pushed out of it; removing is the mirror image. The mean of the kept values is (window sum - top) / (window - k), reduced with gcd.

SHORT OPTIMAL CODE

```python
from bisect import bisect_left
from bisect import bisect_right
import math


def _frac(numerator, denominator):
    divisor = math.gcd(abs(numerator), denominator)
    return f"{numerator // divisor}/{denominator // divisor}"


# BASE QUESTION (optimal): Sorted window with an
# incrementally maintained top-K sum
# Idea: Keep the window values in a sorted list, the total
# window sum, and top_sum = the sum of its K largest elements.
# Inserting x at position p: if p falls into the top K
# region, add x and subtract the element that was pushed
# out of it; removing is the mirror image. The mean of the
# kept values is (window sum - top_sum) / (window - remove_count),
# reduced with gcd.
def trimmed_window_means(nums, window, remove_count):
    window_sorted, window_sum, top_sum, means = [], 0, 0, []

    def insert(value):
        nonlocal window_sum, top_sum
        old_size = len(window_sorted)
        position = bisect_right(window_sorted, value)
        window_sorted.insert(position, value)
        window_sum += value
        if remove_count:
            if old_size < remove_count:
                top_sum += value
            elif position >= old_size + 1 - remove_count:
                top_sum += value - window_sorted[old_size - remove_count]                # window_sorted[old
_size - remove_count] is the element pushed out of the top-K region

    def remove(value):
        nonlocal window_sum, top_sum
        old_size = len(window_sorted)
        position = bisect_left(window_sorted, value)
        window_sum -= value
        if remove_count:
            if old_size <= remove_count:
                top_sum -= value
            elif position >= old_size - remove_count:
                top_sum += window_sorted[old_size - remove_count - 1] - value             # the element just
 below the region joins it
        del window_sorted[position]

    for index, value in enumerate(nums):
        insert(value)
        if index >= window:
            remove(nums[index - window])
        if index >= window - 1:
            means.append(_frac(window_sum - top_sum, window - remove_count))
    return means
```

COMPLEXITY | Time O(n * (log w + w) in memmove, O(n log w) with a balanced tree) | Space O(w)

115 Streaming Insert + Find K-th Largest

MEDIUM

Template move: insort every new value; the k-th largest is sorted_list[-1 - k]

QUESTION

Design a structure with insert(x), which adds a number, and findLargest(k), which returns the k-th largest number inserted so far (k = 0 is the largest), counting repeated values separately, or None when fewer than k + 1 numbers exist.

Function signature

class StreamKth:
    def insert(self, x: float) -> None
    def findLargest(self, k: int) -> float | None

EXAMPLES

Example 1

Input	`s = StreamKth()`
	`s.insert(3)`
	`s.insert(3)`
	`s.insert(2)`
	`s.findLargest(0)`
	`s.findLargest(1)`
	`s.findLargest(2)`
	`s.findLargest(3)`

Output	`s.findLargest(0) -> 3`
	`s.findLargest(1) -> 3`
	`s.findLargest(2) -> 2`
	`s.findLargest(3) -> null`

Why	`Sorted descending the numbers are 3, 3, 2, so the 0th, 1st and 2nd largest are 3, 3 and 2, and there is no 3rd.`

PART 1: Sorted list with bisect (k chosen per query)

Keep all values in an ascending list with insort. The k-th largest (0-indexed as in the examples) is a[-1 - k]. Python's list shift is a fast memmove, which is why this beats a hand-written tree for realistic n; mention a balanced BST or Fenwick tree when n is huge.

PART 2: Fixed k: min-heap of the k largest

If k never changes, only the k largest values can ever matter. A min-heap of size k keeps them and the root is the k-th largest. Pop only when the heap would exceed k.

SHORT OPTIMAL CODE

```python
from bisect import insort


# BASE QUESTION (optimal): Sorted list with bisect (rank
# chosen per query)
# Idea: Keep all values in an ascending list with insort.
# The rank-th largest (0-indexed as in the examples) is a[-1
# - rank]. Python's list shift is a fast memmove, which is
# why this beats a hand-written tree for realistic n;
# mention a balanced BST or Fenwick tree when n is huge.
class StreamKth:
    def __init__(self):
        self.sorted_values = []

    def insert(self, value):
        insort(self.sorted_values, value)

    def findLargest(self, rank):
        return self.sorted_values[-1 - rank] if 0 <= rank < len(self.sorted_values) else None
```

COMPLEXITY | Part 1: Time O(log n) search + O(n) shift per insert, O(1) query | Space O(n)
Part 2: Time O(log k) per insert, O(1) query | Space O(k)

116 Count Sorted Strings With a Given Prefix

MEDIUM

Template move: lo = bisect_left(words, prefix); hi = bisect_left(words, prefix + a very large character); the answer is hi - lo

QUESTION

Given a lexicographically sorted array of strings words and a query string prefix , return the number of strings in words that start with prefix . Because words is sorted, all matching strings occupy a contiguous range; find that range with binary search in O(log n * L) time (where L is the prefix length), not by scanning.

Function signature

```
def count_with_prefix(words: list[str], prefix: str) -> int:
    ...
```

Follow-up (as reported): Many repeated queries: cache counts in a hash map keyed by prefix. All queries unique: build a trie of all words storing subtree sizes, so each query is O(L) .

EXAMPLES

Example 1	
Input	`words = ["app", "apple", "application", "banana", "bat"], prefix = "app"`
Output	`3`
Why	`"app", "apple", "application" all start with "app".`

Example 2	
Input	`words = ["app", "apple", "application", "banana", "bat"], prefix = "ba"`
Output	`2`
Why	`"banana" and "bat".`

Example 3	
Input	`words = ["app", "apple", "application", "banana", "bat"], prefix = "cat"`
Output	`0`
Why	`No string starts with "cat".`

CONSTRAINTS

- 0 <= len(words) <= 10**6 , sorted in non-decreasing lexicographic order.
- 0 <= len(prefix) <= 100 . An empty prefix matches every string.

WHY IT WORKS

The matching words form one contiguous block of the sorted list; find both ends.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Two binary searches
# Idea: The matching words form one contiguous block of
# the sorted list; find both ends.
from bisect import bisect_left


def count_with_prefix(words, prefix):
    first_match = bisect_left(words, prefix)
    after_last_match = bisect_left(words, prefix + "\uffff")
    return after_last_match - first_match
```

COMPLEXITY | Time O(L log n) | Space O(1)

117 Russian Doll Envelopes

MEDIUM

Template move: sort by width ascending and height descending, then the longest increasing run of heights using a tails list and bisect_left

QUESTION

LeetCode 354. Maximum chain of envelopes that strictly fit inside each other. Sort + LIS reduction.

Input: list of (width, height) envelopes.
Output: longest chain where each envelope strictly fits in the next.
Trick: sort by width ascending, height descending (when widths tie); run LIS (Longest Increasing Subsequence) on heights.
LIS with patience-sort gives O(n log n).

EXAMPLES

```
[[5,4],[6,4],[6,7],[2,3]] -> 3.
```

WHY IT WORKS

Sort by (width ascending, height descending). Keep a list `tails` where tails[k] is the smallest height ending a chain of length k + 1; for each height bisect_left finds where it belongs (replace) or extends the list. The final length is the answer.

SHORT OPTIMAL CODE

```python
from bisect import bisect_left


# BASE QUESTION (optimal): Sort, then LIS with binary
# search
# Idea: Sort by (width ascending, height descending). Keep
# a list `tails` where tails[k] is the smallest height
# ending a chain of length k + 1; for each height
# bisect_left finds where it belongs (replace) or extends
# the list. The final length is the answer.
def max_envelopes(envelopes):
    tails = []
    for _, height in sorted(envelopes, key=lambda envelope: (envelope[0], -envelope[1])):
        position = bisect_left(tails, height)
        if position == len(tails):
            tails.append(height)
        else:
            tails[position] = height
    return len(tails)
```

COMPLEXITY | Time O(n log n) | Space O(n)

118 Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.

MEDIUM

Template move: for each kid bisect into the sorted sellers and take the nearer of the two neighbours; the answer is the largest of those distances

QUESTION

Kids and ice cream sellers stand at positions on a line. Find the smallest distance E such that every kid is within E of at least one seller (the kid at position K is served by a seller at P when P - E <= K <= P + E).

Function signature
def smallest_e(kids: list, sellers: list) -> int

EXAMPLES

QUESTION

Example 1	
Input	`[1, 5, 9], [2, 8]`
Output	`3`
Why	`The kid at 5 is 3 away from both sellers (at 2 and 8); the others are 1 away.`

Example 2	
Input	`[4], [4]`
Output	`0`
Why	`A kid standing on a seller needs distance 0.`

WHY IT WORKS

With the sellers sorted, bisect_left gives the first seller at or after the kid; the nearest seller is that one or the one before it. E is the largest of these nearest distances.

SHORT OPTIMAL CODE

```
from bisect import bisect_left


# BASE QUESTION (optimal): Sort the sellers, binary search
# each kid
# Idea: With the sellers sorted, bisect_left gives the
# first seller at or after the kid; the nearest seller is
# that one or the one before it. E is the largest of these
# nearest distances.
def smallest_radius(kids, sellers):
    sellers = sorted(sellers)
    worst = 0
    for kid in kids:
        position = bisect_left(sellers, kid)
        nearest_distance = min(abs(kid - sellers[seller_index]) for seller_index in (position - 1, position) if
 0 <= seller_index < len(sellers))
        worst = max(worst, nearest_distance)
    return worst
```

COMPLEXITY | Time O((N + M) log M) | Space O(1) extra

119 Find Maximum Envelope Nesting — HARD

Template move: exactly the same sort and tails list

QUESTION

Envelopes are given as (width, height). One envelope fits inside another only when both its width and its height are strictly smaller. Return the largest number of envelopes that can be nested one inside the next (Russian dolls).

Function signature

def max_envelopes(envelopes: list) -> int

EXAMPLES

Example 1	
Input	`[(5, 4), (6, 4), (6, 7), (2, 3)]`
Output	`3`
Why	`(2, 3) fits in (5, 4), which fits in (6, 7): 3 nested.`

QUESTION

Example 2	
Input	`[(1, 1), (1, 1), (1, 1)]`
Output	`1`
Why	`Equal envelopes cannot go inside each other.`

WHY IT WORKS

Sorting equal widths by decreasing height means two envelopes of the same width can never both appear in a strictly increasing height sequence. The heights are then fed to the standard LIS: keep the smallest possible tail for each length and replace with bisect_left.

SHORT OPTIMAL CODE

```
from bisect import bisect_left


# BASE QUESTION (optimal): Sort by width ascending and
# height descending, then LIS by binary search
# Idea: Sorting equal widths by decreasing height means
# two envelopes of the same width can never both appear in
# a strictly increasing height sequence. The heights are
# then fed to the standard LIS: keep the smallest possible
# tail for each length and replace with bisect_left.
def maxEnvelopes(envelopes):
    tails = []
    for _, height in sorted(envelopes, key=lambda envelope: (envelope[0], -envelope[1])):
        position = bisect_left(tails, height)
        if position == len(tails):
            tails.append(height)
        else:
            tails[position] = height
    return len(tails)
```

COMPLEXITY | Time O(n log n) | Space O(n)

120 Count of Smaller Numbers After Self — HARD

Template move: go from the right, insort each number into a sorted list; the count is bisect_left of the number before inserting (a Fenwick tree is the fast version)

QUESTION

Given an integer array nums, return an array counts where counts[i] is the number of elements to the right of nums[i] that are strictly smaller than nums[i].

Function signature

def count_smaller_after(nums: list) -> list

EXAMPLES

Example 1	
Input	`[5, 2, 6, 1]`
Output	`[2, 1, 1, 0]`
Why	`5 has two smaller numbers to its right (2 and 1), 2 has one (1), 6 has one (1), 1 has none.`

QUESTION

Example 2	
Input	`[-1]`
Output	`[0]`
Why	`A single number has nothing to its right.`

Example 3	
Input	`[-1, -1]`
Output	`[0, 0]`
Why	`Equal numbers are not smaller.`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- -10^4 <= nums[i] <= 10^4

WHY IT WORKS

Compress values to ranks 1..m. Scan from the right: the answer for x is the number of already inserted values with rank less than rank(x) (a prefix sum query), then insert rank(x). Both operations are O(log m).

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Fenwick tree over compressed
# values, scanning right to left
# Idea: Compress values to ranks 1..m. Scan from the
# right: the answer for x is the number of already
# inserted values with rank less than rank(value) (a prefix
# sum query), then insert rank(value). Both operations are
# O(log rank_count).
def countSmaller(nums):
    ranks = {value: rank + 1 for rank, value in enumerate(sorted(set(nums)))}
    rank_count = len(ranks)
    tree = [0] * (rank_count + 1)
    out = []
    for value in reversed(nums):
        rank_index, total = ranks[value] - 1, 0
        while rank_index > 0:
            total += tree[rank_index]
            rank_index -= rank_index & -rank_index
        out.append(total)
        rank_index = ranks[value]
        while rank_index <= rank_count:
            tree[rank_index] += 1
            rank_index += rank_index & -rank_index
    return out[::-1]
```

COMPLEXITY | Time O(n log n) | Space O(n)

GROUP 15 OF 43 | 8 QUESTIONS | SHORT MAIN CODE

Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

Pattern

pre[i] is the total of the first i items. Any range sum is pre[r + 1] - pre[l], and a balance point is an index where the left total equals the right total. Count something other than values (equal pairs, ones, repeats) by accumulating that instead.

121 Cake Balance Line

EASY

Template move: the same balance test on the cake heights

QUESTION

A cake is described by a list heights , where heights[i] is the amount of cake sitting at position i (positions are left to right). You want to place a single vertical cut exactly at some position i so that the total amount of cake strictly to the left of the cut equals the total amount strictly to the right of the cut (the cake at position i itself is consumed by the cut and counts for neither side). Return the smallest index i where sum(heights[:i]) == sum(heights[i+1:]) , or -1 if no such position exists.

Function signature

```
def cake_balance_index(heights: list[int]) -> int:
    ...
```

Follow-up (as reported): Do it in one pass and O(1) extra space by tracking a running left sum against total - left - heights[i] .

EXAMPLES

Example 1	
Input	`heights = [1, 7, 3, 6, 5, 6]`
Output	`3`
Why	`Left of index 3 is 1+7+3=11; right of index 3 is 5+6=11.`

Example 2	
Input	`heights = [1, 2, 3]`
Output	`-1`
Why	`No cut position balances the two sides.`

Example 3	
Input	`heights = [2, 1, -1]`
Output	`0`
Why	`Left of index 0 is empty (0); right is 1+(-1)=0.`

CONSTRAINTS

- 1 <= len(heights) <= 10^5 .
- -10^4 <= heights[i] <= 10^4 (heights may be negative in this abstracted version).

WHY IT WORKS

The right side is total - left - heights[i].

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Running left sum
# Idea: The right side is total - left - heights[position].
def cake_balance_index(heights):
```

```
    total, left = sum(heights), 0
    for position, height in enumerate(heights):
        if left == total - left - height:
            return position
        left += height
    return -1
```

COMPLEXITY | Time O(n) | Space O(1)

122 Find the Pivot Index (Balanced Split) in O(n) EASY

Template move: the template as written: left total equals total - left - x

QUESTION

Given an integer array nums , return the leftmost index i such that the sum of all elements strictly to the left of i equals the sum of all elements strictly to the right of i . The element at i itself is not counted on either side. For the first index, the left sum is 0 ; for the last index, the right sum is 0 . If no such index exists, return -1 . Your solution must run in O(n) time and O(1) extra space.

Function signature

def pivot_index(nums: list[int]) -> int:

...

Follow-up (as reported): If several pivots exist, return them all. Hint: keep scanning after the first match, maintaining the running left sum.

EXAMPLES

Example 1	
Input	nums = [1, 7, 3, 6, 5, 6]
Output	3
Why	Left of index 3: 1+7+3 = 11. Right of index 3: 5+6 = 11.

Example 2	
Input	nums = [1, 2, 3]
Output	-1
Why	No index balances the two sides.

Example 3	
Input	nums = [2, 1, -1]
Output	0
Why	Left of index 0 is 0; right is 1 + (-1) = 0.

CONSTRAINTS

- 1 <= len(nums) <= 10^4
- -1000 <= nums[i] <= 1000

WHY IT WORKS

The right sum is total - left - nums[i], so one pass is enough.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Running left sum
# Idea: The right sum is total - left - nums[position], so one
# pass is enough.
def pivot_index(nums):
```

```
    total, left = sum(nums), 0
    for position, value in enumerate(nums):
        if left == total - left - value:
            return position
        left += value
    return -1
```

COMPLEXITY | Time O(n) | Space O(1)

123 Parity Alternating Subarray Query

EASY

Template move: pre counts neighbouring pairs of equal parity; a range alternates when pre[r] - pre[l] == 0

QUESTION

Given an array nums, answer many queries alternates(l, r): do the elements nums[l], ..., nums[r] (both ends included) alternate in parity (odd, even, odd, ... or even, odd, even, ...)? Preprocess the array once so that every query takes O(1).

Function signature

class ParityQueries:
  def __init__(self, nums: list)
  def alternates(self, l: int, r: int) -> bool

EXAMPLES

Example 1

Input	`p = ParityQueries([1, 2, 3, 5, 4])`
	`p.alternates(0, 2)`
	`p.alternates(0, 3)`
	`p.alternates(3, 4)`

Output	`p.alternates(0, 2) -> true`
	`p.alternates(0, 3) -> false`
	`p.alternates(3, 4) -> true`

Why	`1, 2, 3 alternates; the neighbours 3 and 5 are both odd, so any range containing both fails; 5, 4 alternates.`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= queries.length <= 10^5
- 0 <= l <= r < nums.length

WHY IT WORKS

bad[i] is the number of positions j < i with nums[j] and nums[j + 1] of the same parity. The pairs inside nums[l..r] are j = l .. r - 1, so the subarray alternates when bad[r] - bad[l] == 0. Use x % 2 (not x & 1 on a language where % can be negative) so negatives work.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Prefix count of equal-parity
# neighbour pairs
# Idea: bad[position] is the number of positions j < position with
# nums[j] and nums[j + 1] of the same parity. The pairs
# inside nums[left..r] are j = left .. right - 1, so the subarray
# alternates when bad[right] - bad[left] == 0. Use x % 2 (not x &
# 1 on a language where % can be negative) so negatives
# work.
class ParityQueries:
    def __init__(self, nums):
        self.bad = [0]
        for position in range(len(nums) - 1):
```

```
            self.bad.append(self.bad[-1] + (nums[position] % 2 == nums[position + 1] % 2))

    def alternates(self, left, right):
        return self.bad[right] - self.bad[left] == 0
```

COMPLEXITY | Time O(n) to build, O(1) per query | Space O(n)

124 Range Duplicate Check with Constant-Time Queries

MEDIUM

Template move: next[i] = next occurrence of the same value; reach[l] = smallest next over l..end; the range l..r is all distinct when reach[l] > r

QUESTION

Preprocess an array once so that each query unique(l, r) answers in O(1) whether all elements from index l to index r (inclusive) are different.

Function signature

class RangeUnique:
  def __init__(self, a: list)
  def unique(self, l: int, r: int) -> bool

EXAMPLES

Example 1

Input	`ru = RangeUnique([1, 2, 3, 1, 4])`
	`ru.unique(0, 2)`
	`ru.unique(0, 3)`
	`ru.unique(1, 4)`

Output	`ru.unique(0, 2) -> true`
	`ru.unique(0, 3) -> false`
	`ru.unique(1, 4) -> true`

Why	`Indexes 0..3 hold 1, 2, 3, 1: the value 1 repeats. Ranges 0..2 and 1..4 have no repeats.`

WHY IT WORKS

Scan right to left with a map from value to its latest index to fill next[i] (infinity if none). Then reach[i] = min(next[i], reach[i + 1]). Query(l, r) is reach[l] > r. If the array changes, rebuild in O(n) (fine when reads dominate), or keep a sorted structure per value when updates are frequent.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Next-occurrence array with
# suffix minimum
# Idea: Scan right to left with a map from value to its
# latest index to fill next[position] (infinity if none). Then
# reach[position] = min(next[position], reach[position + 1]). Query(left, right) is
# reach[left] > right. If the array changes, rebuild in O(length)
# (fine when reads dominate), or keep a sorted structure
# per value when updates are frequent.
class RangeUnique:
    def __init__(self, values):
        length, INFINITY = len(values), float("inf")
        last_index_of, next_same_index = {}, [INFINITY] * length
        for position in range(length - 1, -1, -1):
            next_same_index[position] = last_index_of.get(values[position], INFINITY)
            last_index_of[values[position]] = position
        self.reach = [INFINITY] * (length + 1)
        for position in range(length - 1, -1, -1):
            self.reach[position] = min(next_same_index[position], self.reach[position + 1])

    def unique(self, left, right):
```

```
        return self.reach[left] > right
```

COMPLEXITY | Time O(n) preprocessing, O(1) per query | Space O(n)

125 Find a Horizontal Cut That Bisects Rectangle Area — MEDIUM

Template move: events by height with a running width; accumulate area strip by strip until half the total, then solve inside that strip

QUESTION

Given many disjoint axis-aligned rectangles (x1, y1, x2, y2), return the y of the horizontal line that splits their combined area in half, so the area below the line equals the area above it. Return an exact fraction (no floating-point error).

Function signature

def bisecting_cut(rects: list) -> Fraction

EXAMPLES

Example 1	
Input	`[(0, 0, 2, 2)]`
Output	`1`
Why	`One 2x2 square: the middle line is y = 1.`

Example 2	
Input	`[(0, 0, 1, 1), (0, 2, 1, 4)]`
Output	`5/2`
Why	`Areas 1 and 2, total 3, so half is 1.5: all of the first rectangle (area 1) plus 0.5 of the second, which starts at y = 2: y = 5/2.`

WHY IT WORKS

Make events (y, +width) at each rectangle's bottom and (y, -width) at its top and sort them. Between consecutive event heights the slope is the running sum of widths. Accumulate area slab by slab; in the slab where the accumulated area would reach half the total, the cut is y0 + (half - accumulated) / slope. Fractions keep it exact; slabs with slope zero are skipped, which returns the lowest valid cut.

SHORT OPTIMAL CODE

```
from fractions import Fraction
from collections import defaultdict


# BASE QUESTION (optimal): Sweep the y events and
# interpolate inside one slab
# Idea: Make events (y, +width) at each rectangle's bottom
# and (y, -width) at its top and sort them. Between
# consecutive event heights the slope is the running sum
# of widths. Accumulate area slab by slab; in the slab
# where the accumulated area would reach half the total,
# the cut is y0 + (half - accumulated) / slope. Fractions
# keep it exact; slabs with slope zero are skipped, which
# returns the lowest valid cut.
def bisecting_cut(rectangles):
    total = sum((x2 - x1) * (y2 - y1) for x1, y1, x2, y2 in rectangles)
    half = Fraction(total, 2)
    events = defaultdict(int)
    for x1, y1, x2, y2 in rectangles:
        events[y1] += x2 - x1
        events[y2] -= x2 - x1
    event_heights = sorted(events)
    total_width, area_so_far = 0, Fraction(0)
```

```
    for y0, y1 in zip(event_heights, event_heights[1:]):
        total_width += events[y0]
        gain = total_width * (y1 - y0)
        if total_width and area_so_far + gain >= half:
            return y0 + (half - area_so_far) / total_width
        area_so_far += gain
    return Fraction(event_heights[-1])
```

COMPLEXITY | Time O(R log R) | Space O(R)

126 Check pivot after removing one element

MEDIUM

Template move: prefix sums plus first/last position maps: look up the value whose removal would balance both sides

QUESTION

A pivot index of an array is an index whose left-side sum equals its right-side sum (the pivot element itself is excluded). Remove exactly one element from the array, then ask whether the remaining array has a pivot index. Return one pair (removed index, pivot index), both as indexes in the original array, or None if no removal creates a pivot. Negative numbers and duplicates are allowed.

Function signature

def pivot_after_removal(a: list) -> tuple | None

EXAMPLES

Example 1	
Input	`[2, 1, -1, 3]`
Output	`(0, 3)`
Why	`Removing the 2 at index 0 leaves [1, -1, 3]; the 3 (original index 3) has left sum 1 + (-1) = 0 and nothing on its right.`

Example 2	
Input	`[1, 2, 3]`
Output	`null`
Why	`Whichever element is removed, no pivot appears.`

Example 3	
Input	`[0, 0]`
Output	`(0, 1)`
Why	`Remove either 0: the other one is a pivot of an array with one element.`

WHY IT WORKS

Let P be the prefix sum before i and S the total. If the removed element r lies to the right of the pivot i, the sides are equal exactly when a[r] = S - 2P - a[i]; if r lies to the left, when a[r] = 2P + a[i] - S. Keep the last index of each value (needs an index greater than i) and the first index (needs an index smaller than i).

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Closed-form value to remove,
# found in a dictionary of positions
# Idea: Let P be the prefix sum before i and S the total.
# If the removed element r lies to the right of the pivot
# i, the sides are equal exactly when values[r] = total - 2P -
# values[pivot]; if r lies to the left, when values[r] = 2P + values[pivot] - total.
# Keep the last index of each value (needs an index
# greater than pivot) and the first index (needs an index
```

```
# smaller than pivot).
def pivot_after_removal(values):
    length, total = len(values), sum(values)
    first, last = {}, {}
    for index, value in enumerate(values):
        first.setdefault(value, index)
        last[value] = index
    prefix_sum = 0
    for pivot in range(length):
        need_right = total - 2 * prefix_sum - values[pivot]
        if need_right in last and last[need_right] > pivot:
            return True
        need_left = 2 * prefix_sum + values[pivot] - total
        if need_left in first and first[need_left] < pivot:
            return True
        prefix_sum += values[pivot]
    return False
```

COMPLEXITY | Time O(n) | Space O(n)

127 Gold Chain Split into Two Equal Halves

MEDIUM

Template move: prefix sums with first/last position maps: find two cut points that leave equal halves around one removed link

QUESTION

A gold chain is an array of segment weights. Remove exactly one segment, reconnect the rest, then cut the chain once so that the two parts have equal total weight. Return whether this is possible for some choice of the removed segment and the cut.

Function signature

def gold_chain(a: list) -> bool

EXAMPLES

Example 1	
Input	`[1, 2, 3, 4]`
Output	`true`
Why	`Remove the 4: [1, 2, 3] splits as [1, 2] and [3], both 3.`

Example 2	
Input	`[1, 1, 5]`
Output	`true`
Why	`Remove the 5: [1, 1] splits into 1 and 1.`

Example 3	
Input	`[1, 2, 4]`
Output	`false`
Why	`Removing 1 leaves 6 (halves of 3 need [2] | [4]: no), removing 2 leaves 5, removing 4 leaves 3: nothing works.`

WHY IT WORKS

Let P[j] be the sum of the first j segments. For removed index r with half = (S - a[r]) / 2 (skip if odd): a cut before r exists if half equals some P[j] with 1 <= j <= r; a cut after r exists if half + a[r] equals some P[k] with r < k <= n - 1. Keep the first index and the last index of every prefix value, so each check is one lookup.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Prefix sums with first and last
# occurrence maps
# Idea: Let P[cut_index] be the sum of the first j segments. For
# removed index r with half = (total - segments[removed_index]) / 2 (skip if
# odd): a cut before r exists if half equals some P[cut_index]
# with 1 <= cut_index <= removed_index; a cut after r exists if half + segments[removed_index]
# equals some P[k] with removed_index < k <= segment_count - 1. Keep the first
# index and the last index of every prefix value, so each
# check is one lookup.
def gold_chain(segments):
    segment_count = len(segments)
    if segment_count < 3:
        return False
    prefix_sums = [0]
    for segment in segments:
        prefix_sums.append(prefix_sums[-1] + segment)
    total = prefix_sums[-1]
    first, last = {}, {}
    for cut_index in range(1, segment_count):
        first.setdefault(prefix_sums[cut_index], cut_index)
        last[prefix_sums[cut_index]] = cut_index
    for removed_index in range(segment_count):
        rest = total - segments[removed_index]
        if rest % 2:
            continue
        half = rest // 2
        if half in first and first[half] <= min(removed_index, segment_count - 2):
            return True
        if (half + segments[removed_index]) in last and last[half + segments[removed_index]] > removed_index:
            return True
    return False
```

COMPLEXITY | Time O(n) | Space O(n)

128 Count the number of ways to split a string. MEDIUM

Template move: positions of the ones; the answer multiplies the gap sizes around the k-th and 2k-th one (zero ones: choose two cut points)

QUESTION

A binary string s is split into three non-empty parts by two cuts. Count the ways in which the three parts contain the same number of 1s, and return the count modulo 1,000,000,007.

Function signature

def numWays(s: str) -> int

EXAMPLES

Example 1	
Input	'10101'
Output	4
Why	Four ways to split '10101' so each part has one 1 (for example 1|010|1 and 10|10|1).

Example 2	
Input	'1001'
Output	0
Why	Two 1s cannot be divided into three equal groups.

QUESTION

Example 3	
Input	`'0000'`
Output	`3`
Why	`No 1s: any two cuts work: choose 2 of the 3 gaps = 3 ways.`

WHY IT WORKS

Let ones be the total number of ones. If it is not divisible by 3 the answer is 0; if it is 0, any two of the n - 1 cut positions work, C(n - 1, 2). Otherwise each part holds k = ones / 3 ones; the first cut can fall anywhere in the run of zeros between the k-th and (k + 1)-th one, and the second between the 2k-th and (2k + 1)-th one, so the answer is (gap1 + 1) * (gap2 + 1).

SHORT OPTIMAL CODE

```python
MOD = 10 ** 9 + 7


# BASE QUESTION (optimal): Count the ones, multiply the
# gap sizes
# Idea: Let ones be the total number of ones. If it is not
# divisible by 3 the answer is 0; if it is 0, any two of
# the length - 1 cut positions work, C(length - 1, 2). Otherwise
# each part holds ones_per_part = ones / 3 ones; the first cut can
# fall anywhere in the run of zeros between the ones_per_part-th and
# (ones_per_part + 1)-th one, and the second between the 2k-th and (2k
# + 1)-th one, so the answer is (gap1 + 1) * (gap2 + 1).
def numWays(bits):
    ones = [index for index, bit in enumerate(bits) if bit == "1"]
    length, ones_count = len(bits), len(ones)
    if ones_count % 3:
        return 0
    if ones_count == 0:
        return ((length - 1) * (length - 2) // 2) % MOD if length >= 3 else 0
    ones_per_part = ones_count // 3
    return ((ones[ones_per_part] - ones[ones_per_part - 1]) * (ones[2 * ones_per_part] - ones[2 * ones_per_part
 - 1])) % MOD
```

COMPLEXITY | Time O(n) | Space O(n)

GROUP 16 OF 43 | 8 QUESTIONS | SHORT MAIN CODE

Sorted intervals: merge while they touch, then read the gaps or search

Pattern

Sort by start. Walk once: if the next interval starts before the last one ends, extend the last; otherwise start a new one. The merged list is sorted and disjoint, so gaps are the spaces between neighbours and any point can be found with bisect.

129 Check an Interval Against Existing Intervals for Conflicts — EASY

Template move: bisect on the starts; a new booking conflicts if the previous one ends after its start or the next one starts before its end

QUESTION

Design a calendar of half-open intervals [start, end). book(start, end) books the interval and returns True when it does not overlap any interval booked so far, and returns False (booking nothing) when it does. Intervals that only touch, one ending exactly where the next starts, do not conflict.

Function signature

class Calendar:
    def book(self, start: int, end: int) -> bool

EXAMPLES

Example 1

Input	`c = Calendar()`
	`c.book(10, 20)`
	`c.book(15, 25)`
	`c.book(20, 30)`
	`c.book(5, 10)`
	`c.book(12, 14)`

Output	`c.book(10, 20) -> true`
	`c.book(15, 25) -> false`
	`c.book(20, 30) -> true`
	`c.book(5, 10) -> true`
	`c.book(12, 14) -> false`

Why	`[15, 25) overlaps [10, 20) and [12, 14) lies inside it, so both fail. [20, 30) and [5, 10) only touch the booked interval at an endpoint, so they succeed.`

WHY IT WORKS

Keep the booked intervals sorted by start. Find where the new start would go; it can only conflict with the interval just before (if its end is past the new start) or the one just after (if its start is before the new end). A balanced tree makes the insert O(log n) as well.

SHORT OPTIMAL CODE

```
from bisect import bisect_right


# BASE QUESTION (optimal): Sorted starts with bisect:
# check only two neighbours
# Idea: Keep the booked intervals sorted by start. Find
# where the new start would go; it can only conflict with
# the interval just before (if its end is past the new
# start) or the one just after (if its start is before the
# new end). A balanced tree makes the insert O(log n) as
# well.
class Calendar:
    def __init__(self):
        self.starts, self.ends = [], []
```

```
    def book(self, start, end):
        insert_position = bisect_right(self.starts, start)
        if insert_position > 0 and self.ends[insert_position - 1] > start:
            return False
        if insert_position < len(self.starts) and self.starts[insert_position] < end:
            return False
        self.starts.insert(insert_position, start)
        self.ends.insert(insert_position, end)
        return True
```

COMPLEXITY | Time O(log n) search, O(n) list insert | Space O(n)

130 Find Containing Range

MEDIUM

Template move: merge once, keep the starts in a list, and bisect to find the one interval that could contain x

QUESTION

Given a collection of inclusive ranges [start, end] (overlapping or touching ranges are merged), find(x) returns the merged range that contains the number x, or None when no range contains it.

Function signature

class RangeIndex:
  def __init__(self, ranges: list)
  def find(self, x: int) -> tuple | None

EXAMPLES

Example 1

Input	`r = RangeIndex([(1, 3), (2, 6), (8, 10)])`
	`r.find(5)`
	`r.find(7)`
	`r.find(10)`

Output	`r.find(5) -> (1, 6)`
	`r.find(7) -> null`
	`r.find(10) -> (8, 10)`

Why	`[1, 3] and [2, 6] overlap and merge into [1, 6]; 7 falls in the gap between the merged ranges.`

WHY IT WORKS

After sorting and merging, the starts are increasing, so bisect_right(starts, x) - 1 is the only candidate; it contains x if its end is at least x. Preprocessing is paid once.

SHORT OPTIMAL CODE

```
from bisect import bisect_right


# BASE QUESTION (optimal): Merge once, then binary search
# the starts
# Idea: After sorting and merging, the starts are
# increasing, so bisect_right(starts, point) - 1 is the only
# candidate; it contains x if its end is at least x.
# Preprocessing is paid once.
class RangeIndex:
    def __init__(self, ranges):
        merged = []
        for start, end in sorted(ranges):
            if merged and start <= merged[-1][1]:
                merged[-1][1] = max(merged[-1][1], end)
```

```
            else:
                merged.append([start, end])
        self.starts = [start for start, _ in merged]
        self.merged = merged

    def find(self, point):
        position = bisect_right(self.starts, point) - 1
        if position >= 0 and self.merged[position][1] >= point:
            return tuple(self.merged[position])
        return None
```

COMPLEXITY | Time O(n log n) to build, O(log n) per query | Space O(n)

131 Union of Sorted Interval Lists — MEDIUM

Template move: walk both sorted lists with two indices, always taking the smaller start next, then the same merge test

QUESTION

Two lists of intervals are each sorted and non-overlapping on their own. Return the union of both lists as sorted, non-overlapping intervals. Intervals that touch, such as (1, 3) and (3, 5), are merged unless touching=False.

Function signature

def union_two(a: list, b: list, touching=True) -> list

EXAMPLES

Example 1	
Input	[(1, 3), (6, 9)], [(2, 5), (10, 12)]
Output	[(1, 5), (6, 9), (10, 12)]
Why	(1, 3) and (2, 5) overlap and become (1, 5); (6, 9) and (10, 12) stay.

Example 2	
Input	[(1, 3)], [(3, 5)]
Output	[(1, 5)]
Why	(1, 3) and (3, 5) touch at 3, so they merge.

Example 3	
Input	[(1, 3)], [(3, 5)], touching = false
Output	[(1, 3), (3, 5)]
Why	With touching=False they only merge when they truly overlap.

PART 1: Two-pointer merge

Always take the interval with the smaller start next. If it touches or overlaps the last output interval (start <= last end, closed intervals) extend that one, otherwise append it. Invariant: everything before the pointers is already folded into a canonical output.

PART 2: K lists with a heap (streaming, O(K) memory)

heapq.merge keeps one pointer per list in a heap and yields intervals in start order without materializing the inputs, so it works for lists too large for memory. The same fold step builds the union lazily.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Two-pointer merge
# Idea: Always take the interval with the smaller start
# next. If it touches or overlaps the last output interval
# (start <= last end, closed intervals) extend that one,
# otherwise append it. Invariant: everything before the
# pointers is already folded into a canonical output.
```

```python
def union_two(first, second):
    merged, first_index, second_index = [], 0, 0
    while first_index < len(first) or second_index < len(second):
        if second_index == len(second) or (first_index < len(first) and first[first_index][0] <= second[second_
index][0]):
            next_interval = first[first_index]; first_index += 1
        else:
            next_interval = second[second_index]; second_index += 1
        if merged and next_interval[0] <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], next_interval[1])
        else:
            merged.append([next_interval[0], next_interval[1]])
    return merged
```

COMPLEXITY | Part 1: Time O(m + n) | Space O(m + n) for the output
Part 2: Time O(N log K) | Space O(K) beyond the output

132 Range Module (Insert and Point Query) MEDIUM

Template move: keep sorted lefts and rights; adding bisects to the first interval that touches and the first that starts after, and replaces that block by one merged interval

QUESTION

Design a data structure that tracks a set of half-open ranges [left, right) over the number line. add_range(left, right) adds the interval; overlapping or touching intervals are merged so that only disjoint, non-adjacent intervals remain. query_point(x) returns True when x lies inside a tracked interval; because the intervals are half-open, a point equal to a right endpoint is not inside.

Function signature

class RangeModule:
    def add_range(self, left: float, right: float) -> None
    def query_point(self, x: float) -> bool

EXAMPLES

Example 1

Input	`rm = RangeModule()`
	`rm.add_range(10, 20)`
	`rm.query_point(15)`
	`rm.query_point(20)`
	`rm.query_point(10)`

Output	`rm.query_point(15) -> true`
	`rm.query_point(20) -> false`
	`rm.query_point(10) -> true`

Why	`[10, 20) contains 10 and 15 but not 20.`

Example 2

Input	`rm = RangeModule()`
	`rm.add_range(10, 20)`
	`rm.add_range(20, 30)`
	`rm.query_point(20)`
	`rm.query_point(25)`
	`rm.query_point(30)`

Output	`rm.query_point(20) -> true`
	`rm.query_point(25) -> true`
	`rm.query_point(30) -> false`

Why	`The touching ranges merge into [10, 30), so 20 is now covered; 30 still is not.`

QUESTION

Example 3	
Input	`rm = RangeModule()` `rm.query_point(5)`
Output	`rm.query_point(5) -> false`
Why	`Nothing has been added yet.`

CONSTRAINTS

- Endpoints and query points are integers or floats with -10^9 <= value <= 10^9 .
- For every add_range call, left < right .
- Up to 10^5 total operations.
- Touching intervals merge: add_range(1, 5) followed by add_range(5, 10) yields one interval [1,
- 10) .

WHY IT WORKS

Disjoint sorted intervals mean one bisect locates the only candidate for a point.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Merged intervals with bisect
# Idea: Disjoint sorted intervals mean one bisect locates
# the only candidate for a point.
from bisect import bisect_left, bisect_right


class RangeModule:
    def __init__(self):
        self.lefts = []
        self.rights = []

    def add_range(self, left, right):
        first_overlap = bisect_left(self.rights, left)
        after_last_overlap = bisect_right(self.lefts, right)
        if first_overlap < after_last_overlap:
            left = min(left, self.lefts[first_overlap])
            right = max(right, self.rights[after_last_overlap - 1])
        self.lefts[first_overlap:after_last_overlap] = [left]
        self.rights[first_overlap:after_last_overlap] = [right]

    def query_point(self, point):
        first_overlap = bisect_right(self.lefts, point) - 1
        return first_overlap >= 0 and point < self.rights[first_overlap]
```

COMPLEXITY | Time O(log n) query | Space O(n)

133 Meetings Minus a Do-Not-Schedule Block

MEDIUM

Template move: merge first, then for each merged meeting keep the part before the blocked window and the part after it

QUESTION

You are given a list of meetings , each a (start, end) interval with start < end , and one "do-not- schedule" (DNS) block (dns_start, dns_end) . Consider the set of all time points covered by at least one meeting. Return the smallest list of non-overlapping, sorted intervals that covers exactly those points except any point lying inside the DNS block (treat the DNS block as removing the open coverage on (dns_start, dns_end) ; equivalently split coverage at those bounds). Meetings may overlap and are given in any order. Return the result sorted by start time.

QUESTION

Function signature

```
def subtract_dns(meetings: list[tuple[int, int]],
                 dns: tuple[int, int]) -> list[tuple[int, int]]:
    ...
```

Follow-up (as reported): Generalize to a list of non-overlapping DNS blocks: merge the meetings first, then walk both sorted lists together subtracting each block.

EXAMPLES

Example 1	
Input	`meetings = [(1,7),(5,10),(12,30),(22,40),(50,65)], dns = (20,35)`
Output	`[(1,10),(12,20),(35,40),(50,65)]`
Why	`Merged coverage is [1,10],[12,40],[50,65]. Removing (20,35) splits [12,40] into [12,20] and [35,40].`

Example 2	
Input	`meetings = [(0,10)], dns = (0, 10)`
Output	`[]`
Why	`The DNS block removes the entire covered range.`

Example 3	
Input	`meetings = [(0,5),(6,9)], dns = (100, 200)`
Output	`[(0,5),(6,9)]`
Why	`The DNS block covers nothing that was scheduled.`

CONSTRAINTS

- 1 <= len(meetings) <= 10^5 .
- Coordinates are integers; 0 <= start < end <= 10^9 .

WHY IT WORKS

After merging, every interval keeps at most a left and a right piece around the block.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Merge, then cut
# Idea: After merging, every interval keeps at most a left
# and a right piece around the block.
def merge_intervals(meetings):
    merged = []
    for start, end in sorted(meetings):
        if merged and start <= merged[-1][1]:
            merged[-1] = (merged[-1][0], max(merged[-1][1], end))
        else:
            merged.append((start, end))
    return merged


def subtract_dns(meetings, block):
    block_start, block_end = block
    remaining = []
    for start, end in merge_intervals(meetings):
        if start < block_start:
            remaining.append((start, min(end, block_start)))
        if end > block_end:
            remaining.append((max(start, block_end), end))
    return remaining
```

COMPLEXITY | Time O(n log n) | Space O(n)

134 Merge Intervals

MEDIUM

Template move: the template as written

QUESTION

Merge Intervals. An interval is a representation of a range of values along a number line. For example, [1,3] represents all numbers between 1 and 3, inclusive. Given a collection of intervals, the task is to merge all overlapping intervals. For example, consider the intervals [1,3],[2,6],[8,10],[15,18]. The function should return merged intervals where overlapping intervals are combined. In this case, intervals [1,3] and [2,6] overlap, so they should be merged into [1,6]. Examples intervals = [[1,3], [2,6], [8,10], [15,18]] output: [[1,6], [8,10], [15,18]] intervals = [[1,4], [4,5]] output: [[1,5]] intervals = [[1,4], [2,3]] output: [[1,4]] intervals = [[1,2], [3,4], [5,6], [7,8]] output: [[1,2], [3,4], [5,6], [7,8]]The mergeIntervals function takes an array of intervals and merges all overlapping intervals into a single interval. The process involves sorting the intervals by their start times and then iterating through them to merge where necessary. Here is a step-by-step breakdown: Initial Check: If the input array is empty, the function immediately returns an empty array, as there are no intervals to merge. Sorting: The intervals are sorted by their start times. This ensures that intervals can be processed in a linear sweep, merging adjacent intervals when they overlap. Merging Intervals: Initialize a result list with the first interval. Then, iterate through the sorted intervals starting from the second interval. For each interval, compare it with the last interval in th

EXAMPLES

Example 1	
Input	`[[1, 3], [2, 6], [8, 10], [15, 18]]`
Output	`[[1, 6], [8, 10], [15, 18]]`
Why	`[1, 3] and [2, 6] overlap and merge into [1, 6].`

Example 2	
Input	`[[1, 4], [4, 5]]`
Output	`[[1, 5]]`
Why	`Intervals that touch at an endpoint are merged as well.`

Example 3	
Input	`[[8, 10], [1, 3], [2, 4]]`
Output	`[[1, 4], [8, 10]]`
Why	`Unsorted input is fine.`

WHY IT WORKS

Sort by start. If an interval starts at or before the end of the last output interval, extend that end with max; otherwise append it. The sort guarantees nothing earlier can overlap the new interval.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Sort by start and sweep
# Idea: Sort by start. If an interval starts at or before
# the end of the last output interval, extend that end
# with max; otherwise append it. The sort guarantees
# nothing earlier can overlap the new interval.
def merge_intervals(intervals):
    merged = []
    for start, end in sorted(intervals):
        if merged and start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged
```

COMPLEXITY | Time O(n log n) | Space O(n)

135 Find safe travel intervals between planet influences

HARD

Template move: turn each planet into (centre - radius, centre + radius), merge, and read the gaps (including the infinite ends)

QUESTION

You are planning a space route along a one-dimensional line (the x-axis). You are given a list of planets. Each planet is represented by an integer pair ((c, r)):

- **c** is the coordinate of the planet's center on the x-axis.
- **r** is a non-negative integer radius of its gravitational influence. A point **x** on the x-axis is **unsafe** for a space traveler if it lies within the gravitational influence of at least one planet: [|x - c_i| \le r_i \quad \text{for some planet } i] Equivalently, each planet makes the interval ([c_i - r_i,\; c_i + r_i]) unsafe. Task:
- Consider the entire real line as the possible travel path.
- Compute all **maximal continuous intervals** on this line where the traveler is **safe**, i.e., points not covered by any planet's unsafe interval.
- Return these safe intervals as a sorted list of disjoint intervals. Representation details:
- Represent each safe interval as a pair **`(start, end)`** with **`start < end`**.
- Intervals may be unbounded on the left or right:
- Use **`-inf`** for an interval unbounded to the left (e.g., **`(-inf, a)`**).
- Use **`+inf`** for an interval unbounded to the right (e.g., **`(b, +inf)`**). You may assume the input list of planets can be in any order and may contain overlapping or nested gravitational ranges. Return the safe intervals in increasing order of **`start`**.

You are planning a route along the real x-axis. Each planet is given as a pair (c, r), where c is its center and r is a non-negative radius of gravitational influence. A point x is unsafe if it lies inside at least one planet's influence, meaning |x - c| <= r. So each planet creates an unsafe closed interval [c - r, c + r]. Your task is to return all maximal continuous safe intervals on the real line: regions not covered by any unsafe interval. Represent each safe interval as a tuple (start, end), where the interval means start < x < end. Because JSON/Python literals do not have a direct infinity literal, use None to represent an unbounded side:

- (None, a) means (-infinity, a)
- (b, None) means (b, +infinity)
- (None, None) means the entire real line Return the safe intervals in increasing order. Overlapping, nested, and touching unsafe intervals should be treated as one merged unsafe region.

Constraints
0 <= len(planets) <= 200000-10^9 <= c <= 10^90 <= r <= 10^9

EXAMPLES

```
Input: []
Expected Output: [(None, None)]
```

Explanation: With no planets, no point is unsafe, so the entire real line is safe.

```
Input: [(3, 1), (10, 2)]
Expected Output: [(None, 2), (4, 8), (12, None)]
Explanation: The unsafe intervals are [2, 4] and [8, 12]. Their complement is (-infinity, 2), (4, 8),
    and (12, +infinity).
```

Hints

First convert every planet (c, r) into the unsafe interval [c - r, c + r].Sort the unsafe intervals and merge all overlapping or touching ones. The safe intervals are the gaps before, between, and after the merged intervals.

WHY IT WORKS

Sort the unsafe intervals by start and merge any that overlap or touch (they share a point, which is unsafe). Then the safe intervals are (-inf, first start), each gap (end of one merged interval, start of the next), and (last end, +inf). With no planets the whole line is safe.

SHORT OPTIMAL CODE

```
INFINITY = float("inf")


# BASE QUESTION (optimal): Merge the unsafe intervals,
# then read off the gaps
# Idea: Sort the unsafe intervals by start and merge any
# that overlap or touch (they share a point, which is
# unsafe). Then the safe intervals are (-inf, first
# start), each gap (end of one merged interval, start of
# the next), and (last end, +inf). With no planets the
# whole line is safe.
def safe_intervals(planets):
    merged = []
    for start, end in sorted((centre - radius, centre + radius) for centre, radius in planets):
        if merged and start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    if not merged:
        return [(-INFINITY, INFINITY)]
    safe = [(-INFINITY, merged[0][0])]
    for previous_interval, next_interval in zip(merged, merged[1:]):
        safe.append((previous_interval[1], next_interval[0]))
    safe.append((merged[-1][1], INFINITY))
    return safe
```

COMPLEXITY | Time O(n log n) | Space O(n)

136 Implement a function that processes a stream of interval data. HARD

Template move: the Range Module add: bisect, merge the block, splice in one interval; intervals() returns the pairs

QUESTION

Intervals arrive one at a time. add(start, end) receives a new interval, and intervals() returns the current list of disjoint intervals, sorted by start; intervals that overlap or touch are merged.

Function signature

class IntervalStream:
    def add(self, start: int, end: int) -> None
    def intervals(self) -> list

EXAMPLES

QUESTION

Example 1

Input	`s = IntervalStream()`
	`s.add(1, 3)`
	`s.add(6, 9)`
	`s.intervals()`
	`s.add(2, 5)`
	`s.intervals()`
	`s.add(5, 6)`
	`s.intervals()`

Output	`s.intervals() -> [(1, 3), (6, 9)]`
	`s.intervals() -> [(1, 5), (6, 9)]`
	`s.intervals() -> [(1, 9)]`

Why	`(2, 5) overlaps (1, 3) and extends it to (1, 5); (5, 6) touches (1, 5) at 5 and (6, 9) at 6, so everything merges into (1, 9).`

WHY IT WORKS

Keep the merged intervals sorted by start. A new interval only interacts with the intervals found by two binary searches; replace that slice by the single merged interval. Touching closed intervals merge.

SHORT OPTIMAL CODE

```python
import bisect


# BASE QUESTION (optimal): Sorted disjoint intervals with
# binary search
# Idea: Keep the merged intervals sorted by start. A new
# interval only interacts with the intervals found by two
# binary searches; replace that slice by the single merged
# interval. Touching closed intervals merge.
class IntervalStream:
    def __init__(self):
        self.starts = []
        self.ends = []

    def add(self, start, end):
        first_overlap = bisect.bisect_left(self.ends, start)           # first interval ending at or after star
t
        after_last_overlap = bisect.bisect_right(self.starts, end)          # first interval starting after end
        if first_overlap < after_last_overlap:
            start = min(start, self.starts[first_overlap])
            end = max(end, self.ends[after_last_overlap - 1])
        self.starts[first_overlap:after_last_overlap] = [start]
        self.ends[first_overlap:after_last_overlap] = [end]

    def intervals(self):
        return list(zip(self.starts, self.ends))
```

COMPLEXITY | Time O(log n) to find the spot, O(n) worst-case list shift | Space O(n)

Trie: nested dicts, one character per level

Pattern

Store words as nested dicts, one character per level, with an end marker that holds whatever you want to remember. Insert walks and creates; search walks and fails when a character is missing; prefix queries stop in the middle; longest-match keeps the last end marker you passed.

137 URL Request Handler Router

MEDIUM

Template move: one trie level per URL segment; a node has literal children and one wildcard child; try the literal first

QUESTION

Design a request router. add(pattern) registers a URL pattern whose segments are either literal text or a placeholder written {name}; match(url) returns the registered pattern that handles the URL, or None. When a URL matches both a literal segment and a placeholder, the literal wins.

Function signature

class Router:
    def add(self, pattern: str) -> None
    def match(self, url: str) -> str | None

EXAMPLES

Example 1

Input	`r = Router()`
	`r.add('/users/{userId}/photo/{photoId}')`
	`r.add('/users/me/photo/{photoId}')`
	`r.match('/users/123/photo/456')`
	`r.match('/users/me/photo/9')`
	`r.match('/orders/1')`

Output	`r.match('/users/123/photo/456') -> '/users/{userId}/photo/{photoId}'`
	`r.match('/users/me/photo/9') -> '/users/me/photo/{photoId}'`
	`r.match('/orders/1') -> null`

Why	`The placeholder pattern handles any user id, but for /users/me/... the literal 'me' pattern wins; an unknown path matches nothing.`

WHY IT WORKS

Each trie node has literal children and one wildcard child; the pattern is stored at its last node. Lookup tries the literal child first, and if that whole branch fails it backs up and tries the wildcard child. A match must consume every segment and end on a node that stores a pattern.

SHORT OPTIMAL CODE

```
def _segments(path):
    return [segment for segment in path.strip("/").split("/") if segment != ""]


# BASE QUESTION (optimal): Segment trie with literal-first
# fallback
# Idea: Each trie node has literal children and one
# wildcard child; the pattern is stored at its last node.
# Lookup tries the literal child first, and if that whole
# branch fails it backs up and tries the wildcard child. A
# match must consume every segment and end on a node that
# stores a pattern.
class Router:
```

```
    def __init__(self, patterns=()):
        self.root = {"lit": {}, "wild": None, "handler": None}
        for pattern in patterns:
            self.add(pattern)

    def add(self, pattern):
        node = self.root
        for segment in _segments(pattern):
            if segment.startswith("{") and segment.endswith("}"):
                if node["wild"] is None:
                    node["wild"] = {"lit": {}, "wild": None, "handler": None}
                node = node["wild"]
            else:
                node = node["lit"].setdefault(segment, {"lit": {}, "wild": None, "handler": None})
        if node["handler"] is None:
            node["handler"] = pattern

    def match(self, url):
        url_segments = _segments(url)

        def walk(node, position):
            if position == len(url_segments):
                return node["handler"]
            child = node["lit"].get(url_segments[position])
            if child is not None:
                found = walk(child, position + 1)
                if found is not None:
                    return found
            if node["wild"] is not None:
                return walk(node["wild"], position + 1)
            return None

        return walk(self.root, 0)
```

COMPLEXITY | Time O(L * branching) per lookup, O(total pattern length) to build | Space O(total pattern length)

138 Trie Prefix Search / Autocomplete — MEDIUM

Template move: walk to the prefix node; return the words under it (autocomplete)

QUESTION

Build a trie from a list of words. search_prefix(prefix) returns every word that starts with the prefix, sorted alphabetically (empty list when none).

Function signature

class Trie:
  def insert(self, word: str) -> None
  def search_prefix(self, prefix: str) -> list

EXAMPLES

Example 1	

Input	`t = Trie(['car', 'card', 'care', 'cat', 'dog'])`
	`t.search_prefix('car')`
	`t.search_prefix('ca')`
	`t.search_prefix('x')`

Output	`t.search_prefix('car') -> ['car', 'card', 'care']`
	`t.search_prefix('ca') -> ['car', 'card', 'care', 'cat']`
	`t.search_prefix('x') -> []`

Why	`'car' is itself a word that starts with 'car'; the prefix 'x' matches nothing.`

WHY IT WORKS

Insert each word, marking the last node. For a query walk the prefix; if a step is missing return an empty list. From the prefix node run a DFS that visits children in alphabetical order and records the word at every end node, which yields sorted words.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Trie with a DFS over sorted
# children
# Idea: Insert each word, marking the last node. For a
# query walk the prefix; if a step is missing return an
# empty list. From the prefix node run a DFS that visits
# children in alphabetical order and records the word at
# every end node, which yields sorted words.
class Trie:
    def __init__(self, words=()):
        self.root = {}
        for word in words:
            self.insert(word)

    def insert(self, word):
        node = self.root
        for character in word:
            node = node.setdefault(character, {})
        node["$"] = True

    def search_prefix(self, prefix):
        node = self.root
        for character in prefix:
            if character not in node:
                return []
            node = node[character]
        words = []
        def dfs(node, path):
            if "$" in node:
                words.append("".join(path))
            for character in sorted(key for key in node if key != "$"):
                path.append(character)
                dfs(node[character], path)
                path.pop()
        dfs(node, list(prefix))
        return words
```

COMPLEXITY | Time O(L + output) per query after O(total characters) to build | Space O(total characters)

139 Build Prefix Lookup with a Trie — MEDIUM

Template move: build_trie, then walk to the prefix node and collect every word below it

QUESTION

Build a trie from a list of words. completions(prefix) returns every inserted word that starts with the prefix (including the prefix itself when it is a word), sorted alphabetically, or an empty list when none does.

Function signature

class PrefixTrie:
    def insert(self, word: str) -> None
    def completions(self, prefix: str) -> list

EXAMPLES

QUESTION

Example 1

Input	`t = PrefixTrie(['apple', 'app', 'apply', 'apt', 'bat'])`
	`t.completions('app')`
	`t.completions('ap')`
	`t.completions('z')`

Output	`t.completions('app') -> ['app', 'apple', 'apply']`
	`t.completions('ap') -> ['app', 'apple', 'apply', 'apt']`
	`t.completions('z') -> []`

Why	`'app' is itself a word and a prefix of two more; the prefix 'z' matches nothing.`

WHY IT WORKS

Insert every word into nested dicts, marking word ends. To answer a prefix, walk down one character at a time (no match means no completions), then DFS the subtree collecting words.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Trie with a DFS below the
# prefix node
# Idea: Insert every word into nested dicts, marking word
# ends. To answer a prefix, walk down one character at a
# time (no match means no completions), then DFS the
# subtree collecting words.
class PrefixTrie:
    END = "$"

    def __init__(self, words=()):
        self.root = {}
        for word in words:
            self.insert(word)

    def insert(self, word):
        node = self.root
        for character in word:
            node = node.setdefault(character, {})
        node[self.END] = True

    def completions(self, prefix):
        node = self.root
        for character in prefix:
            if character not in node:
                return []
            node = node[character]
        words, stack = [], [(node, prefix)]
        while stack:
            current_node, text = stack.pop()
            if self.END in current_node:
                words.append(text)
            for character, child in current_node.items():
                if character != self.END:
                    stack.append((child, text + character))
        return sorted(words)
```

COMPLEXITY | Time O(L) to walk the prefix + O(size of the answer) | Space O(total characters)

140 Greedy Longest-Match Dictionary Tokenizer

MEDIUM

Template move: the same longest-match walk, with a priority rule to choose between equal-length matches

QUESTION

Phone-screen coding round (R1). Tokenize a text against a dictionary of **`key:id`** entries using greedy longest-match; emit the matched id when a key matches, otherwise output the unmatched character as a literal. One follow-up on changing the tie-breaking / priority rule for which token wins. A second follow-up appears when the dictionary is very large: discuss how to keep the matcher memory- and lookup-efficient at scale.

Input: a text string and a dictionary mapping key (string) -> id.
Sweep the text left-to-right. At each position, find the longest dictionary key that matches a prefix of the remaining text; emit its id. If no key matches at the current position, emit the single character as a literal and advance by one.
Output the concatenation (or list) of ids and literal characters in order.
Follow-up: the interviewer changes the tie-breaking / priority rule for which key wins when multiple match (e.g., not longest but highest-priority by dictionary order). Adapt the data structure so the rule swap is cheap.
Follow-up: the dictionary becomes very large. Discuss how to keep memory and per-position lookup efficient — a compressed / radix trie (or sharding the key space) instead of holding every key in a flat map.

EXAMPLES

Example 1

Input	`t = Tokenizer({'ab': 1, 'abc': 2, 'c': 3})`
	`t.tokenize('abcab')`
	`t.tokenize('abx')`

Output	`t.tokenize('abcab') -> [2, 1]` `t.tokenize('abx') -> [1, 'x']`
Why	`At each position the longest dictionary key wins: 'abc' (not 'ab' followed by 'c'), then 'ab'. A character that starts no key is emitted as it is.`

WHY IT WORKS

Insert every key into a trie with its id at the last node. From each text position walk down while the next character exists, remembering the last terminal passed. Emit its id and jump past its length, or emit the single character if no terminal was found. For a priority rule keep the terminal with the best priority instead of the last one.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Trie walk keeping the deepest
# (or best-priority) terminal
# Idea: Insert every key into a trie with its id at the
# last node. From each text position walk down while the
# next character exists, remembering the last terminal
# passed. Emit its id and jump past its length, or emit
# the single character if no terminal was found. For a
# priority rule keep the terminal with the best priority
# instead of the last one.
def tokenize_trie(text, table, priority=None):
    root = {}
    for key, token_id in table.items():
        node = root
        for character in key:
            node = node.setdefault(character, {})
        node["$"] = (key, token_id)
    tokens, start = [], 0
    while start < len(text):
        node, best_match, end = root, None, start
        while end < len(text) and text[end] in node:
            node = node[text[end]]
            end += 1
            if "$" in node and (best_match is None or priority is None or priority(node["$"][0]) < priority(bes
t_match[0])):
```

```
                best_match = node["$"]
        if best_match is None:
            tokens.append(text[start])
            start += 1
        else:
            tokens.append(best_match[1])
            start += len(best_match[0])
    return tokens
```

COMPLEXITY | Time O(n * L) | Space O(total key length)

141 Check if all substrings are dictionary words — MEDIUM

Template move: for every start walk the trie; a substring of length 3 or more that leaves the trie or lacks END fails the check

QUESTION

You are given a string **`s`** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **`s`** with length **`>= 3`** is a valid dictionary word (i.e., for all **`i, j`** with **`0 <= i <= j < n`** and **`j - i + 1 >= 3`**, **`s[i..j] in dict`**). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**.
- You may assume dictionary lookups are available (e.g., **`dict.contains(word)`**), or you can preprocess the dictionary. Example:
- If **`s = "cats"`**, you must check **`"cat"`**, **`"ats"`**, and **`"cats"`** (and any other substrings of length >= 3).

EXAMPLES

Example 1	
Input	`'abcd', {'abc', 'abcd', 'bcd'}`
Output	`true`
Why	`Every substring of length 3 or more (abc, bcd, abcd) is in the dictionary.`

Example 2	
Input	`'abcd', {'abc', 'abcd'}`
Output	`false`
Why	`'bcd' is missing from the dictionary.`

WHY IT WORKS

For each start i descend the trie along s[i:]. At depth 1 and 2 only walk; from depth 3 on the current node must be marked as a word end. If the walk leaves the trie (and at least three characters remain from this start) or hits a non-word node at depth >= 3, return False immediately. No substring is built or rehashed.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Walk a trie from each start
# position
# Idea: For each start i descend the trie along text[start:]. At
# depth 1 and 2 only walk; from depth 3 on the current
# node must be marked as a word end. If the walk leaves
# the trie (and at least three characters remain from this
# start) or hits a non-word node at depth >= 3, return
# False immediately. No substring is built or rehashed.
def all_substrings_words(text, words):
    root = {}
    for word in words:
        node = root
        for character in word:
            node = node.setdefault(character, {})
```

```
        node["$"] = True
    for start in range(len(text)):
        node = root
        for depth, character in enumerate(text[start:], 1):
            node = node.get(character)
            if node is None:
                return len(text) - start < 3                # a start with 3 or more characters left has a subs
tring that is not a word
            if depth >= 3 and "$" not in node:
                return False
    return True
```

COMPLEXITY | Time O(n^2) | Space O(dictionary size)

142 Return dictionary words matching a prefix — MEDIUM

Template move: no trie needed: sort the words and bisect to the first match, then read forward while startswith(prefix)

QUESTION

You are given a dictionary of words. Return all words that start with a given prefix. Matching is case-sensitive, duplicates in the dictionary are returned as often as they occur, and the result is an empty list when nothing matches. The index returns the matches in alphabetical order; limit (optional) stops after that many words.

Function signature

class PrefixIndex:
  def __init__(self, words: list)
  def matches(self, prefix: str, limit=None) -> list

EXAMPLES

Example 1

Input	`idx = PrefixIndex(['apple', 'app', 'Apple', 'apply', 'apple', 'bat'])`
	`idx.matches('app')`
	`idx.matches('app', limit=2)`
	`idx.matches('cat')`

Output	`idx.matches('app') -> ['app', 'apple', 'apple', 'apply']`
	`idx.matches('app', limit=2) -> ['app', 'apple']`
	`idx.matches('cat') -> []`

Why	`'Apple' starts with a capital letter so it does not match 'app'; 'apple' is listed twice because it is in the dictionary twice.`

WHY IT WORKS

With the words sorted, the matches are the words from bisect_left(prefix) until the first word that no longer starts with the prefix. The upper edge can be found by a second bisect with prefix + a character larger than any other, or by walking forward while startswith holds.

SHORT OPTIMAL CODE

```
from bisect import bisect_left


# BASE QUESTION (optimal): Sort once, binary search the
# range
# Idea: With the words sorted, the matches are the words
# from bisect_left(prefix) until the first word that no
# longer starts with the prefix. The upper edge can be
# found by a second bisect with prefix + a character
# larger than any other, or by walking forward while
# startswith holds.
class PrefixIndex:
```

```
    def __init__(self, words):
        self.words = sorted(words)

    def matches(self, prefix, limit=None):
        position = bisect_left(self.words, prefix)
        matches = []
        while position < len(self.words) and self.words[position].startswith(prefix):
            matches.append(self.words[position])
            if limit is not None and len(matches) == limit:
                break
            position += 1
        return matches
```

COMPLEXITY | Time O(log W + m) per query after O(W log W) sorting | Space O(W)

143 Implement Trie Insert and Exact-Word Search — MEDIUM

Template move: insert is build_trie; search is walk and then END in node

QUESTION

Implement a trie with insert(word), search(word) and startsWith(prefix). search is true only when the complete word was inserted earlier (a longer stored word that merely starts with it does not count); startsWith is true when some inserted word begins with the prefix.

Function signature

class Trie:
  def insert(self, word: str) -> None
  def search(self, word: str) -> bool
  def startsWith(self, prefix: str) -> bool

EXAMPLES

Example 1

Input	`t = Trie()`
	`t.insert('apple')`
	`t.search('apple')`
	`t.search('app')`
	`t.startsWith('app')`
	`t.insert('app')`
	`t.search('app')`

Output	`t.search('apple') -> true`
	`t.search('app') -> false`
	`t.startsWith('app') -> true`
	`t.search('app') -> true`

Why	`'app' is only a prefix of 'apple' until it is inserted on its own.`

WHY IT WORKS

Each node has children by character and a flag. insert creates missing nodes and marks the last one. search walks the word and requires the flag at the end; startsWith only requires that the walk succeeds.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Trie of nodes with an end-of-
# word flag
# Idea: Each node has children by character and a flag.
# insert creates missing nodes and marks the last one.
# search walks the word and requires the flag at the end;
# startsWith only requires that the walk succeeds.
class Trie:
```

```
    def __init__(self):
        self.root = {"children": {}, "end": False}

    def _walk(self, text):
        node = self.root
        for character in text:
            node = node["children"].get(character)
            if node is None:
                return None
        return node

    def insert(self, word):
        node = self.root
        for character in word:
            node = node["children"].setdefault(character, {"children": {}, "end": False})
        node["end"] = True

    def search(self, word):
        node = self._walk(word)
        return node is not None and node["end"]

    def startsWith(self, prefix):
        node = self._walk(prefix)
        return node is not None and (node["end"] or bool(node["children"]))     # a node counts only if some wo
rd passes through it
```

COMPLEXITY | Time O(L) per operation | Space O(total characters)

144 Implement Longest-Match Text Replacement

HARD

Template move: at each text position walk the trie as far as it goes and remember the last END; replace the longest match and jump past it

QUESTION

You are given a text string and a dictionary of replacement entries. Each dictionary entry has the form **`token:id`**, where **`token`** is a string pattern and **`id`** is the value that should be emitted when that token is matched. Implement a function that scans the text from left to right and produces an output sequence using greedy longest-prefix matching:

1. At the current text position, find all dictionary tokens that match the substring starting at that position.
2. If at least one token matches, choose the longest matching token, append its corresponding **`id`** to the output, and advance by the length of that token.
3. If no token matches, append the original character at the current position to the output and advance by one character.
4. Continue until the entire text has been processed. Example:

```
text = "abcdxyz" dictionary = { "ab": "1", "abc": "2", "xy": "3" }
```

Output:

```
["2", "d", "3", "z"]
```

Explanation: At index **`0`**, both **`"ab"`** and **`"abc"`** match, so choose the longest token **`"abc"`** and output **`"2"`**. Then **`"d"`** has no match, so output it literally. Then **`"xy"`** matches and outputs **`"3"`**. Finally, **`"z"`** is output literally. Follow-up: How would you adapt your design if the matching priority changed from longest match to a configurable priority rule, such as dictionary insertion order, highest score, or a custom comparator?

Part 1: Longest-Match Text ReplacementGiven a text string and a dictionary that maps token strings to replacement id strings, scan the text from left to right. At each position:

1. Find every token that matches the substring starting at that position.
2. If at least one token matches, choose the longest matching token, append its replacement id to the output, and advance by the token length.

QUESTION

3. If no token matches, append the original character at that position as a one-character string and advance by 1. Return the final output as a list of strings.

Constraints

0 <= len(text) <= 1000000 <= len(dictionary) <= 10000Each token in dictionary is non-empty.The sum of all token lengths is at most 100000.Matching is case-sensitive.

EXAMPLES

```
Input: ("abcdxyz", {"ab": "1", "abc": "2", "xy": "3"})
Expected Output: ["2", "d", "3", "z"]
```

Explanation: Both 'ab' and 'abc' match at index 0, so the longer token 'abc' is chosen.

```
Input: ("aaaaa", {"a": "X", "aa": "Y"})
Expected Output: ["Y", "Y", "X"]
```

Explanation: Greedy longest matching picks 'aa' twice, then the final 'a'.

Hints

A trie lets you test all token prefixes starting at one position without checking every token from scratch.While walking forward from a position, remember the last trie node that represented a complete token.

Part 2: Configurable-Priority Text ReplacementImplement the same left-to-right text replacement process, but the match-selection rule is configurable. You are given: - a text string - a list of replacement entries in insertion order - a priority rule Each replacement entry is a tuple: (token, replacement_id, score). At each text position, find all entries whose token matches the substring starting there. If one or more entries match, choose exactly one using the selected priority rule:

- 'longest': choose the longest matching token; if tied, choose the earlier entry in the list - 'insertion': choose the earliest matching entry in the input list - 'score': choose the matching entry with the highest score; if tied, choose the longer token; if still tied, choose the earlier entry in the list Append the chosen replacement_id to the output and advance by the chosen token length. If nothing matches, append the original character and advance by 1. Return the final output as a list of strings.

Constraints

0 <= len(text) <= 1000000 <= len(entries) <= 10000Each token is non-empty.Tokens in entries are unique.The sum of all token lengths i

WHY IT WORKS

Build a trie of the tokens (each token's last node stores its id). At position i walk down the trie along the text; whenever the current node stores an id, record it and its length. When the walk cannot continue, use the last recorded token (advance by its length), or emit the character and advance by one.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Trie walk that remembers the
# last token end
# Idea: Build a trie of the tokens (each token's last node
# stores its id). At position i walk down the trie along
# the text; whenever the current node stores an id, record
# it and its length. When the walk cannot continue, use
# the last recorded token (advance by its length), or emit
# the character and advance by one.
def replace_longest(text, mapping):
    root = {}
    for token, token_id in mapping.items():
        node = root
        for character in token:
            node = node.setdefault(character, {})
        node["$"] = (token_id, len(token))
    output, position = [], 0
```

```
    while position < len(text):
        node, scan_position, longest_match = root, position, None
        while scan_position < len(text) and text[scan_position] in node:
            node = node[text[scan_position]]
            scan_position += 1
            if "$" in node:
                longest_match = node["$"]
        if longest_match:
            output.append(longest_match[0])
            position += longest_match[1]
        else:
            output.append(text[position])
            position += 1
    return output
```

COMPLEXITY | Time O(len(text) * longest token) | Space O(total token length)

GROUP 18 OF 43 | 8 QUESTIONS | SHORT MAIN CODE

A dict for lookup plus an ordered container for who is next (lazy deletion)

Pattern

Keep a dict for O(1) lookup and a deque, list or heap for the order. When something is removed, delete it from the dict only and skip stale entries when you read from the order. LRU caches, waitlists, undo stacks and schedulers all fit.

145 Implement Batched Undo/Redo Layer — EASY

Template move: two stacks (undo, redo) of batches; a batch records (old value, new value) per property; undo applies the old values and moves the batch

QUESTION

You are implementing a simplified document layer for a design tool. The layer stores properties as a **`map<string, string>`**. Implement a class that supports the following operations:

- **`beginBatch()`**: start a batch of edits.
- **`setProperty(key, value)`**: set or overwrite a property inside the current batch.
- **`endBatch()`**: commit the batch as one undoable action.
- **`undo()`**: revert the most recently committed batch.
- **`redo()`**: reapply the most recently undone batch. Requirements:

1. A batch may update the same key multiple times.
2. **`undo()`** must restore the exact document state from before that batch.
3. **`redo()`** must restore the exact document state after that batch.
4. If a property did not exist before the batch, then **`undo()`** should remove that property.
5. If **`undo()`** has been called and then a new batch is committed, the redo history must be cleared.
6. Follow-up: make the history memory-efficient. Instead of storing every intermediate edit in a batch, compact the batch so that each key appears at most once in history while still preserving enough information for correct **`undo()`** and **`redo()`**. A useful compact record for one key is:

```
Change { string key; string value; // final value after the batch string old_value; // value before the ba
tch bool has_old_value; // whether the key existed before the batch }
```

Explain how you would build and use such a compact history so that repeated updates to the same key inside one batch do not waste memory.

Part 1: Implement Batched Undo/Redo for Document PropertiesYou are given a sequence of operations for an initially empty document represented as a map<string, string>. A batch starts with ('begin',), contains zero or more ('set', key, value) operations, and ends with ('end',). Each committed batch must be undoable and redoable as one unit. Operation ('snapshot',) asks you to append the current document to the answer; snapshots may appear while a batch is open and should reflect the live document state. If a key is set multiple times in one batch, undo must restore the value from before the batch began. If a key did not exist before the batch, undo must remove it. If a new non-empty batch is committed after an undo, the redo history is cleared. If undo or redo is impossible, do nothing. If end closes an empty batch, treat it as a no-op.

Constraints

0 <= len(operations) <= 200000The total number of ('set', key, value) operations is at most 200000Keys and values are strings of length 1 to 50Batches are not nestedIf a batch contains no set operations, committing it does not create history

EXAMPLES

```
Input: [('begin',), ('set', 'color', 'red'), ('end',), ('snapshot',), ('undo',), ('snapshot',),
    ('redo',), ('snapshot',)]
Expected Output: [{'color': 'red'}, {}, {'color': 'red'}]
```

QUESTION

Explanation: A single batch is committed, undone, and redone.

```
Input: [('begin',), ('set', 'a', '1'), ('end',), ('begin',), ('set', 'a', '2'), ('set', 'a', '3'),
    ('set', 'b', '9'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]
Expected Output: [{'a': '3', 'b': '9'}, {'a': '1'}, {'a': '3', 'b': '9'}]
```

Explanation: The second batch updates 'a' twice and adds 'b'. Undo must restore the exact pre-batch state.

Hints

For each key touched in the current batch, record its old value only the first time that key is edited in the batch.Use two stacks: one for committed batches that can be undone, and one for undone batches that can be redone. Clear the redo stack when a new non-empty batch is committed.

Part 2: Compact a Batch into Memory-Efficient Change RecordsYou are given the document state before a batch and the raw setProperty edits that happened inside that batch. Multiple edits may target the same key. Build a compact history so that each key appears at most once, while still storing enough informa

WHY IT WORKS

While a batch is open keep {key: (old, new)} with the old value recorded only the first time a key is touched. endBatch pushes it on the undo stack and clears the redo stack. undo writes the old values back (deleting keys that did not exist before) and moves the batch to redo; redo writes the new values. The document is never copied.

SHORT OPTIMAL CODE

```python
_MISSING = object()


# BASE QUESTION (optimal): Store only the changed keys per
# batch
# Idea: While a batch is open keep {key: (old, new)} with
# the old value recorded only the first time a key is
# touched. endBatch pushes it on the undo stack and clears
# the redo stack. undo writes the old values back
# (deleting keys that did not exist before) and moves the
# batch to redo; redo writes the new values. The document
# is never copied.
class Layer:
    def __init__(self):
        self.properties, self.undo_stack, self.redo_stack, self.open_batch = {}, [], [], None

    def beginBatch(self):
        self.open_batch = {}

    def setProperty(self, name, value):
        if self.open_batch is None:
            self.beginBatch()
            self.setProperty(name, value)
            self.endBatch()
            return
        old = self.open_batch[name][0] if name in self.open_batch else self.properties.get(name, _MISSING)
        self.open_batch[name] = (old, value)
        self.properties[name] = value

    def endBatch(self):
        if self.open_batch:
            self.undo_stack.append(self.open_batch)
            self.redo_stack.clear()
        self.open_batch = None

    def _apply(self, batch, use_new):
        for name, (old, new) in batch.items():
            value = new if use_new else old
            if value is _MISSING:
```

```
                self.properties.pop(name, None)
            else:
                self.properties[name] = value

    def undo(self):
        if self.undo_stack:
            batch = self.undo_stack.pop()
            self._apply(batch, False)
            self.redo_stack.append(batch)

    def redo(self):
        if self.redo_stack:
            batch = self.redo_stack.pop()
            self._apply(batch, True)
            self.undo_stack.append(batch)
```

COMPLEXITY | Time O(keys changed in the batch) per operation | Space O(total changed keys)

146 Design a restaurant waitlist system — MEDIUM

Template move: a segment tree of party sizes gives the first entry that fits in O(log n); the dict maps ids to slots

QUESTION

You are implementing the waitlist system for a restaurant. Parties arrive over time, can cancel, and get seated when a table becomes available.

Rules

- Each party has a unique **`partyId`**, a **`name`**, and a **`size`** (number of people).
- Parties are seated in **arrival order**, but only if they **fit** the available table.
- When a table of capacity **c** becomes available, you must seat the **earliest-arrived** party whose **`size <= c`**.
- If no party fits, the table remains unused for that event.

Operations to support Design a

data structure / class that supports the following operations efficiently:

1. **`addParty(name, size) -> partyId`**
- Adds a new party to the end of the waitlist and returns its id.
2. **`cancelParty(partyId) -> bool`**
- Removes the party from the waitlist if present.
- Returns **`true`** if removed, otherwise **`false`**.
3. **`seatTable(capacity) -> partyId or null`**
- Finds and removes the earliest party with **`size <= capacity`**.
- Returns that party's id, or **`null`** if nobody fits.
4. **`getPosition(partyId) -> int`**
- Returns how many parties are ahead of this party in the current waitlist order (0-based).
- If the party is not in the waitlist, return **`-1`**.

Constraints

- Up to **`2 * 10^5`** operations.
- Party size is a small positive integer (e.g., 1–20).
- Aim for better-than-linear time per operation (especially for **`seatTable`** and **`getPosition`**).

Example

(one possible interaction) - addParty("A", 4) -> id1 - addParty("B", 2) -> id2 - seatTable(2) -> id2 (B fits and is earliest among those that fit) - seatTable(4) -> id1

QUESTION

EXAMPLES

Example 1

Input	`w = Waitlist()`
	`w.add(1, 'ann', 4)`
	`w.add(2, 'bob', 2)`
	`w.add(3, 'cy', 3)`
	`w.seat(3)`
	`w.cancel(3)`
	`w.seat(3)`
	`w.seat(4)`

Output	`w.seat(3) -> 2`
	`w.cancel(3) -> true`
	`w.seat(3) -> null`
	`w.seat(4) -> 1`

Why	A table for 3 takes the earliest party that fits: bob (2), not ann (4). After cy cancels, nobody left fits a table for 3; a table for 4 seats ann.

WHY IT WORKS

Each arrival gets the next position. The tree stores, for each range, the smallest waiting party size (infinity for empty or seated slots). seat(c) descends from the root: go left if the left range's minimum is at most c, else right, ending at the leftmost fitting party. Cancel and seat set a leaf to infinity and recompute the path up. The tree has fixed capacity here; double and rebuild to grow.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Segment tree of minimum size
# over arrival order
# Idea: Each arrival gets the next position. The tree
# stores, for each range, the smallest waiting party size
# (infinity for empty or seated slots). seat(c) descends
# from the root: go left if the left range's minimum is at
# most c, else right, ending at the leftmost fitting
# party. Cancel and seat set a leaf to infinity and
# recompute the path up. The tree has fixed capacity here;
# double and rebuild to grow.
class Waitlist:
    INF = float("inf")

    def __init__(self, capacity=1024):
        self.leaf_count = capacity
        self.tree = [self.INF] * (2 * capacity)
        self.arrivals, self.slot_of, self.party_at = 0, {}, {}

    def _set(self, node_index, value):
        node_index += self.leaf_count
        self.tree[node_index] = value
        while node_index > 1:
            node_index //= 2
            self.tree[node_index] = min(self.tree[2 * node_index], self.tree[2 * node_index + 1])

    def add(self, party_id, name, size):
        self.slot_of[party_id] = self.arrivals
        self.party_at[self.arrivals] = party_id
        self._set(self.arrivals, size)
        self.arrivals += 1

    def cancel(self, party_id):
        if party_id not in self.slot_of or self.tree[self.slot_of[party_id] + self.leaf_count] == self.INF:
            return False
        self._set(self.slot_of[party_id], self.INF)
```

```
        return True

    def seat(self, capacity):
        if self.tree[1] > capacity:
            return None
        node_index = 1
        while node_index < self.leaf_count:
            node_index = 2 * node_index if self.tree[2 * node_index] <= capacity else 2 * node_index + 1
        self._set(node_index - self.leaf_count, self.INF)
        return self.party_at[node_index - self.leaf_count]
```

COMPLEXITY | Time O(log n) per operation | Space O(n)

147 Implement Python's itertools.tee — MEDIUM

Template move: one deque per consumer; whoever runs out pulls the next value from the source and appends it to every queue

QUESTION

Implement itertools.tee: given an iterable and n, return n independent iterators that each produce the same values from the one source. The source must be read only once, and a value must be kept only until every iterator has taken it.

Function signature

def tee_queues(iterable, n=2) -> tuple

EXAMPLES

Example 1

Input	`a, b = tee_queues([1, 2, 3])`
	`next(a)`
	`next(a)`
	`next(b)`
	`list(b)`
	`list(a)`

Output	`next(a) -> 1`
	`next(a) -> 2`
	`next(b) -> 1`
	`list(b) -> [2, 3]`
	`list(a) -> [3]`

Why	Iterator a runs ahead by two values; b still sees every value from the start, and neither disturbs the other.

WHY IT WORKS

When a consumer's deque is empty it pulls the next item from the source and appends it to every consumer's deque, then pops its own. Each item is read from the source exactly once and is dropped by a consumer after being read, so memory is only the gap between the leader and the slowest follower.

SHORT OPTIMAL CODE

```
from collections import deque


# BASE QUESTION (optimal): One deque per consumer, filled
# lazily
# Idea: When a consumer's deque is empty it pulls the next
# item from the source and appends it to every consumer's
# deque, then pops its own. Each item is read from the
# source exactly once and is dropped by a consumer after
# being read, so memory is only the gap between the leader
# and the slowest follower.
def tee(iterable, copies=2):
```

```
    source_iterator = iter(iterable)
    queues = [deque() for _ in range(copies)]

    def consumer(own_queue):
        while True:
            if not own_queue:
                try:
                    value = next(source_iterator)
                except StopIteration:
                    return
                for queue in queues:
                    queue.append(value)
            yield own_queue.popleft()

    return tuple(consumer(queue) for queue in queues)
```

COMPLEXITY | Time O(n) total, O(1) amortized per item per consumer | Space O(lag between fastest and slowest consumer)

148 Restaurant Waitlist: join / delete / find_first_match — MEDIUM

Template move: a deque per party size with tickets; delete only removes the name from the dict; find_first_match skips dead tickets

QUESTION

Design a restaurant waitlist. join(name, size) adds a party with its size at the back. delete(name) removes a party (it may be anywhere in the queue). find_first_match(table) returns the name of the party that joined earliest among those whose size fits the table (size at most table), or None.

Function signature

class Waitlist:
  def join(self, name: str, size: int) -> None
  def delete(self, name: str) -> None
  def find_first_match(self, table: int) -> str | None

EXAMPLES

Example 1

Input	`w = Waitlist()`
	`w.join('ann', 4)`
	`w.join('bob', 2)`
	`w.join('cy', 3)`
	`w.find_first_match(3)`
	`w.delete('bob')`
	`w.find_first_match(3)`
	`w.find_first_match(1)`

Output	`w.find_first_match(3) -> 'bob'`
	`w.find_first_match(3) -> 'cy'`
	`w.find_first_match(1) -> null`

Why	A table for 3 skips ann (4 is too big) and takes bob, who joined before cy. After bob leaves it is cy. No party fits a table for 1.

WHY IT WORKS

join stores (sequence number, name) in the queue for its size and in a map from name to its record. delete just removes the record from the map. find_first_match, for each size up to the table size, pops dead entries from the front of its queue, then picks the live front with the smallest sequence number. No one is removed by find.

SHORT OPTIMAL CODE

```
from collections import defaultdict
from collections import deque
```

```python
# BASE QUESTION (optimal): Queue per party size with lazy
# deletion
# Idea: join stores (sequence number, name) in the queue
# for its size and in a map from name to its record.
# delete just removes the record from the map.
# find_first_match, for each size up to the table size,
# pops dead entries from the front of its queue, then
# picks the live front with the smallest sequence number.
# No one is removed by find.
class Waitlist:
    def __init__(self):
        self.last_number, self.live_number, self.queues = 0, {}, defaultdict(deque)

    def join(self, name, size):
        self.last_number += 1
        self.live_number[name] = self.last_number
        self.queues[size].append((self.last_number, name))

    def delete(self, name):
        self.live_number.pop(name, None)

    def find_first_match(self, table_size):
        oldest = None
        for size, queue_of_size in self.queues.items():
            if size > table_size:
                continue
            while queue_of_size and self.live_number.get(queue_of_size[0][1]) != queue_of_size[0][0]:
                queue_of_size.popleft()
            if queue_of_size and (oldest is None or queue_of_size[0][0] < oldest[0]):
                oldest = queue_of_size[0]
        return oldest[1] if oldest else None
```

COMPLEXITY | Time O(1) join and delete, O(S) find (S distinct sizes up to the table size) | Space O(n)

149 Store With Cheapest-Item Removal

MEDIUM

Template move: the order is a heap keyed by (price, timestamp); remove_cheapest pops it

QUESTION

Design a Store that holds items. Each item has a string value , an integer price in cents, and an integer timestamp . Support: add(value, price, timestamp) : insert an item. remove_cheapest() : remove and return the value of the item with the smallest price. If several items share the smallest price, remove the one with the smallest timestamp . Return None if the store is empty. A heap keyed by (price, timestamp) gives O(log n) per operation.

Function signature

```
class Store:
    def __init__(self) -> None:
        ...
    def add(self, value: str, price: int, timestamp: int) -> None:
        ...
    def remove_cheapest(self) -> str | None:
        """Remove and return the value of the lowest-priced item (ties broken by
earliest timestamp)."""
        ...
```

Follow-up (as reported): Improve remove_cheapest from O(log n) to O(1) amortized using the price bound: keep a length-10001 array indexed by price (each slot a queue/count of items) and track the smallest non-empty price index seen so far.

EXAMPLES

QUESTION

Example 1	
Input	`add("a",5,1); add("b",2,2); add("c",8,3); remove_cheapest()`
Output	`"b"`
Why	`"b" has the smallest price (2 cents).`

Example 2	
Input	`add("x",7,10); add("y",7,4); remove_cheapest()`
Output	`"y"`
Why	`Prices tie at 7; "y" wins on the earlier timestamp (4 < 10).`

Example 3	
Input	`remove_cheapest() # on an empty store`
Output	`None`

CONSTRAINTS

- Prices are integers with 1 <= price <= 10000 (i.e. 1 cent up to $100.00).
- Up to 10^6 operations.

WHY IT WORKS

The smallest (price, timestamp) is always at the top of the heap.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): A heap keyed by price and
# timestamp
# Idea: The smallest (price, timestamp) is always at the
# top of the heap.
import heapq


class Store:
    def __init__(self):
        self.heap = []

    def add(self, value, price, timestamp):
        heapq.heappush(self.heap, (price, timestamp, value))

    def remove_cheapest(self):
        if not self.heap:
            return None
        return heapq.heappop(self.heap)[2]
```

COMPLEXITY | Time O(log n) per operation | Space O(n)

150 Round-Robin Job Scheduler — MEDIUM

Template move: a deque of [job, time left]; pop, run for min(quantum, left); push back at the end if unfinished, else record the finish time

QUESTION

The third question was "not really leetcode style" but was a job-scheduling problem solved with a round- robin approach. Reconstruct it as follows. You are given a list of jobs, each with a unique string id and a positive integer burst (the number of time units of CPU it needs). All jobs are available at time 0 and are placed in a ready queue in the given order. The CPU runs jobs round-robin with a fixed time quantum : it takes the job at the front of the queue, runs it for min(quantum, remaining) time units, and then — if the job still has work left — puts it at the back of the queue; otherwise the job completes at that moment. Return the list of (id, completion_time) pairs in the order the jobs complete.

QUESTION

Function signature

def round_robin(jobs: list[tuple[str, int]], quantum: int) -> list[tuple[str, int]]:
    """Simulate round-robin scheduling; return (id, completion_time) pairs in completion order."""
    ...

Follow-up (as reported): Report note ("a constraint was changed"): add per-job arrival times so a job only enters the queue once the clock reaches its arrival. Hint: keep a pointer over arrival-sorted jobs and admit them each time the clock advances.

EXAMPLES

Example 1	
Input	`jobs = [("A", 4), ("B", 3), ("C", 2)], quantum = 2`
Output	`[("C", 6), ("A", 8), ("B", 9)]`
Why	`t=0-2 A (rem 2), t=2-4 B (rem 1), t=4-6 C completes at 6, t=6-8 A completes at 8, t=8-9 B completes at 9.`

Example 2	
Input	`jobs = [("X", 5)], quantum = 2`
Output	`[("X", 5)]`
Why	`One job, run in slices 2+2+1, finishing at time 5.`

Example 3	
Input	`jobs = [("P", 1), ("Q", 1)], quantum = 3`
Output	`[("P", 1), ("Q", 2)]`
Why	`Each finishes within a single quantum: P at 1, Q at 2.`

CONSTRAINTS

- 1 <= len(jobs) <= 10^4
- 1 <= burst <= 10^6 , 1 <= quantum <= 10^6
- Job ids are unique, non-empty strings.

WHY IT WORKS

Run the front job for one slice and send it to the back if it is not finished.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): A queue of jobs
# Idea: Run the front job for one slice and send it to the
# back if it is not finished.
from collections import deque


def round_robin(jobs, quantum):
    queue = deque([job_id, remaining_time] for job_id, remaining_time in jobs)
    clock, done = 0, []
    while queue:
        job = queue.popleft()
        time_slice = min(quantum, job[1])
        clock += time_slice
        job[1] -= time_slice
        if job[1] > 0:
            # not finished: back of the line
            queue.append(job)
        else:
```

```
            done.append((job[0], clock))
    return done
```

COMPLEXITY | Time O(total burst / quantum + n) | Space O(n)

151 Aggregate FizzBuzz Offer Matches

MEDIUM

Template move: a deque of offers per type; a new offer matches against the front of the opposite queue, partial quantities stay at the front

QUESTION

You are given a list of offers in priority order (earliest = highest priority). Each offer is a dict with keys type (either "Fizz" or "Buzz"), name (a string), and quantity (a positive integer). Fizz offers match against Buzz offers on a first-come, first-served basis, and offers may be partially fulfilled. Process the offers in order, maintaining a FIFO queue of unmatched Fizz offers and a FIFO queue of unmatched Buzz offers. When a new offer arrives, match it against the front of the opposite queue repeatedly, each time consuming min(remaining quantities) and recording a match (fizz_name, buzz_name, matched_quantity) ; drop an offer from the queue once its quantity hits 0. If the arriving offer still has quantity left when the opposite queue empties, enqueue it in its own queue. Aggregate matches so the same (fizz_name, buzz_name) pair is reported once with its summed quantity. Return the aggregated matches, in the order each pair was first produced, formatted as: "FizzBuzz! <quantity> <FizzName>:<BuzzName>" .

Function signature

def fizzbuzz_matches(offers: list[dict]) -> list[str]:

...

EXAMPLES

Example 1

Input	`offers = [ {"type": "Fizz", "name": "A", "quantity": 5}, {"type": "Buzz", "name": "X", "quantity": 3}, {"type": "Buzz", "name": "Y", "quantity": 4}, ]`
Output	`["FizzBuzz! 3 A:X", "FizzBuzz! 2 A:Y"]`
Why	`A(5) waits. X(3) matches A -> (A,X,3), A now 2. Y(4) matches A's remaining 2 -> (A,Y,2); Y's leftover 2 waits with no Fizz to match.`

Example 2

Input	`offers = [ {"type": "Fizz", "name": "A", "quantity": 3}, {"type": "Buzz", "name": "X", "quantity": 1}, {"type": "Buzz", "name": "X", "quantity": 2}, ]`
Output	`["FizzBuzz! 3 A:X"]`
Why	`A matches the first X for 1 and the second X for 2. Both are the pair (A, X), so they aggregate to a single "FizzBuzz! 3 A:X".`

CONSTRAINTS

- 0 <= len(offers) <= 10**4 ; quantities are positive integers.

WHY IT WORKS

Match against the opposite queue's front until the offer or the queue is empty.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Two FIFO queues and an ordered
# total
# Idea: Match against the opposite queue's front until the
# offer or the queue is empty.
from collections import deque


def fizzbuzz_matches(offers):
    queues = {"Fizz": deque(), "Buzz": deque()}
```

```
    totals = {}
    for offer in offers:
        mine = queues[offer["type"]]
        other = queues["Buzz" if offer["type"] == "Fizz" else "Fizz"]
        quantity_left = offer["quantity"]
        while quantity_left and other:
            front = other[0]
            matched = min(quantity_left, front[1])
            fizz_name = offer["name"] if offer["type"] == "Fizz" else front[0]
            buzz_name = front[0] if offer["type"] == "Fizz" else offer["name"]
            totals[(fizz_name, buzz_name)] = totals.get((fizz_name, buzz_name), 0) + matched
            quantity_left -= matched
            front[1] -= matched
            if front[1] == 0:
                other.popleft()
        if quantity_left:
            mine.append([offer["name"], quantity_left])
    return [f"FizzBuzz! {quantity} {fizz}:{buzz}" for (fizz, buzz), quantity in totals.items()]
```

COMPLEXITY | Time O(offers + matches) | Space O(offers)

152 Implement LRU Cache. HARD

Template move: dict key -> node plus a doubly linked list; every get/put moves the node to the front; evict from the back (or use OrderedDict.move_to_end)

QUESTION

Your program keeps re-reading the same data from disk, so you want an in-memory cache that holds only the n most recently used items (a least-recently-used cache). Write a class LRUCache(n) with get(key), which returns the stored value or -1, and put(key, value), which stores a value and, when the cache is full, first removes the item that was used least recently. Both get and put count as a use. Both must run in O(1).

Function signature

class LRUCache:
  def __init__(self, n: int)
  def get(self, key) -> int
  def put(self, key, value) -> None

EXAMPLES

Example 1

Input	`c = LRUCache(2)`
	`c.put(1, 1)`
	`c.put(2, 2)`
	`c.get(1)`
	`c.put(3, 3)`
	`c.get(2)`
	`c.put(4, 4)`
	`c.get(1)`
	`c.get(3)`
	`c.get(4)`

Output	`c.get(1) -> 1`
	`c.get(2) -> -1`
	`c.get(1) -> -1`
	`c.get(3) -> 3`
	`c.get(4) -> 4`

Why	`get(1) makes key 1 the most recent, so put(3, 3) evicts key 2; then put(4, 4) evicts key 1 (older than 3).`

PART 1: Hash map plus doubly linked list (built by hand)

Each key maps to a node of a doubly linked list ordered by recency, with sentinel head and tail nodes so no operation has to special-case an empty list. get moves the node to the front; put updates or inserts at the front and, if over capacity, removes the node just before the tail and its map entry.

PART 2: OrderedDict version

OrderedDict remembers insertion order and supports move_to_end and popitem(last=False) in O(1), so it is the hash map and the recency list in one. Good for a quick answer once you can explain what it does underneath.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Hash map plus doubly linked
# list (built by hand)
# Idea: Each key maps to a node of a doubly linked list
# ordered by recency, with sentinel head and tail nodes so
# no operation has to special-case an empty list. get
# moves the node to the front; put updates or inserts at
# the front and, if over capacity, removes the node just
# before the tail and its map entry.
class LRUCache:
    class _Node:
        __slots__ = ("key", "value", "prev", "next")

        def __init__(self, key=None, value=None):
            self.key, self.value, self.prev, self.next = key, value, None, None

    def __init__(self, capacity):
        self.max_size, self.node_by_key = capacity, {}
        self.head, self.tail = self._Node(), self._Node()
        self.head.next, self.tail.prev = self.tail, self.head

    def _remove(self, node):
        node.prev.next, node.next.prev = node.next, node.prev

    def _push_front(self, node):
        node.next, node.prev = self.head.next, self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        node = self.node_by_key.get(key)
        if node is None:
            return -1
        self._remove(node)
        self._push_front(node)
        return node.value

    def put(self, key, value):
        node = self.node_by_key.get(key)
        if node:
            node.value = value
            self._remove(node)
            self._push_front(node)
            return
        if len(self.node_by_key) == self.max_size:
            lru = self.tail.prev
            self._remove(lru)
            del self.node_by_key[lru.key]
        node = self._Node(key, value)
        self.node_by_key[key] = node
        self._push_front(node)
```

COMPLEXITY | Part 1: Time O(1) per operation | Space O(capacity)
Part 2: Time O(1) per operation | Space O(capacity)

GROUP 19 OF 43 | 7 QUESTIONS | SHORT MAIN CODE

Rolling state: the next state comes only from the previous one

Pattern

You never need the whole table, only the previous state: one number, a pair, or one row. Write the state, then one line that builds the new state from the old one, and loop.

153 Climbing Stairs — EASY

Template move: state = the last two counts: a, b = b, a + b, repeated n - 1 times

QUESTION

Given an integer n, representing the number of steps in a staircase, write a function climbStairs that returns the number of distinct ways to climb to the top of the staircase. Each time you can either climb 1 step or 2 steps. You may assume that n is a non-negative integer. Examples Input: n = 1 Output: 1 Explanation: There is only 1 way to climb a staircase with 1 step.

Input: n = 2 Output: 2 Explanation: There are 2 ways to climb a staircase with 2 steps:

1. Climb 1 step + 1 step 2. Climb 2 steps directly Input: n = 3 Output: 3 Explanation: There are 3 ways to climb a staircase with 3 steps:

1. Climb 1 step + 1 step + 1 step 2. Climb 1 step + 2 steps 3. Climb 2 steps + 1 step Input: n = 4 Output: 5 Explanation: There are 5 ways to climb a staircase with 4 steps:

1. Climb 1 step + 1 step + 1 step + 1 step 2. Climb 1 step + 1 step + 2 steps 3. Climb 1 step + 2 steps + 1 step 4. Climb 2 steps + 1 step + 1 step 5. Climb 2 steps + 2 stepsChallenge 1Can you come up with a solution that has a O(n) time complexity?Challenge 2Can you come up with a solution that has a O(1) space complexity?Solution 1: Dynamic Programming Approach In this solution, we use dynamic programming to compute the number of distinct ways to climb n stairs. The idea is to build up the solution incrementally by solving subproblems. We initialize a list ways where ways[i] represents the number of ways to reach the ith step. We start by setting the base cases: ways[0] = 1 (1 way to stay at the ground) ways[1] =

EXAMPLES

Example 1	
Input	**2**
Output	**2**
Why	**Two ways to climb 2 steps: 1+1 or 2.**

Example 2	
Input	**3**
Output	**3**
Why	**Three ways to climb 3 steps: 1+1+1, 1+2, 2+1.**

Example 3	
Input	**5**
Output	**8**
Why	**Eight ways for 5 steps.**

WHY IT WORKS

Keep ways for the previous two steps and advance: (a, b) = (b, a + b). Start with ways(0) = ways(1) = 1. No table and no recursion.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Two running values
# Idea: Keep ways for the previous two steps and advance:
# (previous_ways, current_ways) = (current_ways, previous_ways + current_ways). Start with ways(0) = ways(1) =
1.
# No table and no recursion.
def climbStairs(step_count):
    previous_ways = current_ways = 1
    for _ in range(step_count - 1):
        previous_ways, current_ways = current_ways, previous_ways + current_ways
    return current_ways
```

COMPLEXITY | Time O(n) | Space O(1)

154 Fibonacci Numbers

EASY

Template move: the same pair update starting from (0, 1)

QUESTION

Write a function fib(n) that returns the nth Fibonacci number. The Fibonacci sequence is defined as F(n) = F(n-1) + F(n-2) with F(0) = 1 and F(1) = 1, which produces the following pattern: 1, 1, 2, 3, 5, 8, 13 ... Examples Pythonfib(0)

=> 1 fib(1)

=> 1 fib(2)

=> 2 fib(3)

=> 3 ... fib(10)

=> 89 fib(20)

=> 10946Hint 1The Fibonacci sequence

is defined recursively since it refers to itself.Hint 2We can start with a recursive implementation and optimize it.Hint 3Each call to F(n) has two subproblems: F(n-1) and F(n-2). Can we memoize these calculations?Challenge 1Does your solution work in O(n) time?This question is a classic example of how dynamic programming, or memoization, can help us solve problems faster by reducing repetitive computations. In this first version, we recursively call the fib function twice, and then we call it twice more in every subsequent recursive call, so the total number of operations expands to O(2^n)! To improve the runtime of our program, let's save our prior calculations in a dictionary called memo and pass it into our function. Now, if we've already found fib(n) before, our function will use this stored value instead of recursing again. Time Complexity: The time complexity of the memoized version is O(n). This is because each Fibonacci number from 0 to n is computed only once and then stored in the memo dictionary. The recursive calls will now simply look up the value in the dictionary if it has already been computed, reducing redundant

EXAMPLES

Example 1	
Input	`0`
Output	`1`
Why	`F(0) = 1 and F(1) = 1, as defined in this question.`

Example 2	
Input	`5`
Output	`8`
Why	`1, 1, 2, 3, 5, 8: F(5) = 8.`

Example 3	
Input	`10`
Output	`89`
Why	`F(10) = 89.`

PART 1: Iteration with two variables

Keep the previous two values and advance n times. Memoising the recursion gives the same time with O(n) memory; the loop needs no memory and no recursion.

PART 2: Fast doubling

Use F(2k) = F(k) * (2 * F(k + 1) - F(k)) and F(2k + 1) = F(k)^2 + F(k + 1)^2. Recursing on n // 2 halves the problem each time, so only about log2(n) steps are needed.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Iteration with two variables
# Idea: Keep the previous two values and advance n times.
# Memoising the recursion gives the same time with O(position)
# memory; the loop needs no memory and no recursion.
def fib(position):
    current, following = 0, 1
    for _ in range(position):
        current, following = following, current + following
    return current
```

COMPLEXITY | Part 1: Time O(n) | Space O(1)
Part 2: Time O(log n) big-integer operations | Space O(log n) recursion depth

155 Maximize sum without choosing adjacent elements

MEDIUM

Template move: exactly the house robber update on the array

QUESTION

Given a list of non-negative numbers, choose some of them so that no two chosen numbers are neighbours in the list, and return the largest possible sum of the chosen numbers.

Function signature

def rob_line(nums: list) -> int

EXAMPLES

Example 1	
Input	[2, 7, 9, 3, 1]
Output	12
Why	Choose 2, 9 and 1 (positions 0, 2, 4): 12.

Example 2	
Input	[5]
Output	5
Why	One number: take it.

Example 3	
Input	[]
Output	0
Why	Nothing to choose: 0.

WHY IT WORKS

take is the best total that includes the current element, skip the best that excludes it. For the next element: new take = skip + x (the previous element must have been skipped), new skip = max(take, skip). The answer is the larger of the two at the end.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Two running values (take, skip)
# Idea: take is the best total that includes the current
# element, skip the best that excludes it. For the next
# element: new take = skip + house_value (the previous element must
# have been skipped), new skip = max(take, skip). The
# answer is the larger of the two at the end.
def _house_dp(house_money):
    take = skip = 0
    for house_value in house_money:
        take, skip = skip + house_value, max(take, skip)
    return max(take, skip)
```

COMPLEXITY | Time O(n) | Space O(1)

156 Paint House II — MEDIUM

Template move: state = a whole row of costs, one per colour; new[j] = cost[j] + the smallest previous value that is not colour j (keep the two smallest)

QUESTION

There are n houses in a row, each to be painted one of k colors. costs[i][j] is the cost to paint house i with color j . No two adjacent houses may share the same color. Return the minimum total cost to paint all houses.

Function signature

def min_cost_paint_houses(costs: list[list[int]]) -> int:

    ...

Follow-up (as reported): Reduce the per-row transition from O(k) per cell to O(1) amortized by tracking the two smallest values of the previous row.

EXAMPLES

Example 1	
Input	`costs = [[1, 5, 3], [2, 9, 4]]`
Output	`5`
Why	`Paint house 0 color 0 (1) and house 1 color 2 (4) -> 5; or house 0 color 2 (3) and house 1 color 0 (2) -> 5.`

Example 2	
Input	`costs = [[1, 3], [2, 4]]`
Output	`5`
Why	`1 (house0 color0) + 4 (house1 color1) = 5, the cheapest valid pairing.`

Example 3	
Input	`costs = [[7]]`
Output	`7`
Why	`A single house, one color.`

CONSTRAINTS

- costs is an n x k grid, 0 <= n , k >= 1 (and k >= 2 whenever n >= 2 , so a valid painting exists).
- Costs are non-negative integers. Return 0 for n == 0 .
- Aim for O(n*k) time.

WHY IT WORKS

A cell only needs the cheapest previous colour that differs from its own.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Two smallest values of the
# previous row
# Idea: A cell only needs the cheapest previous colour
# that differs from its own.
def min_cost_paint_houses(costs):
    previous_total = [0] * len(costs[0])
    for house_costs in costs:
        cheapest_two = sorted(range(len(previous_total)), key=lambda colour: previous_total[colour])[:2]
        first = cheapest_two[0]
        second = cheapest_two[1] if len(cheapest_two) > 1 else cheapest_two[0]
        previous_total = [house_costs[colour] + (previous_total[second] if colour == first else previous_total[
first])
                for colour in range(len(house_costs))]
    return min(previous_total)
```

COMPLEXITY | Time O(n k) | Space O(1) extra

157 Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses. MEDIUM

Template move: state = (best if you take this house, best if you skip it): take, skip = skip + x, max(take, skip)

QUESTION

Houses stand in a line and house i holds nums[i] money. A robber cannot rob two houses that are next to each other. Return the largest total amount he can steal.

Function signature

def rob_line(nums: list) -> int

EXAMPLES

Example 1	
Input	[1, 2, 3, 1]
Output	4
Why	Rob houses 0 and 2: 1 + 3 = 4.

Example 2	
Input	[2, 7, 9, 3, 1]
Output	12
Why	Rob houses 0, 2 and 4: 2 + 9 + 1 = 12.

WHY IT WORKS

take is the best total that includes the current element, skip the best that excludes it. For the next element: new take = skip + x (the previous element must have been skipped), new skip = max(take, skip). The answer is the larger of the two at the end.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Two running values (take, skip)
# Idea: take is the best total that includes the current
# element, skip the best that excludes it. For the next
# element: new take = skip + house_value (the previous element must
# have been skipped), new skip = max(take, skip). The
# answer is the larger of the two at the end.
def _house_dp(house_money):
    take = skip = 0
    for house_value in house_money:
        take, skip = skip + house_value, max(take, skip)
    return max(take, skip)
```

COMPLEXITY | Time O(n) | Space O(1)

158 Write SQL code to publish the Fibonacci series. MEDIUM

Template move: WITH RECURSIVE keeps (n, a, b); every new row is (n + 1, b, a + b): the same rolling pair, written as a query

QUESTION

Write a SQL query that publishes the first N Fibonacci numbers (0, 1, 1, 2, 3, 5, ...), one per row and in order, where N is given as the parameter :limit. Use a recursive common table expression.

Function signature

-- result: one column named fibonacci, N rows

EXAMPLES

Example 1	
Input	`:limit = 8`
Output	`0, 1, 1, 2, 3, 5, 8, 13`
Why	`Eight rows: each number is the sum of the two before it.`

Example 2	
Input	`:limit = 1`
Output	`0`
Why	`A single row.`

WHY IT WORKS

The anchor row is (1, 0, 1): the first term is 0 and the next pair is (0, 1). Each recursive step produces (n + 1, b, a + b) while n is below the limit, so the term in column a is the Fibonacci number. Without the WHERE n < limit the recursion never stops. Integer overflow appears around the 93rd term for 64-bit integers.

SHORT OPTIMAL CODE

```
-- BASE QUESTION (optimal): Recursive CTE carrying (n, a, b)
-- Idea: The anchor row is (1, 0, 1): the first term is 0 and
-- the next pair is (0, 1). Each recursive step produces
-- (n + 1, b, a + b) while n is below the limit, so the
-- term in column a is the Fibonacci number. Without the
-- WHERE n < limit the recursion never stops. Integer
-- overflow appears around the 93rd term for 64-bit
-- integers.
WITH RECURSIVE fib(n, a, b) AS (
    SELECT 1, 0, 1
    UNION ALL
    SELECT n + 1, b, a + b FROM fib WHERE n < :limit
)
SELECT a AS fibonacci FROM fib ORDER BY n;
```

COMPLEXITY | Time O(n) | Space O(1) working set (O(n) result)

159 Constrained Dice Roll Sequences HARD

Template move: state = a table [value][run length]; a different value starts a run of 1, the same value extends its run (up to the limit)

QUESTION

You roll a single 6-faced die exactly n times. You are given an array roll_max of length 6, where roll_max[i] is the maximum number of times the value i + 1 may appear consecutively. That is, you may never roll the same value more than roll_max[i] times in a row. Return the number of distinct sequences of n rolls that respect every consecutive-count limit. Because the answer can be very large, return it modulo 10**9 + 7 .

QUESTION

Function signature

def count_dice_sequences(n: int, roll_max: list[int]) -> int:

    ...

Follow-up (as reported): The interviewer's actual twist used two dice per turn (a pair rolled together, with the consecutive limit applied to each die independently). Extend the state to track the run lengths of both dice and count the joint sequences.

EXAMPLES

Example 1	
Input	`n = 2, roll_max = [1, 1, 2, 2, 2, 3]`
Output	`34`
Why	`There are 6*6 = 36 sequences of length 2. Two of them are forbidden: rolling value 1 twice, and rolling value 2 twice (both have roll_max = 1). 36 - 2 = 34.`

Example 2	
Input	`n = 2, roll_max = [1, 1, 1, 1, 1, 1]`
Output	`30`
Why	`No value may repeat consecutively, so the 6 sequences "value v then v" are removed: 36 - 6 = 30.`

Example 3	
Input	`n = 3, roll_max = [1, 1, 1, 2, 2, 3]`
Output	`181`
Why	`Hand-verified against the memoized DP over (rolls_used, last_value, run_length).`

CONSTRAINTS

- 1 <= n <= 5000 roll_max has exactly 6 entries, each 1 <= roll_max[i] <= 15 .
- A naive brute force over all 6**n sequences is far too slow; you must memoize over (rolls_used, last_value, consecutive_count_of_last_value) .

WHY IT WORKS

dp[v][r] counts prefixes ending in value v repeated r times.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): DP over the last value and its
# run length
# Idea: dp[face][run_length] counts prefixes ending in value v
# repeated r times.
def count_dice_sequences(roll_count, roll_max):
    mod = 10**9 + 7
    dp = [[0] * (roll_max[face] + 1) for face in range(6)]
    for face in range(6):
        dp[face][1] = 1
    for _ in range(roll_count - 1):
        total = sum(sum(row) for row in dp) % mod
        next_dp = [[0] * (roll_max[face] + 1) for face in range(6)]
        for face in range(6):
            # a different previous value starts a run of length one
            next_dp[face][1] = (total - sum(dp[face])) % mod
            for run_length in range(1, roll_max[face]):
                next_dp[face][run_length + 1] = dp[face][run_length]
        dp = next_dp
    return sum(sum(row) for row in dp) % mod
```

COMPLEXITY | Time O(n * 6 * R) | Space O(6 * R)

GROUP 20 OF 43 | 7 QUESTIONS | SHORT MAIN CODE

Two pointers: move i and j toward each other (or along two lists)

Pattern

Put one pointer at each end (or one at the start of each of two sorted lists). Look at the two items, decide which pointer moves, and stop when they meet. Most of these are O(n) because each pointer only moves one way.

160 Reverse a linked list. EASY

Template move: three pointers prev, current, next: point current.next back at prev, then move all three one step

QUESTION

Given the head of a singly linked list, reverse the list and return its head. Implement a function reverseList that takes the head of the linked list as input and returns the new head of the reversed list. Refer to the ListNode class/struct and do not comment it out. Example Input: 1 -> 2 -> 3 -> 4 Output: 4 -> 3 -> 2 -> 1Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(1)?

EXAMPLES

Example 1	
Input	`head = 1 -> 2 -> 3`
Output	`vals(reverse_list(head)) -> [3, 2, 1]`
Why	`1 -> 2 -> 3 becomes 3 -> 2 -> 1.`

WHY IT WORKS

Walk with prev and cur. Save cur.next, point cur at prev, then move both forward. When cur is None, prev is the new head. Saving next before the overwrite is the step people forget.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Flip the pointers in one pass
# Idea: Walk with previous_node and cur. Save cur.next, point cur
# at previous_node, then move both forward. When cur is None, previous_node
# is the new head. Saving next before the overwrite is the
# step people forget.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

def reverseList(head):
    previous_node = None
    while head:
        next_node = head.next
        head.next = previous_node
        previous_node, head = head, next_node
    return previous_node
```

COMPLEXITY | Time O(n) | Space O(1)

161 Sort a binary array in linear time. EASY

Template move: left skips 0s, right skips 1s; when left sits on a 1 and right on a 0, swap them

QUESTION

Sort an array that contains only 0s and 1s in place, in one pass over the array (O(n) time) and with O(1) extra space.

Function signature

def sort_binary(arr: list) -> list

EXAMPLES

Example 1	
Input	[1, 0, 1, 0, 0, 1]
Output	[0, 0, 0, 1, 1, 1]
Why	All the 0s move to the front.

Example 2	
Input	[0, 0, 1, 1]
Output	[0, 0, 1, 1]
Why	Already sorted arrays stay as they are.

Example 3	
Input	[1]
Output	[1]
Why	A single value.

PART 1: Two pointers swapping from both ends

Keep lo at the next position for a 0 and hi at the next position for a 1. Advance lo past zeros, retreat hi past ones, and when both are stuck swap them. One pass, in place.

PART 2: Three values (Dutch national flag)

Maintain three regions: [0, low) zeros, [low, mid) ones, (high, end] twos. Examine arr[mid]: swap a 0 to low, skip a 1, swap a 2 to high without advancing mid (the swapped-in value has not been inspected).

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Two pointers swapping from both
# ends
# Idea: Keep zero_slot at the next position for a 0 and one_slot at the
# next position for a 1. Advance zero_slot past zeros, retreat one_slot
# past ones, and when both are stuck swap them. One pass,
# in place.
def sort_binary(bits):
    zero_slot, one_slot = 0, len(bits) - 1
    while zero_slot < one_slot:
        if bits[zero_slot] == 0:
            zero_slot += 1
        elif bits[one_slot] == 1:
            one_slot -= 1
        else:
            bits[zero_slot], bits[one_slot] = bits[one_slot], bits[zero_slot]
            zero_slot += 1
            one_slot -= 1
    return bits
```

COMPLEXITY | Part 1: Time O(n) | Space O(1)
Part 2: Time O(n) | Space O(1)

162 Is this a valid palindrome? EASY

Template move: compare the two ends; skip characters that are not letters or digits; stop when the pointers cross

QUESTION

Given a string s, return true if it is a palindrome, and false otherwise. A string is considered a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Write a function isPalindrome that implements this logic. Examples Input: s = "H123!@321h" Output: true Input: s = "race a car" Output: false Constraints 1 <= s.length <= 2 * 10^5 The input string s can include any printable ASCII characters. Challenge 1Can you come up with a solution that achieves O(n) time complexity?Challenge 2Can you come up with a solution that achieves O(1) space complexity?Solution 1: Filtering and Reversing Approach This solution processes the input string by first filtering out non-alphanumeric characters and converting all remaining characters to lowercase. It then checks if this filtered string reads the same forward and backward. Filtering: Iterate through the string and collect alphanumeric characters while converting them to lowercase. Reversing and Comparing: Create a reversed version of the filtered string and compare it with the original filtered string to check for palindrome properties. Time Complexity: The time complexity is O(n), where n is the length of the input string. This is because filtering the characters and creating the reversed string both require iterating through the input once. Space Complexity: The space complexity is O(n) for storing the filtered characters and their

EXAMPLES

Example 1	
Input	'A man, a plan, a canal: Panama'
Output	true
Why	Ignoring punctuation, spaces and case it reads 'amanaplanacanalpanama'.

Example 2	
Input	'race a car'
Output	false
Why	'raceacar' is not a palindrome.

Example 3	
Input	' '
Output	true
Why	Nothing but a space: nothing to compare, so it is a palindrome.

WHY IT WORKS

Move left and right toward each other, skipping characters that are not letters or digits, and compare the lowercase forms. The first mismatch ends it. Nothing is copied.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Two pointers that skip ignored
# characters
# Idea: Move left and right toward each other, skipping
# characters that are not letters or digits, and compare
# the lowercase forms. The first mismatch ends it. Nothing
# is copied.
def isPalindrome(text):
    left, right = 0, len(text) - 1
    while left < right:
        while left < right and not text[left].isalnum():
            left += 1
        while left < right and not text[right].isalnum():
            right -= 1
        if text[left].lower() != text[right].lower():
```

```
            return False
        left, right = left + 1, right - 1
    return True
```

COMPLEXITY | Time O(n) | Space O(1)

163 Reverse a Sentence

EASY

Template move: swap from both ends to reverse the whole array, then reverse every word the same way

QUESTION

You are given an array of characters arrthat consists of sequences of characters separated by space characters. Each space-delimited sequence of characters defines a word. Implement a function reverseWords that reverses the order of the words in the array in the most efficient manner. Explain your solution and analyze its time and space complexities. Example input: arr = ['p', 'e', 'r', 'f', 'e', 'c', 't', ' ', 'm', 'a', 'k', 'e', 's', ' ', 'p', 'r', 'a', 'c', 't', 'i', 'c', 'e'] output: ['p', 'r', 'a', 'c', 't', 'i', 'c', 'e', ' ', 'm', 'a', 'k', 'e', 's', ' ', 'p', 'e', 'r', 'f', 'e', 'c', 't']Hint 1Even if your language of choice enables this, don't be tempted to use standard functions. Consider how inefficient it is to separately create a string, split the words, and reverse the split.Challenge 1As a general rule, if you're tempted to use a standard or library function, be sure to explain that function's time and space complexity before committing to that solution. You may find a more efficient way to solve the problem yourself. Oftentimes in coding interviews, this is the whole point of the exercise.Challenge 2How have you handled edge cases? For example, empty arrays, arrays with nothing but spaces, arrays with single words only, or with multiple spaces between words, etc.Challenge 3Were you able to achieve a linear time complexity and a constant space complexity?Challenge 4Bonus: how might the mirrorReverse function be implemented with a single index?One possible s

EXAMPLES

Example 1	
Input	`arr = ['t','h','e',' ','s','k','y',' ','i','s',' ','b','l','u','e']`
Output	`''.join(reverse_words(list('the sky is blue'))) -> 'blue is sky the'`
Why	The words come out in reverse order while each word keeps its letters in order (the array is reversed in place).

WHY IT WORKS

Reverse every character; each word is now backwards but in the right place. Walk the array and reverse each maximal run of non-space characters. Everything happens inside the same array.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Reverse the whole array, then
# reverse each word in place
# Idea: Reverse every character; each word is now
# backwards but in the right place. Walk the array and
# reverse each maximal run of non-space characters.
# Everything happens inside the same array.
def reverse_sentence(chars):
    def rev(left, right):
        while left < right:
            chars[left], chars[right] = chars[right], chars[left]
            left, right = left + 1, right - 1

    rev(0, len(chars) - 1)
    start = 0
    for position in range(len(chars) + 1):
        if position == len(chars) or chars[position] == " ":
            rev(start, position - 1)
            start = position + 1
    return chars
```

COMPLEXITY | Time O(n) | Space O(1) extra on the character array

164 Find median of merged RLE arrays

MEDIUM

Template move: walk both sorted run lists, always taking the smaller value, adding its count until you pass the middle position

QUESTION

Two sorted arrays are stored run-length encoded as (value, count) pairs in increasing order of value. Return the median of all their elements together, without expanding the runs (a count can be huge).

Function signature

def median_rle(a: list, b: list) -> float

EXAMPLES

Example 1	
Input	[(1, 2), (5, 1)], [(3, 3)]
Output	3
Why	Combined sequence 1, 1, 3, 3, 3, 5: the two middle elements are 3 and 3.

Example 2	
Input	[(1, 1)], [(2, 1), (9, 1)]
Output	2
Why	Combined sequence 1, 2, 9: the middle one is 2.

Example 3	
Input	[(1, 1000000)], [(5, 1000000)]
Output	3
Why	Counts may be enormous: a million 1s and a million 5s have median 3.

WHY IT WORKS

Let N be the sum of all counts. Walk the two run lists in value order (the merge step of merge sort), adding each run's count to a running total; the value of the run that makes the total pass a rank is the element at that rank. Ask for ranks (N - 1) // 2 and N // 2 and average them.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Merge the runs and walk to the
# two middle ranks
# Idea: Let N be the sum of all counts. Walk the two run
# lists in value order (the merge step of merge sort),
# adding each run's count to a running total; the value of
# the run that makes the total pass a rank is the element
# at that rank. Ask for ranks (total_count - 1) // 2 and total_count // 2 and
# average them.
def median_rle(first_runs, second_runs):
    total_count = sum(run_count for _, run_count in first_runs) + sum(run_count for _, run_count in second_runs
)

    def kth(rank):                                          # zero-based rank in the merged order
        first_index = second_index = elements_passed = 0
        while True:
            take_a = second_index >= len(second_runs) or (first_index < len(first_runs) and first_runs[first_in
dex][0] <= second_runs[second_index][0])
            value, run_count = first_runs[first_index] if take_a else second_runs[second_index]
            elements_passed += run_count
            if elements_passed > rank:
                return value
            first_index, second_index = (first_index + 1, second_index) if take_a else (first_index, second_ind
```

```
ex + 1)

    return kth(total_count // 2) if total_count % 2 else (kth(total_count // 2 - 1) + kth(total_count // 2)) /
2
```

COMPLEXITY | Time O(r1 + r2) | Space O(1)

165 Run-Length Vector Storage + Dot Product

MEDIUM

Template move: one pointer per run list; multiply the values and take min(remaining counts) of the two current runs

QUESTION

Two vectors of the same length are stored run-length encoded as (value, count) pairs. Compute their dot product without expanding them to full arrays.

Function signature

def rle_dot(a: list, b: list) -> int

EXAMPLES

Example 1	
Input	`[(3, 3), (0, 3), (4, 2)], [(1, 4), (2, 4)]`
Output	`25`
Why	`A = [3,3,3,0,0,0,4,4], B = [1,1,1,1,2,2,2,2]: 3*1*3 + 0 + 4*2*2 = 9 + 16 = 25.`

Example 2	
Input	`[(2, 1)], [(5, 1)]`
Output	`10`
Why	`One element each: 2 * 5.`

WHY IT WORKS

Each run is (value, count). Compare the remaining lengths of the two current runs, multiply the values by the overlap, then advance whichever run was exhausted (or both). Never expand the vectors: the work depends on the number of runs, not the logical length.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Two-pointer over runs
# Idea: Each run is (value, count). Compare the remaining
# lengths of the two current runs, multiply the values by
# the overlap, then advance whichever run was exhausted
# (or both). Never expand the vectors: the work depends on
# the number of runs, not the logical length.
def rle_dot(first_runs, second_runs):
    first_index = second_index = 0
    first_left = first_runs[0][1] if first_runs else 0
    second_left = second_runs[0][1] if second_runs else 0
    total = 0
    while first_index < len(first_runs) and second_index < len(second_runs):
        overlap = min(first_left, second_left)
        total += first_runs[first_index][0] * second_runs[second_index][0] * overlap
        first_left -= overlap
        second_left -= overlap
        if first_left == 0:
            first_index += 1
            first_left = first_runs[first_index][1] if first_index < len(first_runs) else 0
        if second_left == 0:
            second_index += 1
            second_left = second_runs[second_index][1] if second_index < len(second_runs) else 0
    return total
```

COMPLEXITY | Time O(r1 + r2) | Space O(1)

166 Count Unique Triplets That Sum to Zero

MEDIUM

Template move: sort, fix the first number, then run the pair template on the rest; skip equal neighbours so each triple counts once

QUESTION

Given an integer array nums , count how many unique triplets (nums[i], nums[j], nums[k]) with three distinct indices i < j < k satisfy nums[i] + nums[j] + nums[k] == 0 . Two triplets are considered the same if they consist of the same multiset of values, regardless of the indices they came from. Return the number of distinct value-triplets (not the number of index combinations).

Function signature

def count_zero_sum_triplets(nums: list[int]) -> int:

...

Follow-up (as reported): Return the actual list of unique triplets instead of the count. Hint: sort the array, fix one element, and use two pointers while skipping duplicate values.

EXAMPLES

Example 1	
Input	nums = [-1, 0, 1, 2, -1, -4]
Output	2
Why	The distinct value-triplets are [-1, -1, 2] and [-1, 0, 1]. Even though [-1, 0, 1] can be formed with different index sets, it is counted once.

Example 2	
Input	nums = [0, 0, 0, 0]
Output	1
Why	The only triplet is [0, 0, 0].

Example 3	
Input	nums = [1, 2, 3]
Output	0
Why	No three elements sum to 0.

CONSTRAINTS

- 0 <= len(nums) <= 3000
- -10^5 <= nums[i] <= 10^5
- The array may contain duplicates.
- If len(nums) < 3 , the answer is 0 .

WHY IT WORKS

Fix the smallest value and sweep the other two inward, skipping duplicates.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Sort and two pointers
# Idea: Fix the smallest value and sweep the other two
# inward, skipping duplicates.
def count_zero_sum_triplets(nums):
    nums = sorted(nums)
    count = 0
    for fixed_index in range(len(nums) - 2):
        if fixed_index and nums[fixed_index] == nums[fixed_index - 1]:
            continue
```

```
        left, right = fixed_index + 1, len(nums) - 1
        while left < right:
            total = nums[fixed_index] + nums[left] + nums[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                count += 1
                left += 1
                right -= 1
                while left < right and nums[left] == nums[left - 1]:
                    left += 1
    return count
```

COMPLEXITY | Time O(n^2) | Space O(1) extra

GROUP 21 OF 43 | 7 QUESTIONS | SHORT MAIN CODE

Walk two strings at once with two indices

Pattern

Keep i on one string and j on the other. Skip characters that do not matter, compare the rest, and decide what to do at the first difference. Substring search, ordering, similarity and move-based transforms are all this loop.

167 Find the First Occurrence of a Substring — EASY

Template move: try every start i; inside, j walks the needle; a full match returns i

QUESTION

Reconstructing the "find substring in array" item as substring search: given two strings haystack and needle , return the index of the first occurrence of needle in haystack , or -1 if needle is not part of haystack .

Function signature

```
def str_str(haystack: str, needle: str) -> int:
    """Return the index of the first occurrence of needle in haystack, or -1."""
    ...
```

Follow-up (as reported): Achieve O(n + m) worst-case time with the Knuth-Morris-Pratt (KMP) failure-function approach instead of the naive O(n * m) scan.

EXAMPLES

Example 1	
Input	`haystack = "sadbutsad", needle = "sad"`
Output	`0`
Why	`"sad" occurs at indices 0 and 6; the first is index 0.`

Example 2	
Input	`haystack = "leetcode", needle = "leeto"`
Output	`-1`
Why	`"leeto" does not occur in "leetcode".`

Example 3	
Input	`haystack = "abc", needle = "c"`
Output	`2`

CONSTRAINTS

- 1 <= len(haystack), len(needle) <= 10^4 haystack and needle consist of only lowercase English letters.
- If needle is the empty string it is considered to occur at index 0 (though the constraints above make it non-empty).

WHY IT WORKS

Compare character by character and stop at the first mismatch.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Naive scan with early exit
# Idea: Compare character by character and stop at the
# first mismatch.
def str_str(haystack, needle):
    haystack_length, needle_length = len(haystack), len(needle)
    for start in range(haystack_length - needle_length + 1):
        matched = 0
```

```
        while matched < needle_length and haystack[start + matched] == needle[matched]:
            matched += 1
        if matched == needle_length:
            return start
    return -1
```

COMPLEXITY | Time O(n * m) worst case | Space O(1)

168 Determine if two words are sorted in lexicographic order. EASY

Template move: walk both words; at the first differing letter compare the letters' ranks in the given alphabet; if one word ends first, it must be the shorter

QUESTION

Words are ordered using a custom alphabet: rank maps each letter to its position in that alphabet. Return whether word a comes before word b (or is equal to it). The first letter where the words differ decides; if one word is a prefix of the other, the shorter one comes first.

Function signature

def in_order(a: str, b: str, rank: dict) -> bool

EXAMPLES

Example 1

Input	`alphabet = 'hlabcdefgijkmnopqrstuvwxyz' # 'h' comes first, then 'l', then a, b, c ...`
	`rank = {c: i for i, c in enumerate(alphabet)}`
	`in_order('hello', 'leetcode', rank)`
	`in_order('word', 'world', rank)`
	`in_order('app', 'apple', rank)`

Output	`in_order('hello', 'leetcode', rank) -> true`
	`in_order('word', 'world', rank) -> false`
	`in_order('app', 'apple', rank) -> true`

Why	`'h' ranks before 'l', so hello comes first. word vs world differ at d versus l, and d ranks after l, so the order is wrong. A prefix always comes first.`

WHY IT WORKS

Find the first position where the words differ and compare the two characters' ranks. If one word is a prefix of the other, the shorter one comes first, so 'app' is before 'apple' but not the reverse. Using a rank dict makes the same code work for any alphabet.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Compare character by character,
# optionally with a custom alphabet
# Idea: Find the first position where the words differ and
# compare the two characters' ranks. If one word is a
# prefix of the other, the shorter one comes first, so
# 'app' is before 'apple' but not the reverse. Using a
# rank dict makes the same code work for any alphabet.
def is_sorted_pair(first, second, alphabet="abcdefghijklmnopqrstuvwxyz"):
    rank = {letter: position for position, letter in enumerate(alphabet)}
    for first_letter, second_letter in zip(first, second):
        if first_letter != second_letter:
            return rank[first_letter] < rank[second_letter]
    return len(first) <= len(second)
```

COMPLEXITY | Time O(min(len a, len b)) | Space O(1)

169 Return all unique substrings of a given string. EASY

Template move: sort all suffixes; the unique substrings are all prefixes minus the shared prefixes of neighbouring suffixes

QUESTION

Return every distinct substring of a string, sorted alphabetically (a substring is a contiguous piece, the whole string and single letters included).

Function signature

def unique_substrings(s: str) -> list

EXAMPLES

Example 1	
Input	`'abc'`
Output	`['a', 'ab', 'abc', 'b', 'bc', 'c']`
Why	`Six different substrings of 'abc'.`

Example 2	
Input	`'aaa'`
Output	`['a', 'aa', 'aaa']`
Why	`'a' appears three times but is listed once.`

WHY IT WORKS

Sort the suffixes; the number of distinct substrings is sum over suffixes of (length) minus sum of the LCP of neighbours in sorted order, because each new suffix adds only the prefixes that the previous one did not already contribute. This counts without storing a single substring.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Count with a suffix array and
# LCP
# Idea: Sort the suffixes; the number of distinct
# substrings is sum over suffixes of (length) minus sum of
# the LCP of neighbours in sorted order, because each new
# suffix adds only the prefixes that the previous one did
# not already contribute. This counts without storing a
# single substring.
def count_unique_substrings(text):
    length = len(text)
    suffixes = sorted(text[start:] for start in range(length))
    total = length * (length + 1) // 2
    for earlier_suffix, later_suffix in zip(suffixes, suffixes[1:]):
        common_length = 0
        while common_length < min(len(earlier_suffix), len(later_suffix)) and earlier_suffix[common_length] ==
later_suffix[common_length]:
            common_length += 1
        total -= common_length
    return total
```

COMPLEXITY | Time O(n^2 log n) with naive sorting (O(n log n) with a real suffix array) | Space O(n)

170 Can board states be transformed? MEDIUM

Template move: skip the X squares in both strings; the remaining letters must match in order, and L may only move left, R only right

QUESTION

A board is a string of 'L', 'R', '#' and 'X'. 'X' is an empty square. An 'L' piece may move left into an empty square, an 'R' piece may move right into an empty square, pieces never jump over each other, and '#' (a wall piece) never moves. Return whether start can be turned into end by any number of moves.

Function signature

def can_transform(start: str, end: str) -> bool

EXAMPLES

Example 1	
Input	`'RXXLRXRXL', 'XRLXXRRLX'`
Output	`true`
Why	`L moves left and R moves right while keeping their order: this classic board is reachable.`

Example 2	
Input	`'XL', 'LX'`
Output	`true`
Why	`L can move left.`

Example 3	
Input	`'LX', 'XL'`
Output	`false`
Why	`L cannot move to the right.`

Example 4	
Input	`'#X', 'X#'`
Output	`false`
Why	`A wall never moves.`

WHY IT WORKS

Skip blanks in both strings and compare the next piece in each: they must be the same letter. An 'L' must not have moved right (its start index is at least its target index) and an 'R' must not have moved left (start index at most target index). If the strings end together the transformation is possible.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Two pointers over the pieces
# Idea: Skip blanks in both strings and compare the next
# piece in each: they must be the same letter. An 'L' must
# not have moved right (its start index is at least its
# target index) and an 'R' must not have moved left (start
# index at most target index). If the strings end together
# the transformation is possible.
def canTransform(start, end):
    if len(start) != len(end):
        return False
    start_index = end_index = 0
    length = len(start)
    while start_index < length or end_index < length:
        while start_index < length and start[start_index] == "X":
            start_index += 1
```

```
        while end_index < length and end[end_index] == "X":
            end_index += 1
        if start_index == length or end_index == length:
            return start_index == end_index
        if start[start_index] != end[end_index]:
            return False
        if start[start_index] == "L" and start_index < end_index:
            return False
        if start[start_index] == "R" and start_index > end_index:
            return False
        start_index, end_index = start_index + 1, end_index + 1
    return True
```

COMPLEXITY | Time O(n) | Space O(1)

171 Decide string transform with directional tokens and walls

MEDIUM

Template move: the same walk with dots as blanks and the movement rule from the statement (walls block)

QUESTION

Two equal-length strings start and target use the characters 'L', 'R', '.' (an empty slot) and 'X' (an immovable wall). An 'L' token can move left into empty slots, an 'R' token can move right into empty slots, tokens can never pass through each other or through a wall, and 'X' never moves. Return whether start can be turned into target by any number of moves.

Function signature

def can_transform(start: str, end: str) -> bool

EXAMPLES

Example 1	
Input	'R..L', '.RL.'
Output	true
Why	R moves one step right and L one step left: both stay in order.

Example 2	
Input	'L.', '.L'
Output	false
Why	L can only move left, never right.

Example 3	
Input	'X.', '.X'
Output	false
Why	A wall cannot move.

WHY IT WORKS

Walk both strings skipping dots. The next symbol must be equal; a wall must be at the same index; an L must satisfy start index >= target index; an R must satisfy start index <= target index. Since the order of symbols is the same, a piece cannot cross a wall.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Compare the non-dot sequences
# with position rules
# Idea: Walk both strings skipping dots. The next symbol
# must be equal; a wall must be at the same index; an L
# must satisfy start index >= target index; an R must
# satisfy start index <= target index. Since the order of
```

```python
# symbols is the same, a piece cannot cross a wall.
def can_transform_walls(start, end):
    if len(start) != len(end):
        return False
    start_index = end_index = 0
    length = len(start)
    while True:
        while start_index < length and start[start_index] == ".":
            start_index += 1
        while end_index < length and end[end_index] == ".":
            end_index += 1
        if start_index == length or end_index == length:
            return start_index == end_index
        if start[start_index] != end[end_index]:
            return False
        if start[start_index] == "X" and start_index != end_index:
            return False
        if start[start_index] == "L" and start_index < end_index:
            return False
        if start[start_index] == "R" and start_index > end_index:
            return False
        start_index, end_index = start_index + 1, end_index + 1
```

COMPLEXITY | Time O(n) | Space O(1)

172 Determine if two sentences are similar. MEDIUM

Template move: zip the two sentences; each pair of words must be equal or listed as similar (store pairs in both directions)

QUESTION

Determine if Two Sentences Are Similar. Two sentences are similar if they have the same length and each pair of corresponding words in the two sentences is similar. The similarity between words is defined by the provided list of similar word pairs. A word is always similar to itself. For example, if we have the list of similar word pairs as [("great", "good"), ("acting","drama"), ("skills","talent")], then the sentences "You have great acting skills" and "You have good drama talent" are similar. Examples sentence1 = ["Let's", "code", "in", "Python"] sentence2 = ["Let's", "program", "in", "Python"] similarPairs = [("code", "program"),] output: true sentence1 = ["I", "love", "to", "play", "football"] sentence2 = ["I", "love", "playing", "soccer"] similarPairs = [("play", "playing"), ("football", "soccer")] output: false, different sentence lengths sentence1 = ["Do", "you", "like", "coffee"] sentence2 = ["Do", "you", "love", "coffee"] similarPairs = [("like", "enjoy"), ("coffee", "tea"),] output: false

"like" is not similar to

"love" based on the given pairs. sentence1 = ["I", "really", "love", "leetcode", "and", "apples"] sentence2 = ["I", "so", "like", "codesignal", "and", "oranges"] similarPairs = [("very", "so"), ("love", "adore"), ("really", "very"), ("leetcode", "codesignal"), ("apples", "oranges"), ("like", "adore"),] output: trueThe provided solution checks if two given sentences are similar by using a graph-based approach. It constructs a graph where words are c

EXAMPLES

Example 1	
Input	`['great', 'acting', 'skills'], ['fine', 'drama', 'talent'], [['great', 'fine'], ['acting', 'drama'], ['skills', 'talent']]`
Output	`true`
Why	`Each word is paired with a similar word.`

QUESTION

Example 2	
Input	`['a'], ['a', 'b'], []`
Output	`false`
Why	`Sentences of different lengths are never similar.`

WHY IT WORKS

Put each pair into a set in both directions, so a word pair is similar if the words are equal or the pair is in the set. Sentences of different length are never similar.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Set of pairs (not transitive)
# Idea: Put each pair into a set in both directions, so a
# word pair is similar if the words are equal or the pair
# is in the set. Sentences of different length are never
# similar.
def areSentencesSimilar(sentence_one, sentence_two, pairs):
    if len(sentence_one) != len(sentence_two):
        return False
    known = set()
    for word_a, word_b in pairs:
        known.add((word_a, word_b))
        known.add((word_b, word_a))
    return all(word_one == word_two or (word_one, word_two) in known for word_one, word_two in zip(sentence_one
, sentence_two))
```

COMPLEXITY | Time O(n + P) | Space O(P)

173 Find Dictionary Matches for a Jammed String — HARD

Template move: compress every word to (letter, run length); a query matches when the letters agree and each run is equal or stretched to at least 3

QUESTION

A word was typed on a keyboard where some keys got stuck, so a letter may be repeated. Given a dictionary of words, matches(query) returns the dictionary words the query could be a jammed version of: the runs of equal letters line up with the word's runs, and each run of the query is either exactly as long as in the word or longer and at least min_stretch (default 3) letters long.

Function signature

class JammedIndex:
    def __init__(self, words: list)
    def matches(self, query: str, min_stretch: int = 3) -> list

EXAMPLES

QUESTION

Example 1

Input	`idx = JammedIndex(['hello', 'help', 'hel'])`
	`idx.matches('heeello')`
	`idx.matches('hello')`
	`idx.matches('heello')`
	`idx.matches('helllo')`

Output	`idx.matches('heeello') -> ['hello']`
	`idx.matches('hello') -> ['hello']`
	`idx.matches('heello') -> []`
	`idx.matches('helllo') -> ['hello']`

Why	`'heeello' stretches the e to 3 letters, which is allowed. 'heello' only stretches it to 2 (shorter than min_stretch), and 'helllo' stretches the l run from 2 to 3 letters, which is allowed.`

WHY IT WORKS

Group words by the string of their distinct run letters ('hello' -> 'helo'). A query can only match words in its own group, so compare run counts only there. Build the index once and reuse it for many queries.

SHORT OPTIMAL CODE

```
def _runs(text):
    runs = []
    for letter in text:
        if runs and runs[-1][0] == letter:
            runs[-1][1] += 1
        else:
            runs.append([letter, 1])
    return runs


# BASE QUESTION (optimal): Index the dictionary by its run
# letters
# Idea: Group words by the string of their distinct run
# letters ('hello' -> 'helo'). A query can only match
# words in its own group, so compare run counts only
# there. Build the index once and reuse it for many
# queries.
class JammedIndex:
    def __init__(self, words):
        self.groups = {}
        for word in words:
            runs = _runs(word)
            self.groups.setdefault("".join(letter for letter, _ in runs), []).append((word, [run_length for _,
run_length in runs]))

    def matches(self, query, min_stretch=3):
        runs = _runs(query)
        key = "".join(letter for letter, _ in runs)
        query_run_lengths = [run_length for _, run_length in runs]
        return [word for word, word_run_lengths in self.groups.get(key, [])
                if all(query_run_length == run_length or (query_run_length > run_length and query_run_length >=
 min_stretch) for query_run_length, run_length in zip(query_run_lengths, word_run_lengths))]
```

COMPLEXITY | Time O(L) per query plus the size of the matching bucket | Space O(total dictionary length)

Simulate a machine: a class with state and one branch per kind of event

Pattern

Put the state in __init__, give it one method that takes an event, and write one branch per event kind. Board games keep a grid or counters and check for a win after each move; game-of-life style updates compute every cell from the old state before changing it.

174 Mario-Style Score Tracker (OOP) EASY

Template move: the template as written: an event name decides how score, lives and size change

QUESTION

Design a class that tracks the state of a platformer character across a sequence of events. The character has a score (starts at 0), a lives count (starts at 3), and a big flag (starts False). Process events with these rules: "coin" : add 100 to the score. "mushroom" : add 1000 to the score and set big = True . "enemy" : add 200 to the score (the character stomps an enemy). "damage" : if big is True , set big = False (no life lost); otherwise decrease lives by 1. Score is unchanged. The character is alive while lives > 0 . Once lives reaches 0, the game is over and any further events are ignored. Implement the class and a method to run a batch of events.

Function signature

```
class MarioGame:
    def __init__(self) -> None:
        """score=0, lives=3, big=False."""
        ...
    def process(self, event: str) -> None:
        """Apply one event, ignoring it if the game is already over."""
        ...
    def run(self, events: list[str]) -> None:
        """Apply a sequence of events in order."""
        ...
    def get_score(self) -> int:
        ...
    def is_alive(self) -> bool:
        ...
```

EXAMPLES

Example 1	
Input	`g = MarioGame(); g.run(["coin", "mushroom", "enemy", "damage", "damage"]); (g.get_score(), g.is_alive())`
Output	`(1300, True)`
Why	`coin -> 100; mushroom -> 1100, big=True; enemy -> 1300; damage -> big=False (no life lost); damage -> small, lives 3->2. Score 1300, still alive.`

Example 2	
Input	`g = MarioGame(); g.run(["damage", "damage", "damage", "coin"]); (g.get_score(), g.is_alive())`
Output	`(0, False)`
Why	`Three small-state hits drop lives 3->0 (game over). The trailing "coin" is ignored.`

CONSTRAINTS

- Event strings are always one of the four listed above.

WHY IT WORKS

One method owns the rules; the game ends when lives hit zero.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): A small state class
# Idea: One method owns the rules; the game ends when
# lives hit zero.
class MarioGame:
    def __init__(self):
        self.score = 0
        self.lives = 3
        self.big = False

    def process(self, event):
        if self.lives == 0:
            return
        if event == "coin":
            self.score += 100
        elif event == "mushroom":
            self.score += 1000
            self.big = True
        elif event == "enemy":
            self.score += 200
        elif event == "damage":
            if self.big:
                self.big = False
            else:
                self.lives -= 1

    def run(self, events):
        for event in events:
            self.process(event)

    def get_score(self):
        return self.score

    def is_alive(self):
        return self.lives > 0
```

COMPLEXITY | Time O(events) | Space O(1)

175 Tic-Tac-Toe System with K Players and 3-in-a-Row Win

MEDIUM

Template move: store the board; after each move count the run through the new cell in four directions and compare with the target length

QUESTION

Design Tic-Tac-Toe for k players on an n x n board. Players move in turn (1, 2, ..., k, then 1 again). move(player, r, c) places a mark and returns 'player p wins' as soon as that player has 3 marks in a straight line (row, column or diagonal), 'draw' when the board is full, and 'continue' otherwise. A move out of turn, off the board or onto a taken cell raises an error.

Function signature

class TicTacToe:
    def __init__(self, n: int, k: int, target: int = 3)
    def move(self, player: int, r: int, c: int) -> str

EXAMPLES

QUESTION

Example 1

Input	`g = TicTacToe(3, 2)`
	`g.move(1, 0, 0)`
	`g.move(2, 1, 0)`
	`g.move(1, 0, 1)`
	`g.move(2, 1, 1)`
	`g.move(1, 0, 2)`

Output	`g.move(1, 0, 0) -> 'continue'`
	`g.move(2, 1, 0) -> 'continue'`
	`g.move(1, 0, 1) -> 'continue'`
	`g.move(2, 1, 1) -> 'continue'`
	`g.move(1, 0, 2) -> 'player 1 wins'`

Why	Player 1 takes the whole top row while player 2 has two in the middle row: player 1 wins.

Example 2

Input	`g = TicTacToe(3, 3)`
	`g.move(1, 0, 0)`
	`g.move(2, 0, 1)`
	`g.move(3, 0, 2)`
	`g.move(1, 1, 1)`

Output	`g.move(1, 0, 0) -> 'continue'`
	`g.move(2, 0, 1) -> 'continue'`
	`g.move(3, 0, 2) -> 'continue'`
	`g.move(1, 1, 1) -> 'continue'`

Why	With three players the turns go 1, 2, 3, 1; nobody has three in a line yet.

WHY IT WORKS

Place the mark, then for each of the four directions walk outward both ways while the cells hold the same player and add the two counts plus one. A total of at least 3 wins immediately. Validate the turn order and empty cell first; a full board with no win is a draw.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Check the four lines through
# the new mark
# Idea: Place the mark, then for each of the four
# directions walk outward both ways while the cells hold
# the same player and add the two counts plus one. A total
# of at least 3 wins immediately. Validate the turn order
# and empty cell first; a full board with no win is a
# draw.
class TicTacToe:
    def __init__(self, size, player_count, target=3):
        self.size, self.player_count, self.target = size, player_count, target
        self.board = [[0] * size for _ in range(size)]
        self.turn, self.moves, self.over = 0, 0, False

    def move(self, player, row, col):
        if self.over or player != self.turn % self.player_count + 1 or not (0 <= row < self.size and 0 <= col <
 self.size) or self.board[row][col]:
            raise ValueError("illegal move")
        self.board[row][col] = player
        self.turn += 1; self.moves += 1
        for row_step, col_step in ((0, 1), (1, 0), (1, 1), (1, -1)):
            line_length = 1
            for sign in (1, -1):
                scan_row, scan_col = row + sign * row_step, col + sign * col_step
```

```
                while 0 <= scan_row < self.size and 0 <= scan_col < self.size and self.board[scan_row][scan_col
] == player:
                    line_length += 1
                    scan_row, scan_col = scan_row + sign * row_step, scan_col + sign * col_step
            if line_length >= self.target:
                self.over = True
                return f"player {player} wins"
        if self.moves == self.size * self.size:
            self.over = True
            return "draw"
        return "continue"
```

COMPLEXITY | Time O(1) per move (O(target) when generalised) | Space O(n^2)

176 Design Tic-Tac-Toe With K Players

MEDIUM

Template move: per player Counters for rows, columns and both diagonals; a move wins when a counter reaches n

QUESTION

Design and implement a Tic-Tac-Toe game engine. You are given:

- **k** players, each with a unique player ID or mark.
- An **`n x n`** board, initially empty.
- Players take turns placing their own mark on an empty cell. Unlike classic Tic-Tac-Toe, a player wins as soon as they form **3 consecutive marks** in a straight line, regardless of how large **n** is. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. Implement a class or set of functions that supports making moves and returns the game status after each move, such as:
- **`"Game Over"`** if a move is attempted after the game has ended.
- **`"<player> won"`** when the latest move causes that player to have 3 consecutive marks.
- A non-terminal status such as **`"Continue"`** if the game should keep going. Your implementation should handle invalid moves, such as placing a mark outside the board or on an occupied cell. Discuss the time and space complexity of your approach, especially for large **n**.

Design a Tic-Tac-Toe game engine for k players on an n x n board. The board starts empty, and players must play in cyclic order: player 1, player 2, ..., player k, then back to player 1. A player wins immediately when their latest valid move creates 3 consecutive marks in a straight line. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. For this problem, implement a function that processes a list of move attempts and returns the status after each attempt. Each move is given as [player, row, col] using 0-based indexing. Return one status string per move:

- "Game Over" if a move is attempted after someone has already won.
- "Invalid move" if the move is not legal. A move is invalid if: - the player ID is not in [1, k] - it is not that player's turn - the cell is outside the board - the cell is already occupied - "<player> won" if the move is valid and causes that player to have 3 consecutive marks - "Continue" if the move is valid and the game should continue Important: an invalid move does not change the board and does not advance the turn. You do not need to detect draws. If no one wins, valid moves simply return "Continue".

Constraints
1 <= n <= 10^91 <= k <= 10^50 <= len(moves) <= 2 * 10^5Each move is a list of three integers: [player, row, col]Row and column in the input may be outside the board and should then be treated as an invalid moveYour approach should be efficient for large n, so avoid scanning the whole board after every move

EXAMPLES

```
Input: (3, 2, [])
Expected Output: []
```

Explanation: No moves are made, so there are no statuses to report.

QUESTION

```
Input: (5, 2, [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2, 1, 1], [1, 0, 2], [2, 2, 2]])
Expected Output: ['Continue', 'Continue', 'Continue', 'Continue', '1 won', 'Game Over']
```

Explanation: Player 1 forms three consecutive marks horizontally at (0,0), (0,1), and (0,2). Any later move returns 'Game Over'.

Hints

Only the latest valid move can create a new 3-in-a-row, so you never need to rescan the entire board.For large boards, store only occupied cells in a hash map keyed by (row, col), then check the 4 line directions around the new move.

WHY IT WORKS

rows[p][r], cols[p][c], diag[p] and anti[p] count each player's marks on those lines. A move increments the counters for its row, column and, if applicable, the diagonals; the player wins when one reaches n. A separate set of occupied cells rejects repeated moves.

SHORT OPTIMAL CODE

```python
from collections import Counter
from collections import defaultdict


# BASE QUESTION (optimal): Counters per player and per
# line
# Idea: rows[p][row], cols[p][col], diag[p] and anti[p] count
# each player's marks on those lines. A move increments
# the counters for its row, column and, if applicable, the
# diagonals; the player wins when one reaches n. A
# separate set of occupied cells rejects repeated moves.
class TicTacToe:
    def __init__(self, size):
        self.size = size
        self.rows, self.cols = defaultdict(Counter), defaultdict(Counter)
        self.diagonal, self.anti_diagonal, self.taken = Counter(), Counter(), set()

    def move(self, row, col, player):
        if (row, col) in self.taken:
            return -1
        self.taken.add((row, col))
        self.rows[player][row] += 1
        self.cols[player][col] += 1
        won = self.rows[player][row] == self.size or self.cols[player][col] == self.size
        if row == col:
            self.diagonal[player] += 1
            won = won or self.diagonal[player] == self.size
        if row + col == self.size - 1:
            self.anti_diagonal[player] += 1
            won = won or self.anti_diagonal[player] == self.size
        return player if won else 0
```

COMPLEXITY | Time O(1) per move | Space O(k * n)

177 Implement iterator merging lists with filtering — MEDIUM

Template move: keep a position per list and a turn counter; advance() looks for the next unblocked, unseen value and stores it for next()/has_next()

QUESTION

Design an iterator over several lists that takes one element from each list in turn (a zigzag), skipping any value in the blocked set and any value that was already returned. hasNext() says whether another value is available and next() returns it.

QUESTION

Function signature

class FilteredZigzag:
  def __init__(self, lists: list, blocked=())
  def hasNext(self) -> bool
  def next(self) -> int

EXAMPLES

Example 1	

Input	`it = FilteredZigzag([[1, 2, 3], [4, 5, 6, 7], [8, 9]], blocked=[2, 5])`
	`out = []`
	`while it.hasNext():`
	`out.append(it.next())`
	`out`

Output	`out -> [1, 4, 8, 9, 3, 6, 7]`
Why	`Order of visit: 1, 4, 8, then 2 (blocked), 5 (blocked), 9, then 3, 6, then 7. The blocked values never appear.`

Example 2	

Input	`it = FilteredZigzag([[1, 1], [1, 2]])`
	`out = []`
	`while it.hasNext():`
	`out.append(it.next())`
	`out`

Output	`out -> [1, 2]`
Why	`Repeated values are returned only once.`

WHY IT WORKS

Keep a pointer per list and a rotation index. _advance() scans round robin for the next value that is not blocked and not already yielded and stores it as the lookahead (or None when everything is exhausted). hasNext() is 'lookahead is not None'; next() returns the lookahead and calls _advance().

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Lazy iterator with one pointer
# per list and a lookahead
# Idea: Keep a pointer per list and a rotation index.
# _advance() scans round robin for the next value that is
# not blocked and not already yielded and stores it as the
# lookahead (or None when everything is exhausted).
# hasNext() is 'lookahead is not None'; next() returns the
# lookahead and calls _advance().
class FilteredZigzag:
    def __init__(self, lists, blocked=()):
        self.lists, self.blocked = lists, set(blocked)
        self.positions, self.turn, self.seen, self.lookahead = [0] * len(lists), 0, set(), None
        self._advance()

    def _advance(self):
        self.lookahead = None
        misses = 0
        while misses < len(self.lists):
            list_index = self.turn
            self.turn = (self.turn + 1) % len(self.lists)
            if self.positions[list_index] < len(self.lists[list_index]):
                value = self.lists[list_index][self.positions[list_index]]
                self.positions[list_index] += 1
                misses = 0
```

```
                if value not in self.blocked and value not in self.seen:
                    self.lookahead = value
                    return
            else:
                misses += 1

    def hasNext(self):
        return self.lookahead is not None

    def next(self):
        value = self.lookahead
        self.seen.add(value)
        self._advance()
        return value
```

COMPLEXITY | Time O(1) amortized per value | Space O(number of lists + distinct ids yielded)

178 Infinite Deck Card Duel — MEDIUM

Template move: an infinite deck is just a random draw; play rounds, compare the highest cards, and decide the champion by the lead in wins

QUESTION

Design an InfiniteDeck that can be drawn from forever: each call to draw() returns a uniformly random integer card value in [1, 13] (cards are never exhausted). Make the deck seedable so games are reproducible. Then simulate a duel between two players. In each round, each player draws hand_size cards. The round winner is the player holding the single highest card that round; if both players' highest cards are equal, the round is a draw (nobody scores). Play num_rounds rounds and tally round wins per player. Finally, declare a champion: the player is champion only if they lead the other in round wins by at least 2; otherwise there is no champion. Implement: InfiniteDeck.draw() returning the next card. round_winner(hand1, hand2) -> 1 , 2 , or 0 (0 meaning draw), based on the highest card in each hand. decide_champion(wins1, wins2) -> 1 , 2 , or 0 (0 meaning no champion), applying the "lead by at least 2" rule. play_game(...) tying it together (uses the deck for randomness).

Function signature

```
import random
class InfiniteDeck:
    def __init__(self, seed: int | None = None) -> None:
        """Create a deck backed by a (optionally seeded) RNG."""
        ...
    def draw(self) -> int:
        """Return a uniformly random card value in [1, 13]."""
        ...
def round_winner(hand1: list[int], hand2: list[int]) -> int:
    """Return 1 if player 1's max card is higher, 2 if player 2's is higher, 0 on a
tie."""
    ...
def decide_champion(wins1: int, wins2: int) -> int:
    """Return the champion (1 or 2) only if they lead by >= 2 round wins, else 0."""
    ...
def play_game(deck: InfiniteDeck, hand_size: int = 5, num_rounds: int = 100) -> int:
    """Play the duel and return the champion via decide_champion, or 0 if none."""
    ...
```

Follow-up (as reported): Extend to n players: replace the two win counters with a dictionary keyed by player id and pick the highest card across all hands each round. Introduce multiple independent InfiniteDeck instances (or one shared deck with per-card metadata) to support probabilistic house rules.

EXAMPLES

QUESTION

Example 1	
Input	`round_winner([3, 7, 2], [7, 1, 9])`
Output	`2`
Why	`Player 2's highest card (9) beats player 1's highest (7).`

Example 2	
Input	`round_winner([5, 5], [5, 4])`
Output	`0`
Why	`Both players' highest card is 5, so the round is a draw.`

CONSTRAINTS

- 1 <= hand_size <= 20 , 1 <= num_rounds <= 100000 , card values in [1, 13] .

WHY IT WORKS

The deck owns the Random; the rules are plain functions of hands and win counts.

SHORT OPTIMAL CODE

```python
import random


# BASE QUESTION (optimal): A seedable deck and pure game
# rules
# Idea: The deck owns the Random; the rules are plain
# functions of hands and win counts.
class InfiniteDeck:
    def __init__(self, seed=None):
        self.rng = random.Random(seed)

    def draw(self):
        return self.rng.randint(1, 13)


def round_winner(hand1, hand2):
    if max(hand1) == max(hand2):
        return 0
    return 1 if max(hand1) > max(hand2) else 2


def decide_champion(wins1, wins2):
    if wins1 - wins2 >= 2:
        return 1
    return 2 if wins2 - wins1 >= 2 else 0


def play_game(deck, hand_size=5, num_rounds=100):
    wins = {1: 0, 2: 0}
    for _ in range(num_rounds):
        first_hand = [deck.draw() for _ in range(hand_size)]
        second_hand = [deck.draw() for _ in range(hand_size)]
        winner = round_winner(first_hand, second_hand)
        if winner:
            wins[winner] += 1
    return decide_champion(wins[1], wins[2])
```

COMPLEXITY | Time O(rounds * hand) | Space O(1)

179 Design an elevator control system

MEDIUM

Template move: each car keeps floor, direction and stops; a request goes to the car with the lowest cost; step() moves every car one floor

QUESTION

Design the control logic for a multi-elevator building. A Request(floor, direction) is a call from a floor (direction +1 for up, -1 for down). Each Elevator keeps its current floor, direction and a set of stops. cost(request) estimates how expensive it is for this car to take the request (the distance, plus a detour penalty when the car is heading away); the dispatcher gives the request to the cheapest car. step() moves the car one floor towards its stops and returns the floor it served, if any.

Function signature

class Elevator:
  def __init__(self, car_id, floor=0)
  def cost(self, req: Request) -> float
  def step(self) -> int | None

EXAMPLES

Example 1

Input	`e = Elevator(1, floor=0)`
	`e.stops.update({3, 5})`
	`[e.step() for _ in range(6)]`

Output	`[e.step() for _ in range(6)] -> [null, null, null, 3, null, 5]`
Why	The car climbs one floor per step and stops at floors 3 and 5 (served when it is on the floor at the start of a step).

Example 2

Input	`idle = Elevator(2, floor=7)`
	`busy = Elevator(3, floor=2)`
	`busy.direction = 1`
	`busy.stops.update({9})`
	`idle.cost(Request(4, -1))`
	`busy.cost(Request(4, 1))`
	`busy.cost(Request(4, -1))`

Output	`idle.cost(Request(4, -1)) -> 3`
	`busy.cost(Request(4, 1)) -> 2`
	`busy.cost(Request(4, -1)) -> 14`

Why	An idle car costs its distance (3). A car already going up that will pass floor 4 going the same way costs its distance (2); going the other way it must finish its run first, so it costs much more.

WHY IT WORKS

Each car keeps a set of stops and a direction. step() moves one floor in its direction and removes a stop it reaches; when no stops remain ahead it reverses, and when none remain at all it goes idle. The controller assigns a hall call to the car with the lowest cost: distance, plus a penalty if the car is moving away or is out of service. Cars that fail drop out and their stops are reassigned.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Elevator (SCAN) per car and
# cost-based assignment of calls
# Idea: Each car keeps a set of stops and a direction.
# step() moves one floor in its direction and removes a
# stop it reaches; when no stops remain ahead it reverses,
# and when none remain at all it goes idle. The controller
# assigns a hall call to the car with the lowest cost:
# distance, plus a penalty if the car is moving away or is
# out of service. Cars that fail drop out and their stops
```

```
# are reassigned.
class Request:
    def __init__(self, floor, direction):
        self.floor, self.direction = floor, direction              # direction: +1 up, -1 down

class Elevator:
    def __init__(self, car_id, floor=0):
        self.car_id, self.floor, self.direction, self.stops, self.in_service = car_id, floor, 0, set(), True

    def cost(self, call):
        if not self.in_service:
            return float("inf")
        distance = abs(self.floor - call.floor)
        if self.direction == 0:
            return distance
        ahead = (call.floor - self.floor) * self.direction >= 0
        return distance if ahead and call.direction == self.direction else distance + 2 * (max(self.stops) - mi
n(self.stops) + 1 if self.stops else 0) + 10

    def step(self):
        served = None
        if self.floor in self.stops:
            self.stops.discard(self.floor)
            served = self.floor
        if not self.stops:
            self.direction = 0
            return served
        up = [stop for stop in self.stops if stop > self.floor]
        down = [stop for stop in self.stops if stop < self.floor]
        if self.direction >= 0 and up or (self.direction < 0 and not down):
            self.direction = 1
        else:
            self.direction = -1
        self.floor += self.direction
        return served
```

COMPLEXITY | Time O(cars) per call, O(1) per car per step | Space O(stops)

180 Simulate In-Place Cellular State Updates

HARD

Template move: encode old and new state in two bits of each cell so every cell is computed from the old state; shift at the end

QUESTION

Conway's Game of Life on a grid of 0 (dead) and 1 (alive) cells. A live cell with 2 or 3 live neighbours (of its 8 neighbours) stays alive, a dead cell with exactly 3 live neighbours becomes alive, and every other cell is dead in the next generation. All cells change at the same time. Do it in place with O(1) extra space; with wrap=True the grid wraps around its edges.

Function signature

def step_in_place(board: list, wrap=False) -> list

EXAMPLES

Example 1	
Input	`[[0, 1, 0], [0, 1, 0], [0, 1, 0]]`
Output	`[[0, 0, 0], [1, 1, 1], [0, 0, 0]]`
Why	The vertical blinker turns into a horizontal one.

Example 2	
Input	`[[1, 1], [1, 1]]`
Output	`[[1, 1], [1, 1]]`
Why	A 2x2 block is stable: every cell has 3 live neighbours.

QUESTION

Example 3	
Input	`[[0, 0, 0], [0, 1, 0], [0, 0, 0]]`
Output	`[[0, 0, 0], [0, 0, 0], [0, 0, 0]]`
Why	`A lonely cell dies of isolation.`

WHY IT WORKS

Bit 0 holds the current state and bit 1 holds the next state. First pass: count neighbors using only bit 0 (value & 1) and set bit 1 by the rules. Second pass: shift every cell right by one to expose the new state. Reading with & 1 means neighbors that were already updated still show their old state.

SHORT OPTIMAL CODE

```
def _life_neighbors(board, row, col, is_live):
    return sum(is_live(board[neighbour_row][neighbour_col]) for neighbour_row in range(row - 1, row + 2) for ne
ighbour_col in range(col - 1, col + 2)
               if (neighbour_row, neighbour_col) != (row, col) and 0 <= neighbour_row < len(board) and 0 <= nei
ghbour_col < len(board[0]))


# BASE QUESTION (optimal): Two-bit encoding in the same
# board
# Idea: Bit 0 holds the current state and bit 1 holds the
# next state. First pass: count neighbors using only bit 0
# (value & 1) and set bit 1 by the rules. Second pass:
# shift every cell right by one to expose the new state.
# Reading with & 1 means neighbors that were already
# updated still show their old state.
def gameOfLife(board):
    rows, cols = len(board), len(board[0])
    for row in range(rows):
        for col in range(cols):
            live_neighbours = _life_neighbors(board, row, col, lambda cell: cell & 1)
            if (board[row][col] & 1 and live_neighbours in (2, 3)) or (not board[row][col] & 1 and live_neighbo
urs == 3):
                board[row][col] |= 2
    for row in range(rows):
        for col in range(cols):
            board[row][col] >>= 1
    return board
```

COMPLEXITY | Time O(m * n) | Space O(1)

GROUP 23 OF 43 | 6 QUESTIONS | SHORT MAIN CODE

Top-K by frequency: Counter plus nsmallest

Pattern

Count with a Counter, then ask heapq.nsmallest for the k best using a key of (negative count, tie-break). That is the whole solution for almost every top-K question; only what you count and the tie-break change.

181 Top-K Frequent in Large Logs — MEDIUM

Template move: count the first word of every line (the speaker); return the names of the k most frequent

QUESTION

A chat log is a list of lines, each starting with the speaker's name ('A says hi'). Return the k speakers who appear most often; ties are broken alphabetically by name.

Function signature

def top_k_speakers(chat_log: list, k: int) -> list

EXAMPLES

Example 1	
Input	['A says hi', 'B says hi', 'A says bye', 'C says hi', 'A says yo'], 2
Output	['A', 'B']
Why	A spoke 3 times, B and C once each; B comes before C alphabetically.

Example 2	
Input	['A says hi', 'B says hi', 'A says bye', 'C says hi', 'A says yo'], 3
Output	['A', 'B', 'C']
Why	Asking for 3 returns everyone, most frequent first.

PART 1: Count, then keep a size-K heap

Parse the speaker from each line and count in a hash map. nsmallest keeps a heap of size k, so it never sorts all d distinct users. Ties break by user name so the output is deterministic.

PART 2: Quickselect on the counts (average O(d))

Partition the (count, user) pairs around a random pivot until exactly the k largest sit on the left. Random pivots make the bad case unlikely; mention median-of-medians if a guaranteed O(d) bound is asked for.

SHORT OPTIMAL CODE

```
from collections import Counter
import heapq


# BASE QUESTION (optimal): Count, then keep a size-K heap
# Idea: Parse the speaker from each line and count in a
# hash map. nsmallest keeps a heap of size k, so it never
# sorts all d distinct users. Ties break by user name so
# the output is deterministic.
def top_k_speakers(chat_log, how_many):
    counts = Counter(line.split(" ", 1)[0] for line in chat_log)
    return [user for user, _ in heapq.nsmallest(how_many, counts.items(), key=lambda entry: (-entry[1], entry[0
]))]
```

COMPLEXITY | Part 1: Time O(n + d log k) | Space O(d)
Part 2: Time O(n + d) average, O(d^2) worst case | Space O(d)

182 Rearrange Tasks With Cooldown

MEDIUM

Template move: only the biggest count matters: answer = max(len(tasks), (biggest - 1) * (n + 1) + how many tasks share the biggest count)

QUESTION

You are given a list of tasks (letters; the same letter means the same kind of task). Every task takes one unit of time, and two tasks of the same kind must have at least n other tasks or idle units between them (the cooldown). The CPU may stay idle. Return the least number of time units needed to finish all the tasks.

Function signature

def least_interval(tasks: list, n: int) -> int

EXAMPLES

Example 1	
Input	['A', 'A', 'A', 'B', 'B', 'B'], 2
Output	8
Why	A B idle A B idle A B: three A's and three B's with a cooldown of 2 take 8 units.

Example 2	
Input	['A', 'A', 'A', 'B', 'B', 'B'], 0
Output	6
Why	No cooldown: just run the 6 tasks back to back.

Example 3	
Input	['A', 'B', 'C'], 5
Output	3
Why	All tasks differ, so no cooldown is ever needed.

WHY IT WORKS

Let m be the highest frequency and k the number of tasks that have that frequency. The skeleton needs (m - 1) * (n + 1) + k slots. If there are more tasks than that, no idle time is needed and the answer is len(tasks). So the answer is max(len(tasks), (m - 1) * (n + 1) + k).

SHORT OPTIMAL CODE

```
from collections import Counter


# BASE QUESTION (optimal): Closed form from the most
# frequent task
# Idea: Let highest_frequency be the highest frequency and tasks_at_highest the number of
# tasks that have that frequency. The skeleton needs (highest_frequency -
# 1) * (cooldown + 1) + tasks_at_highest slots. If there are more tasks than
# that, no idle time is needed and the answer is
# len(tasks). So the answer is max(len(tasks), (highest_frequency - 1) *
# (cooldown + 1) + tasks_at_highest).
def leastInterval(tasks, cooldown):
    frequencies = Counter(tasks).values()
    highest_frequency = max(frequencies, default=0)
    tasks_at_highest = sum(1 for frequency in frequencies if frequency == highest_frequency)
    return max(len(tasks), (highest_frequency - 1) * (cooldown + 1) + tasks_at_highest) if tasks else 0
```

COMPLEXITY | Time O(len(tasks)) | Space O(1) for a fixed alphabet

183 Implement frequency + distance top-K queries

MEDIUM

Template move: count the point tuples; the tie-break is distance from the origin, then x, then y

QUESTION

You will implement solutions for two coding interview questions.

Question

1: Return the top-K points by frequency (with distance tie-break) You are given an array **`points`** of 2D integer points, where each point is represented as **`[x, y]`**. The array may contain duplicates (the same **`[x, y]`** can appear multiple times). Define:

- **`freq(p)`** = number of times point **p** appears in the array.
- **`dist(p)`** = squared Euclidean distance to the origin, **`x^2 + y^2`**. Return **k distinct points** sorted by the following priority:

1. Higher frequency first (**`freq`** descending)
2. If frequencies tie, smaller distance first (**`dist`** ascending)
3. If still tied, sort by **`x`** ascending, then **`y`** ascending

Input

- **`points`**: list of length **n**, each element **`[x, y]`**
- **`k`**: integer, **`1 <= k <= (# of distinct points)`**

Output

- A list of **k** distinct points following the ranking above.

Constraints (typical interview assumptions)

- **`1 <= n <= 2 * 10^5`**
- **`-10^4 <= x, y <= 10^4`** ---

Question

2: Time-based key-value store with “closest timestamp” lookup Design a data structure supporting the following operations:

- **`set(key, value, timestamp)`**: store the string **`value`** for string **`key`** at integer **`timestamp`**.
- Assume timestamps for the same **`key`** are inserted in **non-decreasing** order.
- **`getClosest(key, timestamp) -> value or ""`**:
- If **`key`** has no stored values, return **`""`**.
- Otherwise, return the **`value`** whose stored timestamp is **closest** to the query **`timestamp`**.
- If there is a tie (equal distance to two timestamps), return the value with the **smaller** timestamp.

Example

If for key **`"a"`** you stored timestamps **`[2, 10, 15]`** with values **`["x", "y", "z"]`**:

- **`getClosest("a", 11)`** returns **`"y"`** (10 is closest)
- **`getClosest("a", 14)`** returns **`"z"`** (15 is closest)
- **`getClosest("a", 12)`** returns **`"y"`** (10 and 15 are equally far; choose smaller timestamp)

What to discuss

- Time and space complexity of your approach - How you would improve a “naive” solution (e.g., linear scan) to a more optimal one

Constraints (typical interview assumptions)

- Up to **`2 * 10^5`** total operations - Keys and values are strings; timestamps are integers within typical 32-bit range

QUESTION

Part 1: Top-K Points by Frequency with Distance Tie-BreakGiven a list of 2D integer points, where duplicate points may appear, return the top **k** distinct points ranked by these rules: (1) higher frequency first, (2) if frequencies tie, smaller squared distance to the origin first, and (3) if still tied, smaller **x** first, then smaller **y**. The result must contain exactly **k** distinct points in ranked order.

Constraints

1 <= len(points) <= 2 * 10^5-10^4 <= x, y <= 10^41 <= k <= number of distinct points

EXAMPLES

```
Input: ([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)
Expected Output: [[2, 2], [1, 1]]
Explanation: Point [2,2] appears 3 times and [1,1] appears 2 times, so they are the top 2 distinct
    points.
Input: ([[1,2],[0,2],[1,2],[2,1],[0,2],[2,1],[3,0]], 3)
Expected Output: [[0, 2], [1, 2], [2, 1]]
Explanation: The points [0,2], [1,2], and [2,1] each appear twice. Their squared distances are 4, 5, and
    5, so [0,2] comes first. Between [1,2] and [2,1], x is smaller for [1,2].
```

Hints

First count how many times each point appears. After that, each distinct point only needs to be ranked once.Python can sort tuples directly, so think about building a sort key like **`(-frequency, distance, x, y)`**.

Part 2: Time-Based Key-Value Store with Closest Timestamp QueriesProcess a sequence of operations for a time-based key-value store. Each operation is either **`['set', key, value, timestamp]`** or **`['getClosest', key, timestamp]`**. For every **`getClosest`** operation, return the stored value whose timestamp is closest to the query tim

PART 1: Rank points

Count equal points and keep the best k by frequency, distance, then coordinates.

PART 2: Time store

Keep each key's records sorted and use binary search around the requested time.

SHORT OPTIMAL CODE

```python
import heapq
from bisect import bisect_left, bisect_right, insort
from collections import Counter, defaultdict


def top_points(points, how_many):
    counts = Counter(points)
    best = heapq.nsmallest(
        how_many, counts,
        key=lambda point: (-counts[point], point[0] ** 2 + point[1] ** 2, point),
    )
    return [list(point) for point in best]


class TimeStore:
    def __init__(self):
        self.data = defaultdict(list)

    def set(self, key, value, timestamp):
        insort(self.data[key], (timestamp, value), key=lambda item: item[0])

    def get_closest(self, key, timestamp):
        items = self.data.get(key, [])
        if not items:
            return ""
        position = bisect_left(items, timestamp, key=lambda item: item[0])
        choices = items[max(0, position - 1):position + 1]
        return min(choices, key=lambda item: (abs(item[0] - timestamp), item[0]))[1]
```

```
    def get_at_or_before(self, key, timestamp):
        items = self.data.get(key, [])
        position = bisect_right(items, timestamp, key=lambda item: item[0]) - 1
        return items[position][1] if position >= 0 else ""
```

COMPLEXITY | Part 1: Time O(n + u log k) | Space O(u)
Part 2: Time set O(n), get O(log n) | Space O(n)

184 Find top/bottom-k words in list or stream

MEDIUM

Template move: the same Counter; run nsmallest once with -count and once with +count for the bottom-k

QUESTION

You are given words (strings) either as a finite list or as an unbounded stream. 1)
List version: Given an array **`words`** and an integer **k**, return: - the **k** most frequent distinct words, and - the **k** least frequent distinct words. Specify how you break ties (e.g., lexicographically, or any order). 2)
Stream version (online): Design a data structure that supports:

- **`ingest(word)`**: process the next word in the stream - **`topK(k)`**: return the **k** most frequent distinct words seen so far - **`bottomK(k)`**: return the **k** least frequent distinct words seen so far Discuss time/space complexity and how you would implement these operations efficiently. 3)

Variant (discuss/design): Extend the stream design to support returning the **first k** and/or **last k** words that are currently **non-repeating** (i.e., words with frequency exactly 1) according to stream arrival order.

Part 1: Top and Bottom K Frequent Words in a ListGiven an array of strings words and a non-negative integer k, return two lists: the k most frequent distinct words and the k least frequent distinct words. Use deterministic tie-breaking: if two words have the same frequency, the lexicographically smaller word comes first. If k is larger than the number of distinct words, return all distinct words.
Constraints
0 <= len(words) <= 2 * 10^50 <= k <= 2 * 10^5Each word is a non-empty stringComparison is case-sensitive unless otherwise stated

EXAMPLES

```
Input: (['i', 'love', 'leetcode', 'i', 'love', 'coding'], 2)
Expected Output: [['i', 'love'], ['coding', 'leetcode']]
Explanation: `i` and `love` both appear twice, so lexicographical order gives `['i', 'love']`. `coding`
    and `leetcode` both appear once, so lexicographical order gives `['coding', 'leetcode']`.
Input: (['apple', 'banana', 'apple', 'cherry', 'banana', 'date'], 3)
Expected Output: [['apple', 'banana', 'cherry'], ['cherry', 'date', 'apple']]
```

Explanation: Frequencies are: apple=2, banana=2, cherry=1, date=1. Top 3 are ordered by frequency descending, then lexicographically: **`apple`**, **`banana`**, **`cherry`**. Bottom 3 are ordered by frequency ascending, then lexicographically: **`cherry`**, **`date`**, **`apple`**.

QUESTION

Hints

Count how many times each distinct word appears before deciding the answer.You can sort the distinct words twice: once by (-frequency, word) and once by (frequency, word).

Part 2: Online Top/Bottom K Words from a StreamYou are given a sequence of stream operations. Each operation is a 2-element list of strings: ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]. Process the operations in order. For each query, return the k most frequent or k least frequent distinct words seen so far. Ties are broken lexicographically ascending. If k is larger than the number of distinct words seen so far, return all distinct words.

Constraints

1 <= len(operations) <= 2 * 10^5Each operation is one of ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]0 <= k <= 2 * 10^5Queries may appear before any word is ingested

Examples

Input: [['ingest', 'apple'], ['ingest', 'banana'], ['ingest', 'apple'], ['topK', '2'], ['bottomK', '2']]

Expected Output: [['apple', 'banana'], ['banana', 'apple']]

Explanation: After the ingests, counts are apple=2 and banana=1. topK(2) returns the most frequent first, while bottomK(2) returns the least frequent first.

Input: [['topK', '3'], ['ingest', 'dog'], ['ingest', 'cat'], ['ingest', 'cat'], ['topK', '2'], ['bottomK', '2'], ['topK', '5']]

Expected Output: [[], ['cat', 'dog'], ['dog', 'cat'], ['cat', 'dog']]

Explanation: The first query happens before any words are seen, so it returns []. Later, cat has frequency 2 and dog has frequency 1.

Hints

Keep an exact frequency map for all words seen so far.A max-heap and a min-heap with lazy deletion let you support online updates without fully resorting all words

WHY IT WORKS

One pass counts the words. heapq.nsmallest with key (-count, word) gives the K most frequent and with key (count, word) the K least frequent, each in O(d log k) without a full sort.

SHORT OPTIMAL CODE

```
from collections import Counter
import heapq


# BASE QUESTION (optimal): Counter plus two heaps of size
# K
# Idea: One pass counts the words. heapq.nsmallest with
# key (-count, word) gives the K most frequent and with
# key (count, word) the K least frequent, each in O(d log
# how_many) without a full sort.
def top_bottom(words, how_many):
    counts = Counter(words)
    return (heapq.nsmallest(how_many, counts, key=lambda word: (-counts[word], word)), heapq.nsmallest(how_many
, counts, key=lambda word: (counts[word], word)))
```

COMPLEXITY | Time O(n + d log k) | Space O(d)

185 Solve chat, grid paths, and car rentals

MEDIUM

Template move: the top-K part is the template as written; the other parts are a rolling-column path count and a greedy car assignment

QUESTION

You are given three independent programming tasks. For each, describe and implement an efficient algorithm, and be prepared to analyze its time and space complexity. ---

Q1.

Find top-K most active chat users You are given a log of chat messages from a messaging application.
Input:

- An array **`messages`** of length **`n`**, where each element **`messages[i]`** is the **`user_id`** (string or integer) of the sender of the i-th message.
- An integer **`k`** (1 <= k <= number of distinct users).

Output:

- Return a list of **`k`** user IDs corresponding to the users who have sent the most messages.

Details & requirements:

- The "activity" of a user is defined as the number of messages they have sent (i.e., their frequency in **`messages`**).
- If there are fewer than **`k`** distinct users, return all distinct users.
- If multiple users have the same frequency, you may break ties arbitrarily, or (for determinism) sort tied users by **`user_id`** in ascending order.
- Aim for an algorithm that is more efficient than sorting all users by count when **`k`** is much smaller than the number of distinct users. Design and implement a function, for example:
- **`List<UserId> topKActiveUsers(List<UserId> messages, int k)`** Also explain:
- At least two different approaches (e.g., based on sorting, heaps, or selection algorithms).
- The time and space complexity of each approach. ---

Q2.

Count paths in a 2D grid with constrained moves You have a 2D grid with **`m`** rows and **`n`** columns. Index rows from top to bottom as **`0..m-1`** and columns from left to right as **`0..n-1`**.

- The **start** cell is the bottom-left corner: **`(m-1, 0)`**.
- The **end** cell is the bottom-right corner: **`(m-1, n-1)`**. From any cell **`(r, c)`**, you may move **only** to the next column to the right, in one of up to three ways (if within bounds):
- Right-up: **`(r-1, c+1)`**
- Right: **`(r, c+1)`**
- Right-down: **`(r+1, c+1)`** You cannot move left or stay in the same column. All cells are passable (no obstacles).

Task:

- Given integers **`m`** and **`n`**, compute the total number of distinct paths from the start cell **`(m-1, 0)`** to the end cell **`(m-1, n-1)`** using only the allowed moves.

Requirements:

- Implement a function such as **`long countPaths(int m, int n)`**.
- The result may be large; assume it fits in a 64-bit integer, or, if you prefer, you may return the count modulo a large prime (e.g., 1,000,000,007) — specify which convention you are using.
- First design a straightforward dynamic programming solution using O(m·n) time and O(m·n) space.
- Then describe how to optimize the space usage (e.g., to O(m)) while keeping the same time complexity, and explain why the optimization is correct. ---

Q3.

QUESTION

Minimum cars needed for rentals and assignment You work at a car rental company. You are given a list of rental bookings, each with a pickup and return time.

Input:

- An array of **`R`** rental records. Each record has:
- **`rental_id`**: a unique identifier (e.g., integer or string).
- **`pickup_time`**: an integer start time.
- **`return_time`**: an integer end time, with **`pickup_time < return_time`**. Assume times are on a single timeline (e.g., minutes from some epoch). A car can be reassigned immediately after it is returned, i.e., if one rental ends at time **`t`** and another starts at time **`t`**, the same car can serve both.

Tasks: 1.

Capacity: Compute the **minimum number of cars** required so that all rentals can be fulfilled without time conflicts on any single car. 2.

Assignment: Produce one valid assignment of rentals to cars that uses exactly that minimum number of cars.

Output:

- An integer **`C`** = minimal number of cars needed.
- A mapping from each **`rental_id`** to a **`car_id`** in the range **`1..C`**, such that no two rentals assigned to the same **`car_id`** overlap in time.

Requirements & clarifications:

- Overlap definition: two rentals with intervals **`[s1, e1)`** and **`[s2, e2)`** overlap if their time intervals intersect with non-empty duration. Using half-open intervals, if **`e1 == s2`** they do **not** over

EXAMPLES

Example 1	
Input	`['ann', 'bob', 'ann', 'cy', 'bob', 'ann'], 2`
Output	`['ann', 'bob']`
Why	`Task 1: ann sent 3 messages, bob 2, cy 1; the top 2 are ann and bob.`

Example 2	
Input	`3, 3`
Output	`2`
Why	`Task 2: paths over 3 columns of a 3-row grid that start and end in the bottom row, moving up-right, right or down-right.`

Example 3	
Input	`[(1, 0, 5), (2, 1, 4), (3, 5, 8)]`
Output	`(2, {1: 0, 2: 1, 3: 1})`
Why	`Task 3: rentals are (id, pickup, return). Rental 3 starts at 5, after rental 2 returned its car at 4, so it reuses that car: 2 cars in total.`

PART 1: Active users

Count messages and keep the best k users.

PART 2: Grid paths

Carry one column of path counts into the next column.

PART 3: Rental cars

Keep current return times in a min-heap and reuse a returned car.

SHORT OPTIMAL CODE

```python
import heapq
from collections import Counter


def top_k_users(messages, how_many):
    counts = Counter(messages)
    return heapq.nsmallest(how_many, counts, key=lambda user: (-counts[user], user))


def count_paths(rows, cols, blocked=(), mod=None):
    blocked = set(blocked)
    ways = [0] * rows
    ways[-1] = 1
    for col in range(1, cols):
        following = [0] * rows
        for row in range(rows):
            if (row, col) not in blocked:
                following[row] = ways[row]
                if row:
                    following[row] += ways[row - 1]
                if row + 1 < rows:
                    following[row] += ways[row + 1]
                if mod:
                    following[row] %= mod
        ways = following
    return ways[-1]


def min_cars(rentals):
    returns = []
    for pickup, back in sorted(rentals):
        if returns and returns[0] <= pickup:
            heapq.heapreplace(returns, back)
        else:
            heapq.heappush(returns, back)
    return len(returns)
```

COMPLEXITY | Part 1: Time O(n + u log k) | Space O(u)
Part 2: Time O(m * n) | Space O(m)
Part 3: Time O(n log n) | Space O(n)

186 Top-K Chatters by Word Count

MEDIUM

Template move: count len(body.split()) per speaker instead of lines; ties by name

QUESTION

You are given a chat transcript as a list of message lines. Each line has the form "Name: message text" , where everything before the first ": " is the author's name and everything after it is the message body. Count the number of words each person typed across the entire transcript (a word is a maximal run of non- whitespace characters in the body). A person may appear on many lines; sum their words. Return the k people who typed the most words, as a list of (name, word_count) tuples sorted by word_count descending. Break ties by name in ascending (lexicographic) order.

Function signature

def top_k_chatters(lines: list[str], k: int) -> list[tuple[str, int]]:
    ...

Follow-up (as reported): The interviewer's real ask was to implement the parsing inside a class alongside stubbed API methods. Wrap this in a ChatAnalyzer class that ingests lines incrementally via add_line(line) and answers top_k(k) on demand.

EXAMPLES

QUESTION

Example 1	
Input	`lines = ["Alice: hello there friend", "Bob: hi", "Alice: how are you"], k = 1`
Output	`[("Alice", 6)]`
Why	Alice typed 3 + 3 = 6 words, Bob typed 1. Top 1 is Alice.

Example 2	
Input	`lines = ["Ann: a b", "Bob: c d", "Cara: e"], k = 2`
Output	`[("Ann", 2), ("Bob", 2)]`
Why	Ann and Bob both have 2 words; tie broken alphabetically, so Ann precedes Bob. Cara (1 word) is excluded.

Example 3	
Input	`lines = [], k = 3`
Output	`[]`

CONSTRAINTS

- 0 <= len(lines) <= 10^5
- Every line contains exactly one ": " separating name from body; the name is non-empty and contains no ": " .
- The body may be empty (0 words).
- 1 <= k ; if k exceeds the number of distinct people, return all of them.

WHY IT WORKS

Sum the words per author, then pick the k best with a heap keyed by (-count, name).

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Counter and a bounded selection
# Idea: Sum the words per author, then pick the k best
# with a heap keyed by (-count, name).
import heapq
from collections import Counter


def top_k_chatters(lines, how_many):
    counts = Counter()
    for line in lines:
        name, body = line.split(": ", 1)
        counts[name] += len(body.split())
    return heapq.nsmallest(how_many, counts.items(), key=lambda entry: (-entry[1], entry[0]))
```

COMPLEXITY | Time O(n + p log k) | Space O(p)

GROUP 24 OF 43 | 6 QUESTIONS | SHORT MAIN CODE

Digits and characters: peel one at a time and carry

Pattern

Work one digit or character at a time: n % 10 peels the last digit, n // 10 removes it, a carry moves overflow to the next place, and (ord(c) - 97 + k) % 26 shifts a letter. Build the answer in a list and reverse or join at the end.

187 Caesar Cipher: Recover Key, Encrypt, and Decrypt — EASY

Template move: per letter: (ord(ch) - 97 + k) % 26 + 97; the key comes from one letter pair; decrypt uses -k

QUESTION

A message was scrambled with a Caesar cipher: every lowercase letter was shifted forward by the same secret amount k (0 <= k < 26) with wraparound (z +1 -> a); all non-lowercase characters were left unchanged. You are given one example pair — a known plaintext and its corresponding ciphertext — from which you must recover the secret shift, and then implement the cipher in both directions. Specifically implement: find_shift(plaintext, ciphertext) -> int : return the shift k (0..25) that maps plaintext to ciphertext . The two strings have equal length and contain at least one lowercase letter that pins down k . encrypt(text, k) -> str : shift each lowercase letter forward by k with wraparound; leave other characters unchanged. decrypt(text, k) -> str : the inverse of encrypt — shift each lowercase letter backward by k with wraparound.

Function signature

```
def find_shift(plaintext: str, ciphertext: str) -> int:
    ...
def encrypt(text: str, k: int) -> str:
    ...
def decrypt(text: str, k: int) -> str:
    ...
```

Follow-up (as reported): Given only a ciphertext (no key and no plaintext), how would you break the cipher? Discuss trying all 26 shifts (brute force) or English letter-frequency analysis.

EXAMPLES

Example 1	
Input	`find_shift("abc", "def")`
Output	`3`
Why	`'a'->'d', 'b'->'e', 'c'->'f' is a uniform forward shift of 3.`

Example 2	
Input	`encrypt("hello world", 3)`
Output	`"khoor zruog"`
Why	`Each letter moves forward 3 ('h'->'k', 'o'->'r', 'w'->'z'); the space is unchanged.`

Example 3	
Input	`decrypt("khoor zruog", 3)`
Output	`"hello world"`
Why	`decrypt inverts encrypt, so the original message is recovered.`

Example 4	
Input	`encrypt("xyz", 3)`
Output	`"abc"`
Why	`Wraparound: 'x'->'a', 'y'->'b', 'z'->'c'.`

QUESTION

CONSTRAINTS

- 1 <= len(text) <= 10^5 . text may contain lowercase letters, spaces, digits, and punctuation; only lowercase letters a – z are shifted.
- For find_shift , len(plaintext) == len(ciphertext) and they align character by character.

WHY IT WORKS

The difference of one letter pair is the key; the same helper shifts forward or backward.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Key from one letter pair,
# shared shift helper
# Idea: The difference of one letter pair is the key; the
# same helper shifts forward or backward.
def find_shift(plaintext, ciphertext):
    for plain_letter, cipher_letter in zip(plaintext, ciphertext):
        if "a" <= plain_letter <= "z":
            return (ord(cipher_letter) - ord(plain_letter)) % 26
    return 0


def shift_text(text, shift):
    shifted = []
    for letter in text:
        if "a" <= letter <= "z":
            letter = chr((ord(letter) - 97 + shift) % 26 + 97)
        shifted.append(letter)
    return "".join(shifted)


def encrypt(text, shift):
    return shift_text(text, shift)


def decrypt(text, shift):
    return shift_text(text, -shift)
```

COMPLEXITY | Time O(n) | Space O(n)

188 Solve several coding interview problems — MEDIUM

Template move: the big-number addition on strings, with int(ch) per digit; the other parts are list-meeting and overlap sweeps

QUESTION

Three small tasks. (1) add_strings(a, b): add two non-negative integers given as digit strings and return the sum as a string, without converting the whole numbers. (2) intersection(head_a, head_b): two singly linked lists may share a common tail; return the first shared node, or None. (3) overlap_depth(rects): rectangles (x1, y1, x2, y2) include their left and bottom edges but not their right and top edges; return the largest number of rectangles that cover a single point.

Function signature

def add_strings(a: str, b: str) -> str
def intersection(head_a, head_b)
def overlap_depth(rects: list) -> int

EXAMPLES

QUESTION

Example 1	
Input	`'999', '1'`
Output	`'1000'`
Why	`A carry runs through every digit: 999 + 1 = 1000.`

Example 2	
Input	`'456', '77'`
Output	`'533'`
Why	`456 + 77 = 533.`

Example 3	

Input	`class N:`
	`def __init__(self, val, next=None):`
	`self.val, self.next = val, next`
	`shared = N(8, N(9))`
	`a = N(1, N(2, shared))`
	`b = N(5, shared)`
	`intersection(a, b).val`

Output	`intersection(a, b).val -> 8`
Why	`List a is 1, 2, 8, 9 and list b is 5, 8, 9: they join at the node holding 8.`

Example 4	
Input	`[(0, 0, 4, 4), (2, 2, 6, 6), (3, 3, 5, 5)]`
Output	`3`
Why	`The point (3, 3) lies inside all three rectangles.`

Example 5	
Input	`[(0, 0, 2, 2), (2, 0, 4, 2)]`
Output	`1`
Why	`Rectangles that only share an edge do not overlap.`

PART 1: Big numbers: digit by digit from the right with a carry

Walk both strings from the right, add the two digits and the carry, append total % 10, carry total // 10; after the loop add a final carry digit if needed and reverse. Never convert the whole string to an integer.

PART 2: Chains that meet: two pointers that switch heads at the end

Walk p along a and q along b; when a pointer reaches the end, restart it at the other chain's head. Both pointers then travel m + n steps in total, so they meet at the first shared node (or both become None together).

PART 3: Overlap: sweep over sorted start and end events

Create +1 at each start and -1 at each end, sort the events (ends before starts at the same time for half-open intervals), and track the running count; its maximum is the answer.

SHORT OPTIMAL CODE

```
# Big numbers: digit by digit from the right with a carry
# Idea: Walk both strings from the right, add the two
# digits and the carry, append total % 10, carry total //
# 10; after the loop add a final carry digit if needed and
# reverse. Never convert the whole string to an integer.
def addStrings(first_number, second_number):
    first_index, second_index, carry, digits = len(first_number) - 1, len(second_number) - 1, 0, []
```

```
    while first_index >= 0 or second_index >= 0 or carry:
        total = carry + (int(first_number[first_index]) if first_index >= 0 else 0) + (int(second_number[second
_index]) if second_index >= 0 else 0)
        digits.append(str(total % 10))
        carry = total // 10
        first_index, second_index = first_index - 1, second_index - 1
    return "".join(digits[::-1])


# Chains that meet: two pointers that switch heads at the
# end
# Idea: Walk p along a and q along b; when a pointer
# reaches the end, restart it at the other chain's head.
# Both pointers then travel m + n steps in total, so they
# meet at the first shared node (or both become None
# together).
class LNode:
    def __init__(self, val, nxt=None):
        self.val, self.next = val, nxt

def getIntersectionNode(first_head, second_head):
    walker_a, walker_b = first_head, second_head
    while walker_a is not walker_b:
        walker_a = walker_a.next if walker_a else second_head
        walker_b = walker_b.next if walker_b else first_head
    return walker_a


# Overlap: sweep over sorted start and end events
# Idea: Create +1 at each start and -1 at each end, sort
# the events (ends before starts at the same time for
# half-open intervals), and track the running count; its
# maximum is the answer.
def max_overlap(intervals):
    events = sorted([(start, 1) for start, _ in intervals] + [(end, -1) for _, end in intervals])
    running_count = best = 0
    for _, change in events:
        running_count += change
        best = max(best, running_count)
    return best
```

COMPLEXITY | Part 1: Time O(max(m, n)) | Space O(max(m, n))
Part 2: Time O(m + n) | Space O(1)
Part 3: Time O(n log n) | Space O(n)

189 Check Whether Digits Are Monotone Increasing

MEDIUM

Template move: peel digits from the right; each digit must not exceed the previous one you saw

QUESTION

Return true when the decimal digits of a non-negative integer never decrease from left to right (repeated digits are allowed). Zero and single-digit numbers qualify.

Function signature

def is_monotone_digits(n: int) -> bool

EXAMPLES

Example 1	
Input	`1234`
Output	`true`
Why	`1, 2, 3, 4 only goes up.`

QUESTION

Example 2	
Input	`1223`
Output	`true`
Why	`Equal neighbours are allowed.`

Example 3	
Input	`332`
Output	`false`
Why	`3, 3, 2 goes down at the end.`

Example 4	
Input	`0`
Output	`true`
Why	`Zero qualifies.`

WHY IT WORKS

Take the last digit with n % 10, then the next one, and so on. Because we move right to left, the digit just read must not exceed the one read before it. A single digit and zero are monotone.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Peel digits from the right and
# compare with the previous one
# Idea: Take the last digit with number % 10, then the next
# one, and so on. Because we move right to left, the digit
# just read must not exceed the one read before it. A
# single digit and zero are monotone.
def is_monotone_digits(number):
    previous_digit = 9
    while number:
        digit = number % 10
        if digit > previous_digit:
            return False
        previous_digit = digit
        number //= 10
    return True
```

COMPLEXITY | Time O(d) | Space O(1)

190 Add two big integers from digit lists — MEDIUM

Template move: the template as written

QUESTION

You are given two non-empty singly linked lists (or arrays) representing two non-negative integers. The digits are stored in **reverse order**, and each node contains a single digit from 0 to 9. Your task is to add the two numbers and return the sum as a new linked list, also in reverse order. You may assume:

- Each list has at least one node.
- The integers do not contain leading zeros, except the number 0 itself.

Example

Input:

- List 1: 2 -> 4 -> 3 (represents the integer 342)

QUESTION

- List 2: 5 -> 6 -> 4 (represents the integer 465)

Output:

- 7 -> 0 -> 8 (represents the integer 807, since 342 + 465 = 807)

Another example

Input:

- List 1: 9 -> 9 -> 9 (represents 999)
- List 2: 1 (represents 1)

Output:

- 0 -> 0 -> 0 -> 1 (represents 1000)

Constraints

- Let n be the length of the first list and m be the length of the second list.
- 1 <= n, m <= 10^5.
- Each node value is an integer in the range [0, 9].

Task: Describe an algorithm to perform this addition and construct the resulting linked list in O(n + m) time and O(1) additional space (excluding the space for the output list).

You are given two non-empty arrays of digits representing two non-negative integers. The digits are stored in **reverse order** (least significant digit first), and each element is a single digit from 0 to 9. Add the two numbers and return the sum as a new array of digits, also in reverse order.

Assumptions

- Each array has at least one element.
- The integers do not contain leading zeros, except the number 0 itself.

Example 1

```
Input: l1 = [2, 4, 3], l2 = [5, 6, 4] (342 + 465)
Output: [7, 0, 8] (807)
```

Example 2

```
Input: l1 = [9, 9, 9], l2 = [1] (999 + 1)
Output: [0, 0, 0, 1] (1000)
```

Implement **`add_two_numbers(l1, l2)`** (camelCase **`addTwoNumbers`** in Java/C++/JS) returning the digit array of the sum in reverse order. Aim for O(n + m) time and O(1) extra space beyond the output.

Constraints

1 <= n, m <= 10^5 where n = len(l1), m = len(l2)Each element is an integer in the range [0, 9]Digits are stored in reverse order (least significant digit first)No leading zeros except for the number 0 itself

EXAMPLES

```
Input: ([2, 4, 3], [5, 6, 4])
Expected Output: [7, 0, 8]
Explanation: 342 + 465 = 807, stored reversed as [7, 0, 8].
Input: ([9, 9, 9], [1])
Expected Output: [0, 0, 0, 1]
```

Explanation: 999 + 1 = 1000; carry propagates through all three digits and creates a new leading digit.

Hints

Walk both arrays from index 0 (the least significant digit) toward the end, just like grade-school column addition.Maintain a single carry variable. At each position, sum the available digits plus the carry; append (sum % 10) and set carry = sum // 10.When one array runs out, keep going with just the carry and the remaining longer array. Don't forget a final carry can add one extra digit (e.g. 999 + 1 -> [0,0,0,1]).

WHY IT WORKS

Walk both lists from the right with indices i and j; at each step add a[i] (or 0 once exhausted), b[j] (or 0) and the carry, append sum % 10, and carry = sum // 10. After the loop add a final 1 if the carry remains, and reverse the collected digits.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Digit by digit with a carry
# Idea: Walk both lists from the right with indices first_index and
# second_index; at each step add first_digits[first_index] (or 0 once exhausted), second_digits[second_index] (
or
# 0) and the carry, append sum % 10, and carry = sum //
# 10. After the loop add a final 1 if the carry remains,
# and reverse the collected digits.
def add_digits(first_digits, second_digits):
    first_index, second_index, carry, result = len(first_digits) - 1, len(second_digits) - 1, 0, []
    while first_index >= 0 or second_index >= 0 or carry:
        total = carry + (first_digits[first_index] if first_index >= 0 else 0) + (second_digits[second_index] i
f second_index >= 0 else 0)
        result.append(total % 10)
        carry = total // 10
        first_index, second_index = first_index - 1, second_index - 1
    return result[::-1] or [0]
```

COMPLEXITY | Time O(max(m, n)) | Space O(max(m, n))

191 Check Digits and Combine Ranges — HARD

Template move: palindrome number: reverse only the second half with n % 10 and n // 10 and compare with the first half

QUESTION

You are asked to solve two independent coding tasks.

Task 1:

Check Whether an Integer Reads the Same Backward Given an integer **`x`**, return **`true`** if its decimal representation reads the same from left to right and from right to left; otherwise return **`false`**. Requirements and edge cases:

- Negative integers should return **`false`** because of the leading minus sign.
- You may assume **`x`** fits in a signed 32-bit integer.
- Try to solve it without converting the integer to a string. Example:

```
Input: x = 121
Output: true
```

```
Input: x = -121
Output: false
```

```
Input: x = 10
Output: false
```

Task 2:

Combine Overlapping Ranges Given an array of intervals **`intervals`**, where each interval is represented as **`[start, end]`**, combine all overlapping intervals and return an array of non-overlapping intervals that covers exactly the same ranges. Requirements and edge cases:

- Intervals may be provided in any order.
- Two intervals overlap if they share at least one point. For example, **`[1, 4]`** and **`[4, 5]`** should be combined into **`[1, 5]`**.
- Return the intervals in increasing order by start value. Example:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
```

QUESTION

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
```

Part 1: Check Whether an Integer Reads the Same BackwardGiven an integer x, return True if its decimal representation reads the same from left to right and from right to left. Otherwise, return False. Negative integers are not palindromes because of the leading minus sign. Try to solve the problem using arithmetic operations instead of converting the integer to a string.
Constraints-2^31 <= x <= 2^31 - 1Negative integers must return FalseAn arithmetic-based solution is preferred over string conversion

EXAMPLES

```
Input: 121
Expected Output: True
```

Explanation: 121 reads the same forward and backward.

```
Input: -121
Expected Output: False
```

Explanation: Negative numbers are not palindromes.

Hints

A negative number can never be a palindrome because the minus sign only appears on the left.Try reversing only the second half of the digits instead of the whole number.
Part 2: Combine Overlapping RangesGiven a list of intervals, where each interval is represented as [start, end], merge all overlapping intervals and return a new list of non-overlapping intervals that covers exactly the same ranges. Intervals may appear in any order. Two intervals overlap if they share at least one point, so [1, 4] and [4, 5] should be merged into [1, 5]. Return the result sorted by starting value.
Constraints
0 <= len(intervals) <= 10^4-10^9 <= start <= end <= 10^9Intervals may be unsorted
Examples
Input: [[1, 3], [2, 6], [8, 10], [15, 18]]
Expected Output: [[1, 6], [8, 10], [15, 18]]
Explanation: The first two intervals overlap and merge into [1, 6].
Input: [[1, 4], [4, 5]]
Expected Output: [[1, 5]]
Explanation: The intervals share the point 4, so they overlap.
Hints
Sort the intervals by their start value before trying to merge them.Keep track of the last interval in the output. If the next interval overlaps, extend the end; otherwise, start a new interval.

PART 1: Task 1: reverse only the second half arithmetically

Reject negatives and positive numbers ending in 0. Move digits from x to rev (rev = rev * 10 + x % 10, x //= 10) until rev >= x; then x equals rev (even number of digits) or rev // 10 (odd, middle digit dropped). Only half the digits are processed, so rev cannot overflow.

PART 2: Task 2: sort by start, then one sweep

Sort by start. For each range, if it starts at or before the end of the last merged range extend that range's end with max(); otherwise append it as a new range. Sorting guarantees nothing earlier can overlap the new range.

SHORT OPTIMAL CODE

```
# Task 1: reverse only the second half arithmetically
# Idea: Reject negatives and positive numbers ending in 0.
# Move digits from x to rev (reversed_half = reversed_half * 10 + number % 10, number
```

```python
# //= 10) until reversed_half >= number; then x equals rev (even number
# of digits) or rev // 10 (odd, middle digit dropped).
# Only half the digits are processed, so rev cannot
# overflow.
def isPalindrome(number):
    if number < 0 or (number % 10 == 0 and number != 0):
        return False
    reversed_half = 0
    while number > reversed_half:
        reversed_half = reversed_half * 10 + number % 10
        number //= 10
    return number == reversed_half or number == reversed_half // 10


# Task 2: sort by start, then one sweep
# Idea: Sort by start. For each range, if it starts at or
# before the end of the last merged range extend that
# range's end with max(); otherwise append it as a new
# range. Sorting guarantees nothing earlier can overlap
# the new range.
def merge(ranges):
    merged = []
    for start, end in sorted(ranges):
        if merged and start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged
```

COMPLEXITY | Part 1: Time O(d) | Space O(1)
Part 2: Time O(n log n) | Space O(n)

192 Permutation Sequence

HARD

Template move: factorial number system: divide k - 1 by (n - 1)!, (n - 2)!, ... to pick each digit from the list of unused digits

QUESTION

The set [1, 2, ..., n] has n! distinct permutations. Listing them in strictly increasing (lexicographic) order and numbering them from 1 , return the k -th permutation as a string. Aim for an O(n) -ish construction using the factorial number system rather than generating all permutations.

Function signature

def get_permutation(n: int, k: int) -> str:
    ...

Follow-up (as reported): A classmate asked whether a brute-force "generate all and index" is acceptable; discuss why the factorial-base method avoids O(n!) .

EXAMPLES

Example 1	
Input	`n = 3, k = 3`
Output	`"213"`
Why	`Order is 123, 132, 213, 231, 312, 321; the 3rd is "213".`

Example 2	
Input	`n = 4, k = 9`
Output	`"2314"`
Why	`The 9th permutation of 1234 in lexicographic order is "2314".`

QUESTION

Example 3	
Input	`n = 3, k = 1`
Output	`"123"`
Why	`The first permutation is the sorted sequence.`

CONSTRAINTS

- 1 <= n <= 9
- 1 <= k <= n!

WHY IT WORKS

Each digit is chosen by dividing the remaining index by the size of a block of permutations.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Factorial number system
# Idea: Each digit is chosen by dividing the remaining
# index by the size of a block of permutations.
from math import factorial


def get_permutation(count, rank):
    digits = list(range(1, count + 1))
    rank -= 1
    chosen = []
    for remaining in range(count, 0, -1):
        block = factorial(remaining - 1)
        chosen.append(digits.pop(rank // block))
        rank %= block
    return "".join(map(str, chosen))
```

COMPLEXITY | Time O(n^2) | Space O(n)

GROUP 25 OF 43 | 6 QUESTIONS | SHORT MAIN CODE

Look at the input once and keep only a small summary (and count your calls)

Pattern

When the input is too big to hold, read it one item at a time and keep a tiny summary: a maximum, a set from the smaller file, a sorted window. When asking a question is expensive (an oracle), use binary search on how many items you include so the number of calls stays small.

193 Streaming Points: Emit K Within Distance — MEDIUM

Template move: keep the points in a sorted list; each new point is inserted with bisect and only windows around it are re-checked

QUESTION

Positions (floats) arrive one at a time. Whenever k stored points fit inside a span of d (largest minus smallest is at most d), output those k points and remove them. add(x) returns the groups emitted because of this arrival (a list of lists).

Function signature

class PointStream:
    def __init__(self, k: int, d: float)
    def add(self, x: float) -> list

EXAMPLES

Example 1	

Input	`s = PointStream(3, 2.0)`
	`s.add(1.0)`
	`s.add(5.0)`
	`s.add(2.0)`
	`s.add(1.5)`

Output	`s.add(1.0) -> []`
	`s.add(5.0) -> []`
	`s.add(2.0) -> []`
	`s.add(1.5) -> [[1, 1.5, 2]]`

Why	`After 1, 5, 2 no three points fit in a span of 2 (1 and 5 are too far apart); adding 1.5 gives 1, 1.5, 2 with span 1, which is emitted and removed.`

WHY IT WORKS

Insert with bisect. Because no window existed before, a new window must contain the new point, so test the K windows that include its position. After emitting a group, the points on both sides of the gap become neighbours, so test the windows around that seam too. A balanced tree (sorted container) makes the insert and delete O(log n).

SHORT OPTIMAL CODE

```
from bisect import bisect_left
from bisect import insort


# BASE QUESTION (optimal): Sorted list, check only windows
# touching the change
# Idea: Insert with bisect. Because no window existed
# before, a new window must contain the new point, so test
# the K windows that include its position. After emitting
# a group, the points on both sides of the gap become
# neighbours, so test the windows around that seam too. A
# balanced tree (sorted container) makes the insert and
# delete O(log n).
```

```python
class PointStream:
    def __init__(self, group_size, max_spread):
        self.group_size, self.max_spread, self.points = group_size, max_spread, []

    def _first_window(self, position):
        first_start = max(0, position - self.group_size + 1)
        last_start = min(position, len(self.points) - self.group_size)
        for start in range(first_start, last_start + 1):
            if self.points[start + self.group_size - 1] - self.points[start] <= self.max_spread:
                return start
        return None

    def add(self, point):
        insort(self.points, point)
        emitted, position = [], bisect_left(self.points, point)
        while True:
            start = self._first_window(position)
            if start is None:
                return emitted
            emitted.append(self.points[start:start + self.group_size])
            del self.points[start:start + self.group_size]
            position = start
```

COMPLEXITY | Time O(log n + K) per check, plus list shifting | Space O(n)

194 Two-Day Log Intersection under Memory Limit

MEDIUM

Template move: dict object -> set of event codes for each day; keep the objects that appear on both days with at least two distinct codes

QUESTION

PhD intern coding round 1: two log files with **`(timestamp, obj_id, client_id)`** entries. Find **`obj_id`**s that (a) appear in both files and (b) have at least two distinct **`client_id`**s. Follow-up imposes a memory limit, expecting external sort + two-pointer.

Input: two log files; each line is (timestamp, obj_id, client_id).
Find every obj_id such that:

(a) appears in both files, and
(b) is associated with at least 2 distinct client_ids (aggregated across both files).

Follow-up 1: complexity of set-intersection on the file contents.
Follow-up 2: memory cannot hold either file fully — design a streaming solution.

EXAMPLES

```
File A:  (t1, X, c1), (t2, X, c2), (t3, Y, c1)
File B:  (t4, X, c1), (t5, Z, c2)
-> Only X is in both files; X has 2 distinct clients {c1, c2}.
Output: [X]
```

WHY IT WORKS

Read externally sorted logs together and hold only the client sets for the current object from each day.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Merge two object-sorted streams
# Each row is (timestamp, object_id, client_id). The inputs
# are sorted externally by object id, so memory holds only
# one object's client set from each day at a time.
def log_objects_sorted(day1, day2):
    day_one_rows = iter(day1)
    day_two_rows = iter(day2)

    def next_group(rows, first_row):
```

```
        if first_row is None:
            return None, None
        object_id = first_row[1]
        clients = {first_row[2]}
        for row in rows:
            if row[1] != object_id:
                return (object_id, clients), row
            clients.add(row[2])
        return (object_id, clients), None

    group_a, next_a = next_group(day_one_rows, next(day_one_rows, None))
    group_b, next_b = next_group(day_two_rows, next(day_two_rows, None))
    answer = []
    while group_a and group_b:
        if group_a[0] == group_b[0]:
            if len(group_a[1] | group_b[1]) >= 2:
                answer.append(group_a[0])
            group_a, next_a = next_group(day_one_rows, next_a)
            group_b, next_b = next_group(day_two_rows, next_b)
        elif group_a[0] < group_b[0]:
            group_a, next_a = next_group(day_one_rows, next_a)
        else:
            group_b, next_b = next_group(day_two_rows, next_b)
    return answer
```

COMPLEXITY | Time O(n + m) | Space O(clients for one object)

195 Find shared objects across two log files — MEDIUM

Template move: load only the smaller file's object ids into a set, then stream the big file and keep ids found in the set

QUESTION

You are given two large log files representing activity on two different days. Each line of each log has three fields:

- **`timestamp`**: a time value (you may treat it as an opaque string or integer)
- **`obj_id`**: identifier of an object (string or integer)
- **`client_id`**: identifier of a client (string or integer) The logs are not necessarily sorted by any field. Define an object as **interesting** if:

1. Its **`obj_id`** appears at least once in day 1's log **and** at least once in day 2's log; and 2. Across all of its appearances in both days combined, it is associated with at least **two distinct `client_id`** values.

Tasks

1. Design and implement a function that, given file handles (or streaming iterators) for the two log files, returns the set of all such interesting **`obj_id`**s.
2. For your initial, straightforward in-memory solution (you may assume both logs fit in memory), state the time and space complexity in terms of the total number of log records.
3. Follow-up (memory constrained): Now assume the logs are very large and **cannot** both fit into memory at once.

- You may still sequentially scan each file and you may perform external sorting.
- Describe how you would modify your approach to work under this memory constraint.
- Analyze the time and space complexity of this memory-efficient solution.

Return sorted obj_ids that appear in both day logs and have at least two distinct client_ids across both days combined.
Constraints
Rows are [timestamp, obj_id, client_id] or dicts with those fields

QUESTION

EXAMPLES

```
Input: ([['t1', 'o1', 'c1'], ['t2', 'o2', 'c1']], [['t3', 'o1', 'c2'], ['t4', 'o2', 'c1']])
Expected Output: ['o1']
Input: ([], [['t', 'o', 'c']])
Expected Output: []
```

Hints

Track which days each object appears in and its set of clients.

WHY IT WORKS

Load the object ids of the smaller file into a set, then stream the other file and keep ids that are in the set. Each line is read once; memory depends only on the smaller file.

SHORT OPTIMAL CODE

```
def _obj_ids(lines):
    return {line.split()[1] for line in lines if line.strip()}


# BASE QUESTION (optimal): Hash set of the smaller file's
# ids
# Idea: Load the object ids of the smaller file into a
# set, then stream the other file and keep ids that are in
# the set. Each line is read once; memory depends only on
# the smaller file.
def shared_objects(small_lines, big_lines):
    small = _obj_ids(small_lines)
    return {line.split()[1] for line in big_lines if line.strip() and line.split()[1] in small}
```

COMPLEXITY | Time O(n + m) | Space O(distinct ids of the smaller file)

196 Minimize calls to find all bad test pairs

MEDIUM

Template move: the same group testing: put a whole group in one call and split only the groups that fail

QUESTION

You have (n) atomic tests (t_1, t_2, \dots, t_n). You are given access to a black-box function:

```
bool runTest(set<Test> S)
```

which behaves as follows:

- **`runTest(S) == True`** if the subset (S) contains **no** incompatible (bad) pair of tests.
- **`runTest(S) == False`** if the subset (S) contains **at least one** incompatible (bad) pair of tests. There is some unknown set of unordered **bad pairs** ((t_i, t_j)), where (i\ne j). You do **not** know in advance which pairs are bad or how many bad pairs there are. Your goal is to discover **all** bad pairs while making as few calls to **`runTest`** as possible. You may adaptively choose each subset (S) based on the results of previous calls.

Tasks

1. Design an algorithm that, using only calls to **`runTest`**, finds **all** bad pairs ((t_i, t_j)).
2. Analyze the number of calls to **`runTest`** your algorithm requires, as a function of:

- (n): the total number of tests, and - (k): the total number of bad pairs.

3. Argue why your algorithm is correct (i.e., why it will eventually find every bad pair and not report any good pair as bad). You do not need to provide full production code, but describe the algorithm clearly enough that it could be implemented.

QUESTION

You have n atomic tests numbered 0 to n - 1. Some unknown unordered pairs of tests are incompatible, called bad pairs. A black-box oracle runTest(S) returns True if the subset S contains no bad pair, and False if S contains at least one bad pair. In this non-interactive version, you are given bad_pairs only so your code can simulate the oracle. Implement the following deterministic oracle-discovery strategy and return both the bad pairs it discovers and the number of oracle calls it makes. Strategy:

1. Process vertices from right to left: v = n - 1, n - 2, ..., 0.
2. Maintain all bad pairs already discovered among vertices greater than the current v.
3. For the suffix vertices v + 1 through n - 1, greedily partition them into color classes such that no discovered bad pair appears inside one color class. Since all suffix-suffix bad pairs have already been discovered, every color class is truly compatible internally.
4. For each color class C, find all bad partners of v inside C using recursive group testing:

- Call runTest({v} union C).
- If the oracle returns True, v has no bad pair with any test in C.
- If C has size 1 and the oracle returns False, that single pair is bad.
- Otherwise split C into two halves and recurse on both halves. Return all discovered bad pairs sorted lexicographically, where each pair is represented as [i, j] with i < j, along with the exact number of runTest calls made by this strategy.

Constraints
0 <= n <= 5000 <= len(bad_pairs) <= n * (n - 1) / 2Each bad pair contains two distinct test indices in the range [0, n - 1]bad_pairs may be given in any endpoint order

EXAMPLES

```
Input: (0, [])
Expected Output: ([], 0)
```

Explanation: There are no tests and therefore no oracle calls are needed.

```
Input: (4, [])
Expected Output: ([], 3)
```

Explanation: For v = 2, 1, and 0, the algorithm queries the compatible suffix once and finds no bad pairs. v = 3 has an empty suffix.

Hints

If a candidate set C is internally compatible, then runTest({v} union C) tells you whether v conflicts with at least one element of C.Process tests from right to left so that when handling v, all bad pairs inside the suffix v + 1 ... n - 1 are already known and can be used to build compatible groups.

WHY IT WORKS

Process tests one by one. Keep groups of tests that pass runTest. For a new test t and a group G: if runTest(G + {t}) passes there is no partner in G; otherwise binary search for a partner: the smallest prefix of G that fails with t identifies it, remove it from consideration and repeat until the rest passes. Put t into a group with no partner (or a new group). Pairs inside a group cannot exist, and pairs across groups are found when the later test arrives.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Independent groups plus binary
# search for partners
# Idea: Process tests one by one. Keep groups of tests
# that pass runTest. For a new test test and a group G: if
# runTest(G + {test}) passes there is no partner in G;
# otherwise binary search for a partner: the smallest
# prefix of G that fails with test identifies it, remove it
# from consideration and repeat until the rest passes. Put
# test into a group with no partner (or a new group). Pairs
```

```python
# inside a group cannot exist, and pairs across groups are
# found when the later test arrives.
def find_bad_pairs(tests, runTest):
    groups, pairs = [], set()

    def partners(group, test):
        found, rest = [], list(group)
        while rest and not runTest(set(rest) | {test}):
            low, high = 1, len(rest)
            while low < high:
                middle = (low + high) // 2
                if runTest(set(rest[:middle]) | {test}):
                    low = middle + 1
                else:
                    high = middle
            found.append(rest[low - 1])
            rest = rest[:low - 1] + rest[low:]
        return found

    for test in tests:
        placed = False
        for group in groups:
            found_partners = partners(group, test)
            for partner in found_partners:
                pairs.add((min(partner, test), max(partner, test)))
            if not found_partners and not placed:
                group.append(test)
                placed = True
        if not placed:
            groups.append([test])
    return pairs
```

COMPLEXITY | Time O(n * g + p log n) calls (g = number of groups, p = number of bad pairs) | Space O(n)

197 Find All Bad Pairs Using runTest Oracle — HARD

Template move: for each test, binary search the prefix of earlier tests where runTest first fails; that test is a bad partner; repeat until none is left

QUESTION

PhD intern coding round 2: given a black-box oracle **`runTest(S)`** that returns **`true`** iff a subset **`S`** contains no bad pair, identify all bad pairs across N test items with as few oracle calls as possible.

Items t_1, ..., t_N are given.
Oracle: runTest(S: Set[Item]) -> bool returns true iff S contains zero bad pairs; otherwise false.
Output: every bad pair (t_i, t_j).
Minimize total oracle calls.

EXAMPLES

```
N=4, bad pairs = {(1,2)}.
runTest({1,2,3,4}) -> false -> at least one bad pair somewhere.
runTest({1,3,4})   -> true  -> 2 is in every bad pair containing items from this set.
... binary search to isolate.
```

WHY IT WORKS

Keep groups of tests that pass together. For a new test t and a group G: if runTest(G + {t}) passes there is no partner in G; otherwise binary search for the smallest prefix of G that fails with t, record that partner, remove it and repeat. Put t into a group with no partner (or a new group). Pairs inside a group cannot exist and pairs across groups are found when the later test arrives.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Group the tests and binary
# search for partners
```

```python
# Idea: Keep groups of tests that pass together. For a new
# test test and a group G: if runTest(G + {test}) passes there
# is no partner in G; otherwise binary search for the
# smallest prefix of G that fails with test, record that
# partner, remove it and repeat. Put test into a group with
# no partner (or a new group). Pairs inside a group cannot
# exist and pairs across groups are found when the later
# test arrives.
def find_bad_pairs(tests, runTest):
    groups, pairs = [], set()

    def partners(group, test):
        found, rest = [], list(group)
        while rest and not runTest(set(rest) | {test}):
            low, high = 1, len(rest)
            while low < high:
                middle = (low + high) // 2
                if runTest(set(rest[:middle]) | {test}):
                    low = middle + 1
                else:
                    high = middle
            found.append(rest[low - 1])
            rest = rest[:low - 1] + rest[low:]
        return found

    for test in tests:
        placed = False
        for group in groups:
            found_partners = partners(group, test)
            for partner in found_partners:
                pairs.add((min(partner, test), max(partner, test)))
            if not found_partners and not placed:
                group.append(test)
                placed = True
        if not placed:
            groups.append([test])
    return pairs
```

COMPLEXITY | Time about O(K log N + N * G) calls (K bad pairs, G groups) | Space O(N)

198 Find maximum value in huge text file — HARD

Template move: the template as written; the file is never loaded as a whole

QUESTION

A text file is too large to load into memory (tens or hundreds of gigabytes). Each line holds one signed 64-bit integer. Return the largest value by reading the file sequentially, holding only a small amount in memory.

Function signature

def max_in_file(path: str) -> int

EXAMPLES

Example 1

Input	**nums.txt contains the lines:** **5** **-3** **12** **7**
Output	**max_in_file(path) -> 12**
Why	**Only the current line and the best value so far are in memory, so the file size does not matter.**

PART 1: Single machine: stream the file, keep one running maximum

Never load the file. Read it through a buffered iterator and keep one variable, the maximum so far, started at None so that an all-negative file or a one-line file is handled correctly (do not start at 0). Python ints are unbounded, but in a fixed-width language parse into 64 bits and treat overflow as bad input. Disk read speed is the bottleneck, not the comparison.

PART 2: Parallel: split byte ranges on line boundaries (map), max of the maxima (reduce)

Divide the file into roughly equal byte ranges. Each worker seeks to its start and, unless it starts at byte 0, discards the partial line it landed in (the previous worker finishes that line). It reads lines until the line it just read began past its end offset. Every line is therefore processed exactly once. The reducer takes the max of the partial results and ignores workers whose range was empty.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Single machine: stream the
# file, keep one running maximum
# Idea: Never load the file. Read it through a buffered
# iterator and keep one variable, the maximum so far,
# started at None so that an all-negative file or a one-
# line file is handled correctly (do not start at 0).
# Python ints are unbounded, but in a fixed-width language
# parse into 64 bits and treat overflow as bad input. Disk
# read speed is the bottleneck, not the comparison.
def max_in_file(path):
    best = None
    with open(path, "r", buffering=1 << 20) as file:
        for line in file:
            line = line.strip()
            if not line:
                continue
            value = int(line)
            if best is None or value > best:
                best = value
    return best
```

COMPLEXITY | Part 1: Time O(n) | Space O(1) plus the read buffer
Part 2: Time O(n / workers) plus I/O limits | Space O(workers)

Grid DP: each cell from its neighbours above and to the left

Pattern

Walk the grid row by row and keep only the previous row. A cell's value depends on the cell above, the cell to the left, and the diagonal. Change the condition for a cell to count and the formula that combines the three neighbours.

199 Grid Path Count with Diagonal Moves and Checkpoints — MEDIUM

Template move: advance column by column keeping the number of ways in each row; new[r] = old[r-1] + old[r] + old[r+1]; zero out rows that miss a checkpoint

QUESTION

Count the paths in an n x m grid from the bottom-left cell (n - 1, 0) to the bottom-right cell (n - 1, m - 1). From (r, c) you may move up-right (r - 1, c + 1), right (r, c + 1) or down-right (r + 1, c + 1) and must stay inside the grid. Optionally you are given checkpoint cells that every path has to pass through; with ordered=True they must be visited in the order given.

Function signature

def count_paths(n: int, m: int, checkpoints=(), ordered=False) -> int

EXAMPLES

Example 1	
Input	3, 3
Output	2
Why	Two paths: through (1, 1) and then down-right, or through (2, 1) and then right.

Example 2	
Input	3, 3, checkpoints = [(1, 1)]
Output	1
Why	The path must also pass through (1, 1), the middle cell.

Example 3	
Input	3, 4, checkpoints = [(1, 1), (2, 2)], ordered = true
Output	1
Why	Two checkpoints in different columns, visited in column order.

WHY IT WORKS

ways[r] is the number of paths that are in row r at the current column. The next column is ways'[r] = ways[r - 1] + ways[r] + ways[r + 1]. At a checkpoint column keep only the checkpoint's row. Two checkpoints in one column make the answer 0. For the ordered version the checkpoints must be given by strictly increasing column, because one move always advances one column.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Column DP with a rolling array
# and checkpoint filters
# Idea: ways[row] is the number of paths that are in row r
# at the current column. The next column is ways'[row] =
# ways[row - 1] + ways[row] + ways[row + 1]. At a checkpoint
# column keep only the checkpoint's row. Two checkpoints
# in one column make the answer 0. For the ordered version
# the checkpoints must be given by strictly increasing
# column, because one move always advances one column.
def count_paths(rows, cols, checkpoints=(), ordered=False):
    checkpoint_cells = [tuple(col) for col in checkpoints]
```

```
    by_col = {}
    for row, col in checkpoint_cells:
        if by_col.setdefault(col, row) != row or len({checkpoint_col for _, checkpoint_col in checkpoint_cells}
) != len(checkpoint_cells):
            return 0
    if ordered and [col for _, col in checkpoint_cells] != sorted(col for _, col in checkpoint_cells):
        return 0
    ways = [0] * rows
    ways[rows - 1] = 1
    def keep(column_ways, col):
        if col in by_col:
            column_ways = [ways_value if row == by_col[col] else 0 for row, ways_value in enumerate(column_ways
)]
        return column_ways
    ways = keep(ways, 0)
    for col in range(1, cols):
        ways = keep([(ways[row - 1] if row else 0) + ways[row] + (ways[row + 1] if row + 1 < rows else 0) for r
ow in range(rows)], col)
    return ways[rows - 1]
```

COMPLEXITY | Time O(n * m) | Space O(n)

200 Longest Same-Color Grid Sequence

MEDIUM

Template move: keep four run lengths per cell (horizontal, vertical, two diagonals) from the previous row and the left cell; the answer is the largest

QUESTION

Given a grid of colours (letters such as R, G, B), return the length of the longest run of one colour along a straight line: horizontal, vertical, or diagonal (in either diagonal direction).

Function signature

def longest_same_color_line(g: list) -> int

EXAMPLES

Example 1	
Input	['RRRG', 'RRBB', 'GRRR', 'BGBR']
Output	4
Why	The main diagonal is R, R, R, R from corner to corner: 4. No other line is longer.

Example 2	
Input	['RGB', 'BGR', 'RGB']
Output	3
Why	The longest line is the vertical run of three G in the middle column.

Example 3	
Input	['RR', 'RR']
Output	2
Why	A single colour fills the grid: the longest line is a full row or column.

WHY IT WORKS

Keep, for the previous row, the run length ending at each column in the vertical and both diagonal directions, and a running value for the horizontal run in the current row. A direction extends only when the predecessor has the same colour. The best value seen is the answer.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Four run lengths per cell
# Idea: Keep, for the previous row, the run length ending
# at each column in the vertical and both diagonal
# directions, and a running value for the horizontal run
# in the current row. A direction extends only when the
# predecessor has the same colour. The best value seen is
# the answer.
def longest_same_color_line(colors):
    if not colors or not colors[0]:
        return 0
    width, best = len(colors[0]), 0
    up_run, down_left_run, down_right_run = [0] * width, [0] * width, [0] * width          # vertical, down-lef
t diagonal, down-right diagonal runs of the previous row
    for row_index, row in enumerate(colors):
        new_up_run, new_down_left_run, new_down_right_run = [1] * width, [1] * width, [1] * width
        left_run = 0
        for col_index, color in enumerate(row):
            left_run = left_run + 1 if col_index and row[col_index - 1] == color else 1
            if row_index:
                if colors[row_index - 1][col_index] == color:
                    new_up_run[col_index] = up_run[col_index] + 1
                if col_index and colors[row_index - 1][col_index - 1] == color:
                    new_down_right_run[col_index] = down_right_run[col_index - 1] + 1
                if col_index + 1 < width and colors[row_index - 1][col_index + 1] == color:
                    new_down_left_run[col_index] = down_left_run[col_index + 1] + 1
            best = max(best, left_run, new_up_run[col_index], new_down_left_run[col_index], new_down_right_run[
col_index])
        up_run, down_left_run, down_right_run = new_up_run, new_down_left_run, new_down_right_run
    return best
```

COMPLEXITY | Time O(m * n) | Space O(n)

201 Count Same-Color Squares in Matrix

MEDIUM

Template move: the template as written: a square ends here if the three neighbours match; its size is 1 + min of their sizes

QUESTION

LeetCode 221 variant — count maximal same-color square sub-grids. DP on each cell as the bottom-right corner.

Input: m × n grid of colored cells.

Output: count of square sub-grids whose cells are all the same color (or maximum side length, depending on prompt).

DP: dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) if grid[i][j] == grid[i-1][j] == grid[i][j-1] == grid[i-1][j-1] else 1.

Sum dp[i][j] across the grid for the count variant; max for the side variant.

EXAMPLES

The plain "count all-1 squares" DP generalizes to same-color directly.

WHY IT WORKS

dp[j] holds the best square side ending in the current row at column j. A cell extends the squares of its up, left and up-left neighbours only when all three have the same colour as the cell; otherwise its square has side 1. Add dp[j] to the count (use max instead of the sum for the 'largest side' version). A rolling row keeps the space at O(n).

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): dp on the bottom-right corner
# Idea: dp[col_index] holds the best square side ending in the
# current row at column j. A cell extends the squares of
# its up, left and up-left neighbours only when all three
# have the same colour as the cell; otherwise its square
# has side 1. Add dp[col_index] to the count (use max instead of
# the sum for the 'largest side' version). A rolling row
# keeps the space at O(width).
def count_same_color_squares(colors):
    if not colors:
```

```
        return 0
    width, total = len(colors[0]), 0
    previous_row = [0] * width
    for row_index, row in enumerate(colors):
        current_row = [0] * width
        for col_index, color in enumerate(row):
            if row_index and col_index and colors[row_index - 1][col_index] == color and colors[row_index][col_
index - 1] == color and colors[row_index - 1][col_index - 1] == color:
                current_row[col_index] = 1 + min(previous_row[col_index], current_row[col_index - 1], previous_
row[col_index - 1])
            else:
                current_row[col_index] = 1
            total += current_row[col_index]
        previous_row = current_row
    return total
```

COMPLEXITY | Time O(m * n) | Space O(n)

202 Find the Longest Straight Run of One Color

HARD

Template move: the same four run lengths; extend a run when the neighbour in that direction has the same colour

QUESTION

Given a rectangular grid of colors, return the length of the longest consecutive sequence of cells with the same color along a horizontal, vertical, or diagonal straight line. A sequence cannot change direction or skip a cell. Both diagonal orientations count. The length is the number of cells in the sequence.

Input

- **`grid`**: an array of strings of equal length. Each character represents one cell's color.

Output

Return the maximum length as an integer.

Constraints

and Edge Cases - For this practice version, the grid contains at least one row and one column, with at most **`1000000`** cells in total.

- Colors are uppercase English letters; equal letters mean equal colors.
- A one-cell grid has answer **`1`**.
- Diagonal lines may run down-right or down-left. Reading the same line in reverse does not change its length.
- A connected region that bends is not necessarily one valid sequence.

Example

1

```
grid = ["RRRG", "RRBB", "GRRR", "BGBR"] output = 4
```

The main diagonal contains four consecutive **`R`** cells.

Example

2

```
grid = ["RBG", "BGR", "GRB"] output = 3
```

The down-left diagonal from the top-right cell contains three **`G`** cells. No horizontal or vertical sequence has that length.

EXAMPLES

QUESTION

Example 1	
Input	['RRRG', 'RRBB', 'GRRR', 'BGBR']
Output	4
Why	Four R cells lie on the main diagonal.

Example 2	
Input	['RGB', 'BGR', 'RGB']
Output	3
Why	Three G cells in the middle column.

WHY IT WORKS

run[d][i][j] is the length of the same-color run ending at (i, j) in direction d. It is the previous cell's run plus one when the colors match, else 1. The previous cell is left, up, up-left and up-right for the four directions, so one previous row of each direction is enough.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): DP per direction with a rolling
# row
# Idea: run[d][i][j] is the length of the same-color run
# ending at (i, j) in direction d. It is the previous
# cell's run plus one when the colors match, else 1. The
# previous cell is left, up, up-left and up-right for the
# four directions, so one previous row of each direction
# is enough.
def longest_run(grid):
    if not grid:
        return 0
    width = len(grid[0])
    horiz = [0] * width
    vert = [0] * width
    diag = [0] * width
    anti = [0] * width
    best = 0
    previous_row = None
    for row_index, row in enumerate(grid):
        new_horiz, new_vert, new_diag, new_anti = [1] * width, [1] * width, [1] * width, [1] * width
        for col in range(width):
            if col and row[col] == row[col - 1]:
                new_horiz[col] = new_horiz[col - 1] + 1
            if previous_row is not None:
                if row[col] == previous_row[col]:
                    new_vert[col] = vert[col] + 1
                if col and row[col] == previous_row[col - 1]:
                    new_diag[col] = diag[col - 1] + 1
                if col + 1 < width and row[col] == previous_row[col + 1]:
                    new_anti[col] = anti[col + 1] + 1
            best = max(best, new_horiz[col], new_vert[col], new_diag[col], new_anti[col])
        horiz, vert, diag, anti, previous_row = new_horiz, new_vert, new_diag, new_anti, row
    return best
```

COMPLEXITY | Time O(m * n) | Space O(n)

203 Count same-color squares in a character grid

HARD

Template move: exactly the same recurrence on characters

QUESTION

You are given a 2D grid (matrix) of characters. Each character represents a color: cells with the same character are considered the same color. Formally:

- The grid has **`m`** rows and **`n`** columns.
- **`grid[i][j]`** is an alphabetic character (e.g., **`'a'`**–**`'z'`** or **`'A'`**–**`'Z'`**). A **square** in this grid is defined as a contiguous **`k × k`** submatrix aligned with the grid axes, for some integer **`k >= 1`**. A square is **monochromatic** if **all** of its cells contain the **same** character (i.e., the same color). Task:
- Count and return the total number of monochromatic squares in the grid.
- Count all sizes of squares (1×1, 2×2, ..., up to the largest possible that fits in the grid). Example (just for clarity, not necessarily exhaustive): If the grid is:

```
a a b a a b b b
```

Then some monochromatic squares include:

- All 1×1 cells (each single cell is a monochromatic square).
- The 2×2 square in the top-left corner formed by **`'a'`**. Your function should return the total count of such monochromatic squares for a given input grid.

You are given a 2D grid of characters, where each character represents a color. A square is a contiguous k × k submatrix aligned with the grid axes, for some k >= 1. A square is monochromatic if all of its cells contain the same character. Return the total number of monochromatic squares of all possible sizes and positions in the grid. Every 1 × 1 cell is considered a monochromatic square.
Constraints
0 <= len(grid) <= 1000If grid is non-empty, 0 <= len(grid[0]) <= 1000All rows in grid have the same lengthgrid[i][j] is an alphabetic characterThe total number of cells is at most 1,000,000

EXAMPLES

```
Input: ([])
Expected Output: 0
```

Explanation: An empty grid contains no squares.

```
Input: (["aab", "aab", "bbb"])
Expected Output: 10
```

Explanation: There are 9 single-cell squares and one 2x2 monochromatic square of 'a' in the top-left corner.

Hints

Try defining dp[i][j] as the side length of the largest monochromatic square whose bottom-right corner is cell (i, j).A square larger than 1 can end at (i, j) only if the current cell matches its top, left, and top-left neighboring cells.

WHY IT WORKS

If the cell matches its top, left and top-left neighbours then dp = 1 + min(dp top, dp left, dp top-left); otherwise 1. Summing dp over all cells counts every square exactly once, at its bottom-right corner. Only the previous row is needed.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): DP: side of the largest square
# ending at each cell, summed
# Idea: If the cell matches its top, left and top-left
# neighbours then dp = 1 + min(dp top, dp left, dp top-
# left); otherwise 1. Summing dp over all cells counts
# every square exactly once, at its bottom-right corner.
# Only the previous row is needed.
```

```python
def count_same_color_squares(grid):
    rows, cols = len(grid), len(grid[0])
    previous_row, total = [0] * cols, 0
    for row in range(rows):
        current_row = [1] * cols
        for col in range(cols):
            if row and col and grid[row][col] == grid[row - 1][col] == grid[row][col - 1] == grid[row - 1][col
- 1]:
                current_row[col] = 1 + min(previous_row[col], current_row[col - 1], previous_row[col - 1])
        total += sum(current_row)
        previous_row = current_row
    return total
```

COMPLEXITY | Time O(m * n) | Space O(n) with a rolling row

GROUP 27 OF 43 | 5 QUESTIONS | SHORT MAIN CODE

Nested structures: a stack of contexts, or recursion with a cache

Pattern

When something opens (a bracket, a parenthesis, a placeholder), save the current context on a stack and start a fresh one. When it closes, pop the saved context and fold the result back in. Recursive expansion does the same with a function call and a cache.

204 Decode String — MEDIUM

Template move: the template as written

QUESTION

An encoded string uses k[text] to mean that text is repeated k times; the brackets can be nested. Return the decoded string. (k is a positive integer, possibly with several digits; the input is always valid.)

Function signature

def decode(s: str) -> str

EXAMPLES

Example 1	
Input	'3[a]2[bc]'
Output	'aaabcbc'
Why	3 copies of a, then 2 copies of bc.

Example 2	
Input	'2[abc]3[cd]ef'
Output	'abcabccdcdcdef'
Why	Plain letters after the brackets are kept as they are.

Example 3	
Input	'3[a2[c]]'
Output	'accaccacc'
Why	Nested: the inner 2[c] first (cc), then 3 copies of acc.

PART 1: Stack of (prefix, repeat) frames

On '[' push the text built so far and the repeat count, then start fresh. On ']' pop and append the current text repeated count times to the saved prefix. Digits accumulate into a multi-digit number (the usual bug is reading only one digit). No recursion is needed.

PART 2: Encode (the reverse direction, greedy run-length)

Run-length encode the string into count[char] groups. This is the simple flat inverse; producing the shortest nested encoding (LeetCode 471) needs interval DP and is a hard follow-up.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Stack of (prefix, repeat)
# frames
# Idea: On '[' push the text built so far and the repeat
# count, then start fresh. On ']' pop and append the
# current text repeated count times to the saved prefix.
# Digits accumulate into a multi-digit number (the usual
# bug is reading only one digit). No recursion is needed.
def decodeString(encoded):
    stack, current, pending_count = [], [], 0
```

```
    for char in encoded:
        if char.isdigit():
            pending_count = pending_count * 10 + int(char)
        elif char == "[":
            stack.append((current, pending_count))
            current, pending_count = [], 0
        elif char == "]":
            text_before, repeat_count = stack.pop()
            text_before.append("".join(current) * repeat_count)
            current = text_before
        else:
            current.append(char)
    return "".join(current)
```

COMPLEXITY | Part 1: Time O(output length) | Space O(output + nesting depth)
Part 2: Time O(n) | Space O(n)

205 Template Variable Expansion with Follow-Ups

MEDIUM

Template move: resolve(name) with a cache and a 'being resolved' set; meeting a name that is being resolved is a cycle

QUESTION

Replace **%KEY%** placeholders using a key-value map whose values may contain other placeholders. Clarify malformed delimiters and literal-percent escaping, flatten nested values, and detect cyclic dependencies before expansion.

Input: a key-value map and a string containing placeholders delimited by %, such as %USER%. Replace every placeholder with the corresponding value.
Map values can themselves contain placeholders, so resolve or flatten dependencies before expanding the final string. Define behavior for malformed delimiters such as unmatched percent signs, for a literal % in output, and for nested-looking forms before coding; the prompt intentionally leaves these contracts open.
Follow-up: detect cyclic dependencies among map values. Model keys as a dependency graph and use indegrees plus a queue to determine whether a complete topological resolution is possible.

EXAMPLES

```
Given {USER: "admin", HOME: "/%USER%/home"}:
```

Input: I am %USER% My home is %HOME% Output: I am admin My home is /admin/home

WHY IT WORKS

expand(text) scans the text once: %% becomes %, %NAME% becomes the resolved value of NAME. resolve(name) memoises each value and keeps a visiting set; meeting a name that is already being resolved means a cycle and raises an error.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Depth-first expansion with memo
# and a visiting set
# Idea: expand(text) scans the text once: %% becomes %,
# %NAME% becomes the resolved value of NAME. resolve(name)
# memoises each value and keeps a visiting set; meeting a
# name that is already being resolved means a cycle and
# raises an error.
def expand_template(text, mapping):
    done, visiting = {}, set()
    def resolve(name):
        if name in done:
            return done[name]
        if name in visiting:
            raise ValueError("cycle: " + name)
        if name not in mapping:
            raise KeyError(name)
        visiting.add(name)
```

```
        done[name] = scan(mapping[name])
        visiting.discard(name)
        return done[name]
    def scan(raw):
        pieces, position = [], 0
        while position < len(raw):
            if raw[position] != "%":
                pieces.append(raw[position]); position += 1
            elif raw[position:position + 2] == "%%":
                pieces.append("%"); position += 2
            else:
                closing_percent = raw.find("%", position + 1)
                if closing_percent < 0:
                    raise ValueError("unmatched %")
                pieces.append(resolve(raw[position + 1:closing_percent]))
                position = closing_percent + 1
        return "".join(pieces)
    return scan(text)
```

COMPLEXITY | Time O(total length of the expanded text + keys) | Space O(keys)

206 Expand Recursive Key-Value Placeholders in a Template

MEDIUM

Template move: regex-find {key}, expand each key recursively with a cache and a visiting set to stop cycles

QUESTION

Expand %KEY% placeholders in a template. A placeholder is replaced by its value from the mapping, and placeholders inside the values are expanded too, recursively. An unknown key is left exactly as written. A cycle (A needs B needs A) raises an error.

Function signature

def expand_template(template: str, mapping: dict) -> str

EXAMPLES

Example 1	
Input	'%GREETING%, welcome!', {'NAME': 'Ada', 'GREETING': 'Hello %NAME%'}
Output	'Hello Ada, welcome!'
Why	%GREETING% expands to 'Hello %NAME%', and %NAME% is expanded in turn.

Example 2	
Input	'Hi %WHO%', {}
Output	'Hi %WHO%'
Why	An unknown key is kept as written.

Example 3	
Input	'%A%', {'A': '%B%', 'B': '%A%'}
Output	raises ValueError: cycle at A
Why	A needs B and B needs A: a cycle is reported instead of looping forever.

WHY IT WORKS

expand(key) substitutes every placeholder in the key's value by expand(inner), caches the result, and uses a visiting set to raise on a cycle. Unknown keys stay as written. The template itself is expanded the same way.

SHORT OPTIMAL CODE

```python
import re


_PH = re.compile(r"%([A-Za-z_][A-Za-z0-9_]*)%")


# BASE QUESTION (optimal): Memoised recursive expansion
# with a visiting set
# Idea: expand(key) substitutes every placeholder in the
# key's value by expand(inner), caches the result, and
# uses a visiting set to raise on a cycle. Unknown keys
# stay as written. The template itself is expanded the
# same way.
def expand_template(template, mapping):
    cache, visiting = {}, set()

    def expand_text(text):
        return _PH.sub(lambda match: expand_key(match.group(1)) if match.group(1) in mapping else match.group(0
), text)

    def expand_key(key):
        if key in cache:
            return cache[key]
        if key in visiting:
            raise ValueError(f"cycle at {key}")
        visiting.add(key)
        cache[key] = expand_text(mapping[key])
        visiting.discard(key)
        return cache[key]

    return expand_text(template)
```

COMPLEXITY | Time O(total size of all values + template) | Space O(number of keys)

207 Nested List Weight Sum (LC 339) with Follow-Ups

MEDIUM

Template move: depth counter: '[' adds 1, ']' subtracts 1; each number adds value * depth (read digits and a minus sign as you go)

QUESTION

A nested list holds integers and further lists. The weight of an integer is its depth: 1 at the top level, 2 inside one inner list, 3 inside two, and so on. Return the sum of every integer times its depth. A second version reads the nested list from a string like '[1,[4,[6]]]'.

Function signature

def weight_sum(nested: list) -> int

def weight_sum_string(s: str) -> int

EXAMPLES

Example 1	
Input	`[3, 8, [2, 14], [2, [91]]]`
Output	`320`
Why	`3 + 8 at depth 1, 2 + 14 and 2 at depth 2, and 91 at depth 3: 11 + 32 + 4 + 273 = 320.`

Example 2	
Input	`'[1,[4,[6]]]'`
Output	`27`
Why	`The string version of [1, [4, [6]]]: 1*1 + 4*2 + 6*3 = 27.`

PART 1: Single pass over the string with a depth counter

Every integer is weighted by the number of '[' currently open, so no tree needs to be built. Accumulate digits in a buffer and flush the buffer when a non-digit arrives or the string ends. Forgetting that last flush is the classic bug.

PART 2: Parsed lists with BFS (no recursion depth limit)

Once the string is parsed into Python lists, process one depth level at a time with a queue. BFS never recurses, so a pathologically deep input cannot overflow the call stack.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
# Idea: Every integer is weighted by the number of '['
# currently open, so no tree needs to be built. Accumulate
# digits in a buffer and flush the buffer when a non-digit
# arrives or the string ends. Forgetting that last flush
# is the classic bug.
def nestedWeightSumFromString(text):
    depth = total = 0
    number, sign, has_digits = 0, 1, False
    for char in text + "]":                      # a sentinel flushes a trailing number
        if char.isdigit():
            number, has_digits = number * 10 + int(char), True
            continue
        if has_digits:
            total += sign * number * depth
            number, sign, has_digits = 0, 1, False
        if char == "[":
            depth += 1
        elif char == "]":
            depth -= 1
        elif char == "-":
            sign = -1
    return total
```

COMPLEXITY | Part 1: Time O(n) | Space O(1)
Part 2: Time O(n) | Space O(width)

208 Nested Function Expression Evaluator — MEDIUM

Template move: tokenise, then a recursive parse(i): a number returns itself; a name reads its arguments until ')' and applies the function

QUESTION

Evaluate an expression made of integers and nested function calls: add and mul (any number of arguments), sub and pow (two), div (integer division truncating toward zero), min and max (any number). Return the integer result.

Function signature

def evaluate(expr: str) -> int

EXAMPLES

Example 1	
Input	'add(1, sub(1, 0))'
Output	2
Why	sub(1, 0) is 1, so add(1, 1) = 2.

Example 2	
Input	'add(2, mul(3, pow(4, 5)))'
Output	3074
Why	pow(4, 5) = 1024, times 3 = 3072, plus 2 = 3074.

QUESTION

Example 3	
Input	`'max(3, div(-7, 2), min(5, 9))'`
Output	`5`
Why	`div(-7, 2) truncates toward zero to -3; max of 3, -3 and 5 is 5.`

WHY IT WORKS

Tokenize into identifiers, integers, commas and parentheses. parse(i) returns (value, next index): a number returns itself; an identifier must be followed by '(', then arguments are collected until ')' so any number of arguments works. Apply the function table to the collected values. Keeping the parser and the function table separate makes new functions a one-line change.

SHORT OPTIMAL CODE

```
import math
import re


# BASE QUESTION (optimal): Tokenizer + recursive descent
# over name(arg, arg, ...)
# Idea: Tokenize into identifiers, integers, commas and
# parentheses. parse(position) returns (value, next index): a
# number returns itself; an identifier must be followed by
# '(', then arguments are collected until ')' so any
# number of arguments works. Apply the function table to
# the collected values. Keeping the parser and the
# function table separate makes new functions a one-line
# change.
def evaluate(expr):
    tokens = re.findall(r"-?\d+|[A-Za-z_]\w*|[(),]", expr)
    ops = {
        "add": lambda *a: sum(a),
        "sub": lambda a, b: a - b,
        "mul": lambda *a: math.prod(a),
        "pow": lambda a, b: a ** b,
        "div": lambda a, b: int(a / b),            # truncates toward zero; ask the interviewer about negatives
        "min": min,
        "max": max,
    }

    def parse(position):
        token = tokens[position]
        if re.fullmatch(r"-?\d+", token):
            return int(token), position + 1
        if token not in ops:
            raise ValueError(f"unknown function {token!r}")
        if tokens[position + 1] != "(":
            raise ValueError("expected '('")
        position += 2
        args = []
        while tokens[position] != ")":
            value, position = parse(position)
            args.append(value)
            if tokens[position] == ",":
                position += 1
        return ops[token](*args), position + 1

    value, end = parse(0)
    if end != len(tokens):
        raise ValueError("trailing tokens")
    return value
```

COMPLEXITY | Time O(n) | Space O(depth)

GROUP 28 OF 43 | 5 QUESTIONS | SHORT MAIN CODE

Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

Pattern

The same frontier loop as Tier 1, with three small upgrades: a step can cost 0 or 1 (free steps go to the front of the queue), you can return the moment you reach the goal, and the neighbours of a node can be computed on the fly instead of stored.

209 Shortest Path Nodes and Escaping a Pursuer — MEDIUM

Template move: run it from a, from t and from b; a node is on a shortest a-to-t path when da + dt == da[t]; compare with the pursuer's distance db

QUESTION

In an unweighted, undirected graph (an adjacency dict) Alice is at node a and wants to reach the target t. Return the length (number of edges) of the shortest path from a to t found with BFS. The follow-ups ask for every node that lies on some shortest path, and whether Alice can reach t before a pursuer Bob.

Function signature

def shortest_length(adj: dict, a, t) -> int

EXAMPLES

Example 1	
Input	`{0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2]}, 0, 3`
Output	`2`
Why	`0 -> 1 -> 3 and 0 -> 2 -> 3 both take 2 edges.`

Example 2	
Input	`{0: []}, 0, 0`
Output	`0`
Why	`A start that is the target needs 0 edges.`

WHY IT WORKS

dA, dT and dB come from BFS. The nodes on some shortest path are those with dA[v] + dT[v] == dA[t]. For the escape question walk only those nodes and only while dB[v] > dA[v], layer by layer from Alice; if the target is reached Alice has a safe shortest path.

SHORT OPTIMAL CODE

```
from collections import deque


# BASE QUESTION (optimal): Three BFS runs and a safe
# shortest-path DAG
# Idea: dA, dT and dB come from BFS. The nodes on some
# shortest path are those with dA[neighbour] + dT[neighbour] == dA[target]. For
# the escape question walk only those nodes and only while
# dB[neighbour] > dA[neighbour], layer by layer from Alice; if the target
# is reached Alice has a safe shortest path.
def shortest_path_info(neighbours, alice, target, bob):
    def bfs(start):
        distance = {start: 0}
        queue = deque([start])
        while queue:
            node = queue.popleft()
            for neighbour in neighbours[node]:
```

```
                if neighbour not in distance:
                    distance[neighbour] = distance[node] + 1
                    queue.append(neighbour)
        return distance
    da, dt, db = bfs(alice), bfs(target), bfs(bob)
    if target not in da:
        return None, set(), False
    length = da[target]
    nodes = {node for node in da if node in dt and da[node] + dt[node] == length}
    safe_nodes, frontier = set(), [alice] if db.get(alice, 10 ** 9) > 0 else []
    seen = set(frontier)
    while frontier:
        next_frontier = []
        for node in frontier:
            for neighbour in neighbours[node]:
                if neighbour in nodes and da[neighbour] == da[node] + 1 and neighbour not in seen and db.get(ne
ighbour, 10 ** 9) > da[neighbour]:
                    seen.add(neighbour); next_frontier.append(neighbour)
        frontier = next_frontier
    return length, nodes, target in seen
```

COMPLEXITY | Time O(V + E) | Space O(V)

210 Radio Broadcast Reachability

MEDIUM

Template move: neighbors(u) = every radio within the radius, worked out with squared distances; goal = the target radio, answer = whether it was reached

QUESTION

You are given a list of radios placed on a 2D plane, each described by its (x, y) coordinates. Every radio can broadcast to any other radio within a fixed Euclidean distance radius (inclusive). A message can travel from one radio to another either directly (within radius) or through a chain of intermediate radios that each relay it to a next radio within radius . Given the radios, the broadcast radius , a source index, and a target index, determine whether a message originating at source can reach target through zero or more hops. Return True or False .

Function signature

from typing import List, Tuple
def can_reach(radios: List[Tuple[float, float]],
radius: float,
source: int,
target: int) -> bool:
...

Follow-up (as reported): Model the radios as nodes and connect any two within radius , then run BFS (or DFS / union-find) from source . To avoid floating-point issues, compare squared distances against radius * radius .

EXAMPLES

Example 1	
Input	`radios = [(0, 0), (1, 0), (2, 0), (5, 0)], radius = 1.5, source = 0, target = 2`
Output	`True`
Why	`0->1 (dist 1) and 1->2 (dist 1) are each within 1.5, so 2 is reachable.`

Example 2	
Input	`radios = [(0, 0), (1, 0), (2, 0), (5, 0)], radius = 1.5, source = 0, target = 3`
Output	`False`
Why	`Radio 3 is 3 units from radio 2 (> 1.5) and farther from the rest; no chain of hops reaches it.`

QUESTION

Example 3	
Input	`radios = [(0, 0), (3, 4)], radius = 5.0, source = 0, target = 1`
Output	`True`
Why	`The Euclidean distance is exactly 5.0, which is within radius.`

CONSTRAINTS

- 1 <= len(radios) <= 2000
- Coordinates are integers or floats with -10^4 <= x, y <= 10^4 . radius >= 0 .
- 0 <= source, target < len(radios) .
- Two radios a and b are directly connected when sqrt((ax-bx)^2 + (ay-by)^2) <= radius .
- If source == target , the answer is True .

WHY IT WORKS

Compare squared distances; every radio enters the queue once.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): BFS over the implicit graph
# Idea: Compare squared distances; every radio enters the
# queue once.
from collections import deque


def can_reach(radios, radius, source, target):
    limit = radius * radius
    seen = [False] * len(radios)
    seen[source] = True
    queue = deque([source])
    while queue:
        current = queue.popleft()
        if current == target:
            return True
        for other, (other_x, other_y) in enumerate(radios):
            x_gap, y_gap = radios[current][0] - other_x, radios[current][1] - other_y
            if not seen[other] and x_gap * x_gap + y_gap * y_gap <= limit:
                seen[other] = True
                queue.append(other)
    return False
```

COMPLEXITY | Time O(n^2) | Space O(n)

211 Minimum Direction Violations in a Directed Graph — MEDIUM

Template move: an edge in its own direction costs w = 0, the same edge walked backwards costs w = 1; free steps use appendleft

QUESTION

A directed graph has n nodes. Walking along an edge in its own direction is free; walking against it costs 1 (a direction violation). Return the minimum total cost of a walk from start to end, or -1 if end cannot be reached at all.

Function signature

def min_violations(n: int, edges: list, start: int, end: int) -> int

EXAMPLES

QUESTION

Example 1	
Input	`5, [[0, 1], [2, 1], [2, 3], [4, 3]], 0, 4`
Output	`2`
Why	`Path 0 -> 1 (free), 1 -> 2 against edge 2 -> 1 (cost 1), 2 -> 3 (free), 3 -> 4 against edge 4 -> 3 (cost 1): total 2.`

Example 2	
Input	`3, [[0, 1]], 0, 2`
Output	`-1`
Why	`Node 2 has no edge at all, so it cannot be reached.`

WHY IT WORKS

Add every edge twice: forward with weight 0, backward with weight 1. Shortest path with weights in {0, 1} is a deque BFS: push a zero-weight neighbour to the front and a one-weight neighbour to the back. Plain BFS minimizes edge count, not violations, which is the common mistake.

SHORT OPTIMAL CODE

```python
from collections import deque


# BASE QUESTION (optimal): 0-1 BFS on the doubled graph
# Idea: Add every edge twice: forward with weight 0,
# backward with weight 1. Shortest path with weights in
# {0, 1} is a deque BFS: push a zero-weight neighbour to
# the front and a one-weight neighbour to the back. Plain
# BFS minimizes edge count, not violations, which is the
# common mistake.
def minDirectionViolations(node_count, edges, start, end):
    graph = [[] for _ in range(node_count)]
    for node, neighbour in edges:
        graph[node].append((neighbour, 0))
        graph[neighbour].append((node, 1))
    violations = [float("inf")] * node_count
    violations[start] = 0
    queue = deque([start])
    while queue:
        node = queue.popleft()
        for neighbour, cost in graph[node]:
            if violations[node] + cost < violations[neighbour]:
                violations[neighbour] = violations[node] + cost
                queue.appendleft(neighbour) if cost == 0 else queue.append(neighbour)
    return -1 if violations[end] == float("inf") else violations[end]
```

COMPLEXITY | Time O(n + m) | Space O(n + m)

212 Compute shortest paths with blocked nodes — MEDIUM

Template move: neighbors skip every blocked node; if the source itself is blocked the answer is all -1; unreachable nodes stay -1

QUESTION

Given a graph with nodes and edges and a designated source node s, compute the shortest distance from s to every other node. Some nodes are inaccessible (blocked) and cannot be traversed or reached; treat them as removed from the graph. Return a distance for each node where unreachable nodes (including blocked ones) are -1. Describe your algorithm, data structures, and time/space complexity, and note any differences for directed vs. undirected graphs.

QUESTION

Given n nodes labeled 0..n-1, edges, a source, and blocked nodes, return shortest unweighted distances from the source. Blocked and unreachable nodes must be -1. Set directed=True to treat edges as directed.

Constraints

Nodes are labeled 0 through n-1Edges are unweighted

EXAMPLES

```
Input: (5, [[0, 1], [1, 2], [0, 3], [3, 4]], 0, [3], False)
Expected Output: [0, 1, 2, -1, -1]
Input: (4, [[0, 1], [1, 2], [2, 3]], 0, [], True)
Expected Output: [0, 1, 2, 3]
```

Hints

Treat blocked nodes as removed before BFS.For directed graphs, only add the given edge direction.

WHY IT WORKS

Build adjacency lists (one direction only if directed). If the source is blocked return all -1. Run BFS from the source, skipping blocked neighbours; the distance array starts at -1 and is set when a node is first reached.

SHORT OPTIMAL CODE

```python
from collections import defaultdict
from collections import deque


# BASE QUESTION (optimal): BFS that never enters a blocked
# node
# Idea: Build adjacency lists (one direction only if
# directed). If the source is blocked return all -1. Run
# BFS from the source, skipping blocked neighbours; the
# distance array starts at -1 and is set when a node is
# first reached.
def blocked_distances(node_count, edges, source, blocked, directed=False):
    blocked_set = set(blocked)
    if source in blocked_set:
        return [-1] * node_count
    neighbours = defaultdict(list)
    for node, neighbour in edges:
        neighbours[node].append(neighbour)
        if not directed:
            neighbours[neighbour].append(node)
    distance = [-1] * node_count
    distance[source] = 0
    queue = deque([source])
    while queue:
        node = queue.popleft()
        for neighbour in neighbours[node]:
            if neighbour not in blocked_set and distance[neighbour] == -1:
                distance[neighbour] = distance[node] + 1
                queue.append(neighbour)
    return distance
```

COMPLEXITY | Time O(V + E) | Space O(V + E)

213 Shortest Directed Cycle Through a Node

MEDIUM

Template move: start = v; the goal is an edge that leads back to v, so return dist[u] + 1 the moment a neighbour equals v

QUESTION

Given a directed graph with n nodes and a list of edges (a, b), return the length (number of edges) of the shortest cycle that passes through node v, or -1 if v is on no cycle.

Function signature

def shortest_cycle_through(n: int, edges: list, v: int) -> int

EXAMPLES

Example 1	
Input	3, [(0, 1), (1, 2), (2, 0)], 0
Output	3
Why	0 -> 1 -> 2 -> 0 is a cycle of 3 edges.

Example 2	
Input	3, [(0, 1), (1, 2)], 0
Output	-1
Why	No edge leads back to 0.

Example 3	
Input	3, [(0, 1), (1, 0), (1, 2), (2, 0)], 0
Output	2
Why	A shorter cycle 0 -> 1 -> 0 exists next to the longer one.

WHY IT WORKS

A directed cycle through v is a path v to u followed by an edge u to v. BFS visits nodes in nondecreasing distance, so the first time we see an edge back to v its length is dist[u] + 1 and is the shortest. A self-loop gives 1. Return -1 if BFS ends without closing.

SHORT OPTIMAL CODE

```python
from collections import deque


# BASE QUESTION (optimal): BFS from v, close the cycle on
# the first edge back to v
# Idea: A directed cycle through v is a path v to u
# followed by an edge u to v. BFS visits nodes in
# nondecreasing distance, so the first time we see an edge
# back to v its length is distance_from_start[node] + 1 and is the shortest.
# A self-loop gives 1. Return -1 if BFS ends without
# closing.
def shortest_cycle_through(node_count, edges, start_node):
    neighbours = [[] for _ in range(node_count)]
    for from_node, to_node in edges:
        neighbours[from_node].append(to_node)
    distance_from_start = {start_node: 0}
    queue = deque([start_node])
    while queue:
        node = queue.popleft()
        for next_node in neighbours[node]:
            if next_node == start_node:
                return distance_from_start[node] + 1
            if next_node not in distance_from_start:
                distance_from_start[next_node] = distance_from_start[node] + 1
```

```
                queue.append(next_node)
    return -1
```

COMPLEXITY | Time O(V + E) | Space O(V)

GROUP 29 OF 43 | 5 QUESTIONS | SHORT MAIN CODE

Topological sort (Kahn) and cycle detection

Pattern

Count the incoming edges of every node, start from the nodes with none, and peel them off one by one. If every node got peeled there is a valid order; if some never did, there is a cycle. The questions only differ in what the nodes are called and whether you return the order or a yes/no.

214 Course Scheduling and Dependency Variants — MEDIUM

Template move: build the vertex set from the edges too; return the order or None

QUESTION

Topological-ordering family. Variants: course ordering with validity checks and a nested-list extension; merging two arrays whose internal order defines precedence (return false on a conflict cycle); and a damaged-node dependency-path variant that minimizes broken nodes on a route.

Course-ordering variant

Work through the course-ordering problem, including validity checks, complexity analysis, and test cases.
Extend the result to a nested-list format. The grouping rule is unspecified; clarify what each inner list represents before implementing the extension.

Damaged-node and path variant

Remove damaged nodes and their downstream effects, then find a valid path.
A follow-up minimizes broken nodes on a required route. Clarify the graph direction, path endpoints, and meaning of broken nodes before choosing an algorithm.

Merge-two-ordered-sequences variant

Input: two integer arrays a and b. The order of elements inside each array is an ordering constraint — a = [1, 3, 5] means 1 must come before 3, and 3 before 5.
Determine whether the two arrays can be merged into one sequence that preserves the relative order of both. Return false if the constraints conflict; otherwise return true (or any valid merged sequence).

EXAMPLES

Merge-two-ordered-sequences

```
a = [1, 2, 3]
b = [2, 4, 5]
-> true   (one valid merged order: [1, 2, 3, 4, 5])

a = [1, 6, 4]
b = [4, 1]
-> false  (a requires 1 -> 6 -> 4, b requires 4 -> 1: contradiction)
```

WHY IT WORKS

Build the graph (prerequisite -> dependent, including isolated vertices), count indegrees and keep a queue of vertices with indegree 0. Pop one, append it to the order and lower the indegree of its dependents. If fewer than V vertices were emitted there is a cycle. For the merge question, build the edges from adjacent pairs of both arrays and answer True exactly when a full order exists (a repeated value inside one array is a self-loop, so it fails).

SHORT OPTIMAL CODE

```
from collections import defaultdict
from collections import deque


# BASE QUESTION (optimal): Kahn's algorithm (course order,
```

```
# or can two sequences merge)
# Idea: Build the graph (prerequisite -> dependent,
# including isolated vertices), count indegrees and keep a
# queue of vertices with indegree 0. Pop one, append it to
# the order and lower the indegree of its dependents. If
# fewer than V vertices were emitted there is a cycle. For
# the merge question, build the edges from adjacent pairs
# of both arrays and answer True exactly when a full order
# exists (a repeated value inside one array is a self-
# loop, so it fails).
def topological_order(vertices, edges):
    dependents, indegree = defaultdict(list), {vertex: 0 for vertex in vertices}
    for prerequisite, dependent in edges:
        dependents[prerequisite].append(dependent)
        indegree[dependent] = indegree.get(dependent, 0) + 1
        indegree.setdefault(prerequisite, indegree.get(prerequisite, 0))
    queue = deque(vertex for vertex, incoming in indegree.items() if incoming == 0)
    order = []
    while queue:
        prerequisite = queue.popleft()
        order.append(prerequisite)
        for dependent in dependents[prerequisite]:
            indegree[dependent] -= 1
            if indegree[dependent] == 0:
                queue.append(dependent)
    return order if len(order) == len(indegree) else None
```

COMPLEXITY | Time O(V + E) | Space O(V + E)

215 Validate Race Results (Cycle Detection) — MEDIUM

Template move: edge = winner -> loser; the results are consistent exactly when there is no cycle

QUESTION

You are given a list of pairwise race results as (winner, loser) string pairs, meaning winner beat loser . The results are consistent if they can all be true simultaneously under a strict ranking — equivalently, if the directed graph (edge winner -> loser) contains no cycle. Return True if the results are valid (acyclic) and False otherwise.

Function signature

def results_are_valid(results: list[tuple[str, str]]) -> bool:

...

Follow-up (as reported): Recursive DFS can hit Python's recursion limit on long chains; convert to an iterative DFS with an explicit stack, or use Kahn's topological-sort algorithm.

EXAMPLES

Example 1	
Input	`results = [("A","B"), ("B","C")]`
Output	`True`
Why	`A beat B beat C — a consistent ranking A > B > C, no cycle.`

Example 2	
Input	`results = [("A","B"), ("B","C"), ("C","A")]`
Output	`False`
Why	`A > B > C > A is a contradiction (a cycle).`

QUESTION

Example 3	
Input	`results = []`
Output	`True`
Why	`No results impose no contradiction.`

CONSTRAINTS

- 0 <= len(results) <= 10^5 .
- Player names are non-empty strings; duplicate result pairs may appear.
- An empty result list is valid (True).

WHY IT WORKS

Peel off nodes with no incoming edge; everything peels off exactly when there is no cycle.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Kahn's algorithm
# Idea: Peel off nodes with no incoming edge; everything
# peels off exactly when there is no cycle.
from collections import deque


def results_are_valid(results):
    graph, indegree = {}, {}
    for winner, loser in set(results):
        graph.setdefault(winner, []).append(loser)
        graph.setdefault(loser, [])
        indegree[loser] = indegree.get(loser, 0) + 1
        indegree.setdefault(winner, 0)
    queue = deque(racer for racer in graph if indegree[racer] == 0)
    removed = 0
    while queue:
        racer = queue.popleft()
        removed += 1
        for slower in graph[racer]:
            indegree[slower] -= 1
            if indegree[slower] == 0:
                queue.append(slower)
    return removed == len(graph)
```

COMPLEXITY | Time O(V + E) | Space O(V + E)

216 Solve tree, graph, sliding-window problems — MEDIUM

Template move: the course-schedule part is can-finish: len(order) == n; the zigzag and window parts are separate small loops

QUESTION

Question

LeetCode 103. Binary Tree Zigzag Level Order Traversal – given a binary tree root, return its zigzag level-order traversal. LeetCode 207. Course Schedule – given course prerequisites, determine if all courses can be finished by detecting cycles with topological sort. LeetCode 340. Longest Substring with At Most K Distinct Characters – given string s and integer k, return the length of the longest substring containing at most k distinct characters.
https://leetcode.com/problems/binary-tree-zigzag-level-order-traversal/description/
https://leetcode.com/problems/course-schedule/description/
https://leetcode.com/problems/longest-substring-with-at-most-k-distinct-characters/description/

QUESTION

Part 1: Binary Tree Zigzag Level Order TraversalYou are given a binary tree in level-order array form, where each element is either an integer or None for a missing child. Return the zigzag level-order traversal of the tree's node values. In a zigzag traversal, the first level is read from left to right, the second from right to left, the third from left to right, and so on.

Constraints

0 <= len(root) <= 2000Each non-None node value is an integer in the range [-10^4, 10^4]The input list represents a valid binary tree in level-order form

EXAMPLES

```
Input: ([3, 9, 20, None, None, 15, 7],)
Expected Output: [[3], [20, 9], [15, 7]]
```

Explanation: Level 1 is read left-to-right, level 2 right-to-left, and level 3 left-to-right.

```
Input: ([1],)
Expected Output: [[1]]
```

Explanation: A single-node tree has only one level.

Hints

A breadth-first search naturally processes the tree one level at a time.You can collect each level left-to-right, then reverse every other level before appending it to the answer.

Part 2: Course ScheduleThere are num_courses courses labeled from 0 to num_courses - 1. You are given a list of prerequisite pairs [a, b], meaning you must take course b before course a. Determine whether it is possible to finish all courses. This is equivalent to checking whether the prerequisite graph contains a cycle.

Constraints

1 <= num_courses <= 10^50 <= len(prerequisites) <= 2 * 10^5Each prerequisite pair has length 20 <= course, prerequisite < num_courses

Examples

Input: (2, [[1, 0]])

Expected Output: True

Explanation: Course 0 can be taken first, then course 1.

Input: (2, [[1, 0], [0, 1]])

Expected Output: False

Explanation: The two courses depend on each other, forming a cycle.

Hints

Think of courses as nodes in a directed graph and prerequisites as directed edges.If you repeatedly take courses with indegree 0 and cannot process all nodes, a cycle exists.

Part 3: Longest Substring with At Most K Distinct CharactersGiven a string s and an integer k, return the length of the longest substring that contains at most k distinct characters.

Constraints

0 <= len(s) <= 2 * 10^50 <= k <= len(s)s may contain any standard characters

QUESTION

Examples
Input: ("eceba", 2)
Expected Output: 3
Explanation: The longest valid substring is 'ece', which has length 3.
Input: ("aa", 1)
Expected Output: 2
Explanation: The whole string contains only one distinct character.
Hints
A sliding window can maintain a valid substring while expanding to the right.Keep character counts in the current window, and shrink from the left whenever the number of distinct characters exceeds k.

PART 1: Course schedule: Kahn's topological sort

Count prerequisites per course and start from courses with none. Remove a course, decrease the count of the courses that depend on it, and queue those that reach zero. All courses can be taken exactly when every course is removed; a leftover means a cycle.

PART 2: Longest window: sliding window with a count map

Add s[right] to the counts. While more than K distinct characters are present, remove s[left] (delete the key when its count hits 0) and advance left. The window is always valid after the shrink, so record its length. Each index enters and leaves once.

PART 3: Zigzag: fill each level from the correct end

Keep a direction flag; for each level build a deque, appending on the right when going left to right and on the left otherwise. Flip the flag after each level; no reversal is needed.

SHORT OPTIMAL CODE

```
from collections import defaultdict
from collections import deque


# Course schedule: Kahn's topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
# of the courses that depend on it, and queue those that
# reach zero. All courses can be taken exactly when every
# course is removed; a leftover means a cycle.
def canFinish(course_count, prereqs):
    dependents, indegree = defaultdict(list), [0] * course_count
    for course, prerequisite in prereqs:
        dependents[prerequisite].append(course)
        indegree[course] += 1
    queue = deque(course for course in range(course_count) if indegree[course] == 0)
    taken_count = 0
    while queue:
        ready_course = queue.popleft()
        taken_count += 1
        for nxt in dependents[ready_course]:
            indegree[nxt] -= 1
            if indegree[nxt] == 0:
                queue.append(nxt)
    return taken_count == course_count


# Longest window: sliding window with a count map
# Idea: Add text[right] to the counts. While more than K
# distinct characters are present, remove text[left] (delete
# the key when its count hits 0) and advance left. The
# window is always valid after the shrink, so record its
# length. Each index enters and leaves once.
```

```python
def lengthOfLongestSubstringKDistinct(text, max_distinct):
    count, left, best = {}, 0, 0
    for right, char in enumerate(text):
        count[char] = count.get(char, 0) + 1
        while len(count) > max_distinct:
            count[text[left]] -= 1
            if count[text[left]] == 0:
                del count[text[left]]
            left += 1
        best = max(best, right - left + 1)
    return best


# Zigzag: fill each level from the correct end
# Idea: Keep a direction flag; for each level build a
# deque, appending on the right when going left to right
# and on the left otherwise. Flip the flag after each
# level; no reversal is needed.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def zigzagLevelOrder(root):
    levels, queue, left_to_right = [], deque([root] if root else []), True
    while queue:
        level = deque()
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val) if left_to_right else level.appendleft(node.val)
            queue.extend(child for child in (node.left, node.right) if child)
        levels.append(list(level))
        left_to_right = not left_to_right
    return levels
```

COMPLEXITY | Part 1: Time O(V + E) | Space O(V + E)
Part 2: Time O(n) | Space O(k)
Part 3: Time O(n) | Space O(width)

217 Validate course catalog dependencies — MEDIUM

Template move: first reject prerequisites naming courses that do not exist; then the same peel; report the missing names or the cycle

QUESTION

You are given a set of course ids and prerequisite pairs (u, v), meaning course v requires course u. The catalog is valid when no prerequisite mentions a course that does not exist and there is no cycle. Return (True, [], order) with a possible study order when valid; (False, missing, None) listing the unknown course ids; or (False, [], cycle) with one cycle that was found.

Function signature

def validate_catalog(courses: set, prereqs: list) -> tuple

EXAMPLES

Example 1	
Input	{1, 2, 3, 4}, [(1, 2), (2, 3), (1, 3)]
Output	(true, [], [1, 4, 2, 3])
Why	Course 1 comes first, 4 is unrelated, then 2 and 3.

Example 2	
Input	{1, 2}, [(1, 9)]
Output	(false, [9], null)
Why	Course 9 is not in the catalog.

QUESTION

Example 3	
Input	{1, 2}, [(1, 2), (2, 1)]
Output	(false, [], [1, 2, 1])
Why	1 requires 2 and 2 requires 1: a cycle.

WHY IT WORKS

Report unknown courses (any referenced id not in the set). If none, run Kahn's algorithm: repeatedly remove courses with no remaining prerequisites, building an order. If every course is removed return (True, order). Otherwise the courses left include a cycle: every leftover course still has an unmet prerequisite inside the leftover set, so walk backwards along prerequisites until a course repeats and return that cycle in forward order.

SHORT OPTIMAL CODE

```
from collections import defaultdict
from collections import deque


# BASE QUESTION (optimal): Missing references, then Kahn's
# algorithm, then a cycle from the leftovers
# Idea: Report unknown courses (any referenced id not in
# the set). If none, run Kahn's algorithm: repeatedly
# remove courses with no remaining prerequisites, building
# an order. If every course is removed return (True,
# order). Otherwise the courses left include a cycle:
# every leftover course still has an unmet prerequisite
# inside the leftover set, so walk backwards along
# prerequisites until a course repeats and return that
# cycle in forward order.
def validate_catalog(courses, prereqs):
    course_set = set(courses)
    missing = sorted({course_id for prerequisite, course in prereqs for course_id in (prerequisite, course) if
course_id not in course_set}, key=str)
    if missing:
        return False, missing, None
    dependents, indegree = defaultdict(list), {course: 0 for course in course_set}
    for prerequisite, course in prereqs:
        dependents[prerequisite].append(course)
        indegree[course] += 1
    queue = deque(sorted((course for course in course_set if indegree[course] == 0), key=str))
    order = []
    while queue:
        ready_course = queue.popleft()
        order.append(ready_course)
        for dependent in dependents[ready_course]:
            indegree[dependent] -= 1
            if indegree[dependent] == 0:
                queue.append(dependent)
    if len(order) == len(course_set):
        return True, [], order
    left = {course for course in course_set if indegree[course] > 0}
    prerequisites_of = defaultdict(list)
    for prerequisite, course in prereqs:
        if prerequisite in left and course in left:
            prerequisites_of[course].append(prerequisite)
    start = min(left, key=str)
    walk, walk_position, course_id = [], {}, start
    while course_id not in walk_position:                                    # every leftover course has a leftov
er prerequisite, so this walk never stops early
        walk_position[course_id] = len(walk)
        walk.append(course_id)
        course_id = prerequisites_of[course_id][0]
```

```
    cycle = walk[walk_position[course_id]:] + [course_id]
    return False, [], cycle[::-1]                        # walked against the edges, so reverse to list prerequ
isites first
```

COMPLEXITY | Time O(V + E) | Space O(V + E)

218 Course Schedule — MEDIUM

Template move: edge = prerequisite -> course; return the order, or an empty list when there is a cycle

QUESTION

You are given numCourses and a list of prerequisites, where each element in prerequisites[i] = [a_i, b_i] means you must complete course b_i before taking course a_i. Write a function canFinish that returns True if it's possible to finish all courses, and False if it's not. You need to complete all the courses for the result to be True. Assume there are no duplicate prerequisites, and numCourses is a non-negative number. Examples Input: numCourses = 3, prerequisites = [[2, 1], [1, 0]]
Output: True Explanation: You can complete course 0 first, then course 1, and finally course 2.
Input: numCourses = 4, prerequisites = [[3, 2], [2, 1], [1, 0], [0, 3]]
Output: False Explanation: The prerequisites form a cycle, making it impossible to finish all courses.
Input: numCourses = 5, prerequisites = [[4, 2], [3, 1], [2, 0]]
Output: True Explanation: You can complete the courses in order from course 0 to course 4.Hint 1Try modeling the courses as a graph, with edges representing prerequisites. Consider how you can detect cycles to determine if all courses can be completed.

EXAMPLES

Example 1	
Input	`2, [[1, 0]]`
Output	`true`
Why	`Course 1 requires course 0: finishing both is possible.`

Example 2	
Input	`2, [[1, 0], [0, 1]]`
Output	`false`
Why	`Course 1 requires 0 and course 0 requires 1: a cycle.`

Example 3	
Input	`2, [[1, 0]]`
Output	`[0, 1]`
Why	`A valid order: take 0 before 1.`

WHY IT WORKS

Count each course's unmet prerequisites and queue those with none. Take a course, reduce the counts of the courses that need it, and queue any that reach zero. All courses can be finished exactly when every course was taken; the order taken is a valid schedule.

SHORT OPTIMAL CODE

```
from collections import defaultdict
from collections import deque


# BASE QUESTION (optimal): Kahn's algorithm (topological
# sort)
```

```python
# Idea: Count each course's unmet prerequisites and queue
# those with none. Take a course, reduce the counts of the
# courses that need it, and queue any that reach zero. All
# courses can be finished exactly when every course was
# taken; the order taken is a valid schedule.
def find_order(course_count, prereqs):
    dependents, indegree = defaultdict(list), [0] * course_count
    for course, prerequisite in prereqs:
        dependents[prerequisite].append(course)
        indegree[course] += 1
    queue, order = deque(course for course in range(course_count) if indegree[course] == 0), []
    while queue:
        ready_course = queue.popleft()
        order.append(ready_course)
        for dependent in dependents[ready_course]:
            indegree[dependent] -= 1
            if indegree[dependent] == 0:
                queue.append(dependent)
    return order if len(order) == course_count else []
```

COMPLEXITY | Time O(V + E) | Space O(V + E)

GROUP 30 OF 43 | 5 QUESTIONS | SHORT MAIN CODE

Counting pairs by looking up each partner in a Counter or a bucket

Pattern

For each value, compute the partner it would need (v + k, 2v, the other corner of a square) and look it up in a Counter or a set. When the partner depends on two things, enumerate the pairs once and bucket them by a shared key.

219 Progressive Array Partitioning — MEDIUM

Template move: Counter of the values; stage 1 asks whether all the counts are different: len(set(counts.values())) == len(counts)

QUESTION

The problem is given in stages; stage 1 is: group the equal values of an array together. Return true when every group has a different size (no two values occur the same number of times). The function takes the occurrence count of each value as a dict, so counting stays separate from the rule and later stages can change only the rule.

Function signature

def stage1(counts: dict) -> bool

counts maps value -> how

many times it occurs

EXAMPLES

Example 1

Input	`from collections import Counter`
	`nums = [1, 2, 3, 4, 2, 3, 3, 4, 4, 4]`
	`stage1(Counter(nums))`

Output	`stage1(Counter(nums)) -> true`
Why	`The groups have sizes 1, 2, 3, 4: all different, so true.`

Example 2

Input	`from collections import Counter`
	`nums = [1, 2, 3]`
	`stage1(Counter(nums))`

Output	`stage1(Counter(nums)) -> false`
Why	`Every value occurs once, so all three groups have size 1: false.`

WHY IT WORKS

All counts are different exactly when the set of counts has as many elements as there are distinct values. One pass to count, one pass over the counts.

SHORT OPTIMAL CODE

```python
from collections import Counter


# BASE QUESTION (optimal): Counter, then compare the set
# of counts with the number of values
# Idea: All counts are different exactly when the set of
# counts has as many elements as there are distinct
# values. One pass to count, one pass over the counts.
def distinct_counts(nums):
    counts = Counter(nums)
    return len(set(counts.values())) == len(counts)
```

COMPLEXITY | Time O(n) | Space O(d)

220 K-diff Pairs in an Array

MEDIUM

Template move: the template as written: partner = v + k; for k == 0 count values that appear twice

QUESTION

Given an integer array nums and an integer k , return the number of unique k-diff pairs in the array. A k-diff pair is a pair of integers (nums[i], nums[j]) where: i != j , and abs(nums[i] - nums[j]) == k . Two k-diff pairs (a, b) and (c, d) are considered the same (counted once) if they consist of the same pair of values. Count each distinct value pair only once.

Function signature

```
def count_k_diff_pairs(nums: list[int], k: int) -> int:
    """Return the number of unique value-pairs with absolute difference k."""
    ...
```

EXAMPLES

Example 1	
Input	`nums = [3, 1, 4, 1, 5], k = 2`
Output	`2`
Why	`The unique 2-diff value pairs are (1, 3) and (3, 5).`

Example 2	
Input	`nums = [1, 2, 3, 4, 5], k = 1`
Output	`4`
Why	`(1,2), (2,3), (3,4), (4,5).`

Example 3	
Input	`nums = [1, 3, 1, 5, 4], k = 0`
Output	`1`
Why	`With k = 0 count values that appear at least twice; only 1 does.`

CONSTRAINTS

- 1 <= len(nums) <= 10^4
- -10^7 <= nums[i] <= 10^7
- 0 <= k <= 10^7

WHY IT WORKS

A value v starts a pair when v + k is present; for k = 0 it must appear twice.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Count the values
# Idea: A value v starts a pair when v + difference is present; for
# difference = 0 it must appear twice.
from collections import Counter


def count_k_diff_pairs(nums, difference):
    if difference < 0:
        return 0
    counts = Counter(nums)
    if difference == 0:
        return sum(1 for copies in counts.values() if copies >= 2)
    return sum(1 for value in counts if value + difference in counts)
```

COMPLEXITY | Time O(n) | Space O(n)

221 Count Axis-Aligned Squares Among Grid Points

MEDIUM

Template move: a set of points; two opposite corners fix the other two, so look both up; every square is found twice, divide by 2

QUESTION

The interviewer described a diagram of squares and rectangles and asked which data structure to use and how to count the squares. Framed concretely: you are given a list of distinct integer lattice points in the plane. Count the number of axis-aligned squares whose four corners all appear in points . A square is axis-aligned when its sides are parallel to the x- and y-axes. Return the count of such squares.

Function signature

```
from typing import List, Tuple
def count_axis_aligned_squares(points: List[Tuple[int, int]]) -> int:
    """Return the number of axis-aligned squares whose 4 corners are all in points."""
    ...
```

Follow-up (as reported): A cleaner O(n^2) approach: put all points in a set, then for every pair that could be two opposite corners of a square (equal absolute x- and y-difference), check the other two corners; each square is found via both its diagonals, so divide the total by 2.

EXAMPLES

Example 1	
Input	`points = [(0, 0), (0, 1), (1, 0), (1, 1)]`
Output	`1`
Why	`The four points form exactly one unit square.`

Example 2	
Input	`points = [(0,0),(0,1),(1,0),(1,1),(0,2),(1,2),(2,0),(2,1),(2,2)]`
Output	`5`
Why	`A 3x3 grid of points has four 1x1 squares and one 2x2 square.`

Example 3	
Input	`points = [(0, 0), (2, 0), (0, 1), (2, 1)]`
Output	`0`
Why	`These form a 2x1 rectangle, which is not a square.`

CONSTRAINTS

- 0 <= len(points) <= 2000 ; all points are distinct.
- Each point is (x, y) with -10^9 <= x, y <= 10^9 .
- A square must have positive side length (four distinct corners); degenerate zero-size "squares" do not count.
- Only axis-aligned squares count (tilted squares are ignored).

WHY IT WORKS

Two opposite corners decide the other two, so one set lookup per pair is enough.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Opposite corners and a set
# Idea: Two opposite corners decide the other two, so one
# set lookup per pair is enough.
def count_axis_aligned_squares(points):
    seen = set(points)
    total = 0
    for index, (x_first, y_first) in enumerate(points):
        for x_second, y_second in points[index + 1:]:
```

```
            if abs(x_first - x_second) == abs(y_first - y_second) != 0:
                if (x_first, y_second) in seen and (x_second, y_first) in seen:
                    total += 1
    return total // 2
```

COMPLEXITY | Time O(n^2) | Space O(n)

222 Sum of Two Squares in Two Ways

MEDIUM

Template move: enumerate every pair (a, b) with a <= b once, add a*a + b*b to a Counter, answer = sums counted exactly twice

QUESTION

Given an integer n, find all integers s < n that can be written as a sum of two positive squares in exactly two different ways (the order of the two squares does not matter). For example 50 = 1^2 + 7^2 = 5^2 + 5^2. Return them in increasing order.

Function signature

def two_squares_two_ways(n: int) -> list

EXAMPLES

Example 1	
Input	100
Output	[50, 65, 85]
Why	50 = 1 + 49 = 25 + 25 and 65 = 1 + 64 = 16 + 49 both have exactly two ways.

Example 2	
Input	50
Output	[]
Why	Nothing below 50 qualifies.

CONSTRAINTS

- 1 <= n <= 2^31 - 1

WHY IT WORKS

Loop a from 1 while 2a^2 < n and b from a while a^2 + b^2 < n, adding each sum to a Counter. There are about pi * n / 8 pairs. The answer is the sorted list of sums whose count is exactly 2.

SHORT OPTIMAL CODE

```
from collections import Counter


# BASE QUESTION (optimal): Count every pair once in a hash
# map
# Idea: Loop a from 1 while 2a^2 < limit and b from a while
# a^2 + b^2 < limit, adding each sum to a Counter. There are
# about pi * limit / 8 pairs. The answer is the sorted list of
# sums whose count is exactly 2.
def two_squares_two_ways(limit):
    sum_counts = Counter()
    a = 1
    while 2 * a * a < limit:
        b = a
        while a * a + b * b < limit:
            sum_counts[a * a + b * b] += 1
            b += 1
        a += 1
    return sorted(square_sum for square_sum, ways_count in sum_counts.items() if ways_count == 2)
```

COMPLEXITY | Time O(n) | Space O(n)

223 Maximum-Area Rectangle from Planar Points

HARD

Template move: bucket every pair of points by (sum of coordinates, squared length); two pairs in one bucket are the diagonals of a rectangle

QUESTION

Given 2D points, find the maximum area of a rectangle whose four corners are input points; sides need not be parallel to the axes. Candidates compare it to LC 963 (Minimum Area Rectangle II) but maximizing. Interviewers ask to improve beyond an O(N^3) approach.

Input: a list of 2D coordinate points.
Return the maximum area of any rectangle whose four vertices are all input points.
Rectangle sides do not need to be parallel to the coordinate axes.
Follow-up pressure: start from an O(N^3) idea, then optimize toward a better complexity.

EXAMPLES

Example 1	
Input	`[(1, 1), (1, 3), (3, 1), (3, 3)]`
Output	`4`
Why	`The four points form a 2 x 2 axis-aligned square.`

Example 2	
Input	`[(0, 1), (1, 0), (2, 1), (1, 2)]`
Output	`2`
Why	`A tilted rectangle: a square with diagonal 2 has area 2.`

Example 3	
Input	`[(0, 0), (1, 0), (0, 1)]`
Output	`0`
Why	`Three points never make a rectangle.`

WHY IT WORKS

For every pair (p, q) use the key (p.x + q.x, p.y + q.y, squared length). All pairs with the same key are diagonals of one circle around the same centre, so any two of them form a rectangle. For two diagonals (p, q) and (r, s) the sides from p are r - p and s - p, and the area is the absolute cross product. Keep the maximum; return 0 when no bucket has two pairs.

SHORT OPTIMAL CODE

```
from collections import defaultdict
import itertools


# BASE QUESTION (optimal): Bucket pairs by diagonal
# midpoint and length
# Idea: For every pair (p, q) use the key (p.x + q.x, p.y
# + q.y, squared length). All pairs with the same key are
# diagonals of one circle around the same centre, so any
# two of them form a rectangle. For two diagonals (p, q)
# and (r, s) the sides from p are r - p and s - p, and the
# area is the absolute cross product. Keep the maximum;
# return 0 when no bucket has two pairs.
def max_rect_area(points):
    unique_points = list(set(map(tuple, points)))
    diagonals_by_key = defaultdict(list)
    for first_index in range(len(unique_points)):
        for second_index in range(first_index + 1, len(unique_points)):
            p, q = unique_points[first_index], unique_points[second_index]
```

```
            diagonals_by_key[(p[0] + q[0], p[1] + q[1], (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)].append((p, q)
)
    best = 0
    for pairs in diagonals_by_key.values():
        for (p, q), (r, s) in itertools.combinations(pairs, 2):
            best = max(best, abs((r[0] - p[0]) * (s[1] - p[1]) - (r[1] - p[1]) * (s[0] - p[0])))
    return best
```

COMPLEXITY | Time O(n^2) pairs (worst case O(n^3) if every pair shares a bucket) | Space O(n^2)

GROUP 31 OF 43 | 5 QUESTIONS | SHORT MAIN CODE

Tree DP: build a parent order once, then add up from the leaves

Pattern

Get an order where every parent comes before its children (a BFS order from the root), then sweep it backwards so every child is finished before its parent. Subtree sizes and sums fall out of that one backwards sweep; the re-rooting questions add a second forward sweep.

224 Compute File System Total Size — MEDIUM

Template move: add each file's size while walking down from the root with a stack; a shared directory is counted once (seen set)

QUESTION

A file system is a tree: File(size) is a file and Dir(*children) is a directory holding files and directories. Return the total size of all files. A directory object can be shared (it appears under more than one parent); its files are counted once.

Function signature

def total_size(root) -> int

EXAMPLES

Example 1

Input	`root = Dir(Dir(File(10), File(20)), File(5), Dir(File(30)))` `total_size(root)`
Output	`total_size(root) -> 65`
Why	`10 + 20 + 5 + 30 = 65.`

Example 2

Input	`shared = Dir(File(7))`
	`root = Dir(shared, shared, File(1))`
	`total_size(root)`

Output	`total_size(root) -> 8`
Why	`The same directory appears twice but its 7 is counted once: 7 + 1 = 8.`

WHY IT WORKS

An explicit stack removes the recursion limit, and a set of visited node ids makes each node count once, so shared directories are not double counted and cycles terminate.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Iterative traversal with a
# visited set
# Idea: An explicit stack removes the recursion limit, and
# a set of visited node ids makes each node count once, so
# shared directories are not double counted and cycles
# terminate.
class File:
    def __init__(self, size):
        self.size = size

class Dir:
    def __init__(self, *children):
        self.children = list(children)

def total_size(root):
    total, seen, stack = 0, set(), [root]
    while stack:
```

```
        node = stack.pop()
        if id(node) in seen:
            continue
        seen.add(id(node))
        if isinstance(node, File):
            total += node.size
        else:
            stack.extend(node.children)
    return total
```

COMPLEXITY | Time O(n) | Space O(n)

225 Minimum-Cost Tree Disconnection

MEDIUM

Template move: backwards sweep with two numbers per node: the best cost when the node is cut off, and the cost if it is free

QUESTION

A rooted tree is given as directed edges (parent, child, cost) with node 0 as the root. Cut edges so that no path from the root to any original leaf remains. The price of a cut set is the sum of the costs of the edges cut (costs may be negative). Return the minimum total price.

Function signature

def disconnect_cost(n: int, edges: list) -> int

EXAMPLES

Example 1	
Input	3, [(0, 1, 3), (0, 2, 2)]
Output	5
Why	Two leaves hang off the root: both edges must be cut, 3 + 2 = 5.

Example 2	
Input	3, [(0, 1, 4), (1, 2, 1)]
Output	1
Why	A chain 0 - 1 - 2: cutting the cheaper edge 1 -> 2 (cost 1) already separates the leaf.

Example 3	
Input	3, [(0, 1, -1), (1, 2, -1)]
Output	-2
Why	Negative costs: cutting both edges costs -2, which beats cutting only one.

WHY IT WORKS

For a child c with edge weight w there are two options: cut the edge, paying w plus whatever extra negative-weight edges you can still cut below c (free(c) = sum of min(0, weight) over its subtree), or keep the edge and require best(c), which is impossible when c is a leaf. best(v) is the sum of the cheaper option over its children. With non-negative weights free(c) is 0 and this is the classic recurrence.

SHORT OPTIMAL CODE

```
from collections import defaultdict


# BASE QUESTION (optimal): Tree DP: best(v) = cheapest
# cuts inside v's subtree that disconnect v from its
# leaves
```

```python
# Idea: For a child c with edge weight w there are two
# options: cut the edge, paying w plus whatever extra
# negative-weight edges you can still cut below c (free_total(child)
# = sum of min(0, weight) over its subtree), or keep the
# edge and require best(child), which is impossible when c is
# a leaf. best(v) is the sum of the cheaper option over
# its children. With non-negative weights free_total(child) is 0 and
# this is the classic recurrence.
def disconnect_cost(node_count, edges):
    kids = defaultdict(list)
    for parent, child, weight in edges:
        kids[parent].append((child, weight))
    INFINITY = float("inf")

    def solve(node):
        free_total = 0
        best = 0
        for child, weight in kids[node]:
            child_best, child_free = solve(child)
            free_total += min(0, weight) + child_free
            keep_cost = INFINITY if not kids[child] else child_best
            best += min(weight + child_free, keep_cost)
        return best, free_total

    return solve(0)[0] if kids[0] or node_count > 1 else 0
```

COMPLEXITY | Time O(n) | Space O(h)

226 Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes. MEDIUM

Template move: t[v] = max over children (rank among children sorted by time, longest first, + t[child]); read t at the root

QUESTION

Information starts at the root of a tree. In every iteration a node that already has the information can pass it to one of its children (the tree maps each node to its children as (child, cost) pairs; in the basic problem every cost is 1 iteration). Return the minimum number of iterations until every node has the information.

Function signature

def min_rounds(tree: dict, root) -> int

EXAMPLES

Example 1	
Input	`{'r': [('a', 1), ('b', 1), ('c', 1)], 'a': [('d', 1)]}, 'r'`
Output	`3`
Why	`The root informs a, b and c in iterations 1, 2, 3, while a informs d in iteration 2: everyone knows after 3 iterations.`

Example 2	
Input	`{'r': [('a', 1)], 'a': [('b', 1)]}, 'r'`
Output	`2`
Why	`A path r - a - b needs one iteration per edge.`

WHY IT WORKS

Let T(v) be the rounds needed to inform the subtree of v once v knows. v informs its children one per round. If children are served in decreasing order of T, the child served in round i finishes at i + T(child), so T(v) = max over i of (i + T(child_i)) with i

starting at 1. Computed iteratively so a very deep tree cannot overflow the recursion stack.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Post-order DP: serve the
# slowest subtree first
# Idea: Let T(node) be the rounds needed to inform the
# subtree of v once v knows. v informs its children one
# per round. If children are served in decreasing order of
# T, the child served in round i finishes at i + T(child),
# so T(node) = max over i of (i + T(child_i)) with i starting
# at 1. Computed iteratively so a very deep tree cannot
# overflow the recursion stack.
def minRoundsToInformAll(node_count, children, root=0):
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        stack.extend(children.get(node, []))
    rounds = {}
    for node in reversed(order):
        children_rounds = sorted((rounds[child] for child in children.get(node, [])), reverse=True)
        rounds[node] = max((turn + 1 + child_time for turn, child_time in enumerate(children_rounds)), default=
0)
    return rounds[root]
```

COMPLEXITY | Time O(n log n) | Space O(n)

227 Compute distance-sum from every tree node

HARD

Template move: down[v] = sum of distances to v's subtree; then a forward sweep: ans[child] = ans[parent] - size[child] + (n - size[child])

QUESTION

A tree has n nodes numbered 0 to n - 1 and n - 1 edges (u, v). For every node, return the sum of the distances from that node to all the other nodes.

Function signature

def distance_sums(n: int, edges: list) -> list

EXAMPLES

Example 1	
Input	`4, [(0, 1), (0, 2), (2, 3)]`
Output	`[4, 6, 4, 6]`
Why	`Node 0: 1 + 1 + 2 = 4. Node 1 is a leaf: 1 + 2 + 3 = 6. Node 2: 1 + 2 + 1 = 4. Node 3: 1 + 2 + 3 = 6.`

Example 2	
Input	`3, [(0, 1), (1, 2)]`
Output	`[3, 2, 3]`
Why	`A path of three nodes: the middle one is closest to the others.`

WHY IT WORKS

Root the tree at 0 and compute size[v] (subtree sizes) and ans[0] = the sum of depths. For a child c of v: ans[c] = ans[v] - size[c] + (n - size[c]), because the size[c] nodes below c get one closer and the other n - size[c] nodes get one farther. Done iteratively so deep trees are safe.

SHORT OPTIMAL CODE

```python
from collections import defaultdict


# BASE QUESTION (optimal): Re-rooting with subtree sizes
# Idea: Root the tree at 0 and compute size[neighbour] (subtree
# sizes) and distance_sum[0] = the sum of depths. For a child c of
# v: distance_sum[c] = distance_sum[neighbour] - size[c] + (node_count - size[c]), because
# the size[c] nodes below c get one closer and the other n
# - size[c] nodes get one farther. Done iteratively so
# deep trees are safe.
def sumOfDistancesInTree(node_count, edges):
    adj = defaultdict(list)
    for node, neighbour in edges:
        adj[node].append(neighbour); adj[neighbour].append(node)
    order, parent, depth = [], {0: -1}, {0: 0}
    stack = [0]
    while stack:
        node = stack.pop()
        order.append(node)
        for neighbour in adj[node]:
            if neighbour not in parent:
                parent[neighbour], depth[neighbour] = node, depth[node] + 1
                stack.append(neighbour)
    size = [1] * node_count
    for node in reversed(order):
        if parent[node] != -1:
            size[parent[node]] += size[node]
    distance_sum = [0] * node_count
    distance_sum[0] = sum(depth.values())
    for node in order[1:]:
        distance_sum[node] = distance_sum[parent[node]] - size[node] + (node_count - size[node])
    return distance_sum
```

COMPLEXITY | Time O(n) | Space O(n)

228 Tree Distance Sum (Re-root DP) — HARD

Template move: exactly the same two sweeps (this is the re-rooting version of the question)

QUESTION

Onsite coding round 1: given an undirected tree on N nodes, return for every node the sum of distances to all other nodes. Equivalent to LC 834. Interviewer expects baseline O(n²) DFS followed by the O(n) re-root DP.

Input: undirected tree with N nodes and N-1 edges.
Output: array ans where ans[u] is the sum of shortest-path distances from u to every other node.
Walk through baseline (BFS/DFS from every node, O(n²)) before optimizing.
Final solution should be O(n) via two-pass re-root DP.

EXAMPLES

```
Edges: [[0,1],[0,2],[2,3],[2,4],[2,5]]
ans:   [8, 12, 6, 10, 10, 10]
```

WHY IT WORKS

Root the tree at 0 and get a BFS order. In reverse order compute size[u] and the sum of distances inside u's subtree. For the root that sum is already the answer. In forward order, ans[v] = ans[parent] - size[v] + (n - size[v]): the size[v] nodes inside v's subtree are one step closer and the other n - size[v] nodes one step farther. Everything is iterative, so a chain of 10^5 nodes cannot overflow the stack.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Two passes: subtree sizes, then
# re-root
# Idea: Root the tree at 0 and get a BFS order. In reverse
```

```python
# order compute size[node] and the sum of distances inside
# u's subtree. For the root that sum is already the
# answer. In forward order, distance_sum[neighbour] = distance_sum[parent] - size[neighbour]
# + (node_count - size[neighbour]): the size[neighbour] nodes inside v's subtree
# are one step closer and the other node_count - size[neighbour] nodes one
# step farther. Everything is iterative, so a chain of
# 10^5 nodes cannot overflow the stack.
def tree_distance_sums(node_count, edges):
    if node_count == 0:
        return []
    adj = [[] for _ in range(node_count)]
    for node_a, node_b in edges:
        adj[node_a].append(node_b); adj[node_b].append(node_a)
    parent, order = [-1] * node_count, [0]
    for node in order:
        for neighbour in adj[node]:
            if neighbour != parent[node]:
                parent[neighbour] = node; order.append(neighbour)
    size, distance_inside = [1] * node_count, [0] * node_count
    for node in reversed(order):
        if parent[node] >= 0:
            size[parent[node]] += size[node]
            distance_inside[parent[node]] += distance_inside[node] + size[node]
    distance_sum = [0] * node_count
    distance_sum[0] = distance_inside[0]
    for node in order[1:]:
        distance_sum[node] = distance_sum[parent[node]] + node_count - 2 * size[node]
    return distance_sum
```

COMPLEXITY | Time O(n) | Space O(n)

Counting or reaching an amount: the knapsack loop

Pattern

Make an array indexed by amount. For each item, sweep the amounts and let ways[t] depend on ways[t - item]. Sweep upwards when an item can be reused, downwards when each item can be used once. Swap += for min(...) + 1 (fewest items) or or (reachable).

229 Maximize Shelf Space (Unbounded Knapsack) — MEDIUM

Template move: reachable[t] = any(reachable[t - size]); the answer is the largest reachable total up to the capacity

QUESTION

A shelf has a fixed integer capacity capacity . You have boxes of various integer sizes given in sizes , and you may use any number of each box, including duplicates (each size is available infinitely). Choose boxes so their total size does not exceed capacity , maximizing the total size used. Return that maximum total.

Function signature

def max_shelf_space(capacity: int, sizes: list[int]) -> int:

...

EXAMPLES

Example 1	
Input	`capacity = 10, sizes = [3, 5, 7]`
Output	`10`
Why	`3 + 7 = 10 exactly fills the shelf.`

Example 2	
Input	`capacity = 11, sizes = [5, 3]`
Output	`11`
Why	`5 + 3 + 3 = 11.`

Example 3	
Input	`capacity = 7, sizes = [4, 5]`
Output	`5`
Why	`Options within 7 are 4 or 5 (two boxes would be >= 8). The best is 5.`

CONSTRAINTS

- 0 <= capacity <= 10**4 .
- 1 <= len(sizes) <= 100 , each size is a positive integer.

WHY IT WORKS

A total is reachable if removing one box leaves a reachable total.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Reachable totals DP
# Idea: A total is reachable if removing one box leaves a
# reachable total.
def max_shelf_space(capacity, sizes):
    reachable = [True] + [False] * capacity
    for total in range(1, capacity + 1):
        reachable[total] = any(box_size <= total and reachable[total - box_size] for box_size in sizes)
```

```
    return max(total for total in range(capacity + 1) if reachable[total])
```

COMPLEXITY | Time O(capacity * boxes) | Space O(capacity)

230 Coin Change

MEDIUM

Template move: fewest coins: start with infinity, best[t] = min(best[t], best[t - c] + 1); -1 when it stays infinity

QUESTION

You are given an integer array coins representing different coin denominations and an integer amount representing a total amount of money. Write a function coinChange that returns the fewest number of coins needed to make up that amount. If that amount cannot be made up by any combination of the coins, return -1. You may assume that you have an infinite number of each kind of coin. Examples PythonInput: coins = [1, 2, 5], amount = 11 Output: 3 Explanation: The minimum number of coins needed is 3 (11 = 5 + 5 + 1).
Input: coins = [4, 5], amount = 7 Output: -1 Explanation: It is not possible to make up the amount 7 with coins of denominations 4 and 5.Solution 1: Brute-Force approach The naive approach to solving the coin change problem involves a recursive strategy to explore all possible combinations of coins to find the minimum number needed to make up the target amount. It starts by recursively trying each coin denomination to reduce the target amount. For each coin, the algorithm subtracts the coin's value from the current amount and solves the reduced problem. If the amount becomes zero, it indicates that no more coins are needed, and the function returns zero. If the amount becomes negative, it signals that this path is not feasible, so the function returns infinity or a large number. The algorithm combines the results from each recursive call with the current coin to determine the minimum number of coins required. Finally, after exploring all possibilities, it returns the

EXAMPLES

Example 1	
Input	[1, 2, 5], 11
Output	3
Why	5 + 5 + 1 = 11: 3 coins.

Example 2	
Input	[2], 3
Output	-1
Why	A coin of 2 can never make 3.

Example 3	
Input	[1], 0
Output	0
Why	Zero needs no coins.

WHY IT WORKS

best[0] = 0, and for a from 1 to amount, best[a] = 1 + min(best[a - c]) over coins c <= a (infinity if none works). The answer is best[amount] or -1. Each amount is solved once.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Bottom-up DP over amounts
# Idea: best[0] = 0, and for sub_amount from 1 to amount, best[sub_amount] =
# 1 + min(best[sub_amount - coin]) over coins coin <= sub_amount (infinity if none
# works). The answer is best[amount] or -1. Each amount is
# solved once.
def coinChange(coins, amount):
```

```
    UNREACHABLE = amount + 1
    best = [0] + [UNREACHABLE] * amount
    for sub_amount in range(1, amount + 1):
        for coin in coins:
            if coin <= sub_amount and best[sub_amount - coin] + 1 < best[sub_amount]:
                best[sub_amount] = best[sub_amount - coin] + 1
    return best[amount] if best[amount] != UNREACHABLE else -1
```

COMPLEXITY | Time O(amount * coins) | Space O(amount)

231 Partition an array into two sub-arrays with equal sum. MEDIUM

Template move: subset sum to total / 2, written with a bitset: reach |= reach << x

QUESTION

Given an array of integers, determine if it can be partitioned into two subsets such that the sum of the elements in each subset is equal. A partition is considered valid if it divides the array into two subsets where both subsets have the same sum. Examples Input: [2, 4, 11, 5]
Output: True Explanation: The array can be partitioned into two subsets [2, 4, 5] and [11] with equal sum.
Input: [2, 7, 3, 1]
Output: False Explanation: The array cannot be partitioned into two subsets with equal sums.

EXAMPLES

Example 1	
Input	`[1, 5, 11, 5]`
Output	`true`
Why	`[1, 5, 5] and [11] both add up to 11.`

Example 2	
Input	`[1, 2, 3, 5]`
Output	`false`
Why	`The total is 11, which is odd.`

WHY IT WORKS

reach is an integer whose bit s is set when some subset sums to s; start with bit 0. For each number x do reach |= reach << x. The answer is whether bit total / 2 is set. A boolean array updated from high to low sums is the same idea in the table form.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Subset-sum as a bitset
# Idea: reachable_sums is an integer whose bit s is set when some
# subset sums to s; start with bit 0. For each number x do
# reachable_sums |= reachable_sums << number. The answer is whether bit total / 2
# is set. A boolean array updated from high to low sums is
# the same idea in the table form.
def canPartition(nums):
    total = sum(nums)
    if total % 2:
        return False
    reachable_sums = 1
    for number in nums:
        reachable_sums |= reachable_sums << number
    return bool(reachable_sums >> (total // 2) & 1)
```

COMPLEXITY | Time O(n * sum / 64) with big-integer shifts (O(n * sum) as a boolean table) | Space O(sum)

232 Explain how to find a target sum in an array. MEDIUM

Template move: turn +/- signs into a subset sum of (total + S) / 2, items used once (sweep downwards)

QUESTION

Determine the Number of Ways to Assign Symbols to Reach a Target Sum. You are given a list of non-negative integers, a1, a2, ..., an, and a target sum S. Your task is to assign each integer in the list a symbol + or - and calculate how many different ways there are to arrange these symbols such that the sum of the integers equals S after applying the assigned symbols. For example, if you have the list [1, 1, 1, 1, 1] and the target sum 3, one way to reach the target sum is to assign + to three 1s and - to two 1s, like so: +1 +1 +1 +1 -1. Another way could be +1 -1 +1 +1 +1. You need to return the total number of unique arrangements that meet the target sum S. Examples nums = [1, 1, 1, 1, 1] targetSum = 3 output: 5 nums = [1, 2, 1] targetSum = 2 output: 2 nums = [1, 2, 7, 1, 5, 3] targetSum = 8 output: -1 (Explanation: Since the total sum of the numbers plus the target is odd, it's not possible to reach the target sum.)The original problem asks for the number of ways to assign symbols (+ or -) to a list of numbers such that their sum equals a target sum S. This problem can be converted into a problem of finding subsets of the given numbers whose sum equals (totalSum + S) / 2. This is because if we can find subsets whose sum equals (totalSum + S) / 2, the remaining numbers will naturally form the other subset that sums to (totalSum - S) / 2, which effectively gives us the desired configuration of symbols. NoteWhy (totalSum + S) / 2?Given a list of numbers nums=[a1,a2,...,an]\te

EXAMPLES

Example 1	
Input	[1, 1, 1, 1, 1], 3
Output	5
Why	Put + or - in front of every number; five ways to reach 3 with five 1s.

Example 2	
Input	[1], 1
Output	1
Why	+1 is the only way.

Example 3	
Input	[1, 2], 5
Output	-1
Why	No sign choice gives 5.

WHY IT WORKS

Let need = (total + S) / 2 (return 0 if it is not a non-negative integer). ways[s] counts the subsets with sum s; for each number x update ways from high to low: ways[s] += ways[s - x]. The answer is ways[need]. Zeros double the count, which this loop handles naturally.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Subset-sum counting: choose P
# with sum (total + target) / 2
# Idea: Let wanted_sum = (total + target) / 2 (return 0 if it is not
# a non-negative integer). ways[subset_sum] counts the subsets with
# sum subset_sum; for each number x update ways from high to low:
# ways[subset_sum] += ways[subset_sum - number]. The answer is ways[wanted_sum]. Zeros
# double the count, which this loop handles naturally.
def findTargetSumWays(nums, target):
    total = sum(nums)
    if (total + target) % 2 or total + target < 0 or abs(target) > total:
        return 0
    wanted_sum = (total + target) // 2
```

```
    ways = [1] + [0] * wanted_sum
    for number in nums:
        for subset_sum in range(wanted_sum, number - 1, -1):
            ways[subset_sum] += ways[subset_sum - number]
    return ways[wanted_sum]
```

COMPLEXITY | Time O(n * sum) | Space O(sum)

233 Reverse Coin Change — MEDIUM

Template move: given the ways table, find the coin values: the first amount whose count is wrong must be a coin; add it and rebuild

QUESTION

In the coin change counting problem, dp[i] is the number of ways to make amount i from a set of coins (dp[0] = 1). In the reverse problem you are given the dp array and must find a set of coins that produces it, or None if no coin set does.

Function signature

def coins_from_dp(dp: list) -> list | None

EXAMPLES

Example 1	
Input	[1, 1, 2, 2, 3, 3]
Output	[1, 2]
Why	Coins 1 and 2 make 1, 1, 2, 2, 3, 3 ways for the amounts 0 to 5.

Example 2	
Input	[1, 0, 1, 0, 1]
Output	[2]
Why	Only the coin 2 can make 2 and 4 but not 1 and 3.

Example 3	
Input	[1, 2, 3]
Output	null
Why	No coin set can produce these counts.

CONSTRAINTS

- 1 <= dp.length <= 1000
- dp[0] = 1

WHY IT WORKS

Keep the ways table for the coins found so far. For each amount a from 1 up: if the target equals the current table, no coin; if the target is exactly one larger, a is a coin (the single coin a adds exactly one way at amount a), so add it and update the table; otherwise the input is impossible. dp[0] must be 1. At the end the table must equal the target.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Scan upward and add a coin
# whenever the table falls short
# Idea: Keep the ways table for the coins found so far.
# For each amount a from 1 up: if the target equals the
# current table, no coin; if the target is exactly one
# larger, amount is a coin (the single coin a adds exactly one
# way at amount amount), so add it and update the table;
# otherwise the input is impossible. dp[0] must be 1. At
# the end the table must equal the target.
def ways_table(coins, max_amount):
```

```
    dp = [1] + [0] * max_amount
    for coin in coins:
        for amount in range(coin, max_amount + 1):
            dp[amount] += dp[amount - coin]
    return dp

def coins_from_dp(dp):
    max_amount = len(dp) - 1
    if not dp or dp[0] != 1:
        return None
    current_table, coins = [1] + [0] * max_amount, []
    for amount in range(1, max_amount + 1):
        if dp[amount] == current_table[amount]:
            continue
        if dp[amount] != current_table[amount] + 1:
            return None
        coins.append(amount)
        for updated_amount in range(amount, max_amount + 1):
            current_table[updated_amount] += current_table[updated_amount - amount]
    return coins if current_table == list(dp) else None
```

COMPLEXITY | Time O(n * coins) = O(n^2) worst case | Space O(n)

GROUP 33 OF 43 | 5 QUESTIONS | SHORT MAIN CODE

Prefix sums, Tier 2: running total plus a dict of earlier totals

Pattern

Keep a running total. At each step ask the dict about an earlier total: total - k (subarray sums to k), total % k (sum divisible by k), or the smallest earlier total (best subarray ending here). Store the first index, a count, or the minimum.

234 Solve array-sum and city-community problems — MEDIUM

Template move: count subarrays summing to k with the template as written; the second task is union-find over building cells

QUESTION

Two tasks. (1) count_subarrays(nums, k): count the contiguous subarrays whose elements add up to k. (2) communities(grid): a grid has 1 for a building and 0 for empty land; buildings that touch up, down, left or right belong to the same community. Return the number of communities.

Function signature

def count_subarrays(nums: list, k: int) -> int
def communities(grid: list) -> int

EXAMPLES

Example 1	
Input	`[1, 1, 1], 2`
Output	`2`
Why	`[1, 1] appears twice (positions 0-1 and 1-2).`

Example 2	
Input	`[1, 2, 3], 3`
Output	`2`
Why	`[1, 2] and [3] both add up to 3.`

Example 3	
Input	`[[1, 1, 0], [0, 0, 1], [1, 0, 1]]`
Output	`3`
Why	`Top-left pair, the single building at the right of the middle row joined to the one below it, and the bottom-left: 3 communities.`

PART 1: Task 1: prefix sums in a dictionary

Walk the array keeping the prefix sum. The number of subarrays ending at the current index with sum K equals how many earlier prefixes were equal to prefix - K, so look that up and then record the current prefix. Seed the dictionary with {0: 1} for subarrays that start at index 0. Works with negative numbers and online.

PART 2: Task 2: Union-Find over building cells

Make each building its own set and union it with the building to its right and below. The number of communities is the number of distinct roots. Union-Find also supports adding buildings later.

SHORT OPTIMAL CODE

```
# Task 1: prefix sums in a dictionary
# Idea: Walk the array keeping the prefix sum. The number
# of subarrays ending at the current index with sum K
# equals how many earlier prefixes were equal to prefix -
# K, so look that up and then record the current prefix.
```

```python
# Seed the dictionary with {0: 1} for subarrays that start
# at index 0. Works with negative numbers and online.
def subarraySum(nums, target_sum):
    prefix_counts, prefix, count = {0: 1}, 0, 0
    for number in nums:
        prefix += number
        count += prefix_counts.get(prefix - target_sum, 0)
        prefix_counts[prefix] = prefix_counts.get(prefix, 0) + 1
    return count


# Task 2: Union-Find over building cells
# Idea: Make each building its own set and union it with
# the building to its right and below. The number of
# communities is the number of distinct roots. Union-Find
# also supports adding buildings later.
def communities(grid):
    cells = [(row, col) for row in range(len(grid)) for col in range(len(grid[0])) if grid[row][col] == "1"]
    parent = {cell: cell for cell in cells}

    def find(cell):
        while parent[cell] != cell:
            parent[cell] = parent[parent[cell]]
            cell = parent[cell]
        return cell

    for row, col in cells:
        for neighbour in ((row + 1, col), (row, col + 1)):
            if neighbour in parent:
                parent[find((row, col))] = find(neighbour)
    return len({find(cell) for cell in cells})
```

COMPLEXITY | Part 1: Time O(n) | Space O(n)
Part 2: Time O(m * n * alpha(m * n)) | Space O(m * n)

235 Maximum-Sum Subarray With Equal Endpoints

MEDIUM

Template move: per value keep the earliest start with the smallest total before it; candidate = total now - that earlier total

QUESTION

Given an integer array nums , choose a (contiguous) subarray whose first and last elements are equal and whose sum is as large as possible. A single element is a valid subarray (its first and last elements coincide). Return the chosen subarray itself (as a list). If several subarrays tie for the maximum sum, return the one with the smallest starting index; if they still tie, return the shortest.

Function signature

def max_sum_equal_endpoints(nums: list[int]) -> list[int]:
    ...

Follow-up (as reported): Achieve O(n): for each value, you only ever care about pairing the earliest occurrence (with the best prefix sum) against later occurrences.

EXAMPLES

Example 1	
Input	`nums = [1, 3, -2, 4, 1]`
Output	`[1, 3, -2, 4, 1]`
Why	`The value 1 appears at indices 0 and 4; that whole window sums to 7, beating any single element.`

QUESTION

Example 2	
Input	nums = [5, -10, 5]
Output	[5]
Why	The window from the first 5 to the last 5 sums to 0, which is worse than taking a single 5 (sum 5).

Example 3	
Input	nums = [-3, -1, -2]
Output	[-1]
Why	All values are distinct, so only single elements qualify; -1 is the largest.

CONSTRAINTS

- 1 <= len(nums) <= 10^5 .
- -10^4 <= nums[i] <= 10^4 .

WHY IT WORKS

For each end the best start is the same value with the smallest prefix sum so far.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Best earlier prefix per value
# Idea: For each end the best start is the same value with
# the smallest prefix sum so far.
def max_sum_equal_endpoints(nums):
    prefix = [0]
    for value in nums:
        prefix.append(prefix[-1] + value)
    best_start, best_key = {}, None
    for end_index, value in enumerate(nums):
        if value not in best_start or prefix[end_index] < prefix[best_start[value]]:
            best_start[value] = end_index
        start_index = best_start[value]
        candidate_key = (-(prefix[end_index + 1] - prefix[start_index]), start_index, end_index - start_index)
        if best_key is None or candidate_key < best_key:
            best_key = candidate_key
    return nums[best_key[1]:best_key[1] + best_key[2] + 1] if best_key else []
```

COMPLEXITY | Time O(n) | Space O(n)

236 Maximum Subarray Sum With a Minimum Length Constraint — MEDIUM

Template move: the smallest earlier total is only allowed from min_len positions back, so update it one step late

QUESTION

This is the classic "maximum subarray" problem with the extra constraint the report mentioned ("Max sub array question with a weird constraint"). Given an integer array nums and an integer min_len , find the contiguous subarray (containing at least min_len elements) that has the largest sum, and return that sum.

Function signature

def max_subarray_min_len(nums: list[int], min_len: int) -> int:

"""Return the maximum sum over all contiguous subarrays of length >= min_len."""

...

Follow-up (as reported): The report said each question had a follow-up where a constraint was changed. Now bound the subarray length to the range [min_len, max_len] . Hint: maintain a sliding window of prefix-sum minima with a monotonic deque.

QUESTION

EXAMPLES

Example 1

Input	`nums = [1, -2, 3, 4, -1], min_len = 2`
Output	`7`
Why	`The subarray [3, 4] (length 2) has sum 7, the largest among all subarrays of length >= 2.`

Example 2

Input	`nums = [-1, -2, -3], min_len = 2`
Output	`-3`
Why	`Every element is negative. The best length->=2 subarray is [-1, -2] with sum -3.`

Example 3

Input	`nums = [5, 4, -1, 7, 8], min_len = 1`
Output	`23`
Why	`With min_len = 1 this reduces to ordinary Kadane; the whole array sums to 23.`

CONSTRAINTS

- 1 <= min_len <= len(nums) <= 10^5
- -10^4 <= nums[i] <= 10^4
- The array is guaranteed to be non-empty and min_len never exceeds len(nums) , so a valid subarray always exists.
- Values may be negative; the answer can therefore be negative.

WHY IT WORKS

Sweep the end of the subarray; the best start is the smallest prefix sum that leaves at least min_len elements.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Prefix sums and a running
# minimum
# Idea: Sweep the end of the subarray; the best start is
# the smallest prefix sum that leaves at least min_len
# elements.
def max_subarray_min_len(nums, min_len):
    prefix = [0]
    for number in nums:
        prefix.append(prefix[-1] + number)
    best, smallest = None, float("inf")
    for end in range(min_len, len(nums) + 1):
        # the prefix that is allowed to start a valid subarray
        smallest = min(smallest, prefix[end - min_len])
        candidate = prefix[end] - smallest
        best = candidate if best is None else max(best, candidate)
    return best
```

COMPLEXITY | Time O(n) | Space O(n)

237 Find the maximum subarray sum. MEDIUM

Template move: the dict is just one number: the smallest earlier total; the best subarray ends here with total - smallest (Kadane is the same idea)

QUESTION

Given an array of integers nums, write a function maxSubarraySum to find the maximum sum of a contiguous subarray within the array and return that maximum sum. The subarray must be contiguous, meaning that the elements must appear consecutively in the original array. Examples Input: nums = [2, 3, -2, 4]
Output: 7 Explanation: Maximum sum is 2 + 3 + (-2) + 4 = 7.
Input: nums = [1, -1, -5, -4]
Output: 1 Explanation: The maximum sum is 1, which is the single element with the highest value.Challenge 1Can you come up with a solution with a O(n) time complexity?Challenge 2Can you come up with a solution with a O(1) space complexity?

EXAMPLES

Example 1	
Input	`[-2, 1, -3, 4, -1, 2, 1, -5, 4]`
Output	`6`
Why	`[4, -1, 2, 1] has the largest sum, 6.`

Example 2	
Input	`[1]`
Output	`1`
Why	`A single element.`

Example 3	
Input	`[5, 4, -1, 7, 8]`
Output	`23`
Why	`All numbers are positive: take them all.`

WHY IT WORKS

cur is the best sum of a subarray ending at the current element: either the element alone or the element added to the previous cur. best is the maximum cur seen. A negative running sum is never worth carrying forward.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Kadane's algorithm
# Idea: current_sum is the best_sum sum of a subarray ending at the
# current element: either the element alone or the element
# added to the previous current_sum. best_sum is the maximum current_sum seen.
# A negative running sum is never worth carrying forward.
def maxSubArray(nums):
    current_sum = best_sum = nums[0]
    for number in nums[1:]:
        current_sum = max(number, current_sum + number)
        best_sum = max(best_sum, current_sum)
    return best_sum
```

COMPLEXITY | Time O(n) | Space O(1)

238 Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k. HARD

Template move: key = total % k; store the first index; a hit at least 2 positions back means a subarray whose sum is a multiple of k

QUESTION

Determine if a given array contains a subarray of at least two elements whose sum is a multiple of a specified number k. An array is considered to have a "good subarray" if there exists at least one subarray (consisting of two or more elements) such that the sum of the elements in this subarray is a multiple of k. For example, the array [23, 2, 4, 7] with k = 6 has a "good subarray" ([2, 4]), as the sum 6 is a multiple of k = 6. The array [5, 0, 0, 0] with k = 3 also has a "good subarray" ([0, 0]), since the sum is 0, and 0 is a multiple of 3. Examples nums = [23, 2, 4, 7], k = 6 output: true nums = [5, 0, 0, 0], k = 3 output: true nums = null, k = 1 output: false In these examples, the output is true if there exists a "good subarray" in the given array nums for the specified k, and false otherwise.Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(n)?

EXAMPLES

Example 1	
Input	`[23, 2, 4, 6, 7], 6`
Output	`true`
Why	`[2, 4] adds up to 6, a multiple of 6.`

Example 2	
Input	`[23, 2, 6, 4, 7], 13`
Output	`false`
Why	`No subarray of at least two elements adds up to a multiple of 13.`

Example 3	
Input	`[23, 2, 6, 4, 7], 6`
Output	`true`
Why	`[23, 2, 6, 4, 7] adds up to 42, a multiple of 6.`

WHY IT WORKS

Walk the array keeping the running sum modulo k. If this remainder was seen before at index j and the current index i satisfies i - j >= 2, the elements between form a good subarray. Store only the first index of each remainder (a later one could only give a shorter subarray). For k = 0 compare the raw prefix sums instead.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Prefix sums mod k with the
# first index of each remainder
# Idea: Walk the array keeping the running sum modulo k.
# If this remainder was seen before at index j and the
# current index i satisfies index - j >= 2, the elements
# between form a good subarray. Store only the first index
# of each remainder (a later one could only give a shorter
# subarray). For divisor = 0 compare the raw prefix sums
# instead.
def checkSubarraySum(nums, divisor):
    if not nums or len(nums) < 2:
        return False
    first_index_of, running_sum = {0: -1}, 0
    for index, number in enumerate(nums):
        running_sum += number
        remainder = running_sum if divisor == 0 else running_sum % divisor
        if remainder in first_index_of:
            if index - first_index_of[remainder] >= 2:
```

```
                return True
            else:
                first_index_of[remainder] = index
    return False
```

COMPLEXITY | Time O(n) | Space O(min(n, k))

Group by a canonical key

Pattern

Find a key that is the same for everything that should be grouped (sorted letters, letter counts, shifts between letters, the keyboard row) and put the items into a dict under that key. Equal key means same group.

239 Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard. EASY

Template move: key = the keyboard row of each letter; a word qualifies when its letters map to one row

QUESTION

Given an array of words, return those that can be typed using letters from a single row of an American keyboard (the rows are qwertyuiop, asdfghjkl and zxcvbnm). Ignore upper and lower case and keep the input order.

Function signature

def findWords(words: list) -> list

EXAMPLES

Example 1	
Input	`['Hello', 'Alaska', 'Dad', 'Peace']`
Output	`['Alaska', 'Dad']`
Why	`'Alaska' and 'Dad' use only the middle row; 'Hello' and 'Peace' mix rows.`

Example 2	
Input	`['omk']`
Output	`[]`
Why	`'omk' mixes the top and bottom rows.`

Example 3	
Input	`['adsdf', 'sfd']`
Output	`['adsdf', 'sfd']`
Why	`Both words stay on one row.`

WHY IT WORKS

Build a dict from letter to row index (26 entries). A word qualifies if all of its lowercased letters map to the same row, i.e. the set of rows has size one. Lowercase first so 'Alaska' works.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Map each letter to its row and
# compare sets
# Idea: Build a dict from letter to row index (26
# entries). A word qualifies if all of its lowercased
# letters map to the same row, i.e. the set of rows has
# size one. Lowercase first so 'Alaska' works.
def findWords(words):
    rows = ["qwertyuiop", "asdfghjkl", "zxcvbnm"]
    row_of = {letter: row_number for row_number, row in enumerate(rows) for letter in row}
    return [word for word in words if len({row_of[letter] for letter in word.lower()}) <= 1]
```

COMPLEXITY | Time O(total characters) | Space O(1)

240 Check if all substrings are anagrams of words

MEDIUM

Template move: key = the tuple of letter counts; keep the keys of the allowed words in a set and test every substring of length 3 or more against it

QUESTION

You are given a string **s** (letters only) and access to an English dictionary **dict** (a set of valid words). Return **true** if **every contiguous substring** of **s** with length **>= 3** can be **rearranged (anagrammed)** into a valid dictionary word. Formally, for every substring **t = s[i..j]** with **|t| >= 3**, there exists some word **w in dict** such that **w** is a permutation of **t** (same multiset of characters). Otherwise return **false**. Clarifications:

- Substring means **contiguous**. - "Can be rearranged into a word" means the substring and the dictionary word have exactly the same character counts. Example:
- If **t = "act"**, it passes if the dictionary contains any of **{ "act", "cat", "tac" }**.

EXAMPLES

Example 1	
Input	'abca', ['abc', 'cab', 'bca', 'abca']
Output	true
Why	Every substring of length 3 or more of 'abca' is an anagram of a dictionary word.

Example 2	
Input	'abca', ['abc']
Output	false
Why	'bca' is an anagram of 'abc' but 'abca' has no match.

WHY IT WORKS

Precompute the set of count-vector signatures of the dictionary words. For each start extend one letter at a time, update the count vector in O(1), and look up its tuple (once the length is at least 3). Return False at the first missing signature.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Set of signatures, incremental
# count vectors
# Idea: Precompute the set of count-vector signatures of
# the dictionary words. For each start extend one letter
# at a time, update the count vector in O(1), and look up
# its tuple (once the length is at least 3). Return False
# at the first missing signature.
def all_anagram_words(text, words, alphabet="abcdefghijklmnopqrstuvwxyz"):
    letter_index = {letter: position for position, letter in enumerate(alphabet)}
    signatures = set()
    for word in words:
        counts = [0] * len(alphabet)
        for letter in word:
            counts[letter_index[letter]] += 1
        signatures.add(tuple(counts))
    for start in range(len(text)):
        counts = [0] * len(alphabet)
        for end in range(start, len(text)):
            counts[letter_index[text[end]]] += 1
            if end - start + 1 >= 3 and tuple(counts) not in signatures:
                return False
    return True
```

COMPLEXITY | Time O(n^2 * A) for alphabet size A | Space O(dictionary size)

241 Group anagrams

MEDIUM

Template move: key = the sorted letters (or a tuple of 26 letter counts)

QUESTION

Given a list of words, group together the words that are anagrams of each other (the same letters in a different order). Return the groups in any order.

Function signature

def group_by_sorting(words: list) -> list

EXAMPLES

Example 1	
Input	`['eat', 'tea', 'tan', 'ate', 'nat', 'bat']`
Output	`[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]`
Why	`eat, tea and ate use the same letters; tan and nat; bat is alone.`

Example 2	
Input	`['']`
Output	`[['']]`
Why	`An empty string is its own group.`

Example 3	
Input	`['a']`
Output	`[['a']]`
Why	`One word.`

WHY IT WORKS

Sort the letters of each word and use the resulting string as the key; words with the same key are anagrams. One pass, one dictionary.

SHORT OPTIMAL CODE

```
from collections import defaultdict


# BASE QUESTION (optimal): Dictionary keyed by the sorted
# letters
# Idea: Sort the letters of each word and use the
# resulting string as the key; words with the same key are
# anagrams. One pass, one dictionary.
def groupAnagrams(words):
    groups = defaultdict(list)
    for word in words:
        groups["".join(sorted(word))].append(word)
    return list(groups.values())
```

COMPLEXITY | Time O(n * L log L) | Space O(n * L)

242 Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same. MEDIUM

Template move: key = the differences between neighbouring letters, modulo 26 (a shifted word has the same differences)

QUESTION

Two strings are buddy strings when they have the same length and the distance between each pair of neighbouring letters is the same (the alphabet wraps around after z), so one is the other shifted by a constant. Group the strings into buddy groups and return the groups in any order.

Function signature

def group_buddies(words: list) -> list

EXAMPLES

Example 1	
Input	`['abc', 'bcd', 'acf', 'xyz', 'az', 'ba', 'a', 'z']`
Output	`[['abc', 'bcd', 'xyz'], ['acf'], ['az', 'ba'], ['a', 'z']]`
Why	`'abc', 'bcd' and 'xyz' all step +1, +1; 'az' and 'ba' both step +25 (a to z wraps); single letters 'a' and 'z' form a group.`

WHY IT WORKS

Two strings are buddies when every letter is shifted by the same amount, which means the gaps between neighbouring letters (mod 26) are identical. Use those gaps as the hash key.

SHORT OPTIMAL CODE

```
from collections import defaultdict


# BASE QUESTION (optimal): Group by the tuple of
# successive differences
# Idea: Two strings are buddies when every letter is
# shifted by the same amount, which means the gaps between
# neighbouring letters (mod 26) are identical. Use those
# gaps as the hash key.
def groupBuddyStrings(strings):
    groups = defaultdict(list)
    for word in strings:
        key = tuple((ord(word[position]) - ord(word[position - 1])) % 26 for position in range(1, len(word)))
        groups[key].append(word)
    return list(groups.values())
```

COMPLEXITY | Time O(total characters) | Space O(total characters)

243 Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second? HARD

Template move: same letters used, and the sorted list of letter frequencies must match: compare set(s) == set(t) and the sorted Counter values

QUESTION

Two strings are 'close' if you can turn one into the other with these operations, used any number of times: (1) swap the positions of any two letters; (2) take two letters that occur in the string and swap all of their occurrences (every a becomes b and every b becomes a). Return whether the first string can be transformed into the second.

Function signature

def close_strings(s: str, t: str) -> bool

EXAMPLES

QUESTION

Example 1	
Input	'abc', 'bca'
Output	true
Why	Swapping positions turns 'abc' into 'bca'.

Example 2	
Input	'a', 'aa'
Output	false
Why	The lengths differ.

Example 3	
Input	'cabbba', 'abbccc'
Output	true
Why	Use operation 1 to reorder, then operation 2 to swap the letters a and b, and c and ... until the counts match.

Example 4	
Input	'uau', 'ssx'
Output	false
Why	'u' appears in the first string but not the second, so they cannot be close.

PART 1: Reading 1 (LeetCode 1657): same letter set and same multiset of frequencies

Operation 1 reorders positions freely, so only the multiset of letters matters. Operation 2 exchanges the identities of two letters that exist, which permutes the frequencies among existing letters. Therefore s can become t exactly when both use the same set of letters (a letter cannot be created or destroyed) and their sorted frequency lists are equal.

PART 2: Reading 2 (LeetCode 1153): one-directional conversion, no swaps

Each source letter must map to exactly one target letter (a function), otherwise it is impossible. If s != t and t already uses all 26 letters, no spare letter exists to break mapping cycles, so it is impossible. Otherwise convert in an order that resolves chains, using a spare letter for any cycle.

SHORT OPTIMAL CODE

```
from collections import Counter


# BASE QUESTION (optimal): Reading 1 (LeetCode 1657): same
# letter set and same multiset of frequencies
# Idea: Operation 1 reorders positions freely, so only the
# multiset of letters matters. Operation 2 exchanges the
# identities of two letters that exist, which permutes the
# frequencies among existing letters. Therefore first can
# become second exactly when both use the same set of letters
# (a letter cannot be created or destroyed) and their
# sorted frequency lists are equal.
def close_strings(first, second):
    return set(first) == set(second) and sorted(Counter(first).values()) == sorted(Counter(second).values())
```

COMPLEXITY | Part 1: Time O(n) | Space O(1) for a fixed alphabet
Part 2: Time O(n) | Space O(1)

GROUP 35 OF 43 | 5 QUESTIONS | SHORT MAIN CODE

K-way merge: a heap of the next item from each source

Pattern

Put the first item of every list (with its list number and position) on a heap. Pop the smallest, record it, and push the next item from the same list. After k pops you have the k best without sorting everything.

244 Find k pairs with smallest sums — EASY

Template move: seed with (a[i] + b[0], i, 0); the successor of pair (i, j) is (i, j + 1)

QUESTION

You are given two **sorted** integer arrays **`arr1`** and **`arr2`** (each sorted in non-decreasing order), and an integer **`k`**. Consider all possible pairs formed by taking one element from **`arr1`** and one element from **`arr2`**. For indices **`i`** and **`j`**, the pair is **`(arr1[i], arr2[j])`** with sum **`arr1[i] + arr2[j]`**. Your task is to return the **k pairs `(arr1[i], arr2[j])`** with the smallest sums. If there are fewer than **k** total possible pairs, return all possible pairs. You may return the pairs in any order. ---

Function Signature

(one possible form)

```
kSmallestPairs(arr1: List[int], arr2: List[int], k: int) -> List[Tuple[int, int]]
```

Example

```
arr1 = [1, 7, 11] arr2 = [2, 4, 6] k = 3
```

All possible pairs and their sums are:

- (1, 2) -> 3 - (1, 4) -> 5 - (1, 6) -> 7 - (7, 2) -> 9 - (7, 4) -> 11 - (7, 6) -> 13 - (11, 2) -> 13 - (11, 4) -> 15 - (11, 6) -> 17 The 3 pairs with the smallest sums are:

```
[(1, 2), (1, 4), (1, 6)]
```

(The order among these returned pairs does not matter.) Explain an efficient algorithm to solve this problem and analyze its time and space complexity. You may assume the arrays are large, so a solution that explicitly generates and sorts all **`len(arr1) * len(arr2)`** pairs is too slow.

EXAMPLES

Example 1	
Input	`[1, 7, 11], [2, 4, 6], 3`
Output	`[(1, 2), (1, 4), (1, 6)]`
Why	`Pairs (one from each array) in order of their sum.`

Example 2	
Input	`[1, 1, 2], [1, 2, 3], 2`
Output	`[(1, 1), (1, 1)]`
Why	`Equal numbers give equal pairs.`

WHY IT WORKS

Push (a[i] + b[0], i, 0) for the first min(k, m) values of a. Pop the smallest, record the pair, and push the next pair in the same row, (a[i] + b[j + 1], i, j + 1). After k pops you have the k smallest sums. Nothing larger than needed is ever generated.

SHORT OPTIMAL CODE

```python
import heapq


# BASE QUESTION (optimal): Min-heap over the sorted rows
# Idea: Push (first[first_index] + second[0], first_index, 0) for the first min(pair_count, m)
# values of a. Pop the smallest, record the pair, and push
# the next pair in the same row, (first[first_index] + second[second_index + 1], first_index, second_index +
# 1). After k pops you have the k smallest sums. Nothing
# larger than needed is ever generated.
def kSmallestPairs(first, second, pair_count):
    if not first or not second or pair_count <= 0:
        return []
    heap = [(first[first_index] + second[0], first_index, 0) for first_index in range(min(pair_count, len(first
))))]
    heapq.heapify(heap)
    pairs = []
    while heap and len(pairs) < pair_count:
        _, first_index, second_index = heapq.heappop(heap)
        pairs.append((first[first_index], second[second_index]))
        if second_index + 1 < len(second):
            heapq.heappush(heap, (first[first_index] + second[second_index + 1], first_index, second_index + 1)
)
    return pairs
```

COMPLEXITY | Time O(k log min(k, m)) | Space O(min(k, m))

245 K Largest Elements Across Sorted Lists — MEDIUM

Template move: seed with the LAST item of each list (negated); the successor is the item before it; reverse the result

QUESTION

You are given a list of inner lists, where each inner list is sorted in ascending order. Return the k largest elements taken from the union of all the inner lists, and return them sorted in ascending order. Duplicate values are kept (an element counts once per position it occupies). The interviewer first accepted a simple solution, then asked for an optimized one.

Function signature

```
from typing import List
def k_largest_sorted(lists: List[List[int]], k: int) -> List[int]:
    """Return the k largest elements across all (individually ascending) inner
    lists, in ascending sorted order."""
    ...
```

Follow-up (as reported): Optimize beyond concatenate-and-sort. Hint: because each list is already sorted, seed a max-heap with the last element of every non-empty list and pop k times, pushing the predecessor within a list after each pop; collect the popped values and reverse them.

EXAMPLES

Example 1	
Input	`lists = [[1, 4, 7], [2, 5, 8], [3, 6, 9]], k = 4`
Output	`[6, 7, 8, 9]`
Why	The four largest values overall are 9, 8, 7, 6, returned ascending.

Example 2	
Input	`lists = [[10, 20, 30]], k = 2`
Output	`[20, 30]`

QUESTION

Example 3	
Input	`lists = [[5], [1, 3]], k = 5`
Output	`[1, 3, 5]`
Why	`k exceeds the total count, so all elements are returned sorted.`

Example 4	
Input	`lists = [[1, 2], [3, 4]], k = 0`
Output	`[]`

CONSTRAINTS

- Let m be the number of inner lists and T the total number of elements across all of them.
- 0 <= m <= 10^4 , 0 <= T <= 10^6 .
- Each element is an integer in [-10^9, 10^9] .
- 0 <= k . If k == 0 return [] . If k >= T , return every element sorted ascending.
- Some inner lists may be empty.

WHY IT WORKS

The next largest element is always at the end of one of the lists.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): A max-heap over the ends of the
# lists
# Idea: The next largest element is always at the end of
# one of the lists.
import heapq


def k_largest_sorted(lists, how_many):
    heap = [(-numbers[-1], list_index, len(numbers) - 1) for list_index, numbers in enumerate(lists) if numbers
]
    heapq.heapify(heap)
    largest = []
    while heap and len(largest) < how_many:
        negated, list_index, position = heapq.heappop(heap)
        largest.append(-negated)
        if position:
            heapq.heappush(heap, (-lists[list_index][position - 1], list_index, position - 1))
    return largest[::-1]
```

COMPLEXITY | Time O(m + k log m) | Space O(m)

246 Sort a nearly sorted (or K sorted) array. MEDIUM

Template move: keep a heap of the first k + 1 items; pop the smallest and push the next input item (heapreplace)

QUESTION

In a k-sorted array every element is at most k positions away from where it would be in the sorted array. Sort such an array in O(n log k) time (better than a full sort when k is small).

Function signature

def sort_k_sorted(arr: list, k: int) -> list

EXAMPLES

QUESTION

Example 1	
Input	`[6, 5, 3, 2, 8, 10, 9], 3`
Output	`[2, 3, 5, 6, 8, 9, 10]`
Why	`Every number is within 3 positions of its place in the sorted order.`

Example 2	
Input	`[2, 1, 4, 3], 1`
Output	`[1, 2, 3, 4]`
Why	`k = 1: neighbours may be swapped.`

WHY IT WORKS

Push the first K + 1 elements. Then for each remaining element, pop the smallest into the output and push the new element. At the end pop the heap empty. The smallest of any K + 1 consecutive pending elements is the next element of the sorted order.

SHORT OPTIMAL CODE

```python
import heapq


# BASE QUESTION (optimal): Min-heap of size K + 1
# Idea: Push the first K + 1 elements. Then for each
# remaining element, pop the smallest into the output and
# push the new element. At the end pop the heap empty. The
# smallest of any K + 1 consecutive pending elements is
# the next element of the sorted order.
def sort_nearly(values, max_distance):
    heap = list(values[:max_distance + 1])
    heapq.heapify(heap)
    sorted_values = []
    for value in values[max_distance + 1:]:
        sorted_values.append(heapq.heapreplace(heap, value))
    while heap:
        sorted_values.append(heapq.heappop(heap))
    return sorted_values
```

COMPLEXITY | Time O(n log K) | Space O(K)

247 Return top-k subset sums in descending order — HARD

Template move: heap of (sum, index); each popped state pushes two successors: add the next item, or swap the last item for the next

QUESTION

Given a list of integers (possibly negative), return the k largest subset sums in descending order. The empty subset counts and has sum 0; different subsets with the same sum are listed separately.

Function signature

def top_k_sums(nums: list, k: int) -> list

EXAMPLES

Example 1	
Input	`[1, 2, 3], 4`
Output	`[6, 5, 4, 3]`
Why	`The eight subset sums are 6, 5, 4, 3, 3, 2, 1, 0; the top four are 6, 5, 4, 3.`

QUESTION

Example 2	
Input	`[3, -2, 1], 3`
Output	`[4, 3, 2]`
Why	`With a negative number: 3 + 1 = 4, then 3, then 3 + (-2) + 1 = 2.`

WHY IT WORKS

Every subset sum equals (sum of positives) minus a subset sum of the absolute values, so the k largest sums are the sum of positives minus the k smallest subset sums of the sorted absolute values. Generate those with a heap of (sum, last index): popping a state gives the next smallest sum and pushes at most two successors, 'add the next element' and 'replace the last element with the next'. Each subset is produced once, in nondecreasing order.

SHORT OPTIMAL CODE

```
import heapq


# BASE QUESTION (optimal): Heap of the k smallest sums of
# absolute values
# Idea: Every subset sum equals (sum of positives) minus a
# subset sum of the absolute values, so the k largest sums
# are the sum of positives minus the k smallest subset
# sums of the sorted absolute values. Generate those with
# a heap of (sum, last index): popping a state gives the
# next smallest sum and pushes at most two successors,
# 'add the next element' and 'replace the last element
# with the next'. Each subset is produced once, in
# nondecreasing order.
def top_k_sums(nums, how_many):
    base = sum(number for number in nums if number > 0)
    absolute_values = sorted(abs(number) for number in nums)
    sums = [0]
    heap = [(absolute_values[0], 0)] if absolute_values else []
    while heap and len(sums) < how_many:
        subset_sum, last_index = heapq.heappop(heap)
        sums.append(subset_sum)
        if last_index + 1 < len(absolute_values):
            heapq.heappush(heap, (subset_sum + absolute_values[last_index + 1], last_index + 1))
            heapq.heappush(heap, (subset_sum - absolute_values[last_index] + absolute_values[last_index + 1], l
ast_index + 1))
    return [base - number for number in sums[:how_many]]
```

COMPLEXITY | Time O(n log n + k log k) | Space O(n + k)

248 Merge k sorted linked lists. HARD

Template move: seed with each list head (value, list number, node); the successor of a node is node.next

QUESTION

You are given k linked lists, each sorted in ascending order. Merge them into one sorted linked list and return its head. (In the examples the linked lists are written as arrays.)

Function signature

def merge_k_heap(lists: list) -> ListNode

EXAMPLES

QUESTION

Example 1	
Input	`lists = [[1, 4, 5], [1, 3, 4], [2, 6]]`
Output	`_to_list(merge_k_heap([mk([1, 4, 5]), mk([1, 3, 4]), mk([2, 6])])) -> [1, 1, 2, 3, 4, 4, 5, 6]`
Why	The three sorted lists merge into one sorted list.

Example 2	
Input	`lists = []`
Output	`_to_list(merge_k_heap([])) -> []`
Why	No lists: an empty result.

WHY IT WORKS

Push the head of each non-empty list as (value, list index, node). Pop the smallest, append it to the output, and push its successor. The list index is a tie-breaker so nodes are never compared. The heap never holds more than k items.

SHORT OPTIMAL CODE

```
import heapq


# BASE QUESTION (optimal): Min-heap of the current heads
# Idea: Push the head of each non-empty list as (value,
# list index, node). Pop the smallest, append it to the
# output, and push its successor. The list index is a tie-
# breaker so nodes are never compared. The heap never
# holds more than k items.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

def mergeKLists(lists):
    heap = [(first_node.val, list_index, first_node) for list_index, first_node in enumerate(lists) if first_no
de]
    heapq.heapify(heap)
    dummy = tail = ListNode()
    while heap:
        _, list_index, node = heapq.heappop(heap)
        tail.next = node
        tail = node
        if node.next:
            heapq.heappush(heap, (node.next.val, list_index, node.next))
    return dummy.next
```

COMPLEXITY | Time O(N log k) | Space O(k)

GROUP 36 OF 43 | 5 QUESTIONS | SHORT MAIN CODE

Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

Pattern

Same recursive walk as Tier 1, but now the information also travels down: lower and upper bounds for a binary search tree, or the target nodes you are looking for. The BST rule (left < node < right) lets you go left or right instead of visiting everything.

249 Next Birthday (BST In-Order Successor with Wraparound) EASY

Template move: descend(): go left after a hit (a closer candidate may exist), right otherwise; if no candidate exists wrap around to the leftmost node

QUESTION

This is the "alarm / next-birthday" problem: from a set of birthdays stored in a binary search tree, find the next one that will occur. Each birthday is stored as an integer day-of-year in [1, 365] , and the tree is a valid BST keyed on that value. Given the root of the BST and an integer today , return the next upcoming birthday: the smallest key that is strictly greater than today . If no key is strictly greater than today , the calendar wraps around, so return the smallest key in the whole tree. If the tree is empty, return None .

Function signature

```
from typing import Optional
class TreeNode:
    def __init__(self, val: int, left: "Optional[TreeNode]" = None,
                 right: "Optional[TreeNode]" = None) -> None:
        self.val = val
        self.left = left
        self.right = right
def next_birthday(root: Optional[TreeNode], today: int) -> Optional[int]:
    """Return the smallest key strictly greater than today, wrapping around
    to the smallest key if none is greater. Return None if the tree is empty."""
    ...
```

Follow-up (as reported): If the input were an unsorted binary tree (not a BST), extract all values, sort them, and binary-search for the successor. Hint: bisect_right .

EXAMPLES

Example 1	
Input	`root = above, today = 25`
Output	`30`
Why	`Smallest key strictly greater than 25 is 30.`

Example 2	
Input	`root = above, today = 40`
Output	`10`
Why	`No key > 40, so wrap around to the smallest key, 10.`

Example 3	
Input	`root = above, today = 5`
Output	`10`
Why	`Smallest key greater than 5 is 10.`

QUESTION

Example 4	
Input	`root = None, today = 100`
Output	`None`

CONSTRAINTS

- 0 <= number of nodes <= 10^5 ; all node values are distinct and in [1, 365] .
- 1 <= today <= 365 .
- Aim for O(h) time where h is the tree height (do not sort all nodes if you can avoid it).

WHY IT WORKS

A node above today is a candidate; its left subtree may hold a closer one.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Descend the BST once
# Idea: A node above today is a candidate; its left
# subtree may hold a closer one.
def next_birthday(root, today):
    if root is None:
        return None
    best, node = None, root
    while node:
        if node.val > today:
            best, node = node.val, node.left
        else:
            node = node.right
    if best is not None:
        return best
    node = root
    while node.left:
        node = node.left
    return node.val
```

COMPLEXITY | Time O(h) | Space O(1)

250 Find mode in a trinary search tree

MEDIUM

Template move: in-order walk (left, middle, right); equal values are neighbours, so one running count finds the mode(s)

QUESTION

You are given the root of a **trinary search tree** where each node has up to three children:

```
class Node { int val; Node left; // all values < val Node middle; // all values = val Node right; // all v
alues > val }
```

Assume the tree is intended to satisfy the ordering rules above.

Task

Return the **mode(s)** of the tree: the value(s) that occur most frequently among all nodes.

- If there is one mode, return a single-element list.
- If multiple values tie for highest frequency, return **all** such values (in any order).

Constraints (typical)

- Number of nodes **n** can be large (e.g., up to **`10^5`**).
- Values can be any 32-bit signed integers.

Follow-up

QUESTION

How would your approach change if the tree is **broken** (i.e., it may not respect the **<**, **=**, **>** constraints), but you still must return the correct mode(s)?

Part 1: Find Mode(s) in a Valid Trinary Search TreeYou are given a valid trinary search tree. Each node has an integer value and up to three children: left, middle, and right. For every node, all values in its left subtree are strictly less than the node's value, all values in its middle subtree are equal to the node's value, and all values in its right subtree are strictly greater than the node's value. The tree is encoded using parallel arrays instead of Node objects. Return the value or values that occur most frequently in the tree. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.
Constraints
0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a valid tree with no cyclesThe tree satisfies the trinary search ordering rules recursivelyEach value is a 32-bit signed integer

EXAMPLES

```
Input: ([], [], [], [], -1)
Expected Output: []
```

Explanation: The tree is empty, so there are no modes.

```
Input: ([42], [-1], [-1], [-1], 0)
Expected Output: [42]
```

Explanation: A single-node tree has that node's value as the only mode.

Hints
In a valid trinary search tree, an inorder traversal using left, node, middle, right visits values in nondecreasing order.Because equal values appear consecutively during that traversal, you can track only the current run length and the best run length so far.
Part 2: Find Mode(s) in a Broken Trinary TreeYou are given a trinary tree with integer values. Each node has up to three children: left, middle, and right. Unlike a valid trinary search tree, this tree may be broken: the left, middle, and right children do not necessarily follow any ordering rules. Return the value or values that occur most frequently among all reachable nodes. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.
Constraints
0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a tree with no cyclesNo ordering rule is guaranteed between a node and its childrenEach value is a 32-bit signed integer
Examples
Input: ([], [], [], [], -1)
Expected Output: []
Explanation: The tree is empty, so there are no modes.
Input: ([-2147483648], [-1], [-1], [-1], 0)
Expected Output: [-2147483648]
Explanation: A single-node tree has that node's value as the only mode.
Hints
Without the ordering property, equal values are not guaranteed to be adjacent in any traversal.Use a frequency table while traversing the entire tree.

WHY IT WORKS

Left, node, middle copies and right produces sorted values, so equal values form one run and no frequency dictionary is needed.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Recursive in-order traversal
# Idea: Left, node, middle copies, right produces sorted
# values, so count equal-value runs without a dictionary.
def find_modes(root):
    best = run = 0
    current = None
    modes = []

    def add(value):
        nonlocal best, run, current, modes
        run = run + 1 if value == current else 1
        current = value
        if run > best:
            best, modes = run, [value]
        elif run == best:
            modes.append(value)

    def inorder(node):
        if node is None:
            return
        inorder(node.left)
        add(node.val)
        middle = node.middle
        while middle:
            add(middle.val)
            middle = middle.middle
        inorder(node.right)

    inorder(root)
    return modes
```

COMPLEXITY | Time O(n) | Space O(h) recursion

251 Next Upcoming Birthday in a Family Tree

MEDIUM

Template move: walk the ancestors with a stack and a seen set, keep the best birthday by the circular distance from today

QUESTION

You are given a family tree rooted at "me". Each Person node has a name , a birthday given as (month, day) , and pointers to parent1 and parent2 (either may be None). Following parent pointers reaches the whole ancestry; treat the reachable set of Person nodes as the tree to search (each distinct person counted once, even if reachable by two paths). Given today's date (today_month, today_day) , implement alarm_next_birthday(me, today_month, today_day) that scans every person and returns the (month, day) of the soonest upcoming birthday, where a birthday landing exactly on today counts as upcoming (distance 0) and the calendar wraps around the year. If the tree is empty (me is None), return the empty string "" . Use this ordering key to compare dates: key(m, d) = (m - 1) * 31 + d , and cyclic distance (key(person) - key(today)) mod 372 . Return the person with the smallest such distance; if several people share that soonest date, return that (month, day) .

QUESTION

Function signature

```
from typing import Optional, Union
class Person:
    def __init__(self, name: str, month: int, day: int,
                 parent1: "Optional[Person]" = None,
                 parent2: "Optional[Person]" = None) -> None:
        self.name = name
        self.month = month
        self.day = day
        self.parent1 = parent1
        self.parent2 = parent2
def alarm_next_birthday(me: Optional[Person],
                        today_month: int,
                        today_day: int) -> Union[tuple[int, int], str]:
    """Return (month, day) of the nearest upcoming birthday, or "" if there is
nobody."""
    ...
```

Follow-up (as reported): Support repeated alarms: after reporting one birthday, the next call returns the following-soonest, and once all have been reported, return "" .

EXAMPLES

Example 1	
Input	`me = Me(12,1) with parent1 = Alice(9,25), parent2 = Bob(1,10); today = (9, 12)`
Output	`(9, 25)`
Why	`keys — today=260, Alice=273 (dist 13), Me=342 (dist 82), Bob=10 (dist (10- 260) mod 372 = 122). Alice is soonest.`

Example 2	
Input	`me = None; today = (5, 5)`
Output	`""`
Why	`No people to check.`

Example 3	
Input	`me = Me(3,3) alone; today = (3, 3)`
Output	`(3, 3)`
Why	`The only birthday is today, distance 0.`

CONSTRAINTS

- 1 <= month <= 12 , 1 <= day <= 31 .
- Up to 10^5 people; parent pointers form a DAG (no cycles).

WHY IT WORKS

DFS through both parents, visit each person once and keep the birthday with the smallest wrapped distance from today.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Recursive family-tree walk
# Idea: DFS through both parents, visit each person once,
# and keep the birthday with the smallest wrapped distance.
def next_birthday(me, today_month, today_day):
    today = (today_month - 1) * 31 + today_day
```

```
    seen = set()
    best = None

    def dfs(person):
        nonlocal best
        if person is None or person in seen:
            return
        seen.add(person)
        day = (person.month - 1) * 31 + person.day
        candidate = ((day - today) % 372, person.month, person.day)
        if best is None or candidate < best:
            best = candidate
        dfs(person.parent1)
        dfs(person.parent2)

    dfs(me)
    return "" if best is None else (best[1], best[2])
```

COMPLEXITY | Time O(n) | Space O(n) for seen and recursion

252 Lowest Common Ancestor — MEDIUM

Template move: dfs returns the node if it is p or q; a node that gets an answer from both sides is the ancestor

QUESTION

Given the root of a binary tree and two of its nodes p and q, find their lowest common ancestor: the lowest node that has both p and q as descendants (a node can be a descendant of itself).

Function signature

def lowestCommonAncestor_tree(root: BNode, p: BNode, q: BNode) -> BNode

EXAMPLES

Example 1	

Input	`root = [3, [5, [6], [2]], [1]] (value, left, right)`
	`first query: p = 5, q = 1`
	`second query: p = 5, q = 6`

Output	`lowestCommonAncestor_tree(root, p, q).val -> 3` `lowestCommonAncestor_tree(root, p, p.left).val -> 5`
Why	`5 and 1 are in different subtrees of 3, so 3 is their lowest common ancestor. 5 is an ancestor of 6, and a node may be its own ancestor, so the answer is 5.`

CONSTRAINTS

- Number of nodes in tree: [2, 10^5]
- -10^9 <= Node.val <= 10^9
- All node values are unique
- p != q
- p and q exist in the tree

WHY IT WORKS

dfs(node) returns node if node is p or q, otherwise combines its children: if both children return something, node is the LCA; if only one does, pass that up. The first node that sees both p and q below it is returned all the way to the top.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): One recursive pass
# Idea: dfs(node) returns node if node is p or q,
# otherwise combines its children: if both children return
# something, node is the LCA; if only one does, pass that
```

```
# up. The first node that sees both p and q below it is
# returned all the way to the top.
class BNode:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def lowestCommonAncestor_tree(root, first_target, second_target):
    def dfs(node):
        if node is None or node is first_target or node is second_target:
            return node
        left, right = dfs(node.left), dfs(node.right)
        return node if left and right else left or right

    return dfs(root)
```

COMPLEXITY | Time O(n) | Space O(h)

253 Determine if a given binary tree is a binary search tree (BST). MEDIUM

Template move: check(): pass lo and hi bounds down; fail when a value leaves its allowed window

QUESTION

Given the root of a binary tree, determine whether it is a valid binary search tree: for every node, all values in its left subtree are smaller and all values in its right subtree are larger (strictly).

Function signature

def isValidBST(root: TreeNode) -> bool

EXAMPLES

Example 1	
Input	[2, 1, 3]
Output	true
Why	2 with 1 on the left and 3 on the right.

Example 2	
Input	[5, 1, 4, null, null, 3, 6]
Output	false
Why	The 3 sits in the right subtree of 5 but is smaller than 5.

Example 3	
Input	[2, 2, 2]
Output	false
Why	A node equal to its parent breaks the strict rule.

WHY IT WORKS

Every node must be strictly between the bounds set by its ancestors; each child tightens one bound.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Pass valid bounds down recursively
# Idea: Every node must be strictly between the bounds set
# by its ancestors; each child tightens one bound.
def isValidBST(root):
    def valid(node, low, high):
        if node is None:
            return True
        if not low < node.val < high:
```

```
            return False
        return (valid(node.left, low, node.val)
                and valid(node.right, node.val, high))

    return valid(root, float("-inf"), float("inf"))
```

COMPLEXITY | Time O(n) | Space O(h) recursion

GROUP 37 OF 43 | 5 QUESTIONS | SHORT MAIN CODE

Random pick without repeats: swap the chosen item with the last one and pop

Pattern

To pick a random item and never pick it again in O(1), keep the items in an array. Choose a random index, swap that item with the last one, pop the last, and fix its position in the index dict. The same trick gives O(1) delete and O(1) random access.

254 Design set with O(1) random access

MEDIUM

Template move: add appends and records the position; delete swaps with the last item and pops; get_random picks a random index

QUESTION

Design a data structure ("FancySet") that stores unique integers and supports the following operations, each in average O(1) time:

- **`add(x)`**: Insert value **`x`** if it is not already present. Return whether the insert happened.
- **`delete(x)`**: Remove value **`x`** if it exists. Return whether a deletion happened.
- **`getRandom()`**: Return a uniformly random element currently in the set. Requirements / follow-ups:
- All operations should be average O(1).
- Handle corner cases such as calling **`getRandom()`** on an empty set (define and justify behavior).
- Discuss how you would make this data structure thread-safe. Compare using a single lock for the whole set vs finer-grained locking (e.g., element-level locks), and explain trade-offs.

Design a data structure called FancySet that stores unique integers and supports these operations in average O(1) time: add(x), delete(x), and getRandom(). For this coding problem, implement a function **`solution(operations, values, seed)`** that simulates a newly created empty FancySet and returns the result of every operation. Use the standard dynamic-array + hash-map behavior:

- Store current elements in an array-like list.
- Store each element's current index in a hash map.
- When deleting an element that is not at the end of the list, move the last element into the removed slot, then pop the last slot. To make test cases deterministic, **`getRandom()`** does not use the language's built-in RNG. Instead, keep an internal 32-bit state initialized to **`seed`**. When **`getRandom()`** is called on a non-empty set, update the state with: **`state = (1664525 * state + 1013904223) mod 2^32`** and return the element at index **`state % current_size`** from the current internal list. If **`getRandom()`** is called on an empty set, return **`None`** and do not advance the state. In a real interview, picking a random index from the array gives uniform random access; the fixed generator above is only for reproducible grading. Follow-up discussion (not graded by the code runner): explain how you would make FancySet thread-safe, and compare using one lock for the whole structure versus finer-grained locking.

Constraints

1 <= len(operations) <= 2 * 10^5len(values) == len(operations)Each operation is one of 'add', 'delete', or 'getRandom'For 'add' and 'delete', values[i] contains exactly one integer; for 'getRandom', values[i] is []-10^9 <= x <= 10^9All operations should run in O(1) average time

EXAMPLES

```
Input: (['add','add','getRandom','delete','getRandom','add','getRandom'], [[1],[2],[],[1],[],[2],[]], 7)
Expected Output: [True, True, 1, True, 2, False, 2]
```

Explanation: After inserting 1 and 2, the first deterministic random choice picks index 0, so it returns 1. Deleting 1 leaves only 2. Adding 2 again fails because it is already present.

QUESTION

```
Input: (['getRandom','add','add','delete','delete','getRandom'], [[],[5],[5],[7],[5],[]], 1)
Expected Output: [None, True, False, False, True, None]
```

Explanation: Calling getRandom on an empty set returns None. Adding 5 twice inserts it only once. Deleting 7 fails because it is not present. After deleting 5, the set becomes empty again.

Hints

You need both fast membership/index lookup and fast access by numeric position. What two data structures combine to give that?Deleting from the middle of an array is expensive unless you replace the removed element with the last element and update that last element's stored index.

WHY IT WORKS

add appends to the array and records the index. delete finds the index from the dict, moves the last element into that slot, updates its index in the dict, pops the array and removes the key. getRandom picks a uniform index with random.randrange; every element is equally likely.

SHORT OPTIMAL CODE

```python
import random


# BASE QUESTION (optimal): Array of values plus a dict
# from value to index
# Idea: add appends to the array and records the index.
# delete finds the index from the dict, moves the last
# element into that slot, updates its index in the dict,
# pops the array and removes the key. getRandom picks a
# uniform index with random.randrange; every element is
# equally likely.
class FancySet:
    def __init__(self):
        self.values, self.index_of = [], {}

    def add(self, value):
        if value in self.index_of:
            return False
        self.index_of[value] = len(self.values)
        self.values.append(value)
        return True

    def delete(self, value):
        if value not in self.index_of:
            return False
        slot, last_value = self.index_of[value], self.values[-1]
        self.values[slot] = last_value
        self.index_of[last_value] = slot
        self.values.pop()
        del self.index_of[value]
        return True

    def getRandom(self):
        if not self.values:
            raise IndexError("empty set")
        return self.values[random.randrange(len(self.values))]
```

COMPLEXITY | Time O(1) average per operation | Space O(n)

255 Random Generator Without Repeats

MEDIUM

Template move: the template as written: values 0..n-1, every call returns a new one; raise when empty

QUESTION

Design a class RandomGenerator initialized with a positive integer n . Each call to generate() returns one integer chosen uniformly at random from the values in [0, n-1] that have not yet been returned. After n calls every value in [0, n-1] has been returned exactly once; a further call must raise IndexError (the pool is empty). Each generate() call must run in O(1) time. The intended approach maintains an array of the remaining values and, on each call, swaps a randomly chosen remaining value to the end of the active region and shrinks that region (partial Fisher-Yates).

Function signature

```
import random
class RandomGenerator:
    def __init__(self, n: int) -> None:
        """Prepare to yield a random permutation of [0, n-1], one value per
generate()."""
        ...
    def generate(self) -> int:
        """Return a uniformly random not-yet-returned value in O(1).
        Raise IndexError if all values have already been generated."""
        ...
```

Follow-up (as reported): Add a reset() that makes all n values available again in O(1) amortized.

EXAMPLES

Example 1	
Input	`g = RandomGenerator(3); [g.generate() for _ in range(3)]`
Output	`e.g. [2, 0, 1]`
Why	`The three calls return a permutation of {0, 1, 2} with no repeats; any of the 6 permutations is a valid output. A 4th call would raise IndexError.`

Example 2	
Input	`g = RandomGenerator(1); g.generate()`
Output	`0`
Why	`Only one value exists.`

CONSTRAINTS

- 1 <= n <= 10**6 .

WHY IT WORKS

Swap the drawn value to the end of the active region and shrink it.

SHORT OPTIMAL CODE

```python
import random


# BASE QUESTION (optimal): Partial Fisher-Yates with an
# active region
# Idea: Swap the drawn value to the end of the active
# region and shrink it.
class RandomGenerator:
    def __init__(self, count, seed=None):
        self.count = count
        self.values = list(range(count))
        self.size = count
```

```
        self.rng = random.Random(seed)

    def generate(self):
        if self.size == 0:
            raise IndexError("all values have been generated")
        pick = self.rng.randrange(self.size)
        self.size -= 1
        self.values[pick], self.values[self.size] = self.values[self.size], self.values[pick]
        return self.values[self.size]
```

COMPLEXITY | Time O(1) per call | Space O(n)

256 Random Song Shuffler With a No-Repeat Window — MEDIUM

Template move: the same pool, plus a deque of the last k songs; the played song leaves the pool and returns to it when it drops off the deque

QUESTION

Design a shuffler that plays songs at random from a fixed playlist, subject to one rule: the last k songs played may not be replayed. In other words, a song becomes eligible again only after k other (distinct) plays have occurred since it was last played. Implement the class so that each call to play() runs in O(1) time (no scanning of the playlist). The intended technique is a valid_songs list of currently eligible songs plus a song_to_index dictionary, using swap-and-pop to move a just-played song out of the eligible region and to re-admit the song that is leaving the window.

Function signature

```
import random
class SongShuffler:
    def __init__(self, songs: list[str], k: int) -> None:
        """Initialize with the playlist and the no-repeat window size k."""
        ...
    def play(self) -> str:
        """Return a uniformly random eligible song in O(1); it must not be
        one of the last k songs returned. Update state so the window slides."""
        ...
```

Follow-up (as reported): The interviewer wanted the naive deque solution replaced by O(1). Show why swap-and-pop with an index map achieves constant time per play() .

EXAMPLES

Example 1

Input	`SongShuffler(["A", "B", "C"], k=1); then play(), play(), play(), play()`
Output	`e.g. "B", "A", "C", "B"`
Why	`k=1 forbids repeating the single most recent song. "B" then "A" is fine (A != B). Then "C" (C != A). Then "B" is allowed again because the only forbidden song is the immediately previous one ("C"). Any sequence with no two adjacent equal songs is valid.`

Example 2

Input	`SongShuffler(["X", "Y"], k=0); play(), play()`
Output	`e.g. "Y", "Y"`
Why	`With k=0 there is no restriction, so repeats are allowed.`

CONSTRAINTS

- 1 <= len(songs) <= 10**5 , song titles are distinct strings.
- 0 <= k < len(songs) (so at least one song is always eligible). play() returns any eligible song chosen uniformly at random; every returned song must not be among the previous k returned songs.

WHY IT WORKS

Remove the played song from the eligible array by swapping it with the last element; the song leaving the window is appended back.

SHORT OPTIMAL CODE

```
from collections import deque
import random


# BASE QUESTION (optimal): Eligible array, index map and
# swap-and-pop
# Idea: Remove the played song from the eligible array by
# swapping it with the last element; the song leaving the
# window is appended back.
class SongShuffler:
    def __init__(self, songs, window_size, seed=None):
        self.eligible = list(songs)
        self.index_of = {song: position for position, song in enumerate(self.eligible)}
        self.recent = deque()
        self.window_size = window_size
        self.rng = random.Random(seed)

    def play(self):
        pick = self.rng.randrange(len(self.eligible))
        song = self.eligible[pick]
        last = self.eligible[-1]
        # swap-and-pop: the last song takes the hole
        self.eligible[pick] = last
        self.index_of[last] = pick
        self.eligible.pop()
        del self.index_of[song]
        self.recent.append(song)
        if len(self.recent) > self.window_size:
            returning_song = self.recent.popleft()
            self.index_of[returning_song] = len(self.eligible)
            self.eligible.append(returning_song)
        return song
```

COMPLEXITY | Time O(1) per play | Space O(n)

257 Unique Random Number Generator — MEDIUM

Template move: the lazy version for huge n: a dict 'moved' stores only the swapped slots, so memory stays O(calls)

QUESTION

Design a class UniqueRandom initialized with an integer n . Each call to generate() returns a number drawn uniformly at random from [0, n-1] that has not been returned before, so across successive calls no value ever repeats. After all n distinct values have been produced, further calls return None . Every unused value must remain equally likely on each draw (no bias), and generate() must run in O(1) time per call.

Function signature

```
import random
class UniqueRandom:
    def __init__(self, n: int, seed: int | None = None) -> None:
        ...
    def generate(self) -> int | None:
        """Return a not-yet-returned number in [0, n-1], or None once all n are
exhausted."""
        ...
```

Follow-up (as reported): Use a partial Fisher-Yates shuffle: keep a virtual array 0..n-1 , and on the i -th call swap a random unused slot into position i using a sparse dict of moved entries, giving O(1) time and O(number-of- draws) space.

QUESTION

EXAMPLES

Example 1	
Input	`g = UniqueRandom(3); [g.generate(), g.generate(), g.generate()]`
Output	`a permutation of [0, 1, 2] (e.g. [2, 0, 1])`
Why	`The three calls return all of 0,1,2 in some random order with no repeats.`

Example 2	
Input	`(continuing above) g.generate()`
Output	`None`
Why	`All 3 values already returned, so the pool is exhausted.`

Example 3	
Input	`g = UniqueRandom(1); [g.generate(), g.generate()]`
Output	`[0, None]`
Why	`The only value 0 is returned once, then the generator is exhausted.`

CONSTRAINTS

- 1 <= n <= 10^7 .
- Support an optional seed for reproducible tests.

WHY IT WORKS

Swap a random unused slot into position i, remembering only the moved slots.

SHORT OPTIMAL CODE

```python
import random


# BASE QUESTION (optimal): Lazy Fisher-Yates with a
# dictionary
# Idea: Swap a random unused slot into position i,
# remembering only the moved slots.
class UniqueRandom:
    def __init__(self, count, seed=None):
        self.count = count
        self.drawn = 0
        self.swapped = {}
        self.rng = random.Random(seed)

    def generate(self):
        if self.drawn == self.count:
            return None
        slot = self.drawn
        pick = self.rng.randrange(slot, self.count)
        value = self.swapped.get(pick, pick)
        # the value that sat at slot i takes the place of the one just drawn
        self.swapped[pick] = self.swapped.get(slot, slot)
        self.drawn += 1
        return value
```

COMPLEXITY | Time O(1) per call | Space O(draws)

258 Shuffle array using random integer API

HARD

Template move: Fisher-Yates: for i from the end, j = rand_int(0, i), swap a[i] and a[j]

QUESTION

You have an array nums and no built-in shuffle, only rand_int(l, r), which returns a uniformly random integer in [l, r] (both ends included). Shuffle nums in place so that every one of the n! orders is equally likely.

Function signature

def shuffle(nums: list, rand_int) -> list

EXAMPLES

Example 1

Input	`rand_int = lambda l, r: l # a fake 'random' source that always returns the lowest allowed index`
	`nums = [1, 2, 3, 4]`
	`shuffle(nums, rand_int)`

Output	`shuffle(nums, rand_int) -> [2, 3, 4, 1]`
Why	`Fisher-Yates walks i from the end and swaps nums[i] with nums[rand_int(0, i)]. With the fake source the swaps are (3,0), (2,0), (1,0), giving a predictable result; with a real source each of the 24 orders has probability 1/24.`

WHY IT WORKS

Walk i from n - 1 down to 1, pick j uniformly from 0..i (inclusive) and swap nums[i] with nums[j]. Position i is filled by one of the i + 1 elements still unplaced, each equally likely, so every permutation has probability 1 / n!. Picking j from 0..n-1 every time is the common biased mistake.

SHORT OPTIMAL CODE

```python
import random


def rand_int(low, high):
    return random.randint(low, high)


# BASE QUESTION (optimal): Fisher-Yates
# Idea: Walk i from n - 1 down to 1, pick j uniformly from
# 0..i (inclusive) and swap nums[last_slot] with nums[pick]. Position
# i is filled by one of the last_slot + 1 elements still unplaced,
# each equally likely, so every permutation has
# probability 1 / n!. Picking j from 0..n-1 every time is
# the common biased mistake.
def shuffle(nums):
    for last_slot in range(len(nums) - 1, 0, -1):
        pick = rand_int(0, last_slot)
        nums[last_slot], nums[pick] = nums[pick], nums[last_slot]
    return nums
```

COMPLEXITY | Time O(n) | Space O(1) extra

GROUP 38 OF 43 | 5 QUESTIONS | SHORT MAIN CODE

Build and serialize: write a structure out, read it back with the same walk

Pattern

Encoding is one walk that writes tokens; decoding is the same walk reading tokens back (an iterator or an index). Put a length or a child count before the data so the reader always knows how much to take.

259 Print a Directory Tree — MEDIUM

Template move: build nested dicts from the path parts, then print them with a recursive walk that indents by depth

QUESTION

You are given a list of file/directory paths , each a '/' -separated string (for example "dir/subdir/file.txt"). Reproduce what a Unix directory listing tool prints: build the directory hierarchy implied by the paths and return it as a single newline-separated string. Rules: Each path component is a node. A component that appears under the same parent in several paths must be printed only once. Indent each node with 2 * depth spaces, where the top-level components have depth 0 . Children are printed in the order they are first encountered across the input list (preserve first-seen order; do not sort). The output must not have a trailing newline.

Function signature

```
def print_directory_tree(paths: list[str]) -> str:
    ...
```

Follow-up (as reported): Add box-drawing connectors (, ,) like the real tree command. Hint: track, per level, whether the current node is the last child of its parent.

EXAMPLES

Example 1	
Input	`paths = ["a/b/c", "a/b/d", "a/e"]`
Output	`"a\n b\n c\n d\n e"`
Why	`Rendered: a b c d e`

Example 2	
Input	`paths = ["src/main.py", "src/util.py", "README.md"]`
Output	`"src\n main.py\n util.py\nREADME.md"`
Why	`"src" and its two files come first (first-seen order), then the top-level "README.md".`

Example 3	
Input	`paths = []`
Output	`""`

CONSTRAINTS

- 0 <= len(paths) <= 10^4
- Each path is non-empty, has no leading/trailing '/' , and no empty components.
- If paths is empty, return the empty string "" .

WHY IT WORKS

Insert every path into a dict of dicts, then print depth first with two spaces per level.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Nested dictionaries and a DFS
# Idea: Insert every path into a dict of dicts, then print
```

```python
# depth first with two spaces per level.
def print_directory_tree(paths):
    root = {}
    for path in paths:
        node = root
        for part in path.split("/"):
            node = node.setdefault(part, {})
    lines = []

    def walk(node, depth):
        for name, kids in node.items():
            lines.append("  " * depth + name)
            walk(kids, depth + 1)

    walk(root, 0)
    return "\n".join(lines)
```

COMPLEXITY | Time O(total length) | Space O(total length)

260 Construct a binary tree given its preorder and inorder traversals. MEDIUM

Template move: the first preorder value is the root; its position in a value-to-index map of the inorder list splits left from right

QUESTION

Given two integer arrays, preorder and inorder, which represent the pre-order and in-order traversal of a binary tree, construct and return the binary tree. Pre-order traversal visits nodes in this order: root -> left subtree -> right subtree In-order traversal visits nodes in this order: left subtree -> root -> right subtree You need to construct the binary tree and return its root node. Examples preorder = [3, 9, 20, 15, 7], inorder = [9, 3, 15, 20, 7] output: [3, 9, 20, null, null, 15, 7] Explanation: Given the two arrays, the binary tree constructed is: 3 / \ 9 20 / \ 15 7 preorder = [1, 2], inorder = [2, 1] output: [1, 2]
Explanation: Given the two arrays, the binary tree constructed is: 1 / 2 Constraints: The number of nodes in the tree is in the range [1, 3000]. preorder and inorder consist of unique integers. The elements of inorder will always be the same as the elements of preorder but in a different order.

EXAMPLES

Example 1	
Input	`preorder = [3, 9, 20, 15, 7]` `inorder = [9, 3, 15, 20, 7]`
Output	`values(build_pre_in([3, 9, 20, 15, 7], [9, 3, 15, 20, 7])) -> [3, 9, 20, null, null, 15, 7]`
Why	The first preorder value 3 is the root; in the inorder list 9 is left of it and 15, 20, 7 are right. The tree is shown level by level.

WHY IT WORKS

Map each value to its inorder index. Walk preorder with one shared pointer; for a subtree covering inorder[lo:hi] take the next preorder value as the root, split at its inorder index, and build the left subtree first (preorder visits it before the right). No slicing, no searching.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Hash map of inorder positions
# and index bounds
# Idea: Map each value to its inorder index. Walk preorder
# with one shared pointer; for a subtree covering
# inorder[low:high] take the next preorder value as the root,
# split at its inorder index, and build the left subtree
# first (preorder visits it before the right). No slicing,
# no searching.
```

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def buildTree(preorder, inorder):
    inorder_index = {value: position for position, value in enumerate(inorder)}
    next_preorder = 0

    def build(low, high):
        nonlocal next_preorder
        if low >= high:
            return None
        root = TreeNode(preorder[next_preorder])
        next_preorder += 1
        root_position = inorder_index[root.val]
        root.left = build(low, root_position)
        root.right = build(root_position + 1, high)
        return root

    return build(0, len(inorder))
```

COMPLEXITY | Time O(n) | Space O(n)

261 Write functions to serialize and deserialize a list of strings. MEDIUM

Template move: write len + '#' + text for each string; read the length, then take exactly that many characters

QUESTION

Serialize and Deserialize a List of Strings. You are tasked with writing two functions: serialize and deserialize. The serialize function will take a list of strings as input and return a single string. The deserialize function will take this single string and reconstruct the original list of strings. Details The serialize function should concatenate all strings in the list into a single string, with each original string's length encoded as a single character prefix. The deserialize function should be able to take the single string and accurately split it back into the original list of strings, using the encoded length characters as guides. For example, consider a list ["hello", "world"]. The serialization of this list might be "\x05hello\x05world", where \x05 indicates that the following string has a length of 5 characters. Constraints The maximum length of any string is 65535 (0xFFFF). If a string's length exceeds the maximum, the serialize function should raise an exception. The deserialize function should accurately handle any edge cases, such as empty strings or single-character strings. Assume that all strings consist of ASCII characters only. Examples lst = ['hello', 'world'] serialized = serialize(lst)

serialized output

: "\x05hello\x05world" deserialized = deserialize(serialized)

deserialized output should be

: ['hello', 'world'] lst = [''] serialized = serialize(lst)

serialized output

: "\x00" deserialized = deserialize(serialized)

deserialized output should be

EXAMPLES

QUESTION

Example 1	

Input	`items = ['hello', 'wor#ld', '']`
	`serialize(items)`
	`deserialize(serialize(items)) == items`

Output	`serialize(items) -> '5#hello6#wor#ld0#'` `deserialize(serialize(items)) == items -> true`
Why	`Each string is written as its length, a #, and its text, so a # inside a string cannot confuse the reader.`

WHY IT WORKS

Write length#text for each string. The decoder reads the length first, so every character is safe inside the text.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Length-prefix each string
# Idea: Write length#text for every string. While reading,
# the length tells us exactly where that string ends.
def serialize(strings):
    return "".join(f"{len(text)}#{text}" for text in strings)


def deserialize(data):
    strings = []
    position = 0
    while position < len(data):
        separator = data.index("#", position)
        length = int(data[position:separator])
        start = separator + 1
        strings.append(data[start:start + length])
        position = start + length
    return strings
```

COMPLEXITY | Time O(total characters) | Space O(total characters) for the output

262 Serialize and Deserialize a Graph — HARD

Template move: BFS with a seen set; write one line per node: value, then the values of its neighbours; rebuild by creating all nodes first, then linking

QUESTION

Design a codec that can serialize a connected, undirected graph to a string and deserialize that string back into an identical graph. Each node has a unique integer val and a list of neighbors . The graph may contain cycles. After a round trip, the reconstructed graph must have the same node values and the same adjacency relationships as the original. You are given the reference node class below. Implement serialize (node -> string) and deserialize (string -> node). serialize(None) must round-trip to None via deserialize .

QUESTION

Function signature

```
class Node:
    def __init__(self, val: int = 0, neighbors: list["Node"] | None = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []
class Codec:
    def serialize(self, node: "Node | None") -> str:
        """Encode the whole graph reachable from node into a string."""
        ...
    def deserialize(self, data: str) -> "Node | None":
        """Rebuild the graph from the string produced by serialize."""
        ...
```

Follow-up (as reported): Support directed graphs (asymmetric adjacency). Hint: record each directed edge (u -> v) rather than assuming both directions during deserialization.

EXAMPLES

Example 1	
Input	`adjList = [[]] # a single node with no neighbors`
Output	`A single node with val 1 and an empty neighbor list.`

Example 2	
Input	`node = None # empty graph`
Output	`codec.deserialize(codec.serialize(None)) is None`

CONSTRAINTS

- 0 <= number of nodes <= 100
- Node values are unique.
- The graph is connected when non-empty; you receive any one node as the entry point.

WHY IT WORKS

One token per node with its neighbour values; rebuild by creating the nodes first and linking afterwards.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): BFS to a text, two passes back
# Idea: One token per node with its neighbour values;
# rebuild by creating the nodes first and linking
# afterwards.
from collections import deque


class Node:
    def __init__(self, val=0, neighbors=None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []


class Codec:
    def serialize(self, node):
        if node is None:
            return ""
        seen, queue, parts = {node.val}, deque([node]), []
        while queue:
            current = queue.popleft()
            parts.append(f"{current.val}:" + ",".join(str(neighbour.val) for neighbour in current.neighbors))
            for neighbour in current.neighbors:
```

```
                if neighbour.val not in seen:
                    seen.add(neighbour.val)
                    queue.append(neighbour)
        return ";".join(parts)

    def deserialize(self, data):
        if not data:
            return None
        rows = [part.split(":") for part in data.split(";")]
        nodes = {int(node_value): Node(int(node_value)) for node_value, _ in rows}
        for node_value, neighbour_text in rows:
            nodes[int(node_value)].neighbors = [nodes[int(neighbour_value)] for neighbour_value in neighbour_te
xt.split(",") if neighbour_value]
        return nodes[int(rows[0][0])]
```

COMPLEXITY | Time O(V + E) | Space O(V + E)

263 Implement a decode and encode function of a general N-ary tree. HARD

Template move: write value and child count for each node (preorder); read the count to know how many children to build

QUESTION

A general tree has a value in every node and any number of children. Write encode(root), which turns the tree into a string, and decode(text), which rebuilds the identical tree from that string.

Function signature

def encode(root: Node) -> str

def decode(text: str) -> Node

EXAMPLES

Example 1

Input	`root = Node(1, [Node(3, [Node(5), Node(6)]), Node(2), Node(4)])`
	`text = encode(root)`
	`text`
	`encode(decode(text)) == text`

Output	`text -> '[1, 3, 3, 2, 5, 0, 6, 0, 2, 0, 4, 0]'` `encode(decode(text)) == text -> true`
Why	`Each node is written as its value followed by its number of children, in pre-order: 1 has 3 children, 3 has 2, and so on. Decoding reads exactly that many children for each node, so encoding the decoded tree gives the same text.`

WHY IT WORKS

Write each node before its children with its child count. The decoder recursively reads exactly that many children.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Recursive pre-order with child counts
# Idea: Write each node before its children with its child
# count. The decoder recursively reads exactly that many children.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children or []


def encode(root):
    tokens = []

    def dfs(node):
        tokens.extend((str(node.val), str(len(node.children))))
```

```
        for child in node.children:
            dfs(child)

    if root is not None:
        dfs(root)
    return " ".join(tokens)


def decode(text):
    if not text:
        return None
    tokens = iter(text.split())

    def dfs():
        node = Node(next(tokens))
        child_count = int(next(tokens))
        node.children = [dfs() for _ in range(child_count)]
        return node

    return dfs()
```

COMPLEXITY | Time O(n) | Space O(h) recursion plus output

GROUP 39 OF 43 | 5 QUESTIONS | SHORT MAIN CODE

Games: win or lose with a cache, and guessing with feedback

Pattern

For two-player games, a state is winning if some move leads to a state where the opponent loses; cache every state. For guessing games, pick the guess that shrinks the worst remaining group of candidates the most, then keep only the candidates that agree with the feedback.

264 Choose optimal guesses for green-only Wordle

MEDIUM

Template move: bitmask per (position, letter) of the words that have it; the best guess splits the words into the most even groups

QUESTION

You are guessing a hidden word from a candidate list. After each guess you only learn which positions are green, that is, where your letter equals the answer's letter. best_guess(words) picks the candidate whose answer pattern splits the remaining words most evenly: the one with the smallest expected size of the group left (sum of squared group sizes divided by the number of words); ties go to the alphabetically smaller word.

Function signature

def green(guess: str, answer: str) -> tuple
def best_guess(words: list) -> str

EXAMPLES

Example 1	
Input	`'cat', 'car'`
Output	`(true, true, false)`
Why	`c and a match, the last letters differ.`

Example 2	
Input	`['cat', 'car', 'cap', 'dog']`
Output	`'cap'`
Why	`'cap', 'car' and 'cat' split the other words equally well and 'cap' wins the alphabetical tie-break; 'dog' shares no letter positions with the others, so it tells less.`

Example 3	
Input	`['tea']`
Output	`'tea'`
Why	`With a single candidate the answer is that candidate.`

WHY IT WORKS

Index the words: bits[pos][letter] is an integer whose bit i is set when word i has that letter at pos. For a guess, start with one group containing all words and split it position by position into 'green here' (AND with bits[pos][guess letter]) and 'not green here' (the remainder). Group sizes are popcounts; the score is sum of s^2 over W. Choose the minimum score, ties by the smaller word.

SHORT OPTIMAL CODE

```
from fractions import Fraction
from collections import defaultdict
```

```python
# BASE QUESTION (optimal): Bitsets per (position, letter),
# splitting groups with AND
# Idea: Index the words: bits[pos][letter] is an integer
# whose bit i is set when word i has that letter at pos.
# For a guess, start with one group containing all words
# and split it position by position into 'green here' (AND
# with bits[pos][guess letter]) and 'not green here' (the
# remainder). Group sizes are popcounts; the score is sum
# of s^2 over word_count. Choose the minimum score, ties by the
# smaller word.
def best_guess(words):
    word_count = len(words)
    bits = [defaultdict(int) for _ in range(len(words[0]))]
    for word_index, word in enumerate(words):
        for pos, letter in enumerate(word):
            bits[pos][letter] |= 1 << word_index
    best = None
    for guess in words:
        groups = [(1 << word_count) - 1]
        for pos, letter in enumerate(guess):
            mask = bits[pos][letter]
            next_groups = []
            for group in groups:
                green_part, other_part = group & mask, group & ~mask
                if green_part:
                    next_groups.append(green_part)
                if other_part:
                    next_groups.append(other_part)
            groups = next_groups
        score = Fraction(sum(group.bit_count() ** 2 for group in groups), word_count)
        if best is None or (score, guess) < best:
            best = (score, guess)
    return best[1]
```

COMPLEXITY | Time O(W * L * groups) with bit-parallel work per group | Space O(W * L)

265 Determine winner in stack-merging game — HARD

Template move: state = the sorted tuple of stack sizes; a move merges two whose sum stays within the cap

QUESTION

A game starts with stacks of the given sizes. A move merges two stacks into one if their combined size is at most cap. The player who cannot move loses (in the misere variant, the player who cannot move wins). Return whether the first player wins with best play.

Function signature

def first_player_wins(stacks: list, cap: int, misere=False) -> bool

EXAMPLES

Example 1	
Input	`[1, 1], 2`
Output	`true`
Why	`Merge the two stacks of 1 into 2; the opponent has a single stack and cannot move.`

Example 2	
Input	`[1, 1, 1], 2`
Output	`true`
Why	`Any merge makes a stack of 2 next to a stack of 1, whose sum 3 exceeds the cap: the opponent is stuck.`

QUESTION

Example 3	
Input	[3], 5
Output	false
Why	A single stack: the first player cannot move and loses.

Example 4	
Input	[3], 5, misere = true
Output	true
Why	The same single stack in the misere variant: being unable to move wins.

WHY IT WORKS

Merging does not depend on order, so a sorted tuple identifies the position. Each state is solved once and then looked up. This is the standard fix for 'many move orders reach the same position'.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Memoize on the sorted state
# Idea: Merging does not depend on order, so a sorted
# tuple identifies the position. Each state is solved once
# and then looked up. This is the standard fix for 'many
# move orders reach the same position'.
def merge_game(stacks, height_limit):
    memo = {}

    def wins(state):
        if state in memo:
            return memo[state]
        result = False
        for first in range(len(state)):
            for second in range(first + 1, len(state)):
                if state[first] + state[second] <= height_limit:
                    rest = state[:first] + state[first + 1:second] + state[second + 1:] + (state[first] + state
[second],)
                    if not wins(tuple(sorted(rest))):
                        result = True
                        break
            if result:
                break
        memo[state] = result
        return result

    return wins(tuple(sorted(stacks)))
```

COMPLEXITY | Time O(number of distinct sorted states * k^2) | Space O(number of states)

266 Find secret word with match-count feedback HARD

Template move: guess the word that minimises the size of the biggest feedback group; keep the words with the same match count

QUESTION

Problem

You are given a list of **unique** lowercase words, all of the same length (e.g., length = 6). One of these words is a **secret** word. You can interact with a black-box API:

- **`matchCount(guess) -> int`**: when you submit a word **`guess`** from the list, the API returns the number of positions **`i`** such that **`guess[i] == secret[i]`** (i.e., exact character match at the same index). You are allowed to call **`matchCount`**

QUESTION

at most 10 times.

Task

Design/implement a function (e.g., **`findSecretWord(words)`**) that uses the API to identify the secret word within the guess limit.

Notes / Constraints

- **`words`** contains between 1 and 100 words.
- All words have equal length **`L`** (commonly **`L = 6`**).
- Every guess must be a word from **`words`**.
- After each API response, you may adapt your next guess.

What to return

Depending on the interface, you may either: - return the identified secret word, or - stop once **`matchCount(guess) == L`** (found it). (Your interviewer may treat this as an interactive problem; focus on strategy + correctness under the 10-guess constraint.)

You are given a list of unique lowercase words, and every word has the same length **`L`**. Exactly one word in the list is the secret word. In the original interview version, you interact with a black-box API:

- **`matchCount(guess) -> int`**
- It returns how many positions **`i`** satisfy **`guess[i] == secret[i]`** You may call this API at most 10 times, and every guess must be a word from the given list. In this non-interactive version, implement **`solution(words, secret)`** and simulate the API inside your function. The parameter **`secret`** is provided only so the judge can verify your strategy. Your function should repeatedly make guesses, use the returned match counts to eliminate impossible candidates, and return the discovered secret word. If your strategy does not find the secret within 10 guesses, return an empty string **`""`**.

Constraints**`1 <= len(words) <= 1001 <= len(words[i]) <= 10`**All words are unique, lowercase, and have the same length **`Lsecret`** is guaranteed to be one of the words in **`words`**Your algorithm may use at most 10 simulated **`matchCount`** callsTest data is designed so that a correct elimination strategy can identify the secret within the limit

EXAMPLES

```
Input: (["acckzz", "ccbazz", "eiowzz", "abcczz"], "acckzz")
```

Expected Output: 'acckzz'Explanation: The secret is already one of the candidates. Using match-count feedback, the candidate set shrinks until **`acckzz`** is identified.

```
Input: (["secret"], "secret")
```

Expected Output: 'secret'Explanation: With only one word in the list, the first allowed guess must be the secret.

Hints

After a guess returns **`k`**, any valid remaining candidate must also have exactly **`k`** matching positions with that guess.A strong strategy is to choose the next guess that splits the remaining candidates as evenly as possible, minimizing the size of the largest feedback bucket.

WHY IT WORKS

For each candidate guess count how many other candidates would fall into each match-count group, and pick the guess whose biggest group is smallest (a minimax choice). Query it and keep the words consistent with the answer. With up to 100 words of length 6 this finishes within 10 guesses in practice.

SHORT OPTIMAL CODE

```
from collections import Counter
```

```
def _match(first, second):
    return sum(letter_a == letter_b for letter_a, letter_b in zip(first, second))


# BASE QUESTION (optimal): Guess the word that minimises
# the size of its largest match-count group
# Idea: For each candidate guess count how many other
# candidates would fall into each match-count group, and
# pick the guess whose biggest group is smallest (first
# minimax choice). Query it and keep the words consistent
# with the answer. With up to 100 words of length 6 this
# finishes within 10 guesses in practice.
def find_secret_minimax(words, secret):
    candidates, guesses = list(words), 0
    while True:
        best_guess = min(candidates, key=lambda guess: (max(Counter(_match(guess, word) for word in candidates)
.values()), guess))
        guesses += 1
        match_count = _match(best_guess, secret)
        if match_count == len(secret):
            return guesses
        candidates = [word for word in candidates if _match(best_guess, word) == match_count]
```

COMPLEXITY | Time O(guesses * W^2 * L) | Space O(W)

267 Wordle-Style 5-Letter Minimum-Guess Strategy — HARD

Template move: ask about the letter at a position that splits the remaining words most evenly (min of yes-count and no-count is largest)

QUESTION

A hidden word has 5 lowercase letters. The only thing you can ask is the oracle query(pos, letter), which says whether the letter is at that position (positions 0 to 4). Find the word with as few oracle calls as possible. by_frequency tries letters in order of English frequency; with_dictionary also knows a list of candidate words and always asks the question that splits the candidates most evenly. Both return (word, number of calls).

Function signature

def by_frequency(query) -> (str, int)
def with_dictionary(query, dictionary: list) -> (str, int)

EXAMPLES

Example 1

Input	`secret = 'crane'`
	`query = lambda pos, letter: secret[pos] == letter`
	`by_frequency(query)`
	`with_dictionary(query, ['crane', 'crate', 'slate', 'trace', 'grape'])`

Output	`by_frequency(query) -> ('crane', 31)` `with_dictionary(query, ['crane', 'crate', 'slate', 'trace', 'grape']) -> ('crane', 2)`
Why	`Without a dictionary every position is searched letter by letter; with five candidate words a handful of well-chosen questions are enough.`

WHY IT WORKS

Without a dictionary: for each position ask letters from most to least common and stop at the first yes; if 25 letters said no the 26th is implied and needs no call. With a dictionary: keep the words consistent with every answer so far. Each round pick the (position, letter) pair whose yes-count is closest to half of the candidates, ask it, and keep only the words that agree. The loop ends when one word is left.

SHORT OPTIMAL CODE

```
LETTERS_BY_FREQUENCY = "etaoinshrdlcumwfgypbvkjxqz"


# BASE QUESTION (optimal): Frequency order, or entropy
# split when a dictionary is known
# Idea: Without a dictionary: for each position ask
# letters from most to least common and stop at the first
# yes; if 25 letters said no the 26th is implied and needs
# no call. With a dictionary: keep the words consistent
# with every answer so far. Each round pick the (position,
# letter) pair whose yes-count is closest to half of the
# candidates, ask it, and keep only the words that agree.
# The loop ends when one word is left.
def wordle_guess(query, dictionary=None):
    calls = 0
    def ask(position, letter):
        nonlocal calls
        calls += 1
        return query(position, letter)
    if dictionary:
        words = list(dictionary)
        while len(words) > 1:
            best_probe = max(((position, letter) for position in range(5) for letter in set(word[position] for
word in words)),
                             key=lambda probe: min(sum(word[probe[0]] == probe[1] for word in words), sum(word[probe[
0]] != probe[1] for word in words)))
            answer = ask(*best_probe)
            words = [word for word in words if (word[best_probe[0]] == best_probe[1]) == answer]
        return words[0], calls
    letters = []
    for position in range(5):
        for rank, letter in enumerate(LETTERS_BY_FREQUENCY):
            if rank == 25 or ask(position, letter):
                letters.append(letter)
                break
    return "".join(letters), calls
```

COMPLEXITY | Time at most 5 x 25 calls without a dictionary; about log2(words) with one | Space O(words)

268 Babylon Tile-Stack Merge Game

HARD

Template move: state = the sorted tuple of stacks; a move merges two stacks that share a height or a colour; you win if some merge leaves the opponent without a winning reply

QUESTION

A two-player game on stacks of tiles. Each stack has a height and a colour (the colour of its top tile). A move places one stack on top of another when the two stacks have the same height or the same colour; the new stack has the combined height and the colour of the stack that was placed on top. The player who cannot move loses. Given the stacks as (height, colour) pairs, return whether the player to move wins with best play.

Function signature

def babylon_wins(stacks: list) -> bool

EXAMPLES

Example 1	
Input	`[(1, 'Y'), (1, 'W')]`
Output	`true`
Why	`Two stacks of height 1: the mover merges them, and the opponent has nothing left to move.`

QUESTION

Example 2	
Input	[(1, 'Y')]
Output	false
Why	A single stack: the mover has no move and loses.

Example 3	
Input	[(1, 'Y'), (2, 'W')]
Output	false
Why	Heights differ and colours differ: no move is possible, so the mover loses.

Example 4	
Input	[(1, 'Y'), (1, 'Y'), (1, 'W'), (1, 'W')]
Output	true
Why	Four height-1 stacks of two colours: the mover has a winning plan.

WHY IT WORKS

legal_moves lists the ordered pairs that may merge, apply builds the next state, and wins(state) is true when some move reaches a state where the opponent loses. Sorting the stacks makes equal positions share one cache entry. Playing the game out is then just: pick any move that leads to a state where wins() is False.

SHORT OPTIMAL CODE

```
import itertools


# BASE QUESTION (optimal): Negamax with memoisation on the
# sorted stack tuple
# Idea: legal_moves lists the ordered pairs that may
# merge, apply builds the next state, and wins(state) is
# true when some move reaches a state where the opponent
# loses. Sorting the stacks makes equal positions share
# one cache entry. Playing the game out is then just: pick
# any move that leads to a state where wins() is False.
def babylon_wins(stacks):
    cache = {}
    def wins(state):
        if state in cache:
            return cache[state]
        can_win = False
        for first, second in itertools.permutations(range(len(state)), 2):
            (height_a, colour_a), (height_b, colour_b) = state[first], state[second]
            if height_a == height_b or colour_a == colour_b:
                next_state = tuple(sorted([stack for position, stack in enumerate(state) if position not in (fi
rst, second)] + [(height_a + height_b, colour_a)]))
                if not wins(next_state):
                    can_win = True
                    break
        cache[state] = can_win
        return can_win
    return wins(tuple(sorted(stacks)))
```

COMPLEXITY | Time O(states * moves^2) | Space O(states)

GROUP 40 OF 43 | 5 QUESTIONS | SHORT MAIN CODE

Sliding window: grow on the right, shrink on the left while it is too bad

Pattern

Move the right end one step at a time and add that item to your state. While the window breaks the rule, remove items from the left end. After each step the window is valid, so record its length (or the shortest valid one).

269 Temperature Monitoring Data Structure — MEDIUM

Template move: a deque of the last n readings with a running total; a second deque of decreasing values gives the maximum

QUESTION

Design a monitor over the last n temperature readings. insert(x) adds a reading (the oldest one drops out once more than n are stored), average() returns the mean of the stored readings, and maximum() returns the largest of them in O(1).

Function signature

class Monitor:
    def __init__(self, n: int)
    def insert(self, x: float) -> None
    def average(self) -> float
    def maximum(self) -> float

EXAMPLES

Example 1

Input	`m = Monitor(3)`
	`m.insert(20)`
	`m.insert(22)`
	`m.insert(19)`
	`m.average()`
	`m.insert(25)`
	`m.average()`
	`m.maximum()`
	`m.insert(10)`
	`m.maximum()`

Output	`m.average() -> 20.333333`
	`m.average() -> 22`
	`m.maximum() -> 25`
	`m.maximum() -> 25`

Why	With n = 3 the window slides: after 25 arrives the window is 22, 19, 25 (average 22, maximum 25); after 10 it is 19, 25, 10, so the maximum is still 25.

WHY IT WORKS

Keep the window in a deque and a running sum; on insert add the value and drop the oldest when the window is full. For the maximum keep a second deque of (index, value) with decreasing values: pop smaller ones from the back before pushing, and pop the front when its index leaves the window. The front is the maximum.

SHORT OPTIMAL CODE

```
from collections import deque


# BASE QUESTION (optimal): Running sum plus a monotonic
# deque
# Idea: Keep the window in a deque and a running sum; on
# insert add the value and drop the oldest when the window
# is full. For the maximum keep a second deque of (index,
```

```
# value) with decreasing values: pop smaller ones from the
# back before pushing, and pop the front when its index
# leaves the window. The front is the maximum.
class Monitor:
    def __init__(self, window_size):
        self.window_size, self.window, self.total = window_size, deque(), 0
        self.max_queue, self.readings_seen = deque(), 0

    def insert(self, reading):
        self.window.append(reading)
        self.total += reading
        if len(self.window) > self.window_size:
            self.total -= self.window.popleft()
        while self.max_queue and self.max_queue[-1][1] <= reading:
            self.max_queue.pop()
        self.max_queue.append((self.readings_seen, reading))
        self.readings_seen += 1
        while self.max_queue[0][0] <= self.readings_seen - 1 - self.window_size:
            self.max_queue.popleft()

    def average(self):
        return self.total / len(self.window)

    def maximum(self):
        return self.max_queue[0][1]
```

COMPLEXITY | Time O(1) amortized per operation | Space O(N)

270 Footpath Beauty After Repainting

MEDIUM

Template move: bad = a block that is not the wanted colour; allowed = the repaint budget m; the longest window is the answer

QUESTION

A footpath is built from n blocks laid in a line, painted with one of k colors. block_colors[i] is the color of block i (colors are integers in 1..k). The beauty value for a given favorite color c is the length of the longest run of consecutive blocks that are all color c . You are allowed to repaint at most m blocks (each to any color you choose). Return the maximum possible beauty value for favorite color c after repainting at most m blocks.

Function signature

def max_beauty_after_repaint(block_colors: list[int], c: int, m: int) -> int:

...

Follow-up (as reported): (Reported) Answer q queries, each with a different favorite color, and return the beauty value (with m fixed) for each. Hint: a sliding window per query, or precompute runs. (Reported) The base version with m = 0 was asked first; the repaint version is the twist.

EXAMPLES

Example 1	
Input	`block_colors = [1, 2, 1, 1, 2, 1], c = 1, m = 1`
Output	`4`
Why	`Repaint one of the 2's. The best window has at most one non-c block, e.g. indices 2..5 [1,1,2,1] -> repaint the 2 -> 4.`

Example 2	
Input	`block_colors = [3, 3, 3], c = 3, m = 0`
Output	`3`
Why	`All blocks are already color 3; no repaint needed.`

QUESTION

Example 3	
Input	`block_colors = [2, 2, 2], c = 1, m = 2`
Output	`2`
Why	`Only 2 repaints allowed, so at most 2 consecutive c blocks.`

CONSTRAINTS

- 1 <= n <= 10^5 , block_colors has length n
- 1 <= k , 1 <= c <= k , 1 <= block_colors[i] <= k
- 0 <= m <= n
- With m = 0 , the answer is simply the longest existing run of color c .

WHY IT WORKS

The longest window holding at most m other-colour blocks is the answer.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Sliding window with at most m
# repaints
# Idea: The longest window holding at most m other-colour
# blocks is the answer.
def max_beauty_after_repaint(block_colors, target_color, max_repaints):
    left, other_colored, best = 0, 0, 0
    for right, color in enumerate(block_colors):
        other_colored += color != target_color
        while other_colored > max_repaints:
            other_colored -= block_colors[left] != target_color
            left += 1
        best = max(best, right - left + 1)
    return best
```

COMPLEXITY | Time O(n) | Space O(1)

271 Shortest Subarray with K Distinct Integers — MEDIUM

Template move: state = a count dict; while it holds at least k distinct values, record the window and shrink from the left

QUESTION

Given an array nums and an integer k, return the length of the shortest contiguous subarray that contains at least k different integers, or -1 if there is no such subarray.

Function signature

def shortest_k_distinct(nums: list, k: int) -> int

EXAMPLES

Example 1	
Input	`[1, 2, 1, 3, 2], 3`
Output	`3`
Why	`[2, 1, 3] is the shortest window holding three different numbers.`

Example 2	
Input	`[1, 1, 1], 2`
Output	`-1`
Why	`Only one distinct number exists.`

QUESTION

Example 3	
Input	`[1, 2], 2`
Output	`2`
Why	`The whole array is needed.`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= nums[i] <= 10^9
- 1 <= k <= nums.length

WHY IT WORKS

Expand the right end, counting each value. While the window has at least k distinct values, record its length and shrink from the left (deleting a value from the map when its count reaches 0). Each index is added once and removed once.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Sliding window with a frequency
# map
# Idea: Expand the right end, counting each value. While
# the window has at least k distinct values, record its
# length and shrink from the left (deleting a value from
# the map when its count reaches 0). Each index is added
# once and removed once.
def shortest_k_distinct(nums, distinct_needed):
    if distinct_needed <= 0:
        return 0
    count, best, left = {}, None, 0
    for right, value in enumerate(nums):
        count[value] = count.get(value, 0) + 1
        while len(count) >= distinct_needed:
            best = right - left + 1 if best is None else min(best, right - left + 1)
            leaving_value = nums[left]
            count[leaving_value] -= 1
            if count[leaving_value] == 0:
                del count[leaving_value]
            left += 1
    return -1 if best is None else best
```

COMPLEXITY | Time O(n) | Space O(n)

272 Find the minimum window substring. MEDIUM

Template move: state = how many of each needed letter are still missing; while nothing is missing, record the window and shrink

QUESTION

Given strings s and t, return the shortest substring of s that contains every character of t (a character that appears twice in t must appear at least twice in the substring). Return the empty string if there is no such substring.

Function signature

def min_window(s: str, t: str) -> str

EXAMPLES

Example 1	
Input	`'ADOBECODEBANC', 'ABC'`
Output	`'BANC'`
Why	`BANC is the shortest window that holds an A, a B and a C.`

QUESTION

Example 2	
Input	'a', 'a'
Output	'a'
Why	The whole string is the only window.

Example 3	
Input	'a', 'aa'
Output	''
Why	t needs two a's but s has only one.

WHY IT WORKS

need[c] is how many more of c the window requires, and missing counts the distinct requirements still unmet. Move right adding characters; when missing reaches 0 shrink from the left while the window stays valid, recording the shortest. Removing a needed character that drops below its requirement makes missing go up again.

SHORT OPTIMAL CODE

```
from collections import Counter


# BASE QUESTION (optimal): Sliding window with a 'still
# needed' counter
# Idea: need[c] is how many more of c the window requires,
# and missing counts the distinct requirements still
# unmet. Move right adding characters; when missing
# reaches 0 shrink from the left while the window stays
# valid, recording the shortest. Removing a needed
# character that drops below its requirement makes missing
# go up again.
def minWindow(text, required):
    if not required:
        return ""
    need, missing = Counter(required), len(set(required))
    best, left = "", 0
    for right, char in enumerate(text):
        if char in need:
            need[char] -= 1
            if need[char] == 0:
                missing -= 1
        while missing == 0:
            if best == "" or right - left + 1 < len(best):
                best = text[left:right + 1]
            leaving_char = text[left]
            if leaving_char in need:
                need[leaving_char] += 1
                if need[leaving_char] > 0:
                    missing += 1
            left += 1
    return best
```

COMPLEXITY | Time O(n + m) | Space O(m)

273 Necklace Cut into Two Halves with Equal D/R Counts

HARD

Template move: a fixed-size window of half the length slides around the doubled string, keeping a count of D

QUESTION

A necklace is a circular string of beads D and R. Cut it in two places into two arcs of equal length that contain the same number of D beads (and so the same number of R beads). Return the two cut positions and the two arcs, or None if no such cut exists.

Function signature

def necklace_two_cuts(s: str) -> tuple | None

(cut1, cut2, arc1, arc2)

EXAMPLES

Example 1	
Input	`'DRDR'`
Output	`(0, 2, 'DR', 'DR')`
Why	`Cut at 0 and 2: both arcs are DR.`

Example 2	
Input	`'DDRR'`
Output	`(1, 3, 'DR', 'RD')`
Why	`Linear halves DD and RR are unbalanced, but rotating the cuts by one gives DR and RD.`

Example 3	
Input	`'DDR'`
Output	`null`
Why	`An odd number of beads cannot be split into equal arcs.`

WHY IT WORKS

If each arc has half of the D's and half of the R's, each arc has length n/2, and the total D and R counts must both be even. Slide a window of length n/2 around the circle (use s + s), tracking its D count. The window starting at i and the one starting at i + n/2 are complementary arcs, so their D counts sum to the total. Counts change by at most one per step, so if one is below half and the other above half the count must hit exactly half somewhere. Hence a valid pair of cuts always exists when parity allows.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Fixed-length circular window
# with the discrete intermediate value argument
# Idea: If each arc has half of the D's and half of the
# R's, each arc has length length/2, and the total D and R
# counts must both be even. Slide a window of length length/2
# around the circle (use necklace + necklace), tracking its D count. The
# window starting at i and the one starting at start + length/2 are
# complementary arcs, so their D counts sum to the total.
# Counts change by at most one per step, so if one is
# below half and the other above half the count must hit
# exactly half somewhere. Hence a valid pair of cuts
# always exists when parity allows.
def necklace_two_cuts(necklace):
    length = len(necklace)
    d_total, r_total = necklace.count("D"), necklace.count("R")
    if length % 2 or d_total % 2 or r_total % 2:
        return None
    half, target = length // 2, d_total // 2
    doubled = necklace + necklace
    window = doubled[:half].count("D")
```

```
    for start in range(length):
        if window == target:
            return start, (start + half) % length, doubled[start:start + half], doubled[start + half:start + le
ngth]
        window += (doubled[start + half] == "D") - (doubled[start] == "D")
    return None
```

COMPLEXITY | Time O(n) | Space O(n)

GROUP 41 OF 43 | 5 QUESTIONS | SHORT MAIN CODE

Parent pointers: store who is above each node and climb

Pattern

Keep parent[x] (or manager[x]). Climbing is a while loop that follows parent until it runs out. Questions about common ancestors, cycles and who reports to whom are all small variations of that climb.

274 Tree Node. EASY

Template move: the SQL version of the same idea: p_id is the parent pointer, NULL means root, being someone's parent means inner, else leaf

QUESTION

A binary tree is stored in a table tree_node_table(id, p_id), where p_id is the id of the parent node (NULL for the root). Write a query that labels every node: 'Root' if it has no parent, 'Inner' if it has a parent and at least one child, and 'Leaf' if it has a parent and no children. The output has the columns id and node_types, ordered by id.

Function signature

-- columns: id, node_types

EXAMPLES

Example 1

Input	`CREATE TABLE tree_node_table (id INTEGER, p_id INTEGER);` `INSERT INTO tree_node_table VALUES (1, NULL), (2, 1), (3, 1), (4, 2), (5, 2);`

Output	`1, Root`
	`2, Inner`
	`3, Leaf`
	`4, Leaf`
	`5, Leaf`

Why	`Node 1 has no parent, node 2 has a parent and children 4 and 5, and nodes 3, 4 and 5 have no children.`

Example 2

Input	`CREATE TABLE tree_node_table (id INTEGER, p_id INTEGER);` `INSERT INTO tree_node_table VALUES (1, NULL);`
Output	`1, Root`
Why	`A tree with one node: it has no parent, so it is the Root (not a Leaf).`

WHY IT WORKS

CASE tests in order: no parent means Root, an id that appears as someone's parent means Inner, anything else is a Leaf. The subquery filters out NULL so the IN test is safe (NOT IN with a NULL would return no rows). The table is scanned once and the parent id set is built once.

SHORT OPTIMAL CODE

```
-- BASE QUESTION (optimal): One CASE expression
-- Idea: CASE tests in order: no parent means Root, an id
-- that appears as someone's parent means Inner, anything
-- else is a Leaf. The subquery filters out NULL so the IN
-- test is safe (NOT IN with a NULL would return no rows).
-- The table is scanned once and the parent id set is
-- built once.
SELECT id,
       CASE
           WHEN p_id IS NULL THEN 'Root'
           WHEN id IN (SELECT p_id FROM tree WHERE p_id IS NOT NULL) THEN 'Inner'
```

```
            ELSE 'Leaf'
        END AS node_types
FROM tree
ORDER BY id;
```

COMPLEXITY | Time O(n) with the set of parent ids built once | Space O(n)

275 Employee Hierarchy Add / Delete — MEDIUM

Template move: manager[emp] and a set of reports per manager; delete re-attaches the reports to the deleted person's manager

QUESTION

Maintain a company hierarchy. Org(ceo) starts with the CEO. add(emp, mgr) adds a new employee under an existing manager (False if emp already exists or mgr does not). delete(emp) removes an employee and their direct reports move up to the deleted person's manager. The CEO can only be deleted when exactly one person reports to them, who then becomes the CEO.

Function signature

class Org:
  def __init__(self, ceo)
  def add(self, emp, mgr) -> bool
  def delete(self, emp) -> bool

EXAMPLES

Example 1

Input	`o = Org('ceo')`
	`o.add('vp', 'ceo')`
	`o.add('dev', 'vp')`
	`o.add('qa', 'vp')`
	`o.add('dev', 'ceo')`
	`o.delete('vp')`
	`o.manager['dev']`
	`o.delete('ceo')`

Output	`o.add('vp', 'ceo') -> true`
	`o.add('dev', 'vp') -> true`
	`o.add('qa', 'vp') -> true`
	`o.add('dev', 'ceo') -> false`
	`o.delete('vp') -> true`
	`o.manager['dev'] -> 'ceo'`
	`o.delete('ceo') -> false`

Why	`'dev' already exists so the second add fails. Deleting 'vp' moves dev and qa up to the ceo. The ceo now has two reports, so the ceo cannot be deleted.`

WHY IT WORKS

Keep manager[emp] and reports[emp] (a set). Add inserts into both. Delete reattaches each report to the deleted node's manager and removes the node from its manager's set. Cost depends only on the number of direct reports.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Parent map plus children sets
# Idea: Keep manager[employee] and reports[employee] (a set). Add
# inserts into both. Delete reattaches each report to the
# deleted node's manager and removes the node from its
# manager's set. Cost depends only on the number of direct
# reports.
class Org:
    def __init__(self, ceo):
        self.manager = {ceo: None}
```

```
        self.reports = {ceo: set()}

    def add(self, employee, boss):
        if employee in self.manager or boss not in self.manager:
            return False
        self.manager[employee] = boss
        self.reports[employee] = set()
        self.reports[boss].add(employee)
        return True

    def delete(self, employee):
        if employee not in self.manager:
            return False
        boss, direct_reports = self.manager[employee], self.reports[employee]
        if boss is None and len(direct_reports) != 1:
            return False
        for report in direct_reports:
            self.manager[report] = boss
            if boss is not None:
                self.reports[boss].add(report)
        if boss is not None:
            self.reports[boss].discard(employee)
        del self.manager[employee], self.reports[employee]
        return True
```

COMPLEXITY | Time O(r) per delete where r is the number of reports | Space O(n)

276 Validate parent array forms a tree

MEDIUM

Template move: climb from every node with three states (unseen, on this walk, verified); meeting your own walk is a cycle; there must be exactly one root

QUESTION

Problem

You are given an integer array **`parent`** of length **`n`** describing a directed parent pointer for each node **`i`** (nodes are labeled **`0..n-1`**).

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these **`n`** nodes form a **valid rooted tree**. A valid rooted tree must satisfy all of the following: 1.

Exactly one root (exactly one index **`i`** with **`parent[i] = -1`**). 2.
No cycles (following parent pointers from any node must eventually reach the root). 3.
Connectivity (every node is reachable from the root; equivalently, every node has exactly one simple path to the root).

Input

- **`parent`**: integer array of length **`n`** where each value is either **`-1`** or in **`[0, n-1]`**.

Output

- Return **`true`** if **`parent`** represents a valid rooted tree; otherwise return **`false`**.

Notes

/ Edge Cases - **`n`** can be 1 (then the only valid tree is **`parent[0] = -1`**).

- Self-parenting like **`parent[i] = i`** is invalid.
- Multiple **`-1`** entries (multiple roots) is invalid.
- A cycle among non-root nodes is invalid.
- A disconnected component (some nodes not reachable from the root) is invalid.

You are given an integer array **`parent`** of length **`n`** describing a parent pointer for each node **`i`** where nodes are labeled **`0`** to **`n-1`**.

QUESTION

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these nodes form a valid rooted tree. A valid rooted tree must satisfy all of the following:

1. Exactly one root.
2. No cycles.
3. Every node belongs to the same connected structure and has a path to the root. Return **`True`** if **`parent`** represents a valid rooted tree; otherwise return **`False`**.

Constraints

1 <= n <= 2 * 10^5Each **`parent[i]`** is either **`-1`** or an integer in **`[0, n-1]parent[i] = i`** is invalid

EXAMPLES

```
Input: [-1, 0, 0, 1, 1]
Expected Output: True
```

Explanation: Node 0 is the only root. All other nodes are connected under it, and there are no cycles.

```
Input: [2, -1, 1, 2]
Expected Output: True
Explanation: Node 1 is the root. The paths are 0->2->1, 2->1, and 3->2->1, so every node reaches the
    same root.
```

Hints

First count how many roots there are. A valid tree must have exactly one **`-1`** entry.Build children lists from the parent array, then traverse from the root. If you do not visit every node, the structure is disconnected or contains a cycle away from the root.

WHY IT WORKS

Check there is exactly one -1 and all other values are in range. Then walk up from each unvisited node marking nodes as 'on this walk'. Meeting a node that is on the current walk is a cycle; meeting a node already verified (or the root) ends the walk, and all nodes on it are marked verified. Each node is visited a constant number of times.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): One root, then colour each node
# once while walking up
# Idea: Check there is exactly one -1 and all other values
# are in range. Then walk up from each unvisited node
# marking nodes as 'on this walk'. Meeting a node that is
# on the current walk is a cycle; meeting a node already
# verified (or the root) ends the walk, and all nodes on
# it are marked verified. Each node is visited a constant
# number of times.
def validate_parent_array(parent):
    node_count = len(parent)
    if any(parent_id != -1 and not (0 <= parent_id < node_count) for parent_id in parent) or parent.count(-1) !
= 1:
        return False
    state = [0] * node_count                                    # 0 unseen, 1 on the current walk, 2 verified
    for start in range(node_count):
        path, current = [], start
        while current != -1 and state[current] == 0:
            state[current] = 1
            path.append(current)
            current = parent[current]
        if current != -1 and state[current] == 1:
            return False
        for node in path:
            state[node] = 2
    return True
```

COMPLEXITY | Time O(n) | Space O(n)

277 Root a Tree So It Becomes a Binary Tree — MEDIUM

Template move: the root has no parent so it needs degree <= 2; everyone else spends one edge on its parent; answer = the first node of degree <= 2

QUESTION

You are given a connected, undirected, acyclic graph (i.e. a tree) with n nodes labeled 0 .. n-1 , described by an edge list. It is guaranteed that every node has at most 3 neighbors. Choose a node to be the root. When rooted at node r , one neighbor of each non-root node becomes its parent and the rest become its children; the root has no parent, so all of its neighbors become children. The rooted tree is a binary tree if every node has at most 2 children. Return the smallest label of a node that can serve as such a root. If no node works, return -1 . Key observation (the trick): a non-root node with degree d has d - 1 children, so it needs d <= 3 ; the root has degree children, so the root needs degree <= 2 . Since all nodes already have degree <= 3 , any node with degree <= 2 is a valid root — and leaves (degree 1) always exist in a nonempty tree, so a valid root always exists for n >= 1 .

Function signature

```
from typing import List, Tuple
def find_binary_tree_root(n: int, edges: List[Tuple[int, int]]) -> int:
    """Return the smallest node label that can root the tree as a binary tree
    (every node has at most 2 children), or -1 if none exists."""
    ...
```

Follow-up (as reported): Now nodes must be 2-colored (black/white) so that every parent differs in color from its children and all nodes at the same depth share a color. Determine whether a valid root plus coloring exists and return one. Hint: coloring each node by depth % 2 from any chosen root always satisfies both rules in a tree, so a valid coloring exists whenever a valid root does.

EXAMPLES

Example 1	
Input	`n = 4, edges = [(0, 1), (0, 2), (0, 3)]`
Output	`1`
Why	`Node 0 has degree 3, so as root it would have 3 children (invalid). Nodes 1, 2, 3 have degree 1; the smallest valid root is 1.`

Example 2	
Input	`n = 3, edges = [(0, 1), (1, 2)]`
Output	`0`
Why	`A path; every node has degree <= 2, so the smallest label 0 works.`

Example 3	
Input	`n = 1, edges = []`
Output	`0`
Why	`A single node is trivially a binary tree.`

CONSTRAINTS

- 1 <= n <= 10^5 ; the input is a valid tree with n - 1 edges (for n == 1 , no edges).
- Every node has at most 3 neighbors.
- Return the smallest valid root label.

WHY IT WORKS

Only the root has as many children as its degree; every other node has one fewer.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): First node of degree at most 2
# Idea: Only the root has as many children as its degree;
# every other node has one fewer.
def find_binary_tree_root(node_count, edges):
    degree = [0] * node_count
    for node_a, node_b in edges:
        degree[node_a] += 1
        degree[node_b] += 1
    for node in range(node_count):
        if degree[node] <= 2:
            return node
    return -1
```

COMPLEXITY | Time O(n) | Space O(n)

278 Lowest Common Ancestor of a Binary Tree III

MEDIUM

Template move: two pointers climb; when one falls off the top it restarts at the other node, so both meet at the common ancestor

QUESTION

Given two nodes p and q of a binary tree, return their lowest common ancestor: the deepest node that has both of them as descendants (a node counts as a descendant of itself). Every node has a parent pointer, but you are not given the root.

Function signature

def lca_swap(p: PNode, q: PNode) -> PNode

EXAMPLES

Example 1

Input	`class PNode:`
	`def __init__(self, val):`
	`self.val, self.parent = val, None`
	`root, a, b, c = PNode(3), PNode(5), PNode(1), PNode(6)`
	`a.parent = b.parent = root`
	`c.parent = a`
	`lca_swap(c, b).val`
	`lca_swap(c, a).val`
	`lca_swap(a, b).val`

Output	`lca_swap(c, b).val -> 3`
	`lca_swap(c, a).val -> 5`
	`lca_swap(a, b).val -> 3`

Why	`Node 6 hangs under 5, which hangs under the root 3 next to node 1. The ancestor of 6 and 1 is the root 3; the ancestor of 6 and its own parent 5 is 5 itself.`

CONSTRAINTS

- Number of nodes: [2, 10^5]
- -10^9 <= Node.val <= 10^9
- All node values are unique
- p != q
- p and q exist in the tree

WHY IT WORKS

Move a from p and b from q one step up per round. When a pointer falls off the root, restart it at the other node. After at most two passes both pointers have walked the same total length, so they are equal at the lowest common ancestor (or both None if the nodes are in different trees).

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Two pointers that switch to the
# other node at the end
# Idea: Move a from p and b from q one step up per round.
# When a pointer falls off the root, restart it at the
# other node. After at most two passes both pointers have
# walked the same total length, so they are equal at the
# lowest common ancestor (or both None if the nodes are in
# different trees).
class PNode:
    def __init__(self, val):
        self.val, self.left, self.right, self.parent = val, None, None, None

def lowestCommonAncestor(first_node, second_node):
    pointer_a, pointer_b = first_node, second_node
    while pointer_a is not pointer_b:
        pointer_a = pointer_a.parent if pointer_a else second_node
        pointer_b = pointer_b.parent if pointer_b else first_node
    return pointer_a
```

COMPLEXITY | Time O(h) | Space O(1)

Build or transform a grid with index arithmetic

Pattern

Decide where each output cell comes from with a formula in (r, c), then build the whole grid with one comprehension. Rotation, diamonds, block summaries and prefix tables are all 'cell(r, c) = something computed from r and c'.

279 Filled ASCII Diamond — EASY

Template move: row i has pad = |i - R| underscores on each side and 2 * (R - pad) + 1 x's in the middle

QUESTION

Given a non-negative integer radius R , render a filled diamond on a square grid of side 2*R + 1 . The grid is indexed by (i, j) with 0 <= i, j <= 2*R , and its center is (R, R) . A cell belongs to the diamond (draw 'x') if its Manhattan distance from the center is at most R , i.e. abs(i - R) + abs(j - R) <= R . Every other cell is background (draw '_'). Return the picture as a list of 2*R + 1 strings, one per row (top to bottom), each of length 2*R + 1 .

Function signature

```
def render_diamond(R: int) -> list[str]:
    ...
```

Follow-up (as reported): The naive fill is O(R^2). Can you produce each row directly from its width instead of testing every cell? (Row i is 'x' repeated 2*(R - abs(i-R)) + 1 times, padded with '_' .)

EXAMPLES

Example 1	
Input	`R = 1`
Output	`["_x_", "xxx", "_x_"]`
Why	`Center (1,1). Row 0 has distance 1 from center vertically, so only column 1 (distance 0) is inside.`

Example 2	
Input	`R = 2`
Output	`["__x__", "_xxx_", "xxxxx", "_xxx_", "__x__"]`
Why	`Each row i allows columns within (R - abs(i-R)) of the center.`

Example 3	
Input	`R = 0`
Output	`["x"]`
Why	`A 1x1 grid whose only cell is the center.`

CONSTRAINTS

- 0 <= R <= 500 .
- For R = 0 the result is the single cell ["x"] .

WHY IT WORKS

Row i has 2 * (R - |i - R|) + 1 x's with |i - R| underscores on each side.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Build each row from its width
# Idea: Row i has 2 * (radius - |row_number - radius|) + 1 x's with |row_number - radius|
# underscores on each side.
```

```python
def render_diamond(radius):
    rows = []
    for row_number in range(2 * radius + 1):
        side_underscores = abs(row_number - radius)
        rows.append("_" * side_underscores + "x" * (2 * (radius - side_underscores) + 1) + "_" * side_underscor
es)
    return rows
```

COMPLEXITY | Time O(R^2) characters | Space O(R^2)

280 Connected Crop Arrangement on a Grid

MEDIUM

Template move: fill the cells in snake order (left to right, then right to left) so equal crops stay in one connected strip

QUESTION

A garden is an n x m grid. There are k crop types numbered 1..k, and crop i must fill exactly counts[i - 1] cells; the counts add up to n * m so every cell is used. Fill the grid so that the cells of each crop form one connected region (cells joined up, down, left or right). Return any valid grid.

Function signature

def arrange_crops(n: int, m: int, counts: list) -> list

EXAMPLES

Example 1	
Input	2, 3, [2, 3, 1]
Output	[[1, 1, 2], [3, 2, 2]]
Why	One valid answer: crop 3 is the single cell at the bottom left; crops 1 and 2 are each connected.

Example 2	
Input	1, 4, [1, 3]
Output	[[1, 2, 2, 2]]
Why	A single row: the crops simply sit side by side.

WHY IT WORKS

Generate the snake order of the cells and hand out count[0] cells to crop 1, the next count[1] to crop 2, and so on. Consecutive cells of the snake share an edge, so each crop is connected.

SHORT OPTIMAL CODE

```python
def snake_cells(rows, cols):
    # the cells of an n x m grid in "snake" order: row 0 left to right,
    # row 1 right to left, row 2 left to right, ... Consecutive cells in
    # this order always share an edge.
    path = []
    for row in range(rows):
        for col in range(cols):
            if row % 2 == 0:
                path.append((row, col))
            else:
                path.append((row, cols - 1 - col))
    return path


# BASE QUESTION (optimal): Fill crops along a snake path
# Idea: Generate the snake order of the cells and hand out
# count[0] cells to crop 1, the next count[1] to crop 2,
# and so on. Consecutive cells of the snake share an edge,
```

```
# so each crop is connected.
def arrange_crops(rows, cols, counts):
    assert sum(counts) == rows * cols
    grid = [[0] * cols for _ in range(rows)]
    path = snake_cells(rows, cols)
    path_position = 0
    for crop, cells_for_crop in enumerate(counts, 1):
        for _ in range(cells_for_crop):
            row, col = path[path_position]
            grid[row][col] = crop
            path_position += 1
    return grid
```

COMPLEXITY | Time O(n * m) | Space O(n * m)

281 Solve matrix groups and recipe inventory

MEDIUM

Template move: block (r // k, c // k) holds the minimum of every cell that maps to it; the recipe part is a topological sort

QUESTION

Two tasks. (1) block_mins(matrix, k): cut the matrix into k x k blocks (blocks at the right and bottom edge may be smaller) and return the minimum of every block as a smaller matrix. (2) possible_recipes(recipes, supplies): recipes maps a recipe name to the ingredients it needs, and an ingredient may itself be another recipe; return the recipes that can be made from the supplies.

Function signature

def block_mins(matrix: list, k: int) -> list

def possible_recipes(recipes: dict, supplies: list) -> list

EXAMPLES

Example 1	
Input	`[[3, 1, 4, 1], [5, 9, 2, 6], [5, 3, 5, 8]], 2`
Output	`[[1, 1], [3, 5]]`
Why	Top-left block (3, 1, 5, 9) has minimum 1, top-right (4, 1, 2, 6) has 1; the bottom row forms smaller blocks with minimum 3 and 5.

Example 2	
Input	`{'bread': ['flour', 'yeast'], 'sandwich': ['bread', 'ham'], 'cake': ['flour', 'egg', 'sugar']}, ['flour', 'yeast', 'ham', 'egg']`
Output	`['bread', 'sandwich']`
Why	bread needs flour and yeast (available); sandwich needs bread and ham; cake needs sugar, which is missing.

PART 1: Blocks: one pass updating a minimum per block

Each cell updates the minimum of block (r // k, c // k). No block list is built; the last blocks may be smaller when k does not divide the size.

PART 2: Recipes: Kahn's algorithm on the ingredient graph

For each recipe count the ingredients it still lacks (those not in supplies) and record, for each ingredient, the recipes that need it. Start from the supplies; when an item becomes available decrease the counts of the recipes waiting for it, and queue a recipe when its count hits zero. Recipes on a cycle never reach zero.

SHORT OPTIMAL CODE

```
from collections import defaultdict
from collections import deque
```

```python
# Blocks: one pass updating a minimum per block
# Idea: Each cell updates the minimum of block (r // block_size, c
# // block_size). No block list is built; the last blocks may be
# smaller when k does not divide the size.
def block_mins(matrix, block_size):
    rows, cols = len(matrix), len(matrix[0])
    out = [[float("inf")] * ((cols + block_size - 1) // block_size) for _ in range((rows + block_size - 1) // b
lock_size)]
    for row in range(rows):
        for col in range(cols):
            out[row // block_size][col // block_size] = min(out[row // block_size][col // block_size], matrix[r
ow][col])
    return out


# Recipes: Kahn's algorithm on the ingredient graph
# Idea: For each recipe count the ingredients it still
# lacks (those not in supplies) and record, for each
# ingredient, the recipes that need it. Start from the
# supplies; when an item becomes available decrease the
# counts of the recipes waiting for it, and queue a recipe
# when its count hits zero. Recipes on a cycle never reach
# zero.
def findAllRecipes(recipes, ingredients, supplies):
    need, waiting = {}, defaultdict(list)
    for recipe, ingredient_list in zip(recipes, ingredients):
        need[recipe] = len(set(ingredient_list))
        for ingredient in set(ingredient_list):
            waiting[ingredient].append(recipe)
    queue, made = deque(supplies), []
    while queue:
        item = queue.popleft()
        for recipe in waiting[item]:
            need[recipe] -= 1
            if need[recipe] == 0:
                made.append(recipe)
                queue.append(recipe)
    return sorted(made)
```

COMPLEXITY | Part 1: Time O(m * n) | Space O(blocks)
Part 2: Time O(R + I) | Space O(R + I)

282 Range Sum Query 2D (Immutable Matrix) MEDIUM

Template move: prefix table p[r + 1][c + 1] = cell + p[r][c + 1] + p[r + 1][c] - p[r][c]; a query is four lookups

QUESTION

Given a 2D integer matrix, handle multiple queries of the following type: return the sum of the elements inside the rectangle defined by its upper-left corner (row1, col1) and lower-right corner (row2, col2) , inclusive. Implement a class NumMatrix that is initialized once with the matrix and can then answer many sum_region queries efficiently (each query in O(1) after preprocessing).

Function signature

```
class NumMatrix:
    def __init__(self, matrix: list[list[int]]) -> None:
        """Preprocess the matrix (e.g., build a 2D prefix-sum table)."""
        ...
    def sum_region(self, row1: int, col1: int, row2: int, col2: int) -> int:
        """Return the sum of the submatrix with corners (row1, col1) and (row2, col2),
inclusive."""
        ...
```

QUESTION

Follow-up (as reported): Follow-up constraint change from the report: what if the matrix is mutable (values can be updated between queries)? Hint: switch to a 2D Binary Indexed Tree (Fenwick) for O(log rows * log cols) updates and queries.

EXAMPLES

Example 1	
Input	`matrix = [[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]] sum_region(2, 1, 4, 3)`
Output	`8`
Why	`The 3x3 block rows 2..4, cols 1..3 is [[2,0,1],[1,0,1],[0,3,0]] which sums to 8.`

Example 2	
Input	`same matrix; sum_region(1, 1, 2, 2)`
Output	`11`
Why	`Block [[6,3],[2,0]] sums to 11.`

Example 3	
Input	`same matrix; sum_region(1, 2, 2, 4)`
Output	`12`
Why	`Block [[3,2,1],[0,1,5]] sums to 12.`

CONSTRAINTS

- 1 <= rows, cols <= 200
- -10^5 <= matrix[i][j] <= 10^5
- 0 <= row1 <= row2 < rows , 0 <= col1 <= col2 < cols
- There may be up to 10^4 calls to sum_region .

WHY IT WORKS

P holds the sum of the rectangle from the top-left corner; a query combines four entries.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): 2D prefix sums and inclusion-
# exclusion
# Idea: P holds the sum of the rectangle from the top-left
# corner; a query combines four entries.
class NumMatrix:
    def __init__(self, matrix):
        rows, cols = len(matrix), len(matrix[0]) if matrix else 0
        self.prefix = [[0] * (cols + 1) for _ in range(rows + 1)]
        for row in range(rows):
            for col in range(cols):
                # the cell plus the block above and the block to the left,
                # minus the corner block that was counted twice
                self.prefix[row + 1][col + 1] = (matrix[row][col] + self.prefix[row][col + 1]
                                                 + self.prefix[row + 1][col] - self.prefix[row][col])

    def sum_region(self, top_row, left_col, bottom_row, right_col):
        prefix = self.prefix
        return (prefix[bottom_row + 1][right_col + 1] - prefix[top_row][right_col + 1]
                - prefix[bottom_row + 1][left_col] + prefix[top_row][left_col])
```

COMPLEXITY | Time O(R * C) build, O(1) per query | Space O(R * C)

283 Rotate Any m×n 2D Matrix

MEDIUM

Template move: clockwise: new[r][c] = old[rows - 1 - c][r] on a cols-by-rows result (or zip(*matrix[::-1]))

QUESTION

Rotate an m x n matrix 90 degrees clockwise and return the resulting n x m matrix.

Function signature

def rotate_cw(matrix: list) -> list

EXAMPLES

Example 1	
Input	`[[1, 2, 3], [4, 5, 6]]`
Output	`[[4, 1], [5, 2], [6, 3]]`
Why	`The last row becomes the first column: [4, 1] is the old first column read from the bottom.`

Example 2	
Input	`[[1, 2], [3, 4]]`
Output	`[[3, 1], [4, 2]]`
Why	`A square matrix.`

Example 3	
Input	`[[1, 2, 3]]`
Output	`[[1], [2], [3]]`
Why	`A single row becomes a single column.`

PART 1: Rotate 90 degrees clockwise by reverse + transpose

Rotating clockwise puts the last row first, so reverse the rows and transpose (zip with *). A non-square matrix changes shape from m x n to n x m, which is why it cannot be done in place the way LeetCode 48 is.

PART 2: Other angles, and the in-place square case

Counter-clockwise: transpose then reverse the rows (or reverse each row, then transpose). 180 degrees: reverse the rows and each row. Square matrices can be rotated in place by cycling four cells at a time, layer by layer.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Rotate 90 degrees clockwise by
# reverse + transpose
# Idea: Rotating clockwise puts the last row first, so
# reverse the rows and transpose (zip with *). A non-
# square matrix changes shape from m x n to n x m, which
# is why it cannot be done in place the way LeetCode 48
# is.
def rotate_cw(matrix):
    return [list(row) for row in zip(*matrix[::-1])]
```

COMPLEXITY | Part 1: Time O(mn) | Space O(mn)
Part 2: Time O(mn) | Space O(1) for the in-place square version

GROUP 43 OF 43 | 5 QUESTIONS | SHORT MAIN CODE

Split into pieces, fix each piece, join (and pack words into lines)

Pattern

Parsing questions split the text on a separator, transform every piece with one small rule, and join again. Layout questions go word by word and decide whether the next word still fits on the current line.

284 Implement string formatting and request tracking tasks

EASY

Template move: split, fix each piece, join; the request-tracking part is a small dict

QUESTION

Four small tasks. (A) snake_to_camel(s): convert snake_case to camelCase, ignoring leading, trailing and repeated underscores. (B) format_thousands(n): write an integer with a comma between every group of three digits (keep the minus sign). (C) ocr_valid(t, rot): an OCR scanner may read a label upside down; given the rotation table rot (character -> character seen after turning 180 degrees), return true if t, or t turned 180 degrees (reversed and mapped through rot), is an integer from 1 to 200 without leading zeros. (D) Server: begin(rid, ts) and end(rid, ts) arrive in any order for each request id; drain(watermark) returns the completed requests as 'id begin=b end=e' lines in order of end time, only those with end at most the watermark (all of them when no watermark is given).

Function signature

```
def snake_to_camel(s: str) -> str
def format_thousands(n: int) -> str
def ocr_valid(t: str, rot: dict) -> bool
class Server:
    def begin(self, rid, ts) -> None
    def end(self, rid, ts) -> None
    def drain(self, watermark=None) -> list
```

EXAMPLES

Example 1	
Input	`'make__it_camel_'`
Output	`'makeItCamel'`
Why	`Repeated and trailing underscores are ignored.`

Example 2	
Input	`1234567`
Output	`'1,234,567'`
Why	`A comma before every third digit from the right.`

Example 3	
Input	`-1000`
Output	`'-1,000'`
Why	`The sign stays at the front.`

QUESTION

Example 4

Input	`rot = {'0': '0', '1': '1', '2': '2', '5': '5', '6': '9', '8': '8', '9': '6'}`
	`ocr_valid('186', rot)`
	`ocr_valid('981', rot)`
	`ocr_valid('500', rot)`

Output	`ocr_valid('186', rot) -> true`
	`ocr_valid('981', rot) -> true`
	`ocr_valid('500', rot) -> false`

Why	981 is above 200, but turned upside down it reads 186, which is valid. 500 is too large and its rotation 005 has leading zeros.

Example 5

Input	`sv = Server()`
	`sv.begin('r2', 5)`
	`sv.end('r1', 4)`
	`sv.begin('r1', 1)`
	`sv.end('r2', 9)`
	`sv.drain(8)`
	`sv.drain()`

Output	`sv.drain(8) -> ['r1 begin=1 end=4']` `sv.drain() -> ['r2 begin=5 end=9']`
Why	r1 finished at 4 and r2 at 9. Draining with watermark 8 prints only r1; draining everything then prints r2.

PART 1: Snake to camel

Split on underscores, skip empty pieces, and capitalize every piece after the first.

PART 2: Thousands separators

Take groups of three digits from the right.

PART 3: Rotated OCR

Validate the text directly or reverse it through the rotation map and validate again.

PART 4: Request tracker

Join begin and end events by ID, then heap completed requests by end time.

SHORT OPTIMAL CODE

```python
import heapq


def snake_to_camel(text):
    words = [word for word in text.split("_") if word]
    return "" if not words else words[0] + "".join(word.capitalize() for word in words[1:])


def format_thousands(number):
    sign, digits = ("-", str(-number)) if number < 0 else ("", str(number))
    groups = []
    while digits:
        groups.append(digits[-3:])
        digits = digits[:-3]
    return sign + ",".join(reversed(groups))


def ocr_valid(text, rotations):
    def valid(value):
        return value.isdigit() and value[0] != "0" and 1 <= int(value) <= 200
```

```
    if valid(text):
        return True
    try:
        turned = "".join(rotations[ch] for ch in reversed(text))
    except KeyError:
        return False
    return valid(turned)


class Server:
    def __init__(self):
        self.pending = {}
        self.done = []

    def _add(self, request_id, timestamp, side):
        times = self.pending.setdefault(request_id, [None, None])
        if times[side] is not None:
            raise ValueError("duplicate event")
        times[side] = timestamp
        if None not in times:
            begin, end = self.pending.pop(request_id)
            if end < begin:
                raise ValueError("ends before it begins")
            heapq.heappush(self.done, (end, request_id, begin))

    def begin(self, request_id, timestamp):
        self._add(request_id, timestamp, 0)

    def end(self, request_id, timestamp):
        self._add(request_id, timestamp, 1)

    def drain(self, watermark=None):
        lines = []
        while self.done and (watermark is None or self.done[0][0] <= watermark):
            end, request_id, begin = heapq.heappop(self.done)
            lines.append(f"{request_id} begin={begin} end={end}")
        return lines
```

COMPLEXITY | Part 1: Time O(n) | Space O(n)
Part 2: Time O(d) | Space O(d)
Part 3: Time O(n) | Space O(n)
Part 4: Time O(log n) per completed request | Space O(n)

285 Snake Case to Camel Case with Repository-Wide Refactor

MEDIUM

Template move: the same function applied to every identifier found in the files

QUESTION

Convert a valid snake_case identifier to camelCase, then design a safe repository-wide migration for variables and function names without changing third-party symbols, public interfaces, or the tests' ability to detect regressions.

Implement a conversion from a valid snake_case identifier to camelCase.
The base input is guaranteed valid: no leading underscore, trailing underscore, or doubled separator.
Example: foo_bar -> fooBar.
Extend the discussion to a repository too large for manual review: replace snake-case variables and functions across the codebase.
Explain how to verify that the repository still works after the automated rewrite, including when test-package identifiers are rewritten too.
Preserve third-party functions and public interfaces; explain how the migration distinguishes symbols that may change from contracts that must remain stable.

QUESTION

EXAMPLES

```
foo_bar -> fooBar
```

WHY IT WORKS

snake_to_camel capitalises each part after the first. owned_names collects the functions, arguments and variables the code defines (using the syntax tree), minus an allow-list of names that must stay. rename_identifiers rewrites only NAME tokens that are owned and not after a dot, so strings, comments, attribute accesses and third-party calls are untouched.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Convert names, rename only
# defined identifiers via tokens
# Idea: snake_to_camel capitalises each part after the
# first. owned_names collects the functions, arguments and
# variables the code defines (using the syntax tree),
# minus an allow-list of names that must stay.
# rename_identifiers rewrites only NAME tokens that are
# owned and not after a dot, so strings, comments,
# attribute accesses and third-party calls are untouched.
def snake_to_camel(name):
    head, *rest = name.split("_")
    return head + "".join(part[:1].upper() + part[1:] for part in rest)
```

COMPLEXITY | Time O(size of the source) | Space O(names)

286 Google Doc Line Wrap (Word Wrap with Whitespace and Newlines) MEDIUM

Template move: count lines: go paragraph by paragraph, add a word if it fits (current + 1 + length <= width), else start a new line

QUESTION

EXAMPLES

```
text  = "Hello world\nFoo bar baz qux"
width = 10
Line 1: "Hello"             (5 chars; "world" + space would be 11 -> break)
Line 2: "world"
Line 3: "Foo bar"            (7 chars; "baz" -> "Foo bar baz" = 11 -> break)
Line 4: "baz qux"
Output: 4
```

WHY IT WORKS

Split on newlines first (each paragraph ends its line), then for each word compare cur_len + 1 + len(word) with the width. Only the length is tracked, never the joined line. The empty paragraph produces one empty line, and a word longer than the width sits alone on a line.

SHORT OPTIMAL CODE

```python
# BASE QUESTION (optimal): Greedy with a running line
# length
# Idea: Split on newlines first (each paragraph ends its
# line), then for each word compare cur_len + 1 +
# len(word) with the width. Only the length is tracked,
# never the joined line. The empty paragraph produces one
# empty line, and a word longer than the width sits alone
# on a line.
def wrap_count(text, width):
    total = 0
    for paragraph in text.split("\n"):
        current_length = 0
        lines = 1
```

```
        for word in paragraph.split():
            if current_length == 0:
                current_length = len(word)
            elif current_length + 1 + len(word) <= width:
                current_length += 1 + len(word)
            else:
                lines += 1
                current_length = len(word)
        total += lines
    return total
```

COMPLEXITY | Time O(n) | Space O(1) extra beyond the output

287 Convert Valid Snake Case to Lower Camel Case — HARD

Template move: reformat(): split on '_', keep the first piece, capitalise the others

QUESTION

Given a valid snake_case identifier (one or more lowercase words separated by single underscores, with no leading, trailing or doubled underscores), convert it to lowerCamelCase: keep the first word as it is, remove the underscores and capitalise the first letter of every later word.

Function signature

def snake_to_camel(s: str) -> str

EXAMPLES

Example 1	
Input	'get_http_response_code'
Output	'getHttpResponseCode'
Why	Words are joined and every word after the first is capitalised.

Example 2	
Input	'user_id'
Output	'userId'
Why	Two words.

Example 3	
Input	'name'
Output	'name'
Why	A single word has nothing to change.

WHY IT WORKS

Split on '_', keep the first word, and capitalize the first letter of each other word with a join. For valid input there are no empty parts.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Split on underscores and
# capitalize
# Idea: Split on '_', keep the first word, and capitalize
# the first letter of each other word with a join. For
# valid input there are no empty parts.
def snake_to_camel(snake_name):
    first, *rest = snake_name.split("_")
    return first + "".join(word[:1].upper() + word[1:] for word in rest)
```

COMPLEXITY | Time O(n) | Space O(n)

288 Implement a function to justify a list of words within a given width. HARD

Template move: pack_lines for the words, then spread the extra spaces over the gaps (the left gaps get the remainder); the last line is left-aligned

QUESTION

Given a list of words and a line width, pack the words into lines of exactly that width (full justification): put as many words on a line as fit (with at least one space between them), then spread the extra spaces between the words as evenly as possible, giving the leftmost gaps the extra ones. The last line, and any line with a single word, is left-justified and padded with spaces on the right. Return the list of lines.

Function signature

def fullJustify(words: list, max_width: int) -> list

EXAMPLES

Example 1	
Input	['This', 'is', 'an', 'example', 'of', 'text', 'justification.'], 16
Output	['This is an', 'example of text', 'justification. ']
Why	Three lines of width 16: the first two are spread out, the last is left-aligned and padded.

Example 2	
Input	['a', 'b'], 5
Output	['a b ']
Why	Both words fit on one line, which is the last line: left-aligned with single spaces and padded on the right.

WHY IT WORKS

Pack as many words as fit on a line (counting one space between words). Spread the spare spaces over the gaps, giving the extra ones to the leftmost gaps. The last line and any single-word line are left-justified.

SHORT OPTIMAL CODE

```
# BASE QUESTION (optimal): Greedy line packing
# Idea: Pack as many words as fit on a line (counting one
# space between words). Spread the spare spaces over the
# gaps, giving the extra ones to the leftmost gaps. The
# last line and any single-word line are left-justified.
def fullJustify(words, max_width):
    lines, first = [], 0
    while first < len(words):
        after_last, length = first, 0
        while after_last < len(words) and length + len(words[after_last]) + (after_last - first) <= max_width:
            length += len(words[after_last])
            after_last += 1
        gaps = after_last - first - 1
        if after_last == len(words) or gaps == 0:
            line = " ".join(words[first:after_last])
            line += " " * (max_width - len(line))
        else:
            spaces, extra = divmod(max_width - length, gaps)
            line = ""
            for word_index in range(first, after_last - 1):
                line += words[word_index] + " " * (spaces + (1 if word_index - first < extra else 0))
            line += words[after_last - 1]
        lines.append(line)
        first = after_last
    return lines
```

COMPLEXITY | Time O(total characters) | Space O(total characters)